GitHub Id	PRs merged	Issues opened
xtqqczze	2	1
kilasuit	1	1
davidseibel	1	
doctordns	1	
jhoffmann271	1	
KevinMarquette	1	
klitztuch	1	
markojorge	1	
perjahn	1	
schuelermine	1	
jsilverm		7
mklement0		5
cam1170		2
JustinGrote		2
peetrike		2

2020-April

- New documents
  - PowerShell documentation
    - Using Experimental Features
    - about_PSModulePath
  - Wiki articles
    - The case for and against Invoke-Expression
    - Variables can be assigned values as part of an expression (with limitations)

- Documentation maintenance
  - Now publishing updates to Microsoft Learn on a daily schedule
    - Monday-Friday 3pm Redmond Time (UTC-8)
  - Restructured the Community content
  - Many editorial cleanups
  - Updated Contributor Guide

    - Clarified some formatting rules
    - New information about table formatting

- GitHub stats
    - 74 PRs merged (8 from Community)
    - 79 issues opened (71 from Community)
    - 102 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filling issues. Thank you!

Expand table

GitHub Id	PRs merged	Issues opened
ScSurber	1	1
alexandair	1	
bmkaiser	1	
hyoshioka0128	1	
jpomfret	1	
raydixon	1	
signalwarrant	1	
mklement0		8
reinierk		3
scabon		2
Abdullah0820		2
awooga		2
Damag3d		2
Fiasco123		2
Jasonthurston		2

2020-March

- New documents
  - The PowerShell Docs community pages
    - Community resources page
    - What's new in PowerShell Docs page (this page)
    - PowerShell Infographic added to the Digital Art page
    - PowerShell-Doc contributor guide
  - New PowerShell content
    - Migrating from Windows PowerShell 5.1 to PowerShell 7
    - PowerShell 7 module compatibility list
    - Using PowerShell in Docker
  - New Wiki content
    - PowerShell prevents exceptions for non existent keys for types that implement IDictionary TKey, TValue
    - PowerShell's treatment of namespaces is case insensitive but case preserving

- Documentation maintenance
  - Massive cleanup of broken links
  - Cleanup of old and duplicate issues

- GitHub stats
  - 100 PRs merged (14 from Community)
  - 68 issues opened (56 from Community)
  - 109 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filling issues. Thank you!

- k-takai - 7 PRs
- mklement0 - 5 issues
- juvtib - 4 issues
- iSazonov - 3 issue
- doctordns - 2 issues
- mdorantesm - 2 issues
- qt3m45su0najc7 - 2 issues

2020-February

- New documents
  - about_Parameter_Sets

    - Release history of modules and cmdlets
- PowerShell 7 documentation updates
- Updates to address issues
- Ran PlatyPS to verify parameter info for all cmdlets and versions
- GitHub stats
    - 52 PRs merged (9 from Community)
    - 49 issues opened (42 from Community)
    - 55 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filling issues. Thank you!

- Maamue - 2 PRs
- doctordns - 2 PRs
- mklement0 - 4 issues
- he852100 - 3 issues
- VectorBCO - 2 issues
- metablaster - 2 issues

2020-January

- New documents
    - about_Windows_PowerShell_Compatibility
- PowerShell 7 documentation updates
- Updates to address issues
- GitHub stats
    - 58 PRs merged (7 from Community)
    - 57 issues opened (43 from Community)
    - 70 issues closed

Top Community Contributors

The following people have contributed to PowerShell docs by submitting pull requests or filling issues. Thank you!

- Makovec - 3 PRs
- mklement0 - 9 issues
- mvadu - 2 issues
- razos - 2 issues

- VLoub - 2 issues
- doctordns - 2 issues

PowerShell Digital Art

Article • 03/30/2025

The legends are true! The powerful shell that ensures safe passage to the cloud. But how?

Please enjoy the digital artwork linked below. Demonstrate to your peers that *you have been entrusted with the Scrolls of Monad!*

PowerShell Infographics

- PowerShell Infographic

Comic

- PowerShell Hero Comic (High resolution)
- PowerShell Hero Comic (Print resolution)
- PowerShell Hero Comic (Web resolution)

Wallpaper

- PowerShell Hero Comic Wallpaper (4k resolution)
- PowerShell Hero Pink Wallpaper (4k resolution)
- PowerShell Hero White Wallpaper (4k resolution)

Poster

- PowerShell Hero Poster

PowerShell Hero

- PowerShell Hero Image

Microsoft PowerShell Logo and Digital Art Guidelines

As a general rule, third parties may not use Microsoft logos and artwork without permission. The following are the limited circumstances under which third parties may use the Microsoft PowerShell logo and artwork.

- For non-commercial purposes (documentation or on a website) that reference your connection with Microsoft PowerShell.

Any uses outside of these guidelines as determined by Microsoft is strictly prohibited. Do not use the Microsoft PowerShell logo or artwork in products, product packaging, or other business services for which a formal license is required.

Microsoft reserves the right in its sole discretion to terminate or modify permission to display the logo or artwork, and may request that third parties modify or delete any use of the logo or artwork that, in Microsoft's sole judgment, does not comply with these guidelines or might otherwise impair Microsoft's rights in the logo.

Using Visual Studio Code for PowerShell Development

Visual Studio Code (VS Code) is a cross-platform script editor by Microsoft. Together with the PowerShell extension, it provides a rich and interactive script editing experience, making it easier to write reliable PowerShell scripts. Visual Studio Code with the PowerShell extension is the recommended editor for writing PowerShell scripts.

It supports the following PowerShell versions:

- PowerShell 7.2 and higher (Windows, macOS, and Linux)
- Windows PowerShell 5.1 (Windows-only) with .NET Framework 4.8

> **Note**
>
> Visual Studio Code isn't the same as Visual Studio.

Getting started

Before you begin, make sure PowerShell exists on your system. For modern workloads on Windows, macOS, and Linux, see the following links:

- Installing PowerShell on Linux
- Installing PowerShell on macOS
- Installing PowerShell on Windows

For traditional Windows PowerShell workloads, see Installing Windows PowerShell.

> **Important**
>
> The Windows PowerShell ISE is still available for Windows. However, it's no longer in active feature development. The ISE only works with PowerShell 5.1 and older. As a component of Windows, it continues to be officially supported for security and high-priority servicing fixes. we've no plans to remove the ISE from Windows.

Install VS Code and the PowerShell Extension

1. Install Visual Studio Code. For more information, see the overview Setting up Visual Studio Code.

There are installation instructions for each platform:

- Running Visual Studio Code on Windows
- Running Visual Studio Code on macOS
- Running Visual Studio Code on Linux

2. Install the PowerShell Extension.
   a. Launch the VS Code app by typing `code` in a console or `code-insiders` if you installed Visual Studio Code Insiders.
   b. Launch **Quick Open** on Windows or Linux by pressing `Ctrl` + `P`. On macOS, press `Cmd` + `P`.
   c. In Quick Open, type `ext install powershell` and press **Enter**.
   d. The **Extensions** view opens on the Side Bar. Select the PowerShell extension from Microsoft.
   e. Click the **Install** button on the PowerShell extension from Microsoft.
   f. After the install, if you see the **Install** button turn into **Reload**, Click on **Reload**.
   g. After VS Code has reloaded, you're ready for editing.

For example, to create a new file, click **File** > **New**. To save it, click **File** > **Save** and then provide a filename, such as `HelloWorld.ps1`. To close the file, click the `X` next to the filename. To exit VS Code, **File** > **Exit**.

Installing the PowerShell Extension on Restricted Systems

Some systems are set up to require validation of all code signatures. You may receive the following error:

```
Language server startup failed.
```

This problem can occur when PowerShell's execution policy is set by Windows Group Policy. To manually approve PowerShell Editor Services and the PowerShell extension for VS Code, open a PowerShell prompt and run the following command:

PowerShell

```
Import-Module $HOME\.vscode\extensions\ms-
vscode.powershell*\modules\PowerShellEditorServices\PowerShellEditorServices.psd1
```

You're prompted with **Do you want to run software from this untrusted publisher?** Type `A` to run the file. Then, open VS Code and verify that the PowerShell extension is functioning properly. If you still have problems getting started, let us know in a GitHub issue.

Choosing a version of PowerShell to use with the extension

With PowerShell installing side-by-side with Windows PowerShell, it's now possible to use a specific version of PowerShell with the PowerShell extension. This feature looks at a few well-known paths on different operating systems to discover installations of PowerShell.

Use the following steps to choose the version:

1. Open the **Command Palette** on Windows or Linux with `Ctrl` + `Shift` + `P`. On macOS, use `Cmd` + `Shift` + `P`.
2. Search for **Session**.
3. Click on **PowerShell: Show Session Menu**.
4. Choose the version of PowerShell you want to use from the list.

If you installed PowerShell to a non-typical location, it might not show up initially in the Session Menu. You can extend the session menu by adding your own custom paths as described below.

The PowerShell session menu can also be accessed from the `{}` icon in the bottom right corner of status bar. Hovering on or selecting this icon displays a shortcut to the session menu and a small pin icon. If you select the pin icon, the version number is added to the status bar. The version number is a shortcut to the session menu requiring fewer clicks.

> **Note**
>
> Pinning the version number replicates the behavior of the extension in versions of VS Code before 1.65. The 1.65 release of VS Code changed the APIs the PowerShell extension uses and standardized the status bar for language extensions.

Configuration settings for Visual Studio Code

First, if you're not familiar with how to change settings in VS Code, we recommend reading Visual Studio Code's settings documentation.

After reading the documentation, you can add configuration settings in `settings.json`.

JSON

```
{
    "editor.renderWhitespace": "all",
    "editor.renderControlCharacters": true,
    "files.trimTrailingWhitespace": true,
    "files.encoding": "utf8bom",
```

```
    "files.autoGuessEncoding": true
}
```

If you don't want these settings to affect all files types, VS Code also allows per-language configurations. Create a language-specific setting by putting settings in a `[<language-name>]` field. For example:

JSON

```
{
    "[powershell]": {
        "files.encoding": "utf8bom",
        "files.autoGuessEncoding": true
    }
}
```

> **Tip**
>
> For more information about file encoding in VS Code, see Understanding file encoding. Also, check out How to replicate the ISE experience in VS Code for other tips on how to configure VS Code for PowerShell editing.

Adding your own PowerShell paths to the session menu

You can add other PowerShell executable paths to the session menu through the Visual Studio Code setting: `powershell.powerShellAdditionalExePaths`.

You can do this using the GUI:

1. From the **Command Palette** search for and select **Open User Settings**. Or use the keyboard shortcut on Windows or Linux `Ctrl` + `,`. On macOS, use `Cmd` + `,`.
2. In the **Settings** editor, search for **PowerShell Additional Exe Paths**.
3. Click **Add Item**.
4. For the **Key** (under **Item**), provide your choice of name for this additional PowerShell installation.
5. For the **Value** (under **Value**), provide the absolute path to the executable itself.

You can add as many additional paths as you like. The added items show up in the session menu with the given key as the name.

Alternatively you can add key-value pairs to the object `powershell.powerShellAdditionalExePaths` in your `settings.json`:

JSON

```
{
    "powershell.powerShellAdditionalExePaths": {
        "Downloaded PowerShell": "C:/Users/username/Downloads/PowerShell/pwsh.exe",
        "Built PowerShell": "C:/Users/username/src/PowerShell/src/powershell-win-
core/bin/Debug/net6.0/win7-x64/publish/pwsh.exe"
    },
}
```

> **Note**
>
> Prior to version 2022.5.0 of the extension, this setting was a list of objects with the required keys `exePath` and `versionName`. A breaking change was introduced to support configuration via GUI. If you had previously configured this setting, please convert it the new format. The value given for `versionName` is now the **Key**, and the value given for `exePath` is now the **Value**. You can do this more easily by resetting the value and using the Settings interface.

To set the default PowerShell version, set the value `powershell.powerShellDefaultVersion` to the text displayed in the session menu (the text used for the key):

JSON

```
{
    "powershell.powerShellAdditionalExePaths": {
        "Downloaded PowerShell": "C:/Users/username/Downloads/PowerShell/pwsh.exe",
    },
    "powershell.powerShellDefaultVersion": "Downloaded PowerShell",
}
```

After you've configured this setting, restart VS Code or to reload the current VS Code window from the **Command Palette**, type `Developer: Reload Window`.

If you open the session menu, you now see your additional PowerShell installations.

> **Tip**
>
> If you build PowerShell from source, this is a great way to test out your local build of PowerShell.

Debugging with Visual Studio Code

No-workspace debugging

In VS Code version 1.9 (or higher), you can debug PowerShell scripts without opening the folder that contains the PowerShell script.

1. Open the PowerShell script file with **File** > **Open File...**
2. Set a breakpoint - select a line then press `F9`
3. Press `F5` to start debugging

You should see the Debug actions pane appear that allows you to break into the debugger, step, resume, and stop debugging.

Workspace debugging

Workspace debugging refers to debugging in the context of a folder that you've opened from the **File** menu using **Open Folder...**. The folder you open is typically your PowerShell project folder or the root of your Git repository. Workspace debugging allows you to define multiple debug configurations other than just debugging the currently open file.

Follow these steps to create a debug configuration file:

1. Open the **Debug** view on Windows or Linux by pressing `Ctrl` + `Shift` + `D`. On macOS, press `Cmd` + `Shift` + `D`.

2. Click the **create a launch.json file** link.

3. From the **Select Environment** prompt, choose **PowerShell**.

4. Choose the type of debugging you'd like to use:

    - **Launch Current File** - Launch and debug the file in the currently active editor window
    - **Launch Script** - Launch and debug the specified file or command
    - **Interactive Session** - Debug commands executed from the Integrated Console
    - **Attach** - Attach the debugger to a running PowerShell Host Process

VS Code creates a directory and a file `.vscode\launch.json` in the root of your workspace folder to store the debug configuration. If your files are in a Git repository, you typically want to commit the `launch.json` file. The contents of the `launch.json` file are:

JSON

```
{
  "version": "0.2.0",
  "configurations": [
```

```
        {
            "type": "PowerShell",
            "request": "launch",
            "name": "PowerShell Launch (current file)",
            "script": "${file}",
            "args": [],
            "cwd": "${file}"
        },
        {
            "type": "PowerShell",
            "request": "attach",
            "name": "PowerShell Attach to Host Process",
            "processId": "${command.PickPSHostProcess}",
            "runspaceId": 1
        },
        {
            "type": "PowerShell",
            "request": "launch",
            "name": "PowerShell Interactive Session",
            "cwd": "${workspaceRoot}"
        }
    ]
}
```

This file represents the common debug scenarios. When you open this file in the editor, you see an **Add Configuration...** button. You can click this button to add more PowerShell debug configurations. One useful configuration to add is **PowerShell: Launch Script**. With this configuration, you can specify a file containing optional arguments that are used whenever you press `F5` no matter which file is active in the editor.

After the debug configuration is established, you can select the configuration you want to use during a debug session. Select a configuration from the debug configuration drop-down in the **Debug** view's toolbar.

Troubleshooting the PowerShell extension

If you experience any issues using VS Code for PowerShell script development, see the troubleshooting guide on GitHub.

Useful resources

There are a few videos and blog posts that may be helpful to get you started using the PowerShell extension for VS Code:

Videos

- Using Visual Studio Code as Your Default PowerShell Editor
- Visual Studio Code: deep dive into debugging your PowerShell scripts

Blog posts

- PowerShell Extension
- Write and debug PowerShell scripts in Visual Studio Code
- Debugging Visual Studio Code Guidance
- Debugging PowerShell in Visual Studio Code
- Get started with PowerShell development in Visual Studio Code
- Visual Studio Code editing features for PowerShell development - Part 1
- Visual Studio Code editing features for PowerShell development - Part 2
- Debugging PowerShell script in Visual Studio Code - Part 1
- Debugging PowerShell script in Visual Studio Code - Part 2

PowerShell extension project source code

The PowerShell extension's source code can be found on GitHub.

If you're interested in contributing, Pull Requests are greatly appreciated. Follow along with the developer documentation on GitHub to get started.

Last updated on 12/08/2025

How to replicate the ISE experience in Visual Studio Code

While the PowerShell extension for VS Code doesn't seek complete feature parity with the PowerShell ISE, there are features in place to make the VS Code experience more natural for users of the ISE.

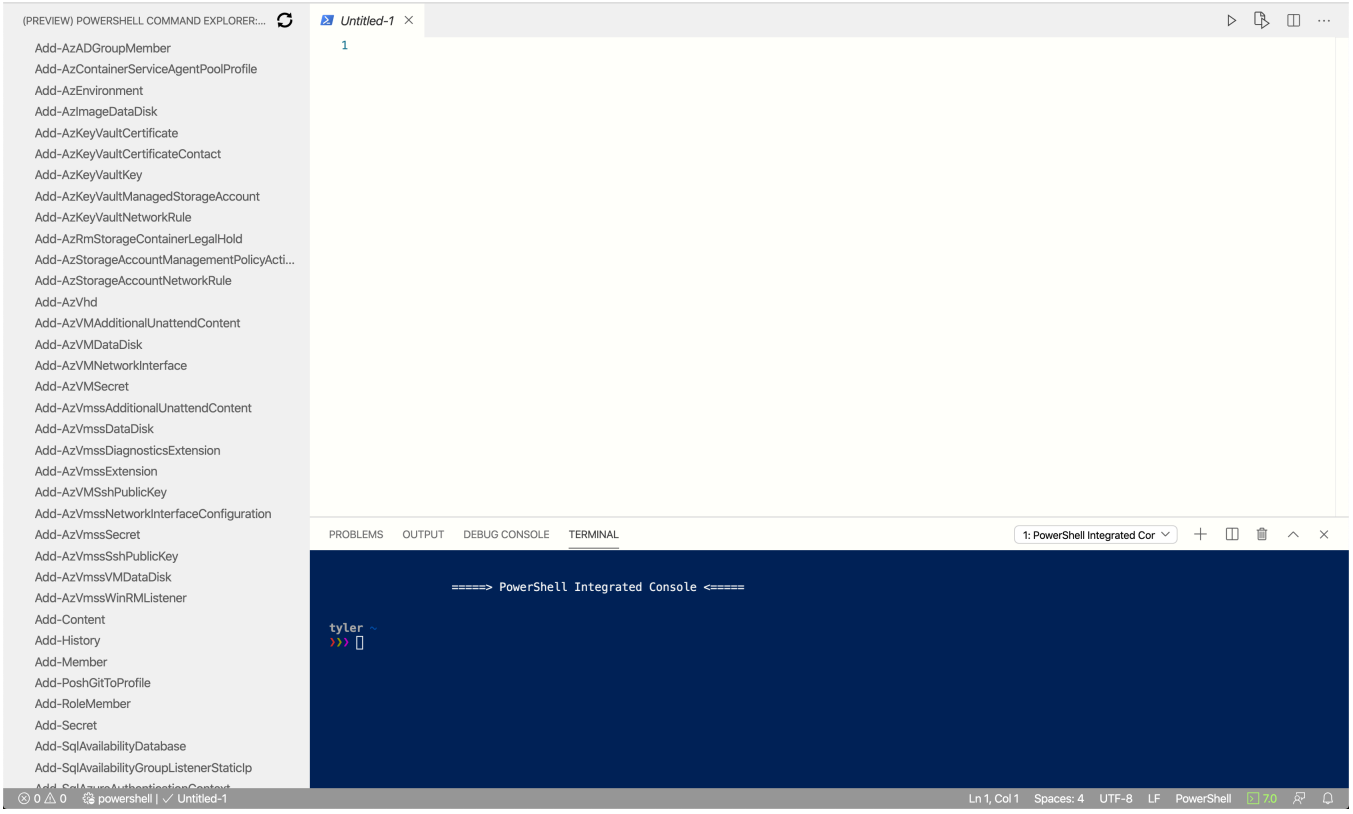
This document tries to list settings you can configure in VS Code to make the user experience a bit more familiar compared to the ISE.

ISE Mode

> **Note**
>
> This feature is available in the PowerShell Preview extension since version 2019.12.0 and in the PowerShell extension since version 2020.3.0.

The easiest way to replicate the ISE experience in Visual Studio Code is by turning on "ISE Mode". To do this, open the command palette (`F1` OR `Ctrl` + `Shift` + `P` OR `Cmd` + `Shift` + `P` on macOS) and type in "ISE Mode". Select "PowerShell: Enable ISE Mode" from the list.

This command automatically applies the settings described below The result looks like this:



ISE Mode configuration settings

ISE Mode makes the following changes to VS Code settings.

- Key bindings

Expand table

Function	ISE Binding	VS Code Binding
Interrupt and break debugger	`Ctrl` + `B`	`F6`
Execute current line/highlighted text	`F8`	`F8`
List available snippets	`Ctrl` + `J`	`Ctrl` + `Alt` + `J`

> **Note**
>
> You can configure your own key bindings in VS Code as well.

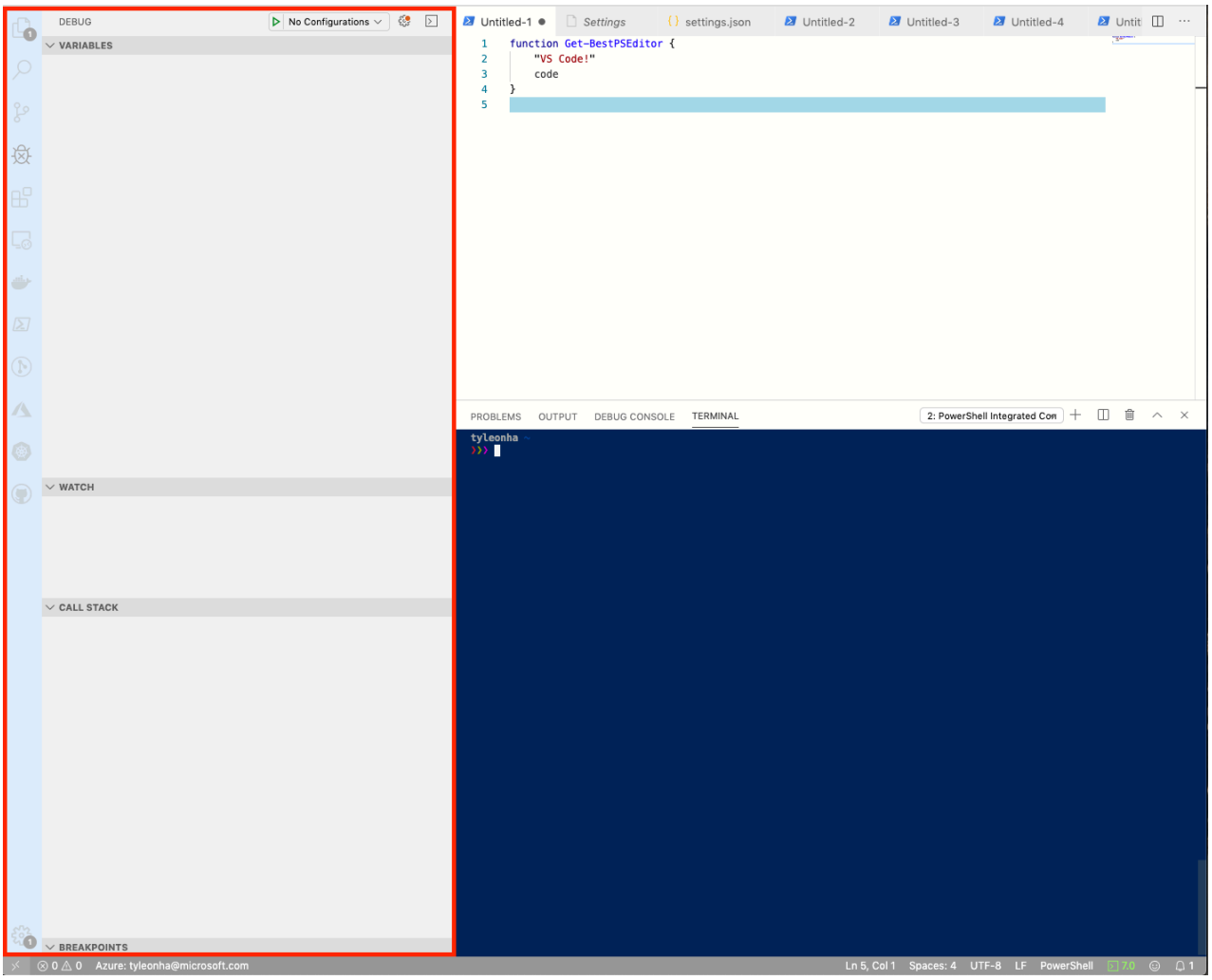
- Simplified ISE-like UI

  If you're looking to simplify the Visual Studio Code UI to look more closely to the UI of the ISE, apply these two settings:

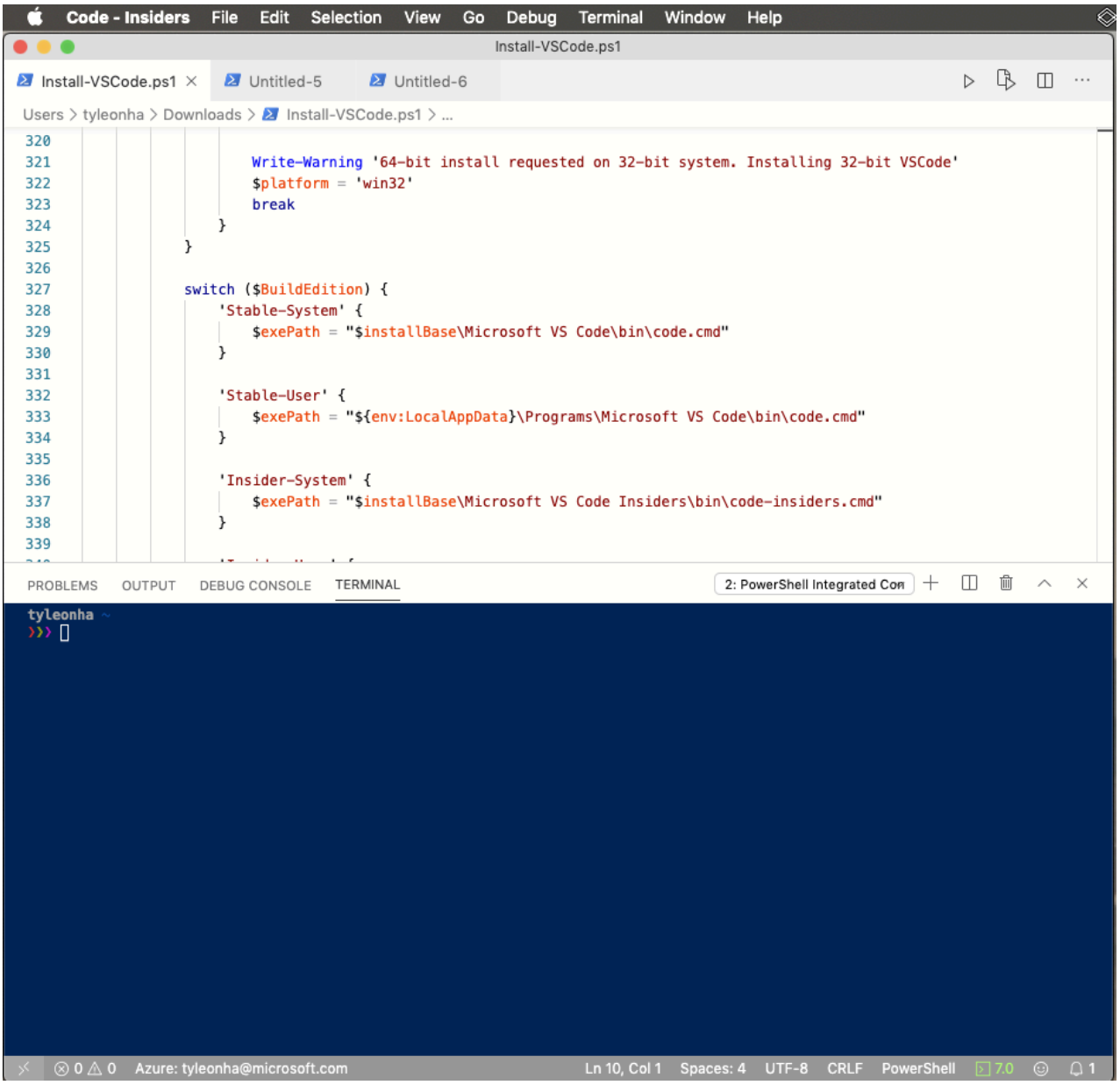
  JSON

```
"workbench.activityBar.visible": false,
"debug.openDebug": "neverOpen",
```

  These settings hide the "Activity Bar" and the "Debug Side Bar" sections shown inside the red box below:



The end result looks like this:



- Tab completion

  To enable more ISE-like tab completion, add this setting:

  JSON

  ```
  "editor.tabCompletion": "on",
  ```

- No focus on console when executing

  To keep the focus in the editor when you execute with `F8`:

  JSON

  ```
  "powershell.integratedConsole.focusConsoleOnExecute": false
  ```

  The default is `true` for accessibility purposes.

- Don't start integrated console on startup

  To stop the integrated console on startup, set:

  ```
  JSON
  "powershell.integratedConsole.showOnStartup": false
  ```

  > **Note**
  >
  > The background PowerShell process still starts to provide IntelliSense, script analysis, symbol navigation, etc., but the console won't be shown.

- Assume files are PowerShell by default

  To make new/untitled files, register as PowerShell by default:

  ```
  JSON
  "files.defaultLanguage": "powershell",
  ```

- Color scheme

  There are a number of ISE themes available for VS Code to make the editor look much more like the ISE.

  In the Command Palette type `theme` to get `Preferences: Color Theme` and press `Enter`. In the drop-down list, select `PowerShell ISE`.

  You can set this theme in the settings with:

  ```
  JSON
  "workbench.colorTheme": "PowerShell ISE",
  ```

- PowerShell Command Explorer

  Thanks to the work of @corbob, the PowerShell extension has the beginnings of its own command explorer.

  In the Command Palette, enter `PowerShell Command Explorer` and press `Enter`.

- Open in the ISE

If you want to open a file in the Windows PowerShell ISE anyway, open the Command Palette, search for "open in ise", then select **PowerShell: Open Current File in PowerShell ISE**.

Other resources

- 4sysops has a great article on configuring VS Code to be more like the ISE.
- Mike F Robbins has a great post on setting up VS Code.

VS Code Tips

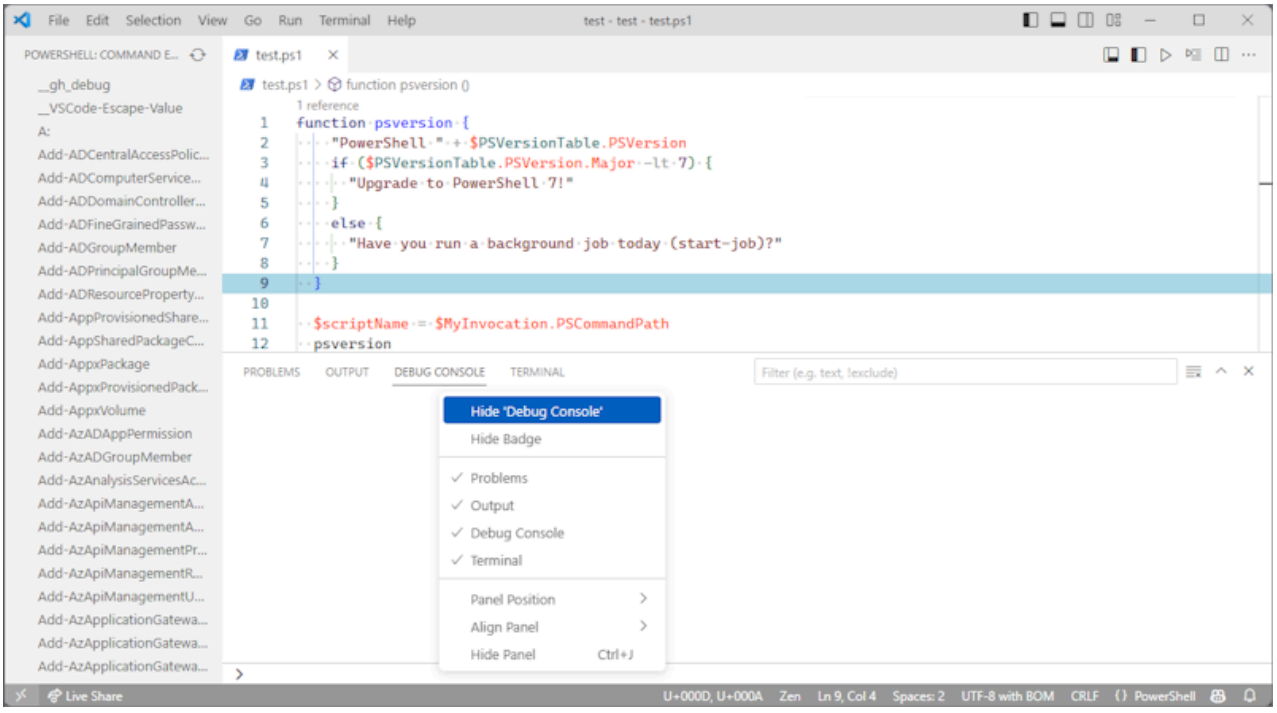
- Command Palette

  The Command Palette is handy way of executing commands in VS Code. Open the command palette using `F1` OR `Ctrl` + `Shift` + `P` OR `Cmd` + `Shift` + `P` on macOS.

  For more information, see the VS Code documentation.

- Hide the Debug Console panel

  The PowerShell extension uses the built-in debugging interface of VS Code to allow for debugging of PowerShell scripts and modules. However, the extension does not use the Debug Console panel. To hide the Debug Console, right-click on **Debug Console** and select **Hide 'Debug Console'**.



For more information about debugging PowerShell with Visual Studio Code, see Using VS Code.

More settings

If you know of more ways to make VS Code feel more familiar for ISE users, contribute to this doc. If there's a compatibility configuration you're looking for, but you can't find any way to enable it, open an issue and ask away!

We're always happy to accept PRs and contributions as well!

Last updated on 12/08/2025

Using Visual Studio Code for remote editing and debugging

For those of you that are familiar with the ISE, you may recall that you could run `psedit file.ps1` from the integrated console to open files - local or remote - right in the ISE.

This feature is also available in the PowerShell extension for VS Code. This guide shows you how to do it.

Prerequisites

This guide assumes that you have:

- A remote resource (ex: a VM, a container) that you have access to
- PowerShell running on it and the host machine
- VS Code and the PowerShell extension for VS Code

This feature works on PowerShell and Windows PowerShell.

This feature also works when connecting to a remote machine via WinRM, PowerShell Direct, or SSH. If you want to use SSH, but are using Windows, check out the Win32 version of SSH !

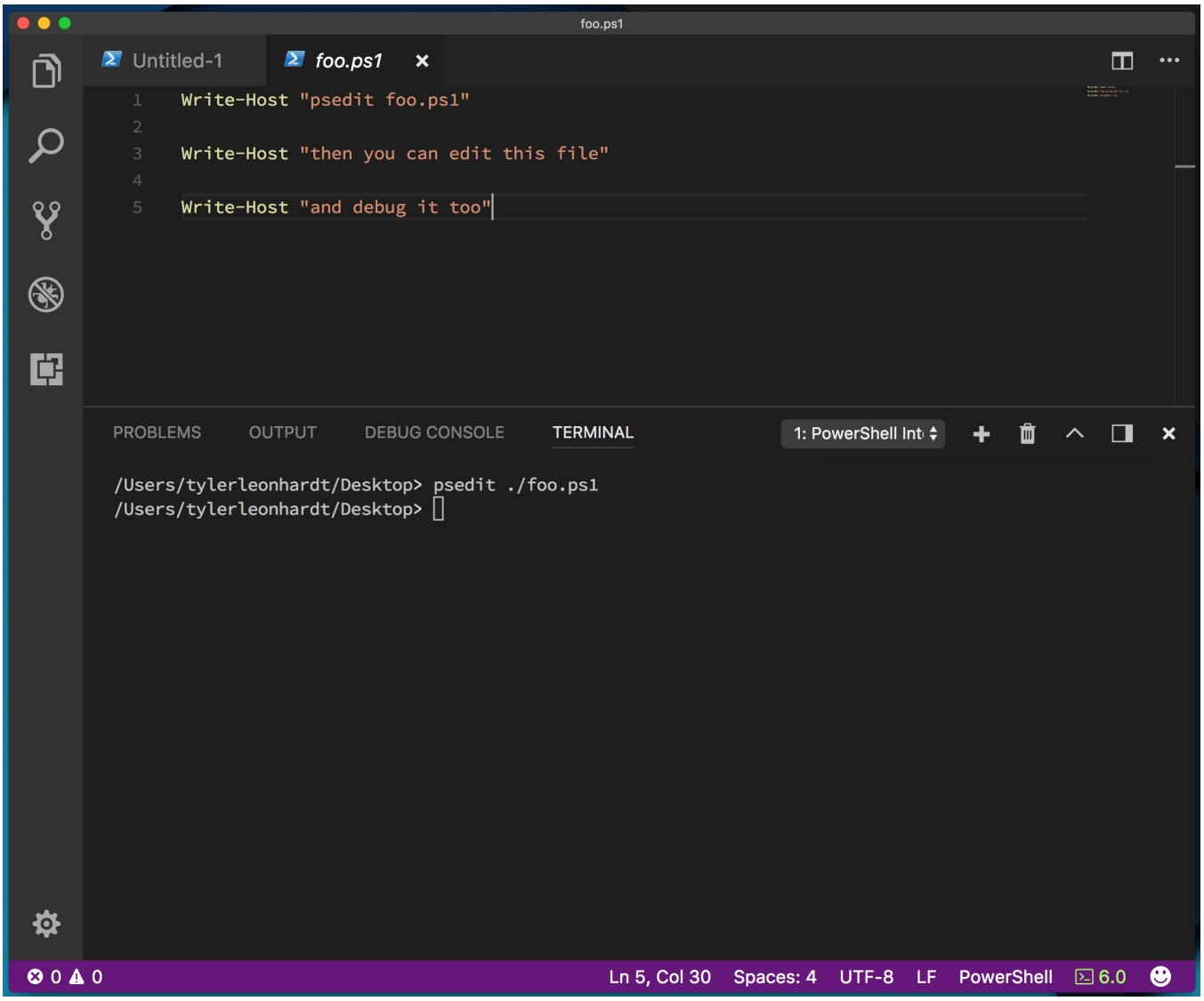
> **Important**
>
> The `Open-EditorFile` and `psedit` commands only work in the **PowerShell Integrated Console** created by the PowerShell extension for VS Code.

Usage examples

These examples show remote editing and debugging from a MacBook Pro to an Ubuntu VM running in Azure. The process is identical on Windows.

Local file editing with Open-EditorFile

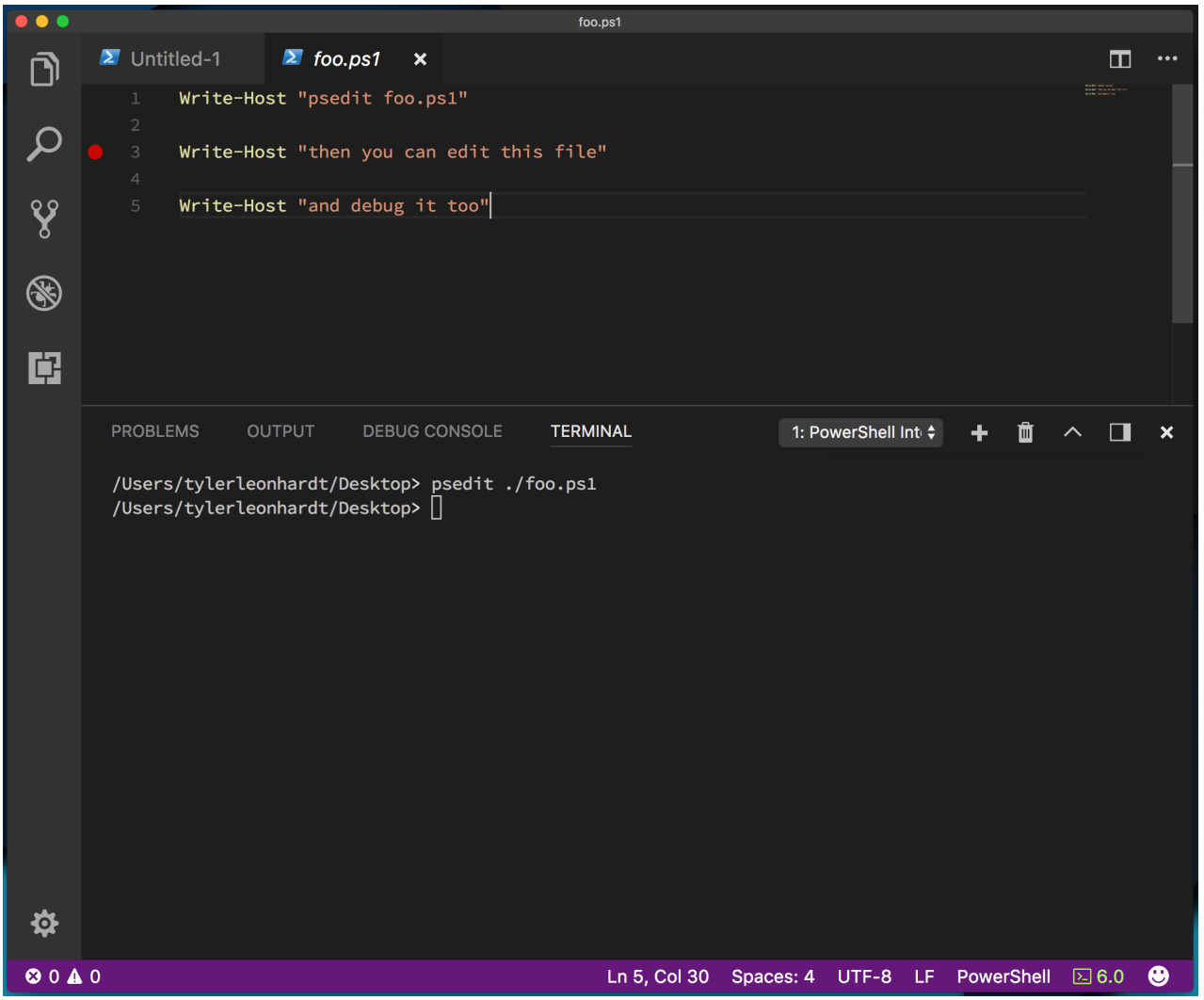
With the PowerShell extension for VS Code started and the PowerShell Integrated Console opened, we can type `Open-EditorFile foo.ps1` or `psedit foo.ps1` to open the local foo.ps1 file right in the editor.



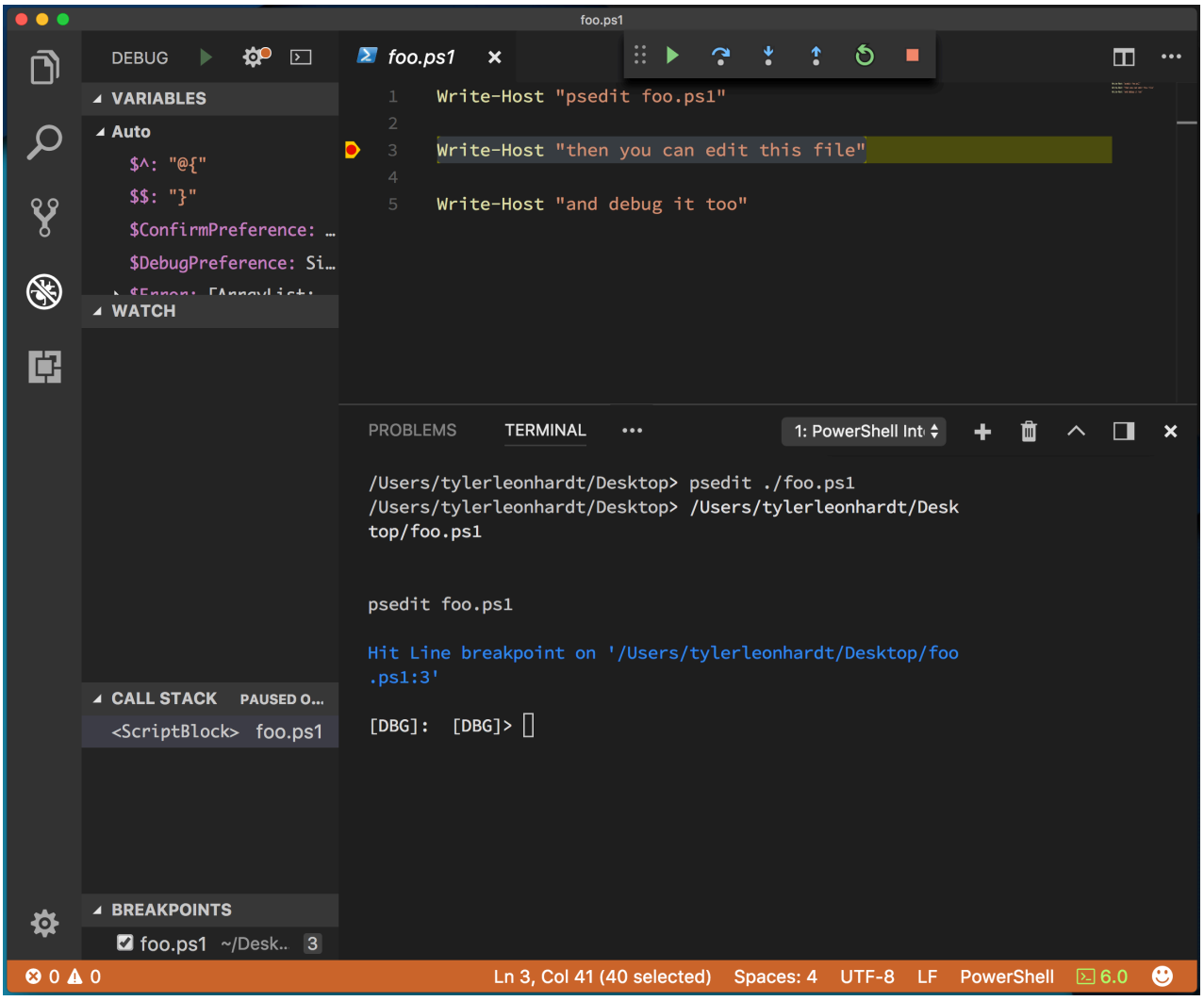
> **Note**
>
> The file `foo.ps1` must already exist.

From there, we can:

- Add breakpoints to the gutter



- Hit F5 to debug the PowerShell script.



While debugging, you can interact with the debug console, check out the variables in the scope on the left, and all the other standard debugging tools.

Remote file editing with Open-EditorFile

Now let's get into remote file editing and debugging. The steps are nearly the same, there's just one thing we need to do first - enter our PowerShell session to the remote server.

There's a cmdlet for to do so. It's called `Enter-PSSession`.

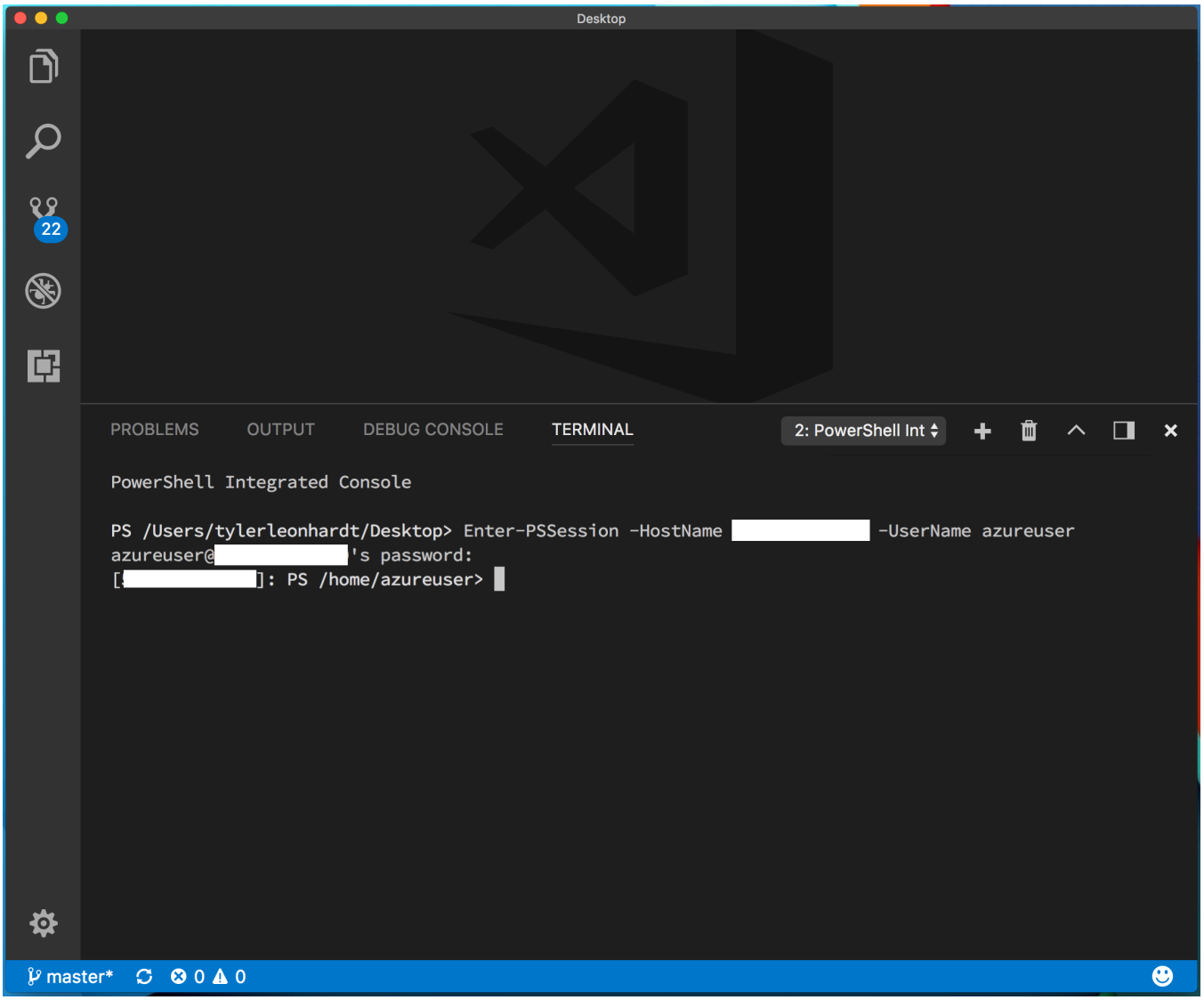
In short:

- `Enter-PSSession -ComputerName foo` starts a session via WinRM
- `Enter-PSSession -ContainerId foo` and `Enter-PSSession -VmId foo` start a session via PowerShell Direct
- `Enter-PSSession -HostName foo` starts a session via SSH

For more information, see the documentation for Enter-PSSession.

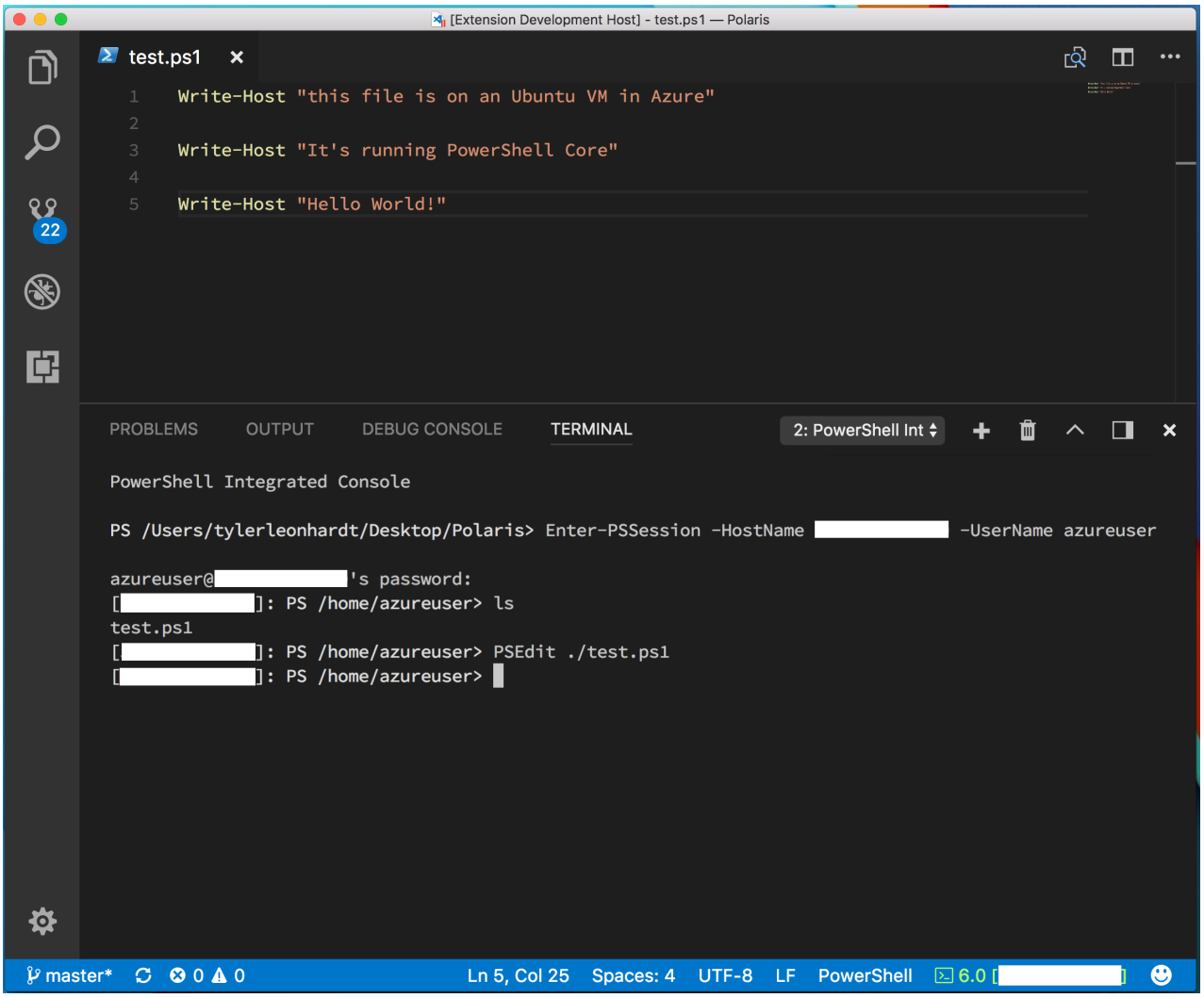
Since we're remoting to an Ubuntu VM in Azure, we're using SSH.

First, in the Integrated Console, run `Enter-PSSession`. You're connected to the remote session when `[<hostname>]` shows up to the left of your prompt.

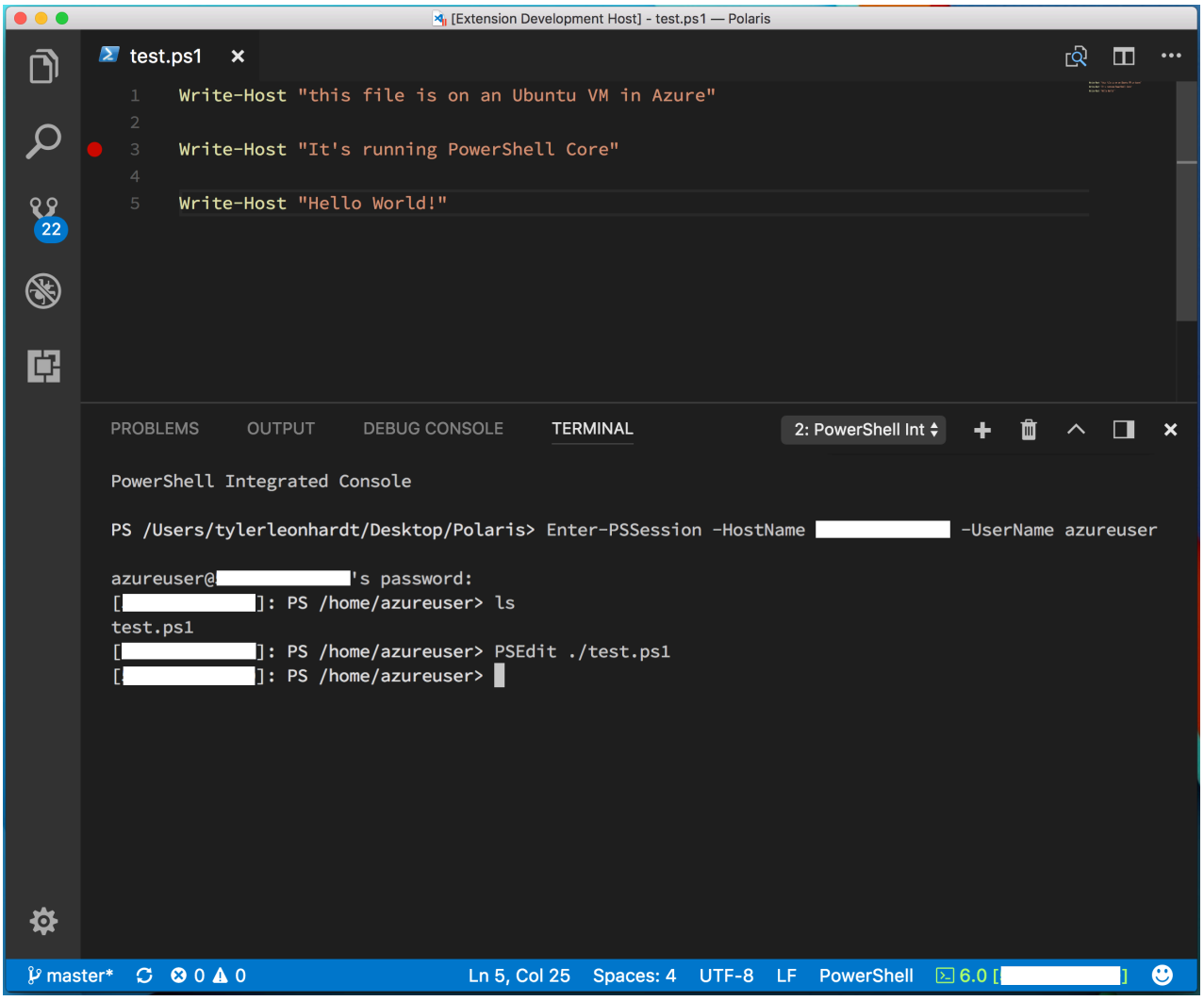


Now, we can do the same steps as if we're editing a local script.

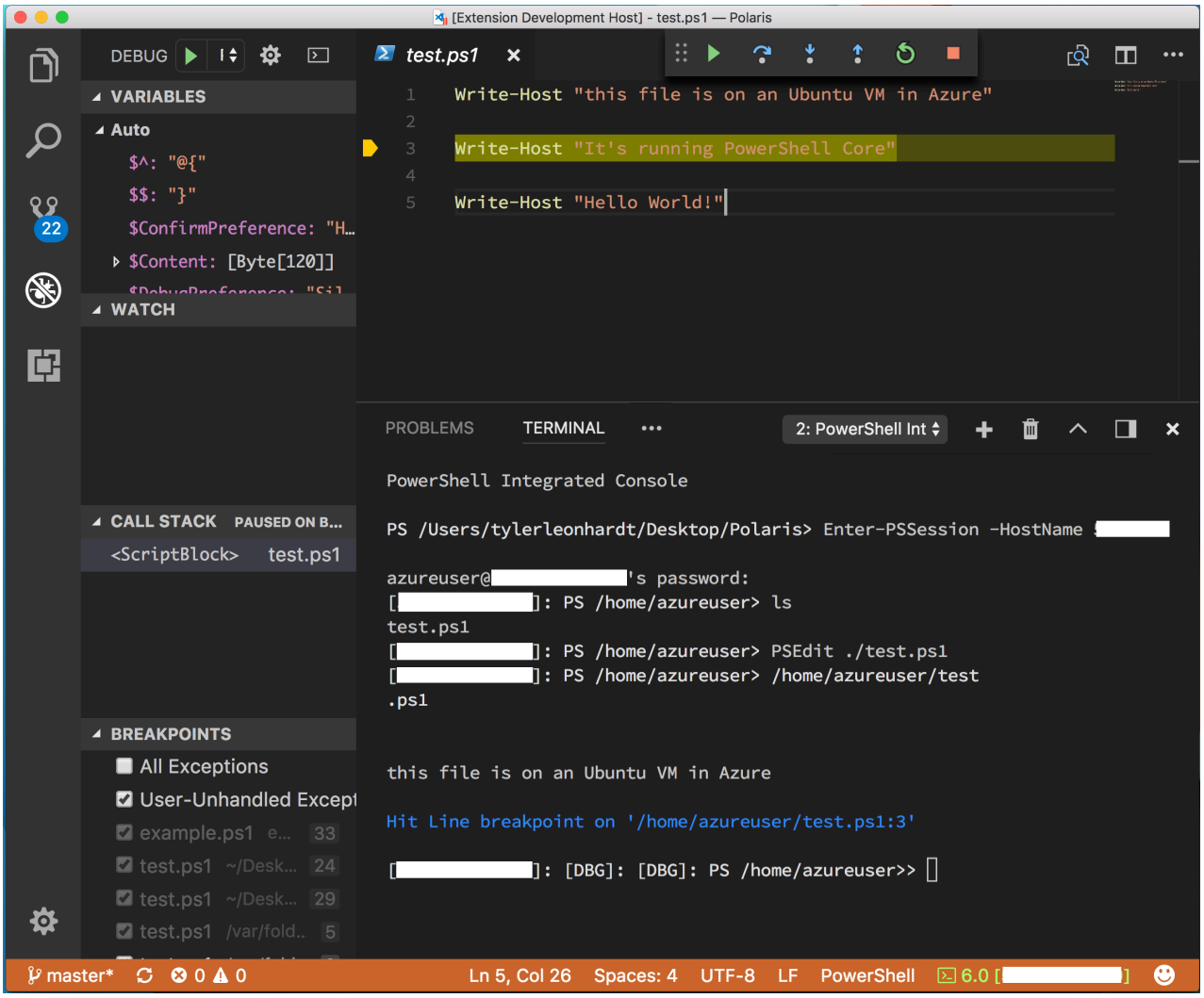
1. Run `Open-EditorFile test.ps1` or `psedit test.ps1` to open the remote `test.ps1` file



2. Edit the file/set breakpoints



3. Start debugging (F5) the remote file



If you have any problems, you can open issues in the GitHub repo.

Last updated on 12/08/2025

Understanding file encoding in VS Code and PowerShell

When using VS Code to create and edit PowerShell scripts, it's important that your files are saved using the correct character encoding format.

What is file encoding and why is it important?

VS Code manages the interface between a human entering strings of characters into a buffer and reading/writing blocks of bytes to the filesystem. When VS Code saves a file, it uses a text encoding to decide what bytes each character becomes. For more information, see about_Character_Encoding.

Similarly, when PowerShell runs a script it must convert the bytes in a file to characters to reconstruct the file into a PowerShell program. Since VS Code writes the file and PowerShell reads the file, they need to use the same encoding system. This process of parsing a PowerShell script goes: *bytes -> characters -> tokens -> abstract syntax tree -> execution*.

Both VS Code and PowerShell are installed with a sensible default encoding configuration. However, the default encoding used by PowerShell has changed with the release of PowerShell 6. To ensure you have no problems using PowerShell or the PowerShell extension in VS Code, you need to configure your VS Code and PowerShell settings properly.

Common causes of encoding issues

Encoding problems occur when the encoding of VS Code or your script file doesn't match the expected encoding of PowerShell. There is no way for PowerShell to automatically determine the file encoding.

You're more likely to have encoding problems when you're using characters not in the 7-bit ASCII character set. For example:

- Extended non-letter characters like em-dash (`–`), non-breaking space ( ) or left double quotation mark (`"`)
- Accented latin characters (`é`, `ü`)
- Non-latin characters like Cyrillic (`д`, `ц`)
- CJK characters (`本`, `화`, `が`)

Common reasons for encoding issues are:

- The encodings of VS Code and PowerShell haven't been changed from their defaults. For PowerShell 5.1 and below, the default encoding is different from VS Code's.
- Another editor has opened and overwritten the file in a new encoding. This often happens with the ISE.
- The file is checked into source control in an encoding that's different from what VS Code or PowerShell expects. This can happen when collaborators use editors with different encoding configurations.

How to tell when you have encoding issues

Often encoding errors present themselves as parse errors in scripts. If you find strange character sequences in your script, this can be the problem. In the example below, an en-dash (`–`) appears as the characters `â€"`:

Output

```
Send-MailMessage : A positional parameter cannot be found that accepts argument
'Testing FuseMail SMTP...'.
At C:\Users\<User>\<OneDrive>\Development\PowerShell\Scripts\Send-
EmailUsingSmtpRelay.ps1:6 char:1
+ Send-MailMessage â&euro;"From $from â&euro;"To $recipient1 â&euro;"Subject
$subject  ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument: (:) [Send-MailMessage],
ParameterBindingException
    + FullyQualifiedErrorId :
PositionalParameterNotFound,Microsoft.PowerShell.Commands.SendMailMessage
```

This problem occurs because VS Code encodes the character `–` in UTF-8 as the bytes `0xE2 0x80 0x93`. When these bytes are decoded as Windows-1252, they're interpreted as the characters `â€"`.

Some strange character sequences that you might see include:

- `â€"` instead of `–` (an en-dash)
- `â€"` instead of `—` (an em-dash)
- `Ã„2` instead of `Ä`
- `Â` instead of ` ` (a non-breaking space)
- `Ã©` instead of `é`

This handy reference lists the common patterns that indicate a UTF-8/Windows-1252 encoding problem.

How the PowerShell extension in VS Code interacts with encodings

The PowerShell extension interacts with scripts in a number of ways:

1. When scripts are edited in VS Code, the contents are sent by VS Code to the extension. The Language Server Protocol mandates that this content is transferred in UTF-8. Therefore, it isn't possible for the extension to get the wrong encoding.
2. When scripts are executed directly in the Integrated Console, they're read from the file by PowerShell directly. If PowerShell's encoding differs from VS Code's, something can go wrong here.
3. When a script that's open in VS Code references another script that isn't open in VS Code, the extension falls back to loading that script's content from the file system. The PowerShell extension defaults to UTF-8 encoding, but uses byte-order mark, or BOM, detection to select the correct encoding.

The problem occurs when assuming the encoding of BOM-less formats (like UTF-8 with no BOM and Windows-1252). The PowerShell extension defaults to UTF-8. The extension can't change VS Code's encoding settings. For more information, see issue #824.

Choosing the right encoding

Different systems and applications can use different encodings:

- In .NET Standard, on the web, and in the Linux world, UTF-8 is now the dominant encoding.
- Many .NET Framework applications use UTF-16. For historical reasons, this is sometimes called "Unicode", a term that now refers to a broad standard that includes both UTF-8 and UTF-16.
- On Windows, many native applications that predate Unicode continue to use Windows-1252 by default.

Unicode encodings also have the concept of a byte-order mark (BOM). BOMs occur at the beginning of text to tell a decoder which encoding the text is using. For multi-byte encodings, the BOM also indicates endianness of the encoding. BOMs are designed to be bytes that rarely occur in non-Unicode text, allowing a reasonable guess that text is Unicode when a BOM is present.

BOMs are optional and their adoption isn't as popular in the Linux world because a dependable convention of UTF-8 is used everywhere. Most Linux applications presume that text input is encoded in UTF-8. While many Linux applications will recognize and correctly handle a BOM, a number don't, leading to artifacts in text manipulated with those applications.

Therefore:

- If you work primarily with Windows applications and Windows PowerShell, you should prefer an encoding like UTF-8 with BOM or UTF-16.
- If you work across platforms, you should prefer UTF-8 with BOM.
- If you work mainly in Linux-associated contexts, you should prefer UTF-8 without BOM.
- Windows-1252 and latin-1 are essentially legacy encodings that you should avoid if possible. However, some older Windows applications may depend on them.
- It's also worth noting that script signing is encoding-dependent, meaning a change of encoding on a signed script will require resigning.

Configuring VS Code

VS Code's default encoding is UTF-8 without BOM.

To set VS Code's encoding, go to the VS Code settings (`Ctrl` + `,`) and set the `"files.encoding"` setting:

JSON

```
"files.encoding": "utf8bom"
```

Some possible values are:

- `utf8`: [UTF-8] without BOM
- `utf8bom`: [UTF-8] with BOM
- `utf16le`: Little endian [UTF-16]
- `utf16be`: Big endian [UTF-16]
- `windows1252`: [Windows-1252]

You should get a dropdown for this in the GUI view, or completions for it in the JSON view.

You can also add the following to autodetect encoding when possible:

JSON

```
"files.autoGuessEncoding": true
```

If you don't want these settings to affect all files types, VS Code also allows per-language configurations. Create a language-specific setting by putting settings in a `[<language-name>]` field. For example:

JSON

```
"[powershell]": {
    "files.encoding": "utf8bom",
    "files.autoGuessEncoding": true
}
```

You may also want to consider installing the Gremlins tracker for Visual Studio Code. This extension reveals certain Unicode characters that easily corrupted because they're invisible or look like other normal characters.

Configuring PowerShell

PowerShell's default encoding varies depending on version:

- In PowerShell 6+, the default encoding is UTF-8 without BOM on all platforms.
- In Windows PowerShell, the default encoding is usually Windows-1252, which is an extension of latin-1 (also known as ISO 8859-1).

In PowerShell 5+ you can find your default encoding with this:

PowerShell

```
[psobject].Assembly.GetTypes() | Where-Object { $_.Name -eq 'ClrFacade'} |
  ForEach-Object {
    $_.GetMethod('GetDefaultEncoding',
[System.Reflection.BindingFlags]'nonpublic,static').Invoke($null, @())
  }
```

The following script can be used to determine what encoding your PowerShell session infers for a script without a BOM.

PowerShell

```
$badBytes = [byte[]]@(0xC3, 0x80)
$utf8Str = [System.Text.Encoding]::UTF8.GetString($badBytes)
$bytes = [System.Text.Encoding]::ASCII.GetBytes('Write-Output "') + [byte[]]@(0xC3,
0x80) + [byte[]]@(0x22)
$path = Join-Path ([System.IO.Path]::GetTempPath()) 'encodingtest.ps1'

try
{
    [System.IO.File]::WriteAllBytes($path, $bytes)

    switch (& $path)
    {
        $utf8Str
        {
            return 'UTF-8'
            break
```

```
        }

        default
        {
            return 'Windows-1252'
            break
        }
    }
}
finally
{
    Remove-Item $path
}
```

It's possible to configure PowerShell to use a given encoding more generally using profile settings. See the following articles:

- @mklement0's answer about PowerShell encoding on Stack Overflow.
- @rkeithhill's blog post about dealing with BOM-less UTF-8 input in PowerShell.

It's not possible to force PowerShell to use a specific input encoding. PowerShell 5.1 and below, running on Windows with the locale set to en-US, defaults to Windows-1252 encoding when there's no BOM. Other locale settings may use a different encoding. To ensure interoperability, it's best to save scripts in a Unicode format with a BOM.

> **Important**
>
> Any other tools you have that touch PowerShell scripts may be affected by your encoding choices or re-encode your scripts to another encoding.

Existing scripts

Scripts already on the file system may need to be re-encoded to your new chosen encoding. In the bottom bar of VS Code, you'll see the label UTF-8. Click it to open the action bar and select **Save with encoding**. You can now pick a new encoding for that file. See VS Code's encoding for full instructions.

If you need to re-encode multiple files, you can use the following script:

PowerShell

```
Get-ChildItem *.ps1 -Recurse | ForEach-Object {
    $content = Get-Content -Path $_
    Set-Content -Path $_.FullName -Value $content -Encoding UTF8 -PassThru -Force
}
```

The PowerShell Integrated Scripting Environment (ISE)

If you also edit scripts using the PowerShell ISE, you need to synchronize your encoding settings there.

The ISE should honor a BOM, but it's also possible to use reflection to set the encoding. Note that this wouldn't be persisted between startups.

Source control software

Some source control tools, such as git, ignore encodings; git just tracks the bytes. Others, like Azure DevOps or Mercurial, may not. Even some git-based tools rely on decoding text.

When this is the case, make sure you:

- Configure the text encoding in your source control to match your VS Code configuration.
- Ensure all your files are checked into source control in the relevant encoding.
- Be wary of changes to the encoding received through source control. A key sign of this is a diff indicating changes but where nothing seems to have changed (because bytes have but characters have not).

Collaborators' environments

On top of configuring source control, ensure that your collaborators on any files you share don't have settings that override your encoding by re-encoding PowerShell files.

Other programs

Any other program that reads or writes a PowerShell script may re-encode it.

Some examples are:

- Using the clipboard to copy and paste a script. This is common in scenarios like:
  - Copying a script into a VM
  - Copying a script out of an email or webpage
  - Copying a script into or out of a Microsoft Word or PowerPoint document
- Other text editors, such as:
  - Notepad
  - vim
  - Any other PowerShell script editor
- Text editing utilities, like:
  - `Get-Content`/`Set-Content`/`Out-File`

    - PowerShell redirection operators like `>` and `>>`
    - `sed` / `awk`
- File transfer programs, like:
    - A web browser, when downloading scripts
    - A file share

Some of these tools deal in bytes rather than text, but others offer encoding configurations. In those cases where you need to configure an encoding, you need to make it the same as your editor encoding to prevent problems.

Other resources on encoding in PowerShell

There are a few other nice posts on encoding and configuring encoding in PowerShell that are worth a read:

- about_Character_Encoding
- @mklement0's summary of PowerShell encoding on Stack Overflow
- Previous issues opened on VS Code-PowerShell for encoding problems:
    - #1308
    - #1628
    - #1680
    - #1744
    - #1751
- The classic *Joel on Software* write up about Unicode
- Encoding in .NET Standard

Last updated on 12/08/2025

Using Visual Studio Code to debug compiled cmdlets

This guide shows you how to interactively debug C# source code for a compiled PowerShell module using Visual Studio Code (VS Code) and the C# extension.

Some familiarity with the Visual Studio Code debugger is assumed.

- For a general introduction to the VS Code debugger, see Debugging in Visual Studio Code.
- For examples of debugging PowerShell script files and modules, see Using Visual Studio Code for remote editing and debugging.

This guide assumes you have read and followed the instructions in the Writing Portable Modules guide.

Creating a build task

Build your project automatically before launching a debugging session. Rebuilding ensures that you debug the latest version of your code.

Configure a build task:

1. In the **Command Palette**, run the **Configure Default Build Task** command.

   Run Configure Default Build Task

2. In the **Select a task to configure** dialog, choose **Create tasks.json file from template**.
3. In the **Select a Task Template** dialog, choose **.NET Core**.

A new `tasks.json` file is created if one doesn't exist yet.

To test your build task:

1. In the **Command Palette**, run the **Run Build Task** command.
2. In the **Select the build task to run** dialog, choose **build**.

Information about DLL files being locked

By default, a successful build doesn't show output in the terminal pane. If you see output that contains the text **Project file doesn't exist**, you should edit the `tasks.json` file. Include the

explicit path to the C# project expressed as `"${workspaceFolder}/myModule"`. In this example, `myModule` is the name of the project folder. This entry must go after the `build` entry in the `args` list as follows:

JSON

```
    {
        "label": "build",
        "command": "dotnet",
        "type": "shell",
        "args": [
            "build",
            "${workspaceFolder}/myModule",
            // Ask dotnet build to generate full paths for file names.
            "/property:GenerateFullPaths=true",
            // Do not generate summary otherwise it leads to duplicate errors in
Problems panel
            "/consoleloggerparameters:NoSummary",
        ],
        "group": "build",
        "presentation": {
            "reveal": "silent"
        },
        "problemMatcher": "$msCompile"
    }
```

When debugging, your module DLL is imported into the PowerShell session in the VS Code terminal. The DLL becomes locked. The following message is displayed when you run the build task without closing the terminal session:

Output

```
Could not copy "obj\Debug\netstandard2.0\myModule.dll" to
"bin\Debug\netstandard2.0\myModule.dll"`.
```

Terminal sessions must be closed before you rebuild.

Setting up the debugger

To debug the PowerShell cmdlet, you need to set up a custom launch configuration. This configuration is used to:

- Build your source code
- Start PowerShell with your module loaded
- Leave PowerShell open in the terminal pane

When you invoke your cmdlet in the terminal session, the debugger stops at any breakpoints set in your source code.

Configuring launch.json for PowerShell

1. Install the C# for Visual Studio Code extension
2. In the Debug pane, add a debug configuration
3. In the `Select environment` dialog, choose `.NET Core`
4. The `launch.json` file is opened in the editor. With your cursor inside the `configurations` array, you see the `configuration` picker. If you don't see this list, select **Add Configuration**.
5. To create a default debug configuration, select **Launch .NET Core Console App**:

   Launch .NET Core Console App
6. Edit the `name`, `program`, `args`, and `console` fields as follows:

   JSON

```
{
     "name": "PowerShell cmdlets: pwsh",
     "type": "coreclr",
     "request": "launch",
     "preLaunchTask": "build",
     "program": "pwsh",
     "args": [
         "-NoExit",
         "-NoProfile",
         "-Command",
         "Import-Module
${workspaceFolder}/myModule/bin/Debug/netstandard2.0/myModule.dll",
     ],
     "cwd": "${workspaceFolder}",
     "stopAtEntry": false,
     "console": "integratedTerminal"
}
```

The `program` field is used to launch `pwsh` so that the cmdlet being debugged can be run. The `-NoExit` argument prevents the PowerShell session from exiting as soon as the module is imported. The path in the `Import-Module` argument is the default build output path when you've followed the Writing Portable Modules guide. If you've created a module manifest (`.psd1` file), you should use the path to that instead. The `/` path separator works on Windows,

Linux, and macOS. You must use the integrated terminal to run the PowerShell commands you want to debug.

> **Note**
>
> If the debugger doesn't stop at any breakpoints, look in the Visual Studio Code Debug Console for a line that says:
>
> ```
> Loaded '/path/to/myModule.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.
> ```
>
> If you see this, add `"justMyCode": false` to your launch config (at the same level as `"console": "integratedTerminal"`.

Configuring launch.json for Windows PowerShell

This launch configuration works for testing your cmdlets in Windows PowerShell (`powershell.exe`). Create a second launch configuration with the following changes:

1. `name` should be `PowerShell cmdlets: powershell`
2. `type` should be `clr`
3. `program` should be `powershell`

   It should look like this:

   JSON

   ```
   {
       "name": "PowerShell cmdlets: powershell",
       "type": "clr",
       "request": "launch",
       "preLaunchTask": "build",
       "program": "powershell",
       "args": [
           "-NoExit",
           "-NoProfile",
           "-Command",
           "Import-Module
   ${workspaceFolder}/myModule/bin/Debug/netstandard2.0/myModule.dll",
       ],
       "cwd": "${workspaceFolder}",
       "stopAtEntry": false,
   ```

```
    "console": "integratedTerminal"
}
```

Launching a debugging session

Now everything is ready to begin debugging.

- Place a breakpoint in the source code for the cmdlet you want to debug:

  A breakpoint shows as a red dot in the gutter

- Ensure that the relevant **PowerShell cmdlets** configuration is selected in the configuration drop-down menu in the **Debug** view:

  Select the launch configuration

- Press `F5` or click on the **Start Debugging** button

- Switch to the terminal pane and invoke your cmdlet:

  Invoke the cmdlet

- Execution stops at the breakpoint:

  Executions halts at breakpoint

You can step through the source code, inspect variables, and inspect the call stack.

To end debugging, click **Stop** in the debug toolbar or press `Shift` + `F5`. The shell used for debugging exits and releases the lock on the compiled DLL file.

Last updated on 12/08/2025

PowerShell scripting performance considerations

PowerShell scripts that leverage .NET directly and avoid the pipeline tend to be faster than idiomatic PowerShell. Idiomatic PowerShell uses cmdlets and PowerShell functions, often leveraging the pipeline, and resorting to .NET only when necessary.

> **Note**
>
> Many of the techniques described here aren't idiomatic PowerShell and may reduce the readability of a PowerShell script. Script authors are advised to use idiomatic PowerShell unless performance dictates otherwise.

Suppressing output

There are many ways to avoid writing objects to the pipeline.

- Assignment or file redirection to `$null`
- Casting to `[void]`
- Pipe to `Out-Null`

The speeds of assigning to `$null`, casting to `[void]`, and file redirection to `$null` are almost identical. However, calling `Out-Null` in a large loop can be significantly slower, especially in PowerShell 5.1.

PowerShell

```
$tests = @{
    'Assign to $null' = {
        $arrayList = [System.Collections.ArrayList]::new()
        foreach ($i in 0..$args[0]) {
            $null = $arraylist.Add($i)
        }
    }
    'Cast to [void]' = {
        $arrayList = [System.Collections.ArrayList]::new()
        foreach ($i in 0..$args[0]) {
            [void] $arraylist.Add($i)
        }
    }
    'Redirect to $null' = {
        $arrayList = [System.Collections.ArrayList]::new()
        foreach ($i in 0..$args[0]) {
            $arraylist.Add($i) > $null
```

```
        }
    }
    'Pipe to Out-Null' = {
        $arrayList = [System.Collections.ArrayList]::new()
        foreach ($i in 0..$args[0]) {
            $arraylist.Add($i) | Out-Null
        }
    }
}

10kb, 50kb, 100kb | ForEach-Object {
    $groupResult = foreach ($test in $tests.GetEnumerator()) {
        $ms = (Measure-Command { & $test.Value $_ }).TotalMilliseconds

        [pscustomobject]@{
            Iterations        = $_
            Test              = $test.Key
            TotalMilliseconds = [Math]::Round($ms, 2)
        }

        [GC]::Collect()
        [GC]::WaitForPendingFinalizers()
    }

    $groupResult = $groupResult | Sort-Object TotalMilliseconds
    $groupResult | Select-Object *, @{
        Name       = 'RelativeSpeed'
        Expression = {
            $relativeSpeed = $_.TotalMilliseconds /
$groupResult[0].TotalMilliseconds
            [Math]::Round($relativeSpeed, 2).ToString() + 'x'
        }
    }
}
```

These tests were run on a Windows 11 machine in PowerShell 7.3.4. The results are shown below:

Output

```
Iterations Test              TotalMilliseconds RelativeSpeed
---------- ----              ----------------- -------------
     10240 Assign to $null              36.74 1x
     10240 Redirect to $null            55.84 1.52x
     10240 Cast to [void]               62.96 1.71x
     10240 Pipe to Out-Null             81.65 2.22x
     51200 Assign to $null             193.92 1x
     51200 Cast to [void]              200.77 1.04x
     51200 Redirect to $null           219.69 1.13x
     51200 Pipe to Out-Null            329.62 1.7x
    102400 Redirect to $null           386.08 1x
    102400 Assign to $null             392.13 1.02x
```

```
    102400 Cast to [void]               405.24 1.05x
    102400 Pipe to Out-Null             572.94 1.48x
```

The times and relative speeds can vary depending on the hardware, the version of PowerShell, and the current workload on the system.

Array addition

Generating a list of items is often done using an array with the addition operator:

PowerShell

```
$results = @()
$results += Get-Something
$results += Get-SomethingElse
$results
```

> **Note**
>
> In PowerShell 7.5, array addition was optimized and no longer creates a new array for each operation. The performance considerations described here still apply to PowerShell versions prior to 7.5. For more information, see What's New in PowerShell 7.5.

Array addition is inefficient because arrays have a fixed size. Each addition to the array creates a new array big enough to hold all elements of both the left and right operands. The elements of both operands are copied into the new array. For small collections, this overhead may not matter. Performance can suffer for large collections.

There are a couple of alternatives. If you don't actually require an array, instead consider using a typed generic list (`[List<T>]`):

PowerShell

```
$results = [System.Collections.Generic.List[Object]]::new()
$results.AddRange((Get-Something))
$results.AddRange((Get-SomethingElse))
$results
```

The performance impact of using array addition grows exponentially with the size of the collection and the number additions. This code compares explicitly assigning values to an array with using array addition and using the `Add(T)` method on a `[List<T>]` object. It defines explicit assignment as the baseline for performance.

PowerShell

```powershell
$tests = @{
    'PowerShell Explicit Assignment' = {
        param($Count)

        $result = foreach($i in 1..$Count) {
            $i
        }
    }
    '.Add(T) to List<T>' = {
        param($Count)

        $result = [Collections.Generic.List[int]]::new()
        foreach($i in 1..$Count) {
            $result.Add($i)
        }
    }
    '+= Operator to Array' = {
        param($Count)

        $result = @()
        foreach($i in 1..$Count) {
            $result += $i
        }
    }
}

5kb, 10kb, 100kb | ForEach-Object {
    $groupResult = foreach($test in $tests.GetEnumerator()) {
        $ms = (Measure-Command { & $test.Value -Count $_ }).TotalMilliseconds

        [pscustomobject]@{
            CollectionSize    = $_
            Test              = $test.Key
            TotalMilliseconds = [Math]::Round($ms, 2)
        }

        [GC]::Collect()
        [GC]::WaitForPendingFinalizers()
    }

    $groupResult = $groupResult | Sort-Object TotalMilliseconds
    $groupResult | Select-Object *, @{
        Name       = 'RelativeSpeed'
        Expression = {
            $relativeSpeed = $_.TotalMilliseconds /
$groupResult[0].TotalMilliseconds
            [Math]::Round($relativeSpeed, 2).ToString() + 'x'
        }
    }
}
```

These tests were run on a Windows 11 machine in PowerShell 7.3.4.

Output

```
CollectionSize Test                           TotalMilliseconds RelativeSpeed
-------------- ----                           ----------------- -------------
          5120 PowerShell Explicit Assignment             26.65 1x
          5120 .Add(T) to List<T>                        110.98 4.16x
          5120 += Operator to Array                      402.91 15.12x
         10240 PowerShell Explicit Assignment              0.49 1x
         10240 .Add(T) to List<T>                        137.67 280.96x
         10240 += Operator to Array                     1678.13 3424.76x
        102400 PowerShell Explicit Assignment             11.18 1x
        102400 .Add(T) to List<T>                       1384.03 123.8x
        102400 += Operator to Array                   201991.06 18067.18x
```

When you're working with large collections, array addition is dramatically slower than adding to a `List<T>`.

When using a `[List<T>]` object, you need to create the list with a specific type, like `[string]` or `[int]`. When you add objects of a different type to the list, they are cast to the specified type. If they can't be cast to the specified type, the method raises an exception.

PowerShell

```
$intList = [System.Collections.Generic.List[int]]::new()
$intList.Add(1)
$intList.Add('2')
$intList.Add(3.0)
$intList.Add('Four')
$intList
```

Output

```
MethodException:
Line |
   5 |  $intList.Add('Four')
     |  ~~~~~~~~~~~~~~~~~~~~
     | Cannot convert argument "item", with value: "Four", for "Add" to type
     "System.Int32": "Cannot convert value "Four" to type "System.Int32".
     Error: "The input string 'Four' was not in a correct format.""

1
2
3
```

When you need the list to be a collection of different types of objects, create it with `[Object]` as the list type. You can enumerate the collection inspect the types of the objects in it.

PowerShell

```
$objectList = [System.Collections.Generic.List[Object]]::new()
$objectList.Add(1)
$objectList.Add('2')
$objectList.Add(3.0)
$objectList | ForEach-Object { "$_ is $($_.GetType())" }
```

Output

```
1 is int
2 is string
3 is double
```

If you do require an array, you can call the `ToArray()` method on the list or you can let PowerShell create the array for you:

PowerShell

```
$results = @(
    Get-Something
    Get-SomethingElse
)
```

In this example, PowerShell creates an `[ArrayList]` to hold the results written to the pipeline inside the array expression. Just before assigning to `$results`, PowerShell converts the `[ArrayList]` to an `[Object[]]`.

Type-safe collections

PowerShell is a loosely typed language, which makes coding easier but can have performance implications. Consider using type-safe (or type-specific) collections. Type-safe collections consume less memory and are faster. Compare the following examples:

PowerShell

```
$Stopwatch = [System.Diagnostics.Stopwatch]::StartNew()
$ListInt = [System.Collections.Generic.List[int]]::new()
for ($i = 0; $i -lt 1mb; $i++) {
    $ListInt.Add($i)
}
$Stopwatch.Stop()
Write-Host "Time to add 1mb integers to List[int]:
$($Stopwatch.Elapsed.TotalSeconds) seconds."
```

Output

```
Time to add 1mb integers to List[int]: 9.8841501 seconds.
```

Creating a list of `[int]` is faster than creating a list of `[Object]`.

PowerShell

```
$Stopwatch = [System.Diagnostics.Stopwatch]::StartNew()
$ListObject = [System.Collections.Generic.List[Object]]::new()
for ($i = 0; $i -lt 1mb; $i++) {
    $ListObject.Add($i)
}
$Stopwatch.Stop()
Write-Host "Time to add 1mb integers to List[Object]:
$($Stopwatch.Elapsed.TotalSeconds) seconds."
```

Output

```
Time to add 1mb integers to List[Object]: 10.5677782 seconds.
```

String addition

Strings are immutable. Each addition to the string actually creates a new string big enough to hold the contents of both the left and right operands, then copies the elements of both operands into the new string. For small strings, this overhead may not matter. For large strings, this can affect performance and memory consumption.

There are at least two alternatives:

- The -join operator concatenates strings
- The .NET `[StringBuilder]` class provides a mutable string

The following example compares the performance of these three methods of building a string.

PowerShell

```
$tests = @{
    'StringBuilder' = {
        $sb = [System.Text.StringBuilder]::new()
        foreach ($i in 0..$args[0]) {
            $sb = $sb.AppendLine("Iteration $i")
        }
        $sb.ToString()
    }
    'Join operator' = {
        $string = @(
            foreach ($i in 0..$args[0]) {
                "Iteration $i"
            }
        ) -join "`n"
        $string
```

```
    }
    'Addition Assignment +=' = {
        $string = ''
        foreach ($i in 0..$args[0]) {
            $string += "Iteration $i`n"
        }
        $string
    }
}

10kb, 50kb, 100kb | ForEach-Object {
    $groupResult = foreach ($test in $tests.GetEnumerator()) {
        $ms = (Measure-Command { & $test.Value $_ }).TotalMilliseconds

        [pscustomobject]@{
            Iterations        = $_
            Test              = $test.Key
            TotalMilliseconds = [Math]::Round($ms, 2)
        }

        [GC]::Collect()
        [GC]::WaitForPendingFinalizers()
    }

    $groupResult = $groupResult | Sort-Object TotalMilliseconds
    $groupResult | Select-Object *, @{
        Name       = 'RelativeSpeed'
        Expression = {
            $relativeSpeed = $_.TotalMilliseconds /
$groupResult[0].TotalMilliseconds
            [Math]::Round($relativeSpeed, 2).ToString() + 'x'
        }
    }
}
```

These tests were run on a Windows 11 machine in PowerShell 7.4.2. The output shows that the `-join` operator is the fastest, followed by the `[StringBuilder]` class.

Output

```
Iterations Test                  TotalMilliseconds RelativeSpeed
---------- ----                  ----------------- -------------
     10240 Join operator                     14.75 1x
     10240 StringBuilder                     62.44 4.23x
     10240 Addition Assignment +=           619.64 42.01x
     51200 Join operator                     43.15 1x
     51200 StringBuilder                    304.32 7.05x
     51200 Addition Assignment +=         14225.13 329.67x
    102400 Join operator                     85.62 1x
    102400 StringBuilder                    499.12 5.83x
    102400 Addition Assignment +=         67640.79 790.01x
```

The times and relative speeds can vary depending on the hardware, the version of PowerShell, and the current workload on the system.

Processing large files

The idiomatic way to process a file in PowerShell might look something like:

PowerShell

```
Get-Content $path | Where-Object Length -GT 10
```

This can be an order of magnitude slower than using .NET APIs directly. For example, you can use the .NET `[StreamReader]` class:

PowerShell

```
try {
    $reader = [System.IO.StreamReader]::new($path)
    while (-not $reader.EndOfStream) {
        $line = $reader.ReadLine()
        if ($line.Length -gt 10) {
            $line
        }
    }
}
finally {
    if ($reader) {
        $reader.Dispose()
    }
}
```

You could also use the `ReadLines` method of `[System.IO.File]`, which wraps `StreamReader`, simplifies the reading process:

PowerShell

```
foreach ($line in [System.IO.File]::ReadLines($path)) {
    if ($line.Length -gt 10) {
        $line
    }
}
```

Looking up entries by property in large collections

It's common to need to use a shared property to identify the same record in different collections, like using a name to retrieve an ID from one list and an email from another.

Iterating over the first list to find the matching record in the second collection is slow. In particular, the repeated filtering of the second collection has a large overhead.

Given two collections, one with an **Id** and **Name**, the other with **Name** and **Email**:

PowerShell

```
$Employees = 1..10000 | ForEach-Object {
    [pscustomobject]@{
        Id   = $_
        Name = "Name$_"
    }
}

$Accounts = 2500..7500 | ForEach-Object {
    [pscustomobject]@{
        Name  = "Name$_"
        Email = "Name$_@fabrikam.com"
    }
}
```

The usual way to reconcile these collections to return a list of objects with the **Id**, **Name**, and **Email** properties might look like this:

PowerShell

```
$Results = $Employees | ForEach-Object -Process {
    $Employee = $_

    $Account = $Accounts | Where-Object -FilterScript {
        $_.Name -eq $Employee.Name
    }

    [pscustomobject]@{
        Id    = $Employee.Id
        Name  = $Employee.Name
        Email = $Account.Email
    }
}
```

However, that implementation has to filter all 5000 items in the `$Accounts` collection once for every item in the `$Employee` collection. That can take minutes, even for this single-value lookup.

Instead, you can make a Hash Table that uses the shared **Name** property as a key and the matching account as the value.

PowerShell

```
$LookupHash = @{}
foreach ($Account in $Accounts) {
```

```
    $LookupHash[$Account.Name] = $Account
}
```

Looking up keys in a hash table is much faster than filtering a collection by property values. Instead of checking every item in the collection, PowerShell can check if the key is defined and use its value.

PowerShell

```
$Results = $Employees | ForEach-Object -Process {
    $Email = $LookupHash[$_.Name].Email
    [pscustomobject]@{
        Id    = $_.Id
        Name  = $_.Name
        Email = $Email
    }
}
```

This is much faster. While the looping filter took minutes to complete, the hash lookup takes less than a second.

Use Write-Host carefully

The `Write-Host` command should only be used when you need to write formatted text to the host console, rather than writing objects to the **Success** pipeline.

`Write-Host` can be an order of magnitude slower than `[Console]::WriteLine()` for specific hosts like `pwsh.exe`, `powershell.exe`, or `powershell_ise.exe`. However, `[Console]::WriteLine()` isn't guaranteed to work in all hosts. Also, output written using `[Console]::WriteLine()` doesn't get written to transcripts started by `Start-Transcript`.

JIT compilation

PowerShell compiles the script code to bytecode that's interpreted. Beginning in PowerShell 3, for code that's repeatedly executed in a loop, PowerShell can improve performance by Just-in-time (JIT) compiling the code into native code.

Loops that have fewer than 300 instructions are eligible for JIT-compilation. Loops larger than that are too costly to compile. When the loop has executed 16 times, the script is JIT-compiled in the background. When the JIT-compilation completes, execution is transferred to the compiled code.

Avoid repeated calls to a function

Calling a function can be an expensive operation. If you're calling a function in a long running tight loop, consider moving the loop inside the function.

Consider the following examples:

PowerShell

```
$tests = @{
    'Simple for-loop'       = {
        param([int] $RepeatCount, [random] $RanGen)

        for ($i = 0; $i -lt $RepeatCount; $i++) {
            $null = $RanGen.Next()
        }
    }
    'Wrapped in a function' = {
        param([int] $RepeatCount, [random] $RanGen)

        function Get-RandomNumberCore {
            param ($Rng)

            $Rng.Next()
        }

        for ($i = 0; $i -lt $RepeatCount; $i++) {
            $null = Get-RandomNumberCore -Rng $RanGen
        }
    }
    'for-loop in a function' = {
        param([int] $RepeatCount, [random] $RanGen)

        function Get-RandomNumberAll {
            param ($Rng, $Count)

            for ($i = 0; $i -lt $Count; $i++) {
                $null = $Rng.Next()
            }
        }

        Get-RandomNumberAll -Rng $RanGen -Count $RepeatCount
    }
}

5kb, 10kb, 100kb | ForEach-Object {
    $Rng = [random]::new()
    $groupResult = foreach ($test in $tests.GetEnumerator()) {
        $ms = Measure-Command { & $test.Value -RepeatCount $_ -RanGen $Rng }

        [pscustomobject]@{
            CollectionSize    = $_
            Test              = $test.Key
            TotalMilliseconds = [Math]::Round($ms.TotalMilliseconds,2)
        }
```

```
        [GC]::Collect()
        [GC]::WaitForPendingFinalizers()
    }

    $groupResult = $groupResult | Sort-Object TotalMilliseconds
    $groupResult | Select-Object *, @{
        Name       = 'RelativeSpeed'
        Expression = {
            $relativeSpeed = $_.TotalMilliseconds /
$groupResult[0].TotalMilliseconds
            [Math]::Round($relativeSpeed, 2).ToString() + 'x'
        }
    }
}
```

The **Basic for-loop** example is the base line for performance. The second example wraps the random number generator in a function that's called in a tight loop. The third example moves the loop inside the function. The function is only called once but the code still generates the same amount of random numbers. Notice the difference in execution times for each example.

Output

```
CollectionSize Test                    TotalMilliseconds RelativeSpeed
-------------- ----                    ----------------- -------------
          5120 for-loop in a function               9.62 1x
          5120 Simple for-loop                     10.55 1.1x
          5120 Wrapped in a function               62.39 6.49x
         10240 Simple for-loop                     17.79 1x
         10240 for-loop in a function              18.48 1.04x
         10240 Wrapped in a function              127.39 7.16x
        102400 for-loop in a function             179.19 1x
        102400 Simple for-loop                    181.58 1.01x
        102400 Wrapped in a function             1155.57 6.45x
```

Avoid wrapping cmdlet pipelines

Most cmdlets are implemented for the pipeline, which is a sequential syntax and process. For example:

PowerShell

```
cmdlet1 | cmdlet2 | cmdlet3
```

Initializing a new pipeline can be expensive, therefore you should avoid wrapping a cmdlet pipeline into another existing pipeline.

Consider the following example. The `Input.csv` file contains 2100 lines. The `Export-Csv` command is wrapped inside the `ForEach-Object` pipeline. The `Export-Csv` cmdlet is invoked for every iteration of the `ForEach-Object` loop.

PowerShell

```
$measure = Measure-Command -Expression {
    Import-Csv .\Input.csv | ForEach-Object -Begin { $Id = 1 } -Process {
        [pscustomobject]@{
            Id   = $Id
            Name = $_.opened_by
        } | Export-Csv .\Output1.csv -Append
    }
}

'Wrapped = {0:N2} ms' -f $measure.TotalMilliseconds
```

Output

```
Wrapped = 15,968.78 ms
```

For the next example, the `Export-Csv` command was moved outside of the `ForEach-Object` pipeline. In this case, `Export-Csv` is invoked only once, but still processes all objects passed out of `ForEach-Object`.

PowerShell

```
$measure = Measure-Command -Expression {
    Import-Csv .\Input.csv | ForEach-Object -Begin { $Id = 2 } -Process {
        [pscustomobject]@{
            Id   = $Id
            Name = $_.opened_by
        }
    } | Export-Csv .\Output2.csv
}

'Unwrapped = {0:N2} ms' -f $measure.TotalMilliseconds
```

Output

```
Unwrapped = 42.92 ms
```

The unwrapped example is **372 times faster**. Also, notice that the first implementation requires the **Append** parameter, which isn't required for the later implementation.

Avoid unnecessary collection enumeration

The PowerShell comparison operators have a convience feature when comparing collections. When the left-hand value in the expression is a collection, the operator returns the elements of the collection that match the right-hand value of the expression.

This feature provides a simple way to filter a collection. For example:

PowerShell

```
PS> $Collection = 1..99
PS> ($Collection -like '*1*') -join ' '

1 10 11 12 13 14 15 16 17 18 19 21 31 41 51 61 71 81 91
```

However, when you use a collection comparison in a conditional statement that only expects a boolean result, this feature can result in poor performance.

Take for example:

PowerShell

```
if ($Collection -like '*1*') { 'Found' }
```

In this example, PowerShell compares the right-hand value to every value in the collection and returns a collection of results. Since the result isn't empty, the non-null result evaluates as `$true`. The condition is true when the first match is found, but PowerShell still enumerates the entire collection. This enumeration can have a significant performance impact for large collections.

One way to improve performance is to use the Where() method of the collection. The `Where()` method stops evaluating the collection after it finds the first match.

PowerShell

```
# Create an array of 1048576 items
$Collection = foreach ($x in 1..1MB) { $x }
(Measure-Command { if ($Collection -like '*1*') { 'Found' } }).TotalMilliseconds
633.3695
(Measure-Command { if ($Collection.Where({ $_ -like '*1*' }, 'first')) { 'Found' }
}).TotalMilliseconds
2.607
```

For a million items, using the `Where()` method is significantly faster.

Object creation

Creating objects using the `New-Object` cmdlet can be slow. The following code compares the performance of creating objects using the `New-Object` cmdlet to the `[pscustomobject]` type accelerator.

PowerShell

```
Measure-Command {
    $test = 'PSCustomObject'
    for ($i = 0; $i -lt 100000; $i++) {
        $resultObject = [pscustomobject]@{
            Name = 'Name'
            Path = 'FullName'
        }
    }
} | Select-Object @{n='Test';e={$test}},TotalSeconds

Measure-Command {
    $test = 'New-Object'
    for ($i = 0; $i -lt 100000; $i++) {
        $resultObject = New-Object -TypeName psobject -Property @{
            Name = 'Name'
            Path = 'FullName'
        }
    }
} | Select-Object @{n='Test';e={$test}},TotalSeconds
```

Output

```
Test           TotalSeconds
----           ------------
PSCustomObject         0.48
New-Object             3.37
```

PowerShell 5.0 added the `new()` static method for all .NET types. The following code compares the performance of creating objects using the `New-Object` cmdlet to the `new()` method.

PowerShell

```
Measure-Command {
    $test = 'new() method'
    for ($i = 0; $i -lt 100000; $i++) {
        $sb = [System.Text.StringBuilder]::new(1000)
    }
} | Select-Object @{n='Test';e={$test}},TotalSeconds

Measure-Command {
    $test = 'New-Object'
    for ($i = 0; $i -lt 100000; $i++) {
        $sb = New-Object -TypeName System.Text.StringBuilder -ArgumentList 1000
```

```
    }
} | Select-Object @{n='Test';e={$test}},TotalSeconds
```

Output

```
Test          TotalSeconds
----          ------------
new() method          0.59
New-Object            3.17
```

Use OrderedDictionary to dynamically create new objects

There are situations where we may need to dynamically create objects based on some input, the perhaps most commonly used way to create a new **PSObject** and then add new properties using the `Add-Member` cmdlet. The performance cost for small collections using this technique may be negligible however it can become very noticeable for big collections. In that case, the recommended approach is to use an `[OrderedDictionary]` and then convert it to a **PSObject** using the `[pscustomobject]` type accelerator. For more information, see the *Creating ordered dictionaries* section of about_Hash_Tables.

Assume you have the following API response stored in the variable `$json`.

JSON

```
{
  "tables": [
    {
      "name": "PrimaryResult",
      "columns": [
        { "name": "Type", "type": "string" },
        { "name": "TenantId", "type": "string" },
        { "name": "count_", "type": "long" }
      ],
      "rows": [
        [ "Usage", "63613592-b6f7-4c3d-a390-22ba13102111", "1" ],
        [ "Usage", "d436f322-a9f4-4aad-9a7d-271fbf66001c", "1" ],
        [ "BillingFact", "63613592-b6f7-4c3d-a390-22ba13102111", "1" ],
        [ "BillingFact", "d436f322-a9f4-4aad-9a7d-271fbf66001c", "1" ],
        [ "Operation", "63613592-b6f7-4c3d-a390-22ba13102111", "7" ],
        [ "Operation", "d436f322-a9f4-4aad-9a7d-271fbf66001c", "5" ]
      ]
    }
  ]
}
```

Now, suppose you want to export this data to a CSV. First you need to create new objects and add the properties and values using the `Add-Member` cmdlet.

PowerShell

```
$data = $json | ConvertFrom-Json
$columns = $data.tables.columns
$result = foreach ($row in $data.tables.rows) {
    $obj = [psobject]::new()
    $index = 0

    foreach ($column in $columns) {
        $obj | Add-Member -MemberType NoteProperty -Name $column.name -Value
$row[$index++]
    }

    $obj
}
```

Using an `OrderedDictionary`, the code can be translated to:

PowerShell

```
$data = $json | ConvertFrom-Json
$columns = $data.tables.columns
$result = foreach ($row in $data.tables.rows) {
    $obj = [ordered]@{}
    $index = 0

    foreach ($column in $columns) {
        $obj[$column.name] = $row[$index++]
    }

    [pscustomobject] $obj
}
```

In both cases the `$result` output would be same:

Output

```
Type        TenantId                             count_
----        --------                             ------
Usage       63613592-b6f7-4c3d-a390-22ba13102111 1
Usage       d436f322-a9f4-4aad-9a7d-271fbf66001c 1
BillingFact 63613592-b6f7-4c3d-a390-22ba13102111 1
BillingFact d436f322-a9f4-4aad-9a7d-271fbf66001c 1
Operation   63613592-b6f7-4c3d-a390-22ba13102111 7
Operation   d436f322-a9f4-4aad-9a7d-271fbf66001c 5
```

The latter approach becomes exponentially more efficient as the number of objects and member properties increases.

Here is a performance comparison of three techniques for creating objects with 5 properties:

PowerShell

```
$tests = @{
    '[ordered] into [pscustomobject] cast' = {
        param([int] $Iterations, [string[]] $Props)

        foreach ($i in 1..$Iterations) {
            $obj = [ordered]@{}
            foreach ($prop in $Props) {
                $obj[$prop] = $i
            }
            [pscustomobject] $obj
        }
    }
    'Add-Member'                            = {
        param([int] $Iterations, [string[]] $Props)

        foreach ($i in 1..$Iterations) {
            $obj = [psobject]::new()
            foreach ($prop in $Props) {
                $obj | Add-Member -MemberType NoteProperty -Name $prop -Value $i
            }
            $obj
        }
    }
    'PSObject.Properties.Add'               = {
        param([int] $Iterations, [string[]] $Props)

        # this is how, behind the scenes, `Add-Member` attaches
        # new properties to our PSObject.
        # Worth having it here for performance comparison

        foreach ($i in 1..$Iterations) {
            $obj = [psobject]::new()
            foreach ($prop in $Props) {
                $obj.psobject.Properties.Add(
                    [psnoteproperty]::new($prop, $i))
            }
            $obj
        }
    }
}

$properties = 'Prop1', 'Prop2', 'Prop3', 'Prop4', 'Prop5'

1kb, 10kb, 100kb | ForEach-Object {
    $groupResult = foreach ($test in $tests.GetEnumerator()) {
        $ms = Measure-Command { & $test.Value -Iterations $_ -Props $properties }
```

```
        [pscustomobject]@{
            Iterations        = $_
            Test              = $test.Key
            TotalMilliseconds = [Math]::Round($ms.TotalMilliseconds, 2)
        }

        [GC]::Collect()
        [GC]::WaitForPendingFinalizers()
    }

    $groupResult = $groupResult | Sort-Object TotalMilliseconds
    $groupResult | Select-Object *, @{
        Name       = 'RelativeSpeed'
        Expression = {
            $relativeSpeed = $_.TotalMilliseconds /
$groupResult[0].TotalMilliseconds
            [Math]::Round($relativeSpeed, 2).ToString() + 'x'
        }
    }
}
```

And these are the results:

Output

```
Iterations Test                                TotalMilliseconds RelativeSpeed
---------- ----                                ----------------- -------------
      1024 [ordered] into [pscustomobject] cast             22.00 1x
      1024 PSObject.Properties.Add                         153.17 6.96x
      1024 Add-Member                                      261.96 11.91x
     10240 [ordered] into [pscustomobject] cast             65.24 1x
     10240 PSObject.Properties.Add                        1293.07 19.82x
     10240 Add-Member                                     2203.03 33.77x
    102400 [ordered] into [pscustomobject] cast            639.83 1x
    102400 PSObject.Properties.Add                       13914.67 21.75x
    102400 Add-Member                                    23496.08 36.72x
```

Related links

- $null
- System.Void
- Out-Null
- List<T>
- Add(T) method
- System.String
- System.Int32
- System.Object

- ToArray() method
- System.Collections.ArrayList
- System.Text.StringBuilder
- System.IO.StreamReader
- File::ReadLines() method
- Write-Host
- Add-Member

Last updated on 01/21/2026

PowerShell module authoring considerations

This document includes some guidelines related to how a module is authored for best performance.

Module Manifest Authoring

A module manifest that doesn't use the following guidelines can have a noticeable impact on general PowerShell performance even if the module isn't used in a session.

Command auto-discovery analyzes each module to determine which commands the module exports and this analysis can be expensive. The results of module analysis are cached per user, but the cache isn't available on first run, which is a typical scenario with containers. During module analysis, if the exported commands can be fully determined from the manifest, more expensive analysis of the module can be avoided.

Guidelines

- In the module manifest, don't use wildcards in the `AliasesToExport`, `CmdletsToExport`, and `FunctionsToExport` entries.
- If the module doesn't export commands of a particular type, specify this explicitly in the manifest by specifying `@()`. A missing or `$null` entry is equivalent to specifying the wildcard `*`.

The following should be avoided where possible:

PowerShell

```
@{
    FunctionsToExport = '*'

    # Also avoid omitting an entry, it's equivalent to using a wildcard
    # CmdletsToExport = '*'
    # AliasesToExport = '*'
}
```

Instead, use:

PowerShell

```
@{
    FunctionsToExport = 'Format-Hex', 'Format-Octal'
    CmdletsToExport = @()  # Specify an empty array, not $null
    AliasesToExport = @()  # Also ensure all three entries are present
}
```

Avoid CDXML

When deciding how to implement your module, there are three primary choices:

- Binary (usually C#)
- Script (PowerShell)
- CDXML (an XML file wrapping CIM)

If the speed of loading your module is important, CDXML is roughly an order of magnitude slower than a binary module.

A binary module loads the fastest because it's compiled ahead of time and can use NGen to JIT compile once per machine.

A script module typically loads a bit more slowly than a binary module because PowerShell must parse the script before compiling and executing it.

A CDXML module is typically much slower than a script module because it must first parse an XML file which then generates quite a bit of PowerShell script that's then parsed and compiled.

Last updated on 12/08/2025

Optimize performance using parallel execution

PowerShell provides several options for the creation of parallel invocations.

- `Start-Job` runs each job in a separate process, each with a new instance of PowerShell. In many cases, a linear loop is faster. Also, serialization and deserialization can limit the usefulness of the objects returned. This command is built in to all versions of PowerShell.
- `Start-ThreadJob` is a cmdlet found in the **ThreadJob** module. This command uses PowerShell runspaces to create and manage thread-based jobs. These jobs are lighter-weight than the jobs created by `Start-Job` and avoid potential loss of type fidelity required by cross-process serialization and deserialization. The **ThreadJob** module comes with PowerShell 7 and higher. For Windows PowerShell 5.1, you can install this module from the PowerShell Gallery.
- Use the **System.Management.Automation.Runspaces** namespace from the PowerShell SDK to create your own parallel logic. Both `ForEach-Object -Parallel` and `Start-ThreadJob` use PowerShell runspaces to execute the code in parallel.
- Workflows are a feature of Windows PowerShell 5.1. Workflows aren't available in PowerShell 7.0 and higher. Workflows are a special type of PowerShell script that can run in parallel. They're designed for long-running tasks and can be paused and resumed. Workflows aren't recommended for new development. For more information, see about_Workflows.
- `ForEach-Object -Parallel` is a feature of PowerShell 7.0 and higher. Like `Start-ThreadJob`, it uses PowerShell runspaces to create and manage thread-based jobs. This command is designed for use in a pipeline.

Limit execution concurrency

Running scripts in parallel doesn't guarantee improved performance. For example, the following scenarios can benefit from parallel execution:

- Compute intensive scripts on multi-threaded multi-core processors
- Scripts that spend time waiting for results or doing file operations, as long as those operations don't block each other.

It's important to balance the overhead of parallel execution with the type of work done. Also, there are limits to the number of invocations that can run in parallel.

The `Start-ThreadJob` and `ForEach-Object -Parallel` commands have a **ThrottleLimit** parameter to limit the number of jobs running at one time. As more jobs are started, they're queued and wait until the current number of jobs drops below the throttle limit. As of PowerShell 7.1, `ForEach-Object -Parallel` reuses runspaces from a runspace pool by default. The **ThrottleLimit** parameter sets the runspace pool size. The default runspace pool size is 5. You can still create a new runspace for each iteration using the UseNewRunspace switch.

The `Start-Job` command doesn't have a **ThrottleLimit** parameter. You have to manage the number of jobs running at one time.

Measure performance

The following function, `Measure-Parallel`, compares the speed of the following parallel execution approaches:

- `Start-Job` - creates a child PowerShell process behind the scenes
- `Start-ThreadJob` - runs each job in a separate thread
- `ForEach-Object -Parallel` - runs each job in a separate thread
- `Start-Process` - invokes an external program asynchronously

> ⓘ **Note**
>
> This approach only makes sense if your parallel tasks only consist of a single call to an external program, as opposed to running a block of PowerShell code. Also, the only way to capture output with this approach is by redirecting to a file.

PowerShell

```powershell
function Measure-Parallel {
    [CmdletBinding()]
    param(
        [ValidateRange(2, 2147483647)]
        [int] $BatchSize = 5,

        [ValidateSet('Job', 'ThreadJob', 'Process', 'ForEachParallel', 'All')]
        [string[]] $Approach,
```

```
        # pass a higher count to run multiple batches
        [ValidateRange(2, 2147483647)]
        [int] $JobCount = $BatchSize
    )

    $noForEachParallel = $PSVersionTable.PSVersion.Major -lt 7
    $noStartThreadJob = -not (Get-Command -ErrorAction Ignore Start-ThreadJob)

    # Translate the approach arguments into their corresponding hashtable keys (see
below).
    if ('All' -eq $Approach) { $Approach = 'Job', 'ThreadJob', 'Process',
'ForEachParallel' }
    $approaches = $Approach.ForEach({
            if ($_ -eq 'ForEachParallel') { 'ForEach-Object -Parallel' }
            else { $_ -replace '^', 'Start-' }
        })

    if ($noStartThreadJob) {
        if ($interactive -or $approaches -contains 'Start-ThreadJob') {
            Write-Warning "Start-ThreadJob is not installed, omitting its test."
            $approaches = $approaches.Where({ $_ -ne 'Start-ThreadJob' })
        }
    }
    if ($noForEachParallel) {
        if ($interactive -or $approaches -contains 'ForEach-Object -Parallel') {
            Write-Warning 'ForEach-Object -Parallel require PowerShell v7+, omitting
its test.'
            $approaches = $approaches.Where({ $_ -ne 'ForEach-Object -Parallel' })
        }
    }

    # Simulated input: Create 'f0.zip', 'f1'.zip', ... file names.
    $zipFiles = 0..($JobCount - 1) -replace '^', 'f' -replace '$', '.zip'

    # Sample executables to run - here, the native shell is called to simply
    # echo the argument given.
    $exe = if ($env:OS -eq 'Windows_NT') { 'cmd.exe' } else { 'sh' }

    # The list of its arguments *as a single string* - use '{0}' as the placeholder
    # for where the input object should go.
    $exeArgList = if ($env:OS -eq 'Windows_NT') {
        '/c "echo {0} > NUL:"'
     } else {
        '-c "echo {0} > /dev/null"'
    }

    # A hashtable with script blocks that implement the 3 approaches to parallelism.
    $approachImpl = [ordered] @{}

    # child-process-based job
    $approachImpl['Start-Job'] = {
        param([array] $batch)
        $batch |
            ForEach-Object {
                Start-Job {
```

```
                    Invoke-Expression ($using:exe + ' ' + ($using:exeArgList -f
$args[0]))
                } -ArgumentList $_
            } |
            Receive-Job -Wait -AutoRemoveJob | Out-Null
    }

    # thread-based job - requires the ThreadJob module
    if (-not $noStartThreadJob) {
        # If Start-ThreadJob is available, add an approach for it.
        $approachImpl['Start-ThreadJob'] = {
            param([array] $batch)
            $batch |
                ForEach-Object {
                    Start-ThreadJob -ThrottleLimit $BatchSize {
                        Invoke-Expression ($using:exe + ' ' + ($using:exeArgList -f
$args[0]))
                    } -ArgumentList $_
                } |
                Receive-Job -Wait -AutoRemoveJob | Out-Null
        }
    }

    # ForEach-Object -Parallel job
    if (-not $noForEachParallel) {
        $approachImpl['ForEach-Object -Parallel'] = {
            param([array] $batch)
            $batch | ForEach-Object -ThrottleLimit $BatchSize -Parallel {
                Invoke-Expression ($using:exe + ' ' + ($using:exeArgList -f $_))
            }
        }
    }

    # direct execution of an external program
    $approachImpl['Start-Process'] = {
        param([array] $batch)
        $batch |
            ForEach-Object {
                Start-Process -NoNewWindow -PassThru $exe -ArgumentList ($exeArgList
-f $_)
            } |
            Wait-Process
    }

    # Partition the array of all indices into subarrays (batches)
    $batches = @(
        0..([math]::Ceiling($zipFiles.Count / $batchSize) - 1) | ForEach-Object {
            , $zipFiles[($_ * $batchSize)..($_ * $batchSize + $batchSize - 1)]
        }
    )

    $tsTotals = foreach ($appr in $approaches) {
        $i = 0
        $tsTotal = [timespan] 0
        $batches | ForEach-Object {
```

```
            Write-Verbose "$batchSize-element '$appr' batch"
            $ts = Measure-Command { & $approachImpl[$appr] $_ | Out-Null }
            $tsTotal += $ts
            if (++$i -eq $batches.Count) {
                # last batch processed.
                if ($batches.Count -gt 1) {
                    Write-Verbose ("'$appr' processing $JobCount items finished in "
+
                        "$($tsTotal.TotalSeconds.ToString('N2')) secs.")
                }
                $tsTotal # output the overall timing for this approach
            }
        }
    }

    # Output a result object with the overall timings.
    $oht = [ordered] @{}
    $oht['JobCount'] = $JobCount
    $oht['BatchSize'] = $BatchSize
    $oht['BatchCount'] = $batches.Count
    $i = 0
    foreach ($appr in $approaches) {
        $oht[($appr + ' (secs.)')] = $tsTotals[$i++].TotalSeconds.ToString('N2')
    }
    [pscustomobject] $oht
}
```

The following example uses `Measure-Parallel` to run 20 jobs in parallel, 5 at a time, using all available approaches.

PowerShell

```
Measure-Parallel -Approach All -BatchSize 5 -JobCount 20 -Verbose
```

The following output comes from a Windows computer running PowerShell 7.5.1. Your timing can vary based on many factors, but the ratios should provide a sense of relative performance.

Output

```
VERBOSE: 5-element 'Start-Job' batch
VERBOSE: 5-element 'Start-Job' batch
VERBOSE: 5-element 'Start-Job' batch
VERBOSE: 5-element 'Start-Job' batch
VERBOSE: 'Start-Job' processing 20 items finished in 7.58 secs.
VERBOSE: 5-element 'Start-ThreadJob' batch
VERBOSE: 5-element 'Start-ThreadJob' batch
VERBOSE: 5-element 'Start-ThreadJob' batch
VERBOSE: 5-element 'Start-ThreadJob' batch
VERBOSE: 'Start-ThreadJob' processing 20 items finished in 2.37 secs.
VERBOSE: 5-element 'Start-Process' batch
VERBOSE: 5-element 'Start-Process' batch
```

```
VERBOSE: 5-element 'Start-Process' batch
VERBOSE: 5-element 'Start-Process' batch
VERBOSE: 'Start-Process' processing 20 items finished in 0.26 secs.
VERBOSE: 5-element 'ForEach-Object -Parallel' batch
VERBOSE: 5-element 'ForEach-Object -Parallel' batch
VERBOSE: 5-element 'ForEach-Object -Parallel' batch
VERBOSE: 5-element 'ForEach-Object -Parallel' batch
VERBOSE: 'ForEach-Object -Parallel' processing 20 items finished in 0.79 secs.

JobCount                        : 20
BatchSize                       : 5
BatchCount                      : 4
Start-Job (secs.)               : 7.58
Start-ThreadJob (secs.)         : 2.37
Start-Process (secs.)           : 0.26
ForEach-Object -Parallel (secs.) : 0.79
```

Conclusions

- The `Start-Process` approach performs best because it doesn't have the overhead of job management. However, as previously noted, this approach has fundamental limitations.
- The `ForEach-Object -Parallel` adds the least overhead, followed by `Start-ThreadJob`.
- `Start-Job` has the most overhead because of the hidden PowerShell instances it creates for each job.

Acknowledgments

Much of the information is this article is based on the answers from Santiago Squarzon and mklement0 in this Stack Overflow post.

You might also be interested in the PSParallelPipeline module created by Santiago Squarzon.

Further reading

- Start-Job
- about_Jobs
- Start-ThreadJob
- ForEach-Object
- Start-Process

Last updated on 04/21/2026

Troubleshoot PowerShell startup issues

Sometimes, PowerShell can have problems before it's even ready to use. Startup issues can be difficult to troubleshoot, especially when you want to use PowerShell to help. There are three main phases of startup:

1. Process creation
2. PowerShell SessionState initialization
3. Profile processing

The most common problems include:

- Long startup time or slow performance
- Errors
- Crashes

The steps of the startup sequence

It's helpful to understand the steps PowerShell goes through during startup. This way, you can narrow down where the issue is happening.

Step 1: Process creation

The process creation has a few steps:

1. Create a Host window

   On Windows, the Host can be Windows Terminal, the Windows Console Host, Visual Studio Code, or any other hosting application. Problems that occur here are usually unrelated to PowerShell, but also extremely rare.

2. Start the process host process

   Problems that occur here are usually caused by a corrupted executable or an issue in the operating system.

3. Prepare .NET

   PowerShell is .NET-based and that needs to fully load. Depending on which PowerShell version you are trying to start, you either get the full, Windows-integrated .NET Framework with Windows PowerShell 5.1 or the newer .NET included in PowerShell 7.

   During first-time startup of PowerShell, PowerShell and .NET run optimization tasks. This optimization task is only run once, during the first startup after installation, upgrade, or if

the cache is empty. Startup will take longer during this first-time optimization. Failure during optimization can create a corrupted cache. A corrupted PowerShell cache can cause issues with command discovery and module loading.

Step 2: PowerShell SessionState initialization

Loading the PowerShell binaries and initializing the engine involves processing the PowerShell configuration and some cached data.

1. Process configuration files: `powershell.config.json` and PSSession configuration files used by JEA and other remoting scenarios. These files may contain settings that can affect the language mode, available commands and modules, and some policy settings.
2. Check Group Policies and Windows Security policies. Windows Group Policies can override settings in the `powershell.config.json`. Security Policies can enable features like WDAC (Windows Defender Application Control), which can also constrain the language mode available.
3. Load the default modules (Microsoft.PowerShell.Core and PSReadLine) and any modules and assemblies defined in the PSSession configuration.

For more information about PowerShell security features, see the following articles:

- PowerShell security features
- about_Language_Modes

Step 3: Profile processing

Finally, PowerShell runs the available profile files. Profile scripts are run in the following order:

1. All Hosts All Users
2. Current Host All Users
3. All Hosts Current User
4. Current Host Current User

> **Note**
>
> Profile scripts aren't run for remote sessions.

For more information about profiles, see about_Profiles.

Narrow the scope of the issue

It's helpful to remove variables and narrow down the specific scope of where the issue happens. The easiest variable to eliminate is the profile. The profile often contains custom code, especially in the user-specific profile scripts.

Try running PowerShell with the profile disabled:

PowerShell

```
# PS 5.1:
powershell -NoProfile

# PS 7.*:
pwsh -NoProfile
```

Next you should see if the problem is version-specific. Try running your profile in both Windows PowerShell 5.1 and PowerShell 7. Windows PowerShell and PowerShell 7 store the profile in different locations. Your profile may not be the same for both versions. Compare the files to understand the differences. You can try installing your PowerShell profile in Windows PowerShell 5.1. However, be aware that some PowerShell 7 commands and modules aren't compatible with Windows PowerShell 5.1.

You can test your PowerShell 7 profile in Windows PowerShell 5.1 without overwriting your existing profile.

1. Start Windows PowerShell 5.1 with the profile disabled.
2. Manually dot-source your PowerShell 7 profile file into the Windows PowerShell 5.1 session.

   PowerShell

   ```
   . $env:USERPROFILE\Documents\PowerShell\Microsoft.PowerShell_profile.ps1
   ```

3. Observe whether the issue occurs.

If the issue persists, then you know problem is an environmental issue outside of the profile.

Try running the profile on a different device. If the profile works correctly on another device, then you know the issue is specific to your original device.

Troubleshoot common environmental problems

Crashes during startup

If the PowerShell console crashes during startup, especially early and with no feedback, you could have a corrupted process cache. This is a rare condition that you can resolve by clearing the cache. There are two cache locations that can be cleared:

- User Cache: `$env:LOCALAPPDATA\Microsoft\Windows\Caches`
- System Cache: `$env:windir\System32\Config\SystemProfile\AppData\Local\Microsoft\Windows\Caches`

Delete the contents of the user cache folder first, then try starting PowerShell again. If the problem persists, delete the contents of the system cache and try again.

You may also need to delete the PowerShell analysis cache. You can find the cache files in the following locations:

- Windows PowerShell: `$env:windir\System32\Config\SystemProfile\AppData\Local\Microsoft\Windows\PowerShell`
- PowerShell 7: `$env:LOCALAPPDATA\Microsoft\PowerShell`

Delete only the following file patterns:

- `ModuleAnalysisCache-*`
- `StartupProfileData-*`

The cached data is recreated the next time you start PowerShell.

If the problem persists in Windows PowerShell 5.1, you may need to repair the .NET Framework installation. For more information, see Repair the .NET Framework.

Troubleshoot common profile problems

This section describes some common problems that can occur during PowerShell startup, and how to troubleshoot them.

Profile takes too long to run

First you must define what's "*too long.*" PowerShell is only doing what the scripts tell it to do. Check all profile paths. There may be multiple profile scripts being run. Review the code to understand it's trying to do.

- Determine where the delay occurs

  If there are profile scripts for the **AllUsers** scope, you might not be able to edit those files. Work with your system administrator to review those files. For the **CurrentUser** scope

profile scripts, edit those files to add timing messages to help you find where the delay occurs. For example, you can add the following line at various points in your profile script.

```
PowerShell
Write-Host "$(Get-Date -Format 'HH:mm:ss.fff') | Profile: Step X"
```

- Reduce dependencies

  Reduce the number of modules that need to be loaded to execute your profile. Run `Get-Module` after profile runs to see the modules that were loaded during startup. By default, PowerShell loads the Microsoft.PowerShell.Core and PSReadLine modules. Any additional modules were loaded by your profile scripts.

  Modules can be loaded explicitly by using `Import-Module` or implicitly by using commands defined in those modules. Consider whether you need a command or a module loaded during startup.

- Avoid installing modules in a redirected folder

  In many situations on Windows, your `Documents` folder can be redirected to a network file share or to OneDrive. Network file access can be slow, especially if the network is congested or the server is under heavy load. Depending on how OneDrive is configured, it can also introduce delays or cause errors during profile exectution.

  You have a few options to mitigate this problem:
  - Don't redirect your `Documents` folder, but that might not be desired
  - Configure your `Modules` folder in OneDrive to *always* be kept on disk. This prevents errors and delayed load times.
  - Install modules in the **AllUsers** scope, which is outside of the user profile folder.

- Measure the actual performance

  If you don't know how long your profile takes to run, you can't determine whether it's too long. The `Measure-Command` cmdlet can show how long a command or script block takes to run.

  Steve Lee, the PowerShell Dev Manager, has a blog post that describes how to measure the performance of your profile. It includes instructions for establishing a baseline for performance, how to get detailed timing information, and ways to optimize your profile. See Optimizing your $Profile.

PowerShell 7 starts slowly in an isolated network

In this scenario, your Windows computer is on a network that's not connected to the internet. For interactive PowerShell sessions, PowerShell loads the PSReadLine module automatically. PSReadLine is a signed module. PowerShell must verify the digital signature of the module. This verification can cause delays in a disconnected environment. To test this theory, start PowerShell 7 in *non-interactive* mode:

```PowerShell
pwsh.exe -noninteractive
```

If PowerShell starts quickly in non-interactive mode, then the problem is likely caused by Certificate Revocation List (CRL) checks. As part of the process of verifying a digital signature, The .NET runtime checks the CRL to ensure that the signing certificate is still valid. In a disconnected environment, you computer can't access the CRL on internet. The default timeout for CRL checks is 15 seconds. This means that every time PowerShell loads a signed module, it can take up to 15 seconds to timeout.

There are three possible workarounds for this problem:

- Firewall or proxy exemption

  Allowing direct internet access for CRL checking prevents the problem. In an environment where you can control access to the internet access, you can configure your firewall or proxy to allow access to the CRL Urls. This is the easiest solution with the least impact. The firewall logs should show the Url that PowerShell attempted to reach.

- Reduce CRL Timeout

  Reducing the CRL lookup timeout is possible, but doing so risks other lookups to fail, that can't complete in the time specified. For details about how to change the timeout, see Manage Network Retrieval and Path Validation.

- Remove CRL checking

  The CRL checking settings are managed by Group Policy. For more information, see Manage Trusted Publishers.

  > ⚠ **Warning**
  >
  > It's possible to disable the CRL check however, it's not recommended. Disabling CRL checking prevents you from actually revoking compromised certificates.

ERROR: Cannot dot-source this command because it was defined in a different language mode

PowerShell works with application control systems, such as AppLocker and Windows Defender Application Control (WDAC), by automatically running in **ConstrainedLanguage** mode. ConstrainedLanguage mode restricts some features that are potentially dangerous. However, there are times when you need **FullLanguage** mode to use certain commands or features. Scripts can run in **FullLanguage** mode when they're trusted by the policy. Trust can be indicated through file signing or other policy mechanisms configured in AppLocker or WDAC.

When you start a PowerShell session that's managed under application control, you get the following error:

```
Cannot dot-source this command because it was defined in a different language mode.
To invoke this
command without importing its contents, omit the '.' operator.
```

Under application control, PowerShell is running in **ConstrainedLanguage** mode. This error occurs when and your profile script is *exempted* or is *signed* to run in **FullLanguage** mode. When PowerShell is running in **ConstrainedLanguage** mode, it can't dot-source code trusted to run in **FullLanguage** mode.

To resolve the problem, you must remove the exemption or signature from the profile script. If you need code that must run in **FullLanguage** mode during your profile, move it into another script file that's exempted or signed. Call (don't dot-source) that script file from within your profile.

For more information on this issue, see PowerShell Constrained Language mode and the Dot-Source Operator.

Further reading

- about_Language_Modes
- Measure-Command

Last updated on 01/14/2026

Portable Modules

Windows PowerShell is written for .NET Framework while PowerShell Core is written for .NET Core. Portable modules are modules that work in both Windows PowerShell and PowerShell Core. While .NET Framework and .NET Core are highly compatible, there are differences in the available APIs between the two. There are also differences in the APIs available in Windows PowerShell and PowerShell Core. Modules intended to be used in both environments need to be aware of these differences.

Porting an existing module

Porting a PSSnapIn

PowerShell SnapIns aren't supported in PowerShell Core. However, it's trivial to convert a PSSnapIn to a PowerShell module. Typically, the PSSnapIn registration code is in a single source file of a class that derives from PSSnapIn. Remove this source file from the build; it's no longer needed.

Use New-ModuleManifest to create a new module manifest that replaces the need for the PSSnapIn registration code. Some values from the **PSSnapIn** (such as **Description**) can be reused within the module manifest.

The **RootModule** property in the module manifest should be set to the name of the assembly (`.dll`) implementing the cmdlets.

The .NET Portability Analyzer (aka APIPort)

To port modules written for Windows PowerShell to work with PowerShell Core, start with the .NET Portability Analyzer. Run this tool against your compiled assembly to determine if the .NET APIs used in the module are compatible with .NET Framework, .NET Core, and other .NET runtimes. The tool suggests alternate APIs if they exist. Otherwise, you may need to add runtime checks and restrict capabilities not available in specific runtimes.

Creating a new module

If creating a new module, the recommendation is to use the .NET CLI.

Installing the PowerShell Standard module template

Once the .NET CLI is installed, install a template library to generate a simple PowerShell module. The module will be compatible with Windows PowerShell, PowerShell Core, Windows, Linux, and macOS.

The following example shows how to install the template:

PowerShell

```
dotnet new install Microsoft.PowerShell.Standard.Module.Template
```

Output

```
The following template packages will be installed:
   Microsoft.PowerShell.Standard.Module.Template

Success: Microsoft.PowerShell.Standard.Module.Template::0.1.3 installed the
following templates:
Template Name               Short Name  Language  Tags
--------------------------  ----------  --------  ------------------------
PowerShell Standard Module  psmodule    [C#]      Library/PowerShell/Module
```

Creating a new module project

After the template is installed, you can create a new PowerShell module project using that template. In this example, the sample module is called 'myModule'.

```
PS> mkdir myModule

    Directory: C:\Users\Steve

Mode LastWriteTime Length Name
---- ------------- ------ ----
d----- 8/3/2018 2:41 PM myModule

PS> cd myModule
PS C:\Users\Steve\myModule> dotnet new psmodule
The template "PowerShell Standard Module" was created successfully.

Processing post-creation actions...
Restoring  C:\Users\Steve\myModule\myModule.csproj:
  Determining projects to restore...
  Restored C:\Users\Steve\myModule\myModule.csproj (in 184 ms).
Restore succeeded.
```

Building the module

Use standard .NET CLI commands to build the project.

PowerShell

```
dotnet build
```

Output

```
PS C:\Users\Steve\myModule> dotnet build
MSBuild version 17.6.3+07e294721 for .NET
  Determining projects to restore...
  All projects are up-to-date for restore.
  PowerShellPG -> C:\Users\Steve\myModule\bin\Debug\netstandard2.0\myModule.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.36
```

Testing the module

After building the module, you can import it and execute the sample cmdlet.

PowerShell

```
Import-Module .\bin\Debug\netstandard2.0\myModule.dll
Test-SampleCmdlet -?
Test-SampleCmdlet -FavoriteNumber 7 -FavoritePet Cat
```

Output

```
NAME
    Test-SampleCmdlet

SYNTAX
    Test-SampleCmdlet [-FavoriteNumber] <int> [[-FavoritePet] {Cat | Dog | Horse}]
[<CommonParameters>]


ALIASES
    None


REMARKS
    None


FavoriteNumber FavoritePet
```

```
-------------- -----------
             7 Cat
```

Debugging the module

For a guide on setting up Visual Studio Code to debug the module, see Using Visual Studio Code for debugging compiled cmdlets.

Supporting technologies

The following sections describe in detail some of the technologies used by this template.

.NET Standard Library

.NET Standard is a formal specification of .NET APIs that are available in all .NET implementations. Managed code targeting .NET Standard works with the .NET Framework and .NET Core versions that are compatible with that version of the .NET Standard.

> **Note**
>
> Although an API may exist in .NET Standard, the API implementation in .NET Core may throw a `PlatformNotSupportedException` at runtime, so to verify compatibility with Windows PowerShell and PowerShell Core, the best practice is to run tests for your module within both environments. Also run tests on Linux and macOS if your module is intended to be cross-platform.

Targeting .NET Standard helps ensure that, as the module evolves, incompatible APIs don't accidentally get introduced into the module. Incompatibilities are discovered at compile time instead of runtime.

However, it isn't required to target .NET Standard for a module to work with both Windows PowerShell and PowerShell Core, as long as you use compatible APIs. The Intermediate Language (IL) is compatible between the two runtimes. You can target .NET Framework 4.6.1, which is compatible with .NET Standard 2.0. If you don't use APIs outside of .NET Standard 2.0, then your module works with PowerShell Core 6 without recompilation.

PowerShell Standard Library

The PowerShell Standard library is a formal specification of PowerShell APIs available in all PowerShell versions greater than or equal to the version of that standard.

For example, PowerShell Standard 5.1 is compatible with both Windows PowerShell 5.1 and PowerShell Core 6.0 or newer.

We recommend you compile your module using PowerShell Standard Library. The library ensures the APIs are available and implemented in both Windows PowerShell and PowerShell Core 6. PowerShell Standard is intended to always be forwards-compatible. A module built using PowerShell Standard Library 5.1 will always be compatible with future versions of PowerShell.

Module Manifest

Indicating Compatibility With Windows PowerShell and PowerShell Core

After validating that your module works with both Windows PowerShell and PowerShell Core, the module manifest should explicitly indicate compatibility by using the CompatiblePSEditions property. A value of `Desktop` means that the module is compatible with Windows PowerShell, while a value of `Core` means that the module is compatible with PowerShell Core. Including both `Desktop` and `Core` means that the module is compatible with both Windows PowerShell and PowerShell Core.

> **Note**
>
> `Core` doesn't automatically mean that the module is compatible with Windows, Linux, and macOS. The **CompatiblePSEditions** property was introduced in PowerShell v5. Module manifests that use the **CompatiblePSEditions** property fail to load in versions prior to PowerShell v5.

Indicating OS Compatibility

First, validate that your module works on Linux and macOS. Next, indicate compatibility with those operating systems in the module manifest. This makes it easier for users to find your module for their operating system when published to the PowerShell Gallery.

Within the module manifest, the `PrivateData` property has a `PSData` sub-property. The optional `Tags` property of `PSData` takes an array of values that show up in PowerShell Gallery. The PowerShell Gallery supports the following compatibility values:

Expand table

Tag	Description
PSEdition_Core	Compatible with PowerShell Core 6
PSEdition_Desktop	Compatible with Windows PowerShell
Windows	Compatible with Windows
Linux	Compatible with Linux (no specific distro)
macOS	Compatible with macOS

Example:

PowerShell

```
@{
    GUID = "4ae9fd46-338a-459c-8186-07f910774cb8"
    Author = "Microsoft Corporation"
    CompanyName = "Microsoft Corporation"
    Copyright = "(C) Microsoft Corporation. All rights reserved."
    HelpInfoUri = "https://go.microsoft.com/fwlink/?linkid=855962"
    ModuleVersion = "1.2.4"
    PowerShellVersion = "3.0"
    ClrVersion = "4.0"
    RootModule = "PackageManagement.psm1"
    Description = 'PackageManagement (a.k.a. OneGet) is a new way to discover and
install software packages from around the web.
 it's a manager or multiplexer of existing package managers (also called package
providers) that unifies Windows package management with a single Windows PowerShell
interface. With PackageManagement, you can do the following.
  - Manage a list of software repositories in which packages can be searched,
acquired and installed
  - Discover software packages
  - Seamlessly install, uninstall, and inventory packages from one or more software
repositories'

    CmdletsToExport = @(
        'Find-Package',
        'Get-Package',
        'Get-PackageProvider',
        'Get-PackageSource',
        'Install-Package',
        'Import-PackageProvider'
        'Find-PackageProvider'
        'Install-PackageProvider'
        'Register-PackageSource',
        'Set-PackageSource',
        'Unregister-PackageSource',
        'Uninstall-Package'
        'Save-Package'
    )
```

```
    FormatsToProcess  = @('PackageManagement.format.ps1xml')

    PrivateData = @{
        PSData = @{
            Tags = @('PackageManagement', 'PSEdition_Core', 'PSEdition_Desktop',
'Windows', 'Linux', 'macOS')
            ProjectUri = 'https://oneget.org'
        }
    }
}
```

Dependency on Native Libraries

Modules intended for use across different operating systems or processor architectures may depend on a managed library that itself depends on some native libraries.

Prior to PowerShell 7, one would have to have custom code to load the appropriate native dll so that the managed library can find it correctly.

With PowerShell 7, native binaries to load are searched in sub-folders within the managed library's location following a subset of the .NET RID Catalog notation.

```
managed.dll folder
    |
    |--- 'win-x64' folder
    |       |--- native.dll
    |
    |--- 'win-x86' folder
    |       |--- native.dll
    |
    |--- 'win-arm' folder
    |       |--- native.dll
    |
    |--- 'win-arm64' folder
    |       |--- native.dll
    |
    |--- 'linux-x64' folder
    |       |--- native.so
    |
    |--- 'linux-x86' folder
    |       |--- native.so
    |
    |--- 'linux-arm' folder
    |       |--- native.so
    |
    |--- 'linux-arm64' folder
    |       |--- native.so
    |
```

```
    |--- 'osx-x64' folder
    |       |--- native.dylib
```

Last updated on 12/08/2025

How to create a Standard Library binary module

I recently had an idea for module that I wanted to implement as a binary module. I have yet to create one using the PowerShell Standard Library so this felt like a good opportunity. I used the Creating a cross-platform binary module guide to create this module without any roadblocks. We're going to walk that same process and I'll add a little extra commentary along the way.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

What's the PowerShell Standard Library?

The PowerShell Standard Library allows us to create cross platform modules that work in both PowerShell and Windows PowerShell 5.1.

Why binary modules?

When you are writing a module in C# you give up easy access to PowerShell cmdlets and functions. But if you are creating a module that doesn't depend on a lot of other PowerShell commands, the performance benefit can be significant. PowerShell was optimized for the administrator, not the computer. By switching to C#, you get to shed the overhead added by PowerShell.

For example, we have a critical process that does a lot of work with JSON and hashtables. We optimized the PowerShell as much as we could but the process still takes 12 minutes to complete. The module already contained a lot of C# style PowerShell. This makes conversion to a binary module clean and simple. By converting to a binary module, we reduced the process time from over 12 minutes to under four minutes.

Hybrid modules

You can mix binary cmdlets with PowerShell advanced functions. Everything you know about script modules applies the same way. The empty `psm1` file is included so you can add other PowerShell functions later.

Almost all of the compiled cmdlets that I have created started out as PowerShell functions first. All of our binary modules are really hybrid modules.

Build scripts

I kept the build script simple here. I generally use a large `Invoke-Build` script as part of my CI/CD pipeline. It does more magic like running Pester tests, running **PSScriptAnalyzer**, managing versioning, and publishing to the PSGallery. Once I started using a build script for my modules, I was able to find lots of things to add to it.

Planning the module

The plan for this module is to create a `src` folder for the C# code and structure the rest like I would for a script module. This includes using a build script to compile everything into an `Output` folder. The folder structure looks like this:

```
MyModule
├───src
├───Output
│   └───MyModule
├───MyModule
│   ├───Data
│   ├───Private
│   └───Public
└───Tests
```

Getting Started

First I need to create the folder and create the git repo. I'm using `$module` as a placeholder for the module name. This should make it easier for you to reuse these examples if needed.

PowerShell

```powershell
$module = 'MyModule'
New-Item -Path $module -Type Directory
Set-Location $module
git init
```

Then create the root level folders.

PowerShell

```
New-Item -Path 'src' -Type Directory
New-Item -Path 'Output' -Type Directory
New-Item -Path 'Tests' -Type Directory
New-Item -Path $module -Type Directory
```

Binary module setup

This article is focused on the binary module so that's where we'll start. This section pulls examples from the Creating a cross-platform binary module guide. Review that guide if you need more details or have any issues.

First thing we want to do is check the version of the dotnet core SDK that we have installed. I'm using 2.1.4, but you should have 2.0.0 or newer before continuing.

PowerShell

```
PS> dotnet --version
2.1.4
```

I'm working out of the `src` folder for this section.

PowerShell

```
Set-Location 'src'
```

Using the dotnet command, create a new class library.

PowerShell

```
dotnet new classlib --name $module
```

This created the library project in a subfolder but I don't want that extra level of nesting. I'm going to move those files up a level.

PowerShell

```
Move-Item -Path .\$module\* -Destination .\
Remove-Item $module -Recurse
```

Set the .NET core SDK version for the project. I have the 2.1 SDK so I'm going to specify `2.1.0`. Use `2.0.0` if you're using the 2.0 SDK.

PowerShell

```
dotnet new globaljson --sdk-version 2.1.0
```

Add the **PowerShell Standard Library** NuGet package to the project. Make sure you use the most recent version available for the level of compatibility that you need. I would default to the latest version but I don't think this module leverages any features newer than PowerShell 3.0.

PowerShell

```
dotnet add package PowerShellStandard.Library --version 7.0.0-preview.1
```

We should have a src folder that looks like this:

PowerShell

```
PS> Get-ChildItem
    Directory: \MyModule\src

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        7/14/2018   9:51 PM                obj
-a----        7/14/2018   9:51 PM             86 Class1.cs
-a----        7/14/2018  10:03 PM            259 MyModule.csproj
-a----        7/14/2018  10:05 PM             45 global.json
```

Now we're ready to add our own code to the project.

Building a binary cmdlet

We need to update the `src\Class1.cs` to contain this starter cmdlet:

C#

```
using System;
using System.Management.Automation;

namespace MyModule
{
    [Cmdlet( VerbsDiagnostic.Resolve , "MyCmdlet")]
    public class ResolveMyCmdletCommand : PSCmdlet
    {
        [Parameter(Position=0)]
        public Object InputObject { get; set; }

        protected override void EndProcessing()
        {
            this.WriteObject(this.InputObject);
            base.EndProcessing();
        }
```

```
    }
}
```

Rename the file to match the class name.

PowerShell

```
Rename-Item .\Class1.cs .\ResolveMyCmdletCommand.cs
```

Then we can build our module.

PowerShell

```
PS> dotnet build

Microsoft (R) Build Engine version 15.5.180.51428 for .NET Core
Copyright (C) Microsoft Corporation. All rights reserved.

Restore completed in 18.19 ms for C:\workspace\MyModule\src\MyModule.csproj.
MyModule -> C:\workspace\MyModule\src\bin\Debug\netstandard2.0\MyModule.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.19
```

We can call `Import-Module` on the new dll to load our new cmdlet.

PowerShell

```
PS> Import-Module .\bin\Debug\netstandard2.0\$module.dll
PS> Get-Command -Module $module

CommandType Name                    Version Source
----------- ----                    ------- ------
Cmdlet      Resolve-MyCmdlet        1.0.0.0 MyModule
```

If the import fails on your system, try updating .NET to 4.7.1 or newer. The Creating a cross-platform binary module guide goes into more details on .NET support and compatibility for older versions of .NET.

Module manifest

It's cool that we can import the dll and have a working module. I like to keep going with it and create a module manifest. We need the manifest if we want to publish to the PSGallery later.

From the root of our project, we can run this command to create the module manifest that we need.

PowerShell

```powershell
$manifestSplat = @{
    Path              = ".\$module\$module.psd1"
    Author            = 'Kevin Marquette'
    NestedModules     = @('bin\MyModule.dll')
    RootModule        = "$module.psm1"
    FunctionsToExport = @('Resolve-MyCmdlet')
}
New-ModuleManifest @manifestSplat
```

I'm also going to create an empty root module for future PowerShell functions.

PowerShell

```powershell
Set-Content -Value '' -Path ".\$module\$module.psm1"
```

This allows me to mix both normal PowerShell functions and binary cmdlets in the same project.

Building the full module

I compile everything together into an output folder. We need to create a build script to do that. I would normally add this to an `Invoke-Build` script, but we can keep it simple for this example. Add this to a `build.ps1` at the root of the project.

PowerShell

```powershell
$module = 'MyModule'
Push-Location $PSScriptRoot

dotnet build $PSScriptRoot\src -o $PSScriptRoot\output\$module\bin
Copy-Item "$PSScriptRoot\$module\*" "$PSScriptRoot\output\$module" -Recurse -Force

Import-Module "$PSScriptRoot\Output\$module\$module.psd1"
Invoke-Pester "$PSScriptRoot\Tests"
```

These commands build our DLL and place it into our `output\$module\bin` folder. It then copies the other module files into place.

```
Output
└───MyModule
```

```
    ├──MyModule.psd1
    ├──MyModule.psm1
    └──bin
        ├──MyModule.deps.json
        ├──MyModule.dll
        └──MyModule.pdb
```

At this point, we can import our module with the psd1 file.

PowerShell

```powershell
Import-Module ".\Output\$module\$module.psd1"
```

From here, we can drop the `.\Output\$module` folder into our `$Env:PSModulePath` directory and it autoloads our command whenever we need it.

Update: dotnet new PSModule

I learned that the `dotnet` tool has a `PSModule` template.

All the steps that I outlined above are still valid, but this template cuts many of them out. It's still a fairly new template that's still getting some polish placed on it. Expect it to keep getting better from here.

This is how you use install and use the PSModule template.

PowerShell

```powershell
dotnet new -i Microsoft.PowerShell.Standard.Module.Template
dotnet new psmodule
dotnet build
Import-Module "bin\Debug\netstandard2.0\$module.dll"
Get-Module $module
```

This minimally-viable template takes care of adding the .NET SDK, PowerShell Standard Library, and creates an example class in the project. You can build it and run it right away.

Important details

Before we end this article, here are a few other details worth mentioning.

Unloading DLLs

Once a binary module is loaded, you can't really unload it. The DLL file is locked until you unload it. This can be annoying when developing because every time you make a change and

want to build it, the file is often locked. The only reliable way to resolve this is to close the PowerShell session that loaded the DLL.

VS Code reload window action

I do most of my PowerShell dev work in VS Code. When I'm working on a binary module (or a module with classes), I've gotten into the habit of reloading VS Code every time I build. `Ctrl` + `Shift` + `P` pops the command window and `Reload Window` is always at the top of my list.

Nested PowerShell sessions

One other option is to have good Pester test coverage. Then you can adjust the build.ps1 script to start a new PowerShell session, perform the build, run the tests, and close the session.

Updating installed modules

This locking can be annoying when trying to update your locally installed module. If any session has it loaded, you have to go hunt it down and close it. This is less of an issue when installing from a PSGallery because module versioning places the new one in a different folder.

You can set up a local PSGallery and publish to that as part of your build. Then do your local install from that PSGallery. This sounds like a lot of work, but this can be as simple as starting a docker container. I cover a way to do that in my post on Using a NuGet server for a PSRepository.

Final thoughts

I didn't touch on the C# syntax for creating a cmdlet, but there is plenty of documentation on it in the Windows PowerShell SDK. It's definitely something worth experimenting with as a stepping stone into more serious C#.

Last updated on 12/08/2025

Choosing the right PowerShell NuGet package for your .NET project

Alongside the `pwsh` executable packages published with each PowerShell release, the PowerShell team also maintains several packages available on NuGet. These packages allow targeting PowerShell as an API platform in .NET.

As a .NET application that provides APIs and expects to load .NET libraries implementing its own (binary modules), it's essential that PowerShell be available in the form of a NuGet package.

Currently there are several NuGet packages that provide some representation of the PowerShell API surface area. Which package to use with a particular project isn't always clear. This article sheds some light on a few common scenarios for PowerShell-targeting .NET projects and how to choose the right NuGet package to target for your PowerShell-oriented .NET project.

Hosting vs referencing

Some .NET projects seek to write code to be loaded into a preexisting PowerShell runtime (such as `pwsh`, `powershell.exe`, the PowerShell Integrated Console, or the ISE), while others want to run PowerShell in their own applications.

- **Referencing** is for when a project, usually a module, is intended to be loaded into PowerShell. You must compile the module against the APIs that PowerShell provides in order to interact with it. But PowerShell supplies the implementation by loading it in. A project can use reference assemblies or the actual runtime assemblies as a compilation target, but must ensure that it doesn't publish any of these assemblies with its build.
- **Hosting** is when a project needs its own implementation of PowerShell, usually because it's a standalone application that needs to run PowerShell. In this case, pure reference assemblies can't be used. Instead, a concrete PowerShell implementation must be depended upon. Because a concrete PowerShell implementation must be used, a specific version of PowerShell must be chosen for hosting; a single host application can't multi-target PowerShell versions.

Publishing projects that target PowerShell as a reference

> ⓘ Note

We use the term **publish** in this article to refer to running `dotnet publish`, which places a .NET library into a directory with all of its dependencies, ready for deployment to a particular runtime.

Set the PrivateAssets attribute to prevent publishing project dependencies that are just being used as compilation reference targets:

XML

```
<PackageReference Include="PowerShellStandard.Library" Version="5.1.0.0"
PrivateAssets="all" />
```

If you don't set this attribute and use a reference assembly as your target, you can see issues related to using the reference assembly's default implementation instead of the actual implementation. For example, you can receive a `NullReferenceException`, since reference assemblies often mock the implementation API by returning `null`.

Key kinds of PowerShell-targeting .NET projects

While any .NET library or application can embed PowerShell, there are some common scenarios that use PowerShell APIs:

- **Implementing a PowerShell binary module**

  PowerShell binary modules are .NET libraries loaded by PowerShell that must implement PowerShell APIs like the PSCmdlet or CmdletProvider types in order to expose cmdlets or providers respectively. Because they're loaded in, modules seek to compile against references to PowerShell without publishing it in their build. It's also common for modules to want to support multiple PowerShell versions and platforms, ideally with a minimum of overhead of disk space, complexity, or repeated implementation. For more information, see about_Modules.

- **Implementing a PowerShell Host**

  A PowerShell Host provides an interaction layer for the PowerShell runtime. It's a specific form of *hosting*, where a PSHost is implemented as a new user interface to PowerShell. For example, the PowerShell ConsoleHost provides a terminal user interface for PowerShell executables, while the PowerShell Editor Services Host and the ISE Host both provide an editor-integrated partially graphical user interface around PowerShell. While it's possible to load a host onto an existing PowerShell process, it's much more common for a host

implementation to act as a standalone PowerShell implementation that redistributes the PowerShell engine.

- **Calling into PowerShell from another .NET application**

  As with any application, you can call PowerShell as a subprocess to run workloads. However, as a .NET application, it's also possible to invoke PowerShell in-process to get back full .NET objects for use within the calling application. This is a more general form of *hosting*, where the application holds its own PowerShell implementation for internal use. For example, you could create a service or daemon running PowerShell to manage machine state, or a web application that runs PowerShell on request to do some work like managing cloud deployments.

- **Unit testing PowerShell modules from .NET**

  Usually, modules and other libraries designed to expose functionality to PowerShell should be tested from PowerShell. However, it's sometimes necessary to unit test APIs written for a PowerShell module from .NET.

PowerShell NuGet packages at a glance

The following NuGet packages expose PowerShell APIs:

- PowerShellStandard.Library, a reference assembly that enables building a single assembly that you can load in multiple PowerShell runtimes.
- Microsoft.PowerShell.SDK, the way to target and rehost the whole PowerShell SDK
- The System.Management.Automation package, the core PowerShell runtime and engine implementation, that can be useful in minimal hosted implementations and for version-specific targeting scenarios.
- The **Windows PowerShell reference assemblies**, the way to target and effectively rehost Windows PowerShell (PowerShell versions 5.1 and below).

> **Note**
>
> The PowerShell NuGet package isn't a .NET library package at all, but instead provides the PowerShell dotnet global tool implementation. Don't use this package in any projects, since it only provides an executable.

PowerShellStandard.Library

The PowerShell Standard library is a reference assembly that captures the intersection of the APIs of PowerShell v7 and v5.1. It provides a compile-time-checked API surface to compile .NET code against, allowing .NET projects to target PowerShell v7 and v5.1 without risking calling an API that that's not available.

Use PowerShell Standard to write PowerShell modules or other code only intended to be run after loading it into a PowerShell process. Because it's a reference assembly, PowerShell Standard contains no implementation itself, so it provides no functionality for standalone applications.

Using PowerShell Standard with different .NET runtimes

PowerShell Standard targets the .NET Standard 2.0 target framework, which is an abstraction layer designed to provide a common surface area shared by .NET Framework and .NET Core. It allows you to target a single framework to produce a single assembly that works with multiple PowerShell versions, but has the following consequences:

- The PowerShell instance loading the module or library must be running a minimum of .NET 4.6.1; .NET 4.6 and .NET 4.5.2 don't support .NET Standard.

  > **Note**
  >
  > A newer Windows PowerShell version doesn't mean a newer .NET Framework version. Windows PowerShell 5.1 can run on .NET 4.5.2.

- To work with a PowerShell running .NET Framework 4.7.1 or below, the .NET 4.6.1 NETStandard.Library implementation is required to provide the `netstandard.dll` and other shim assemblies in older .NET Framework versions.

PowerShell 6 (and higher) provides its own shim assemblies for type forwarding from .NET Framework 4.6.1 (and higher) to .NET Core. As long as a module uses only APIs that exist in .NET Core, PowerShell 6 (and higher) can load and run it if it was built for .NET Framework 4.6.1 (the `net461` runtime target).

Binary modules using PowerShell Standard to target multiple PowerShell versions with a single published DLL have two options:

1. Publishing an assembly built for the `net461` target runtime involves:

   - Publishing the project for the `net461` runtime

- Also compiling against the `netstandard2.0` runtime (without using its build output) to ensure that all APIs used are also present in .NET Core.

2. Publishing an assembly build for the `netstandard2.0` target runtime requires:

   - Publishing the project for the `netstandard2.0` runtime
   - Taking the `net461` dependencies of NETStandard.Library and copying them into the project assembly's publish location so that the assembly is type-forwarded corrected in .NET Framework.

To build PowerShell modules or libraries targeting older .NET Framework versions, you might find it preferable to target multiple .NET runtimes. Doing so publishes an assembly for each target runtime. The correct assembly must be loaded at module load time, for example with a small `.psm1` file as the root module.

Testing PowerShell Standard projects in .NET

When you test your module in .NET with test runners like xUnit, remember that compile-time checks can only go so far. You must test your module against the relevant PowerShell platforms.

To test APIs built against PowerShell Standard in .NET, you should add `Microsoft.PowerShell.SDK` as a testing dependency with .NET Core (with the version set to match the desired PowerShell version), and the appropriate Windows PowerShell reference assemblies with .NET Framework.

For more information on PowerShell Standard and using it to write a binary module that works in multiple PowerShell versions, see this blog post. Also see the PowerShell Standard repository on GitHub.

Microsoft.PowerShell.SDK

`Microsoft.PowerShell.SDK` is a meta-package that pulls together all of the components of the PowerShell SDK into a single NuGet package. A self-contained .NET application can use Microsoft.PowerShell.SDK to run arbitrary PowerShell functionality without depending on any external PowerShell installations or libraries.

> **Note**
>
> The PowerShell SDK just refers to all the component packages that make up PowerShell, and which can be used for .NET development with PowerShell.

A given `Microsoft.PowerShell.SDK` version contains the concrete implementation of the same version of the PowerShell application. Version 7.0 contains the implementation of PowerShell 7.0. Commands or scripts behave like running them in PowerShell 7.0. However, running PowerShell commands from the SDK isn't the same as running them from `pwsh`. For example, Start-Job depends on the `pwsh` executable being available, so it doesn't work with `Microsoft.PowerShell.SDK` by default.

Targeting `Microsoft.PowerShell.SDK` from a .NET application allows you to integrate with all of PowerShell's implementation assemblies, such as `System.Management.Automation`, `Microsoft.PowerShell.Management`, and other module assemblies.

Publishing an application targeting `Microsoft.PowerShell.SDK` includes all these assemblies, and any dependencies PowerShell requires. It also includes other assets that PowerShell required in its build, such as the module manifests for `Microsoft.PowerShell.*` modules and the `ref` directory required by Add-Type.

`Microsoft.PowerShell.SDK` is best suited for:

- Implementation of PowerShell hosts.
- xUnit testing of libraries targeting PowerShell reference assemblies.
- Invoking PowerShell in-process from a .NET application.

You can use `Microsoft.PowerShell.SDK` as a reference target for a .NET project to create a PowerShell module or assembly loaded by PowerShell that depends on APIs only present in a particular version of PowerShell. An assembly published for a specific version of `Microsoft.PowerShell.SDK` is only safe to load and use in that version of PowerShell. To target multiple PowerShell versions with specific APIs, multiple builds are required, each targeting their own version of `Microsoft.PowerShell.SDK`.

> **Note**
>
> The PowerShell SDK is only available for PowerShell v6 and higher. To provide equivalent functionality with Windows PowerShell, use the Windows PowerShell reference assemblies described later in this article.

System.Management.Automation

The `System.Management.Automation` package is the heart of the PowerShell SDK. It exists on NuGet, primarily, as an asset for `Microsoft.PowerShell.SDK` to pull in. However, it can also be

used directly as a package for smaller hosting scenarios and version-targeting modules.

Specifically, the `System.Management.Automation` package is the preferred when:

- You're only looking to use PowerShell language functionality from the `System.Management.Automation.Language` namespace, such as the PowerShell parser, AST, and AST visitor APIs.
- You only wish to execute specific commands from the `Microsoft.PowerShell.Core` module and can execute them in a session state created with the CreateDefault2 factory method.

Additionally, `System.Management.Automation` is a useful reference assembly when:

- You wish to target APIs that are only present within a specific PowerShell version
- You aren't depending on types occurring outside the `System.Management.Automation` assembly, such as types exported by cmdlets in `Microsoft.PowerShell.*` modules.

Windows PowerShell reference assemblies

For Windows PowerShell versions 5.1 and older, there's no SDK to provide an implementation of PowerShell, since Windows PowerShell's implementation is a part of Windows. Instead, the Windows PowerShell reference assemblies provide both reference targets and a way to rehost Windows PowerShell, acting the same as the PowerShell SDK does for version 6 and higher. Windows PowerShell reference assemblies have a different package for each version of Windows PowerShell:

- PowerShell 5.1
- PowerShell 4
- PowerShell 3

You can find information about how to use the Windows PowerShell reference assemblies in the Windows PowerShell SDK.

Real-world examples using these NuGet packages

Different PowerShell tooling projects target different PowerShell NuGet packages depending on their needs. Listed here are some notable examples.

PSReadLine

PSReadLine, the PowerShell module that provides much of PowerShell's rich console experience, targets PowerShell Standard as a dependency rather than a specific PowerShell

version, and targets the `net461` .NET runtime in its csproj.

PowerShell v6 (and higher) supplies its own shim assemblies that allow a DLL targeting the `net461` runtime to *just work* when loaded. The shim simplifies the delivery and module layout of PSReadLine. PowerShell Standard ensures that the module only uses APIs that are available in both Windows PowerShell 5.1 and PowerShell 6 (and higher), which allows the module to ship with only a single assembly.

The .NET 4.6.1 target does mean that Windows PowerShell running on .NET 4.5.2 and .NET 4.6 isn't supported though.

PowerShell Editor Services

PowerShell Editor Services (PSES) is the backend for the PowerShell extension for Visual Studio Code. This PowerShell module gets loaded by a PowerShell and then takes over that process to rehost PowerShell within itself, while also providing Language Service Protocol and Debug Adapter features.

PSES provides concrete implementation targets for `netcoreapp2.1` to target PowerShell 6+ (since PowerShell 7's `netcoreapp3.1` runtime is backwards compatible) and `net461` to target Windows PowerShell 5.1, but contains most of its logic in a second assembly that targets `netstandard2.0` and PowerShell Standard. This design allows it to pull in dependencies required for .NET Core and .NET Framework platforms, while still simplifying most of the codebase behind a uniform abstraction.

Because PSES targets PowerShell Standard, it requires a runtime implementation to be tested correctly. To do this, PSES's xUnit tests pull in `Microsoft.PowerShell.SDK` and `Microsoft.PowerShell.5.ReferenceAssemblies` to provide a PowerShell implementation in the test environment.

Since PSES can't support .NET 4.6 and older, it performs a check at runtime before calling any of the APIs that could cause a crash on the lower .NET Framework runtimes.

PSScriptAnalyzer

PSScriptAnalyzer, the linter for PowerShell, must target syntactic elements only introduced in certain versions of PowerShell. Because recognition of these syntactic elements is accomplished by implementing an AstVisitor2, it's not possible to use PowerShellStandard and also implement AST visitor methods for newer PowerShell syntaxes.

Instead, PSScriptAnalyzer targets each PowerShell version as a build configuration and produces a separate DLL for each version. This increases build size and complexity, but allows:

- Version-specific API targeting
- Version-specific functionality to be implemented with essentially no runtime cost
- Total support for Windows PowerShell running on .NET Framework 4.5.2 and higher

Summary

This article listed and discussed the NuGet packages you can target and the reasons for using each one. The general recommendations are:

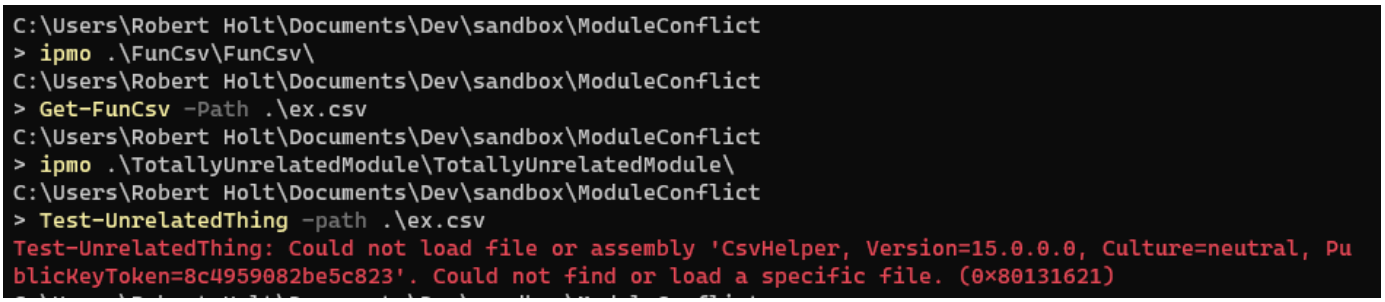
- PowerShell **modules** should compile against PowerShell Standard if they only require APIs common to different PowerShell versions.
- PowerShell **hosts and applications** that need to run PowerShell internally should target:
  - The PowerShell SDK for PowerShell v6 and higher
  - Or the relevant Windows PowerShell reference assemblies for Windows PowerShell
- PowerShell modules that need **version-specific APIs** should target the PowerShell SDK or Windows PowerShell reference assemblies for the required PowerShell versions. Use them as reference assemblies so you don't publish the PowerShell dependencies.

Last updated on 09/08/2026

Resolving PowerShell module assembly dependency conflicts

When writing a binary PowerShell module in C#, it's natural to take dependencies on other packages or libraries to provide functionality. Taking dependencies on other libraries is desirable for code reuse. PowerShell always loads assemblies into the same context. This presents issues when a module's dependencies conflict with already-loaded DLLs and may prevent using two otherwise unrelated modules in the same PowerShell session.

If you've had this problem, you've seen an error message like this:



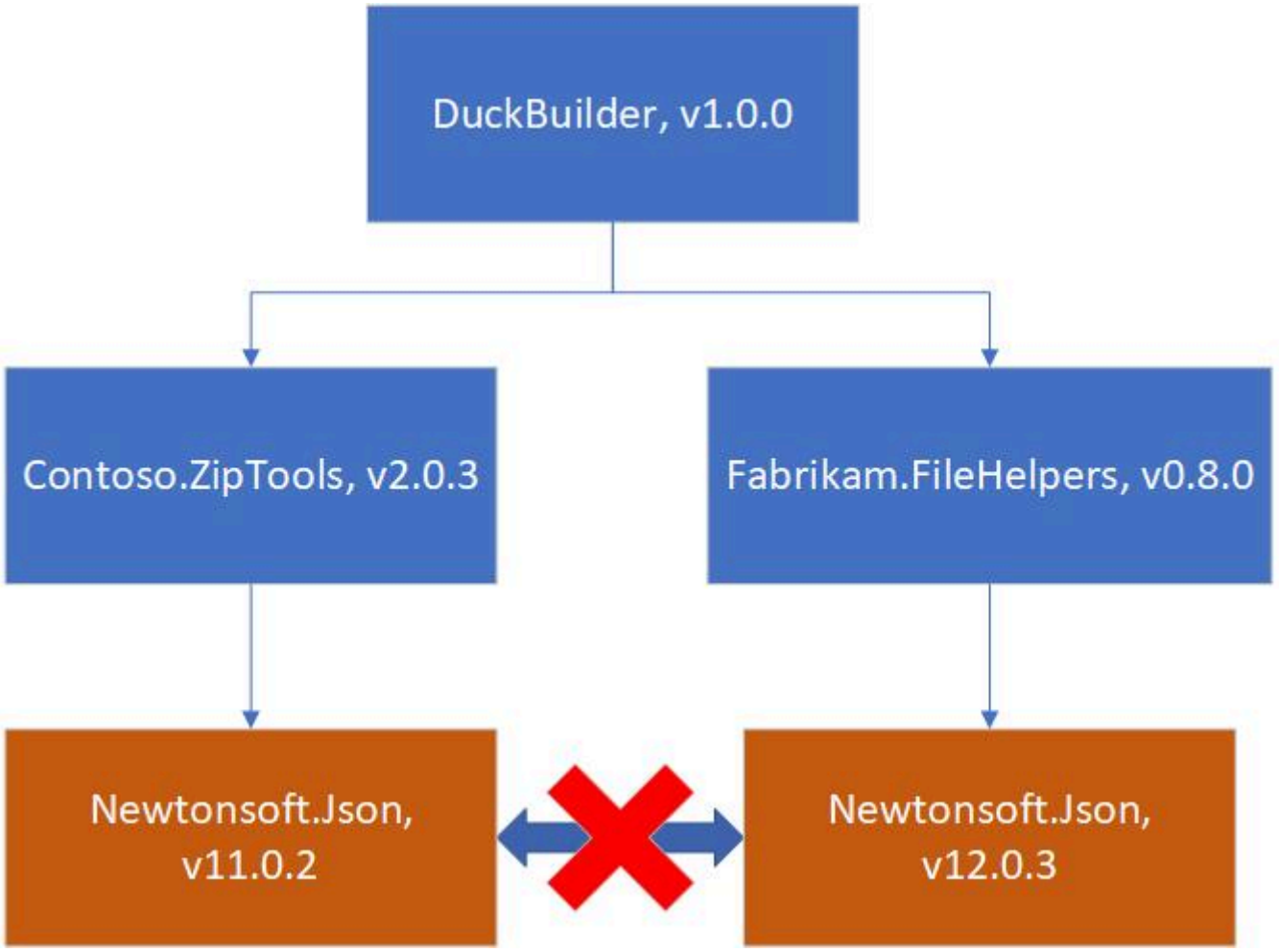
This article looks at some ways dependency conflicts occur in PowerShell and ways to mitigate dependency conflict issues. Even if you're not a module author, there are some tricks in here that might help you with dependency conflicts occurring in modules that you use.

Why do dependency conflicts occur?

In .NET, dependency conflicts occur when two versions of the same assembly are loaded into the same *Assembly Load Context*. This term means slightly different things on different .NET platforms, which is covered later in this article. This conflict is a common problem that occurs in any software where versioned dependencies are used.

Conflict issues are compounded by the fact that a project almost never deliberately or directly depends on two versions of the same dependency. Instead, the project has two or more dependencies that each require a different version of the same dependency.

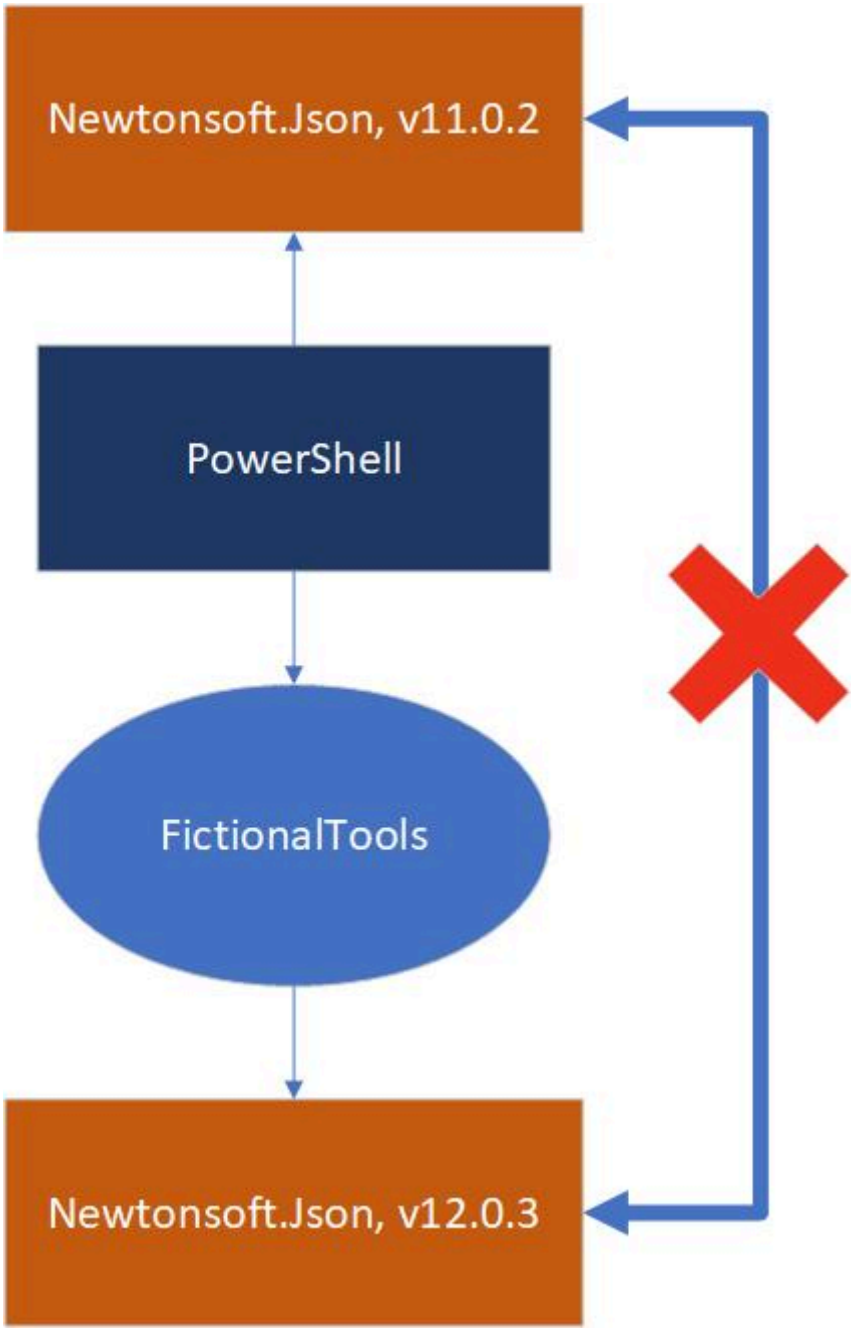
For example, say your .NET application, `DuckBuilder`, brings in two dependencies, to perform parts of its functionality and looks like this:



Because `Contoso.ZipTools` and `Fabrikam.FileHelpers` both depend on different versions of **Newtonsoft.Json**, there may be a dependency conflict depending on how each dependency is loaded.

Conflicting with PowerShell's dependencies

In PowerShell, the dependency conflict issue is magnified because PowerShell's own dependencies are loaded into the same shared context. This means the PowerShell engine and all loaded PowerShell modules must not have conflicting dependencies. A classic example of this is **Newtonsoft.Json**:



In this example, the module `FictionalTools` depends on **Newtonsoft.Json** version `12.0.3`, which is a newer version of **Newtonsoft.Json** than `11.0.2` that ships in the example PowerShell.

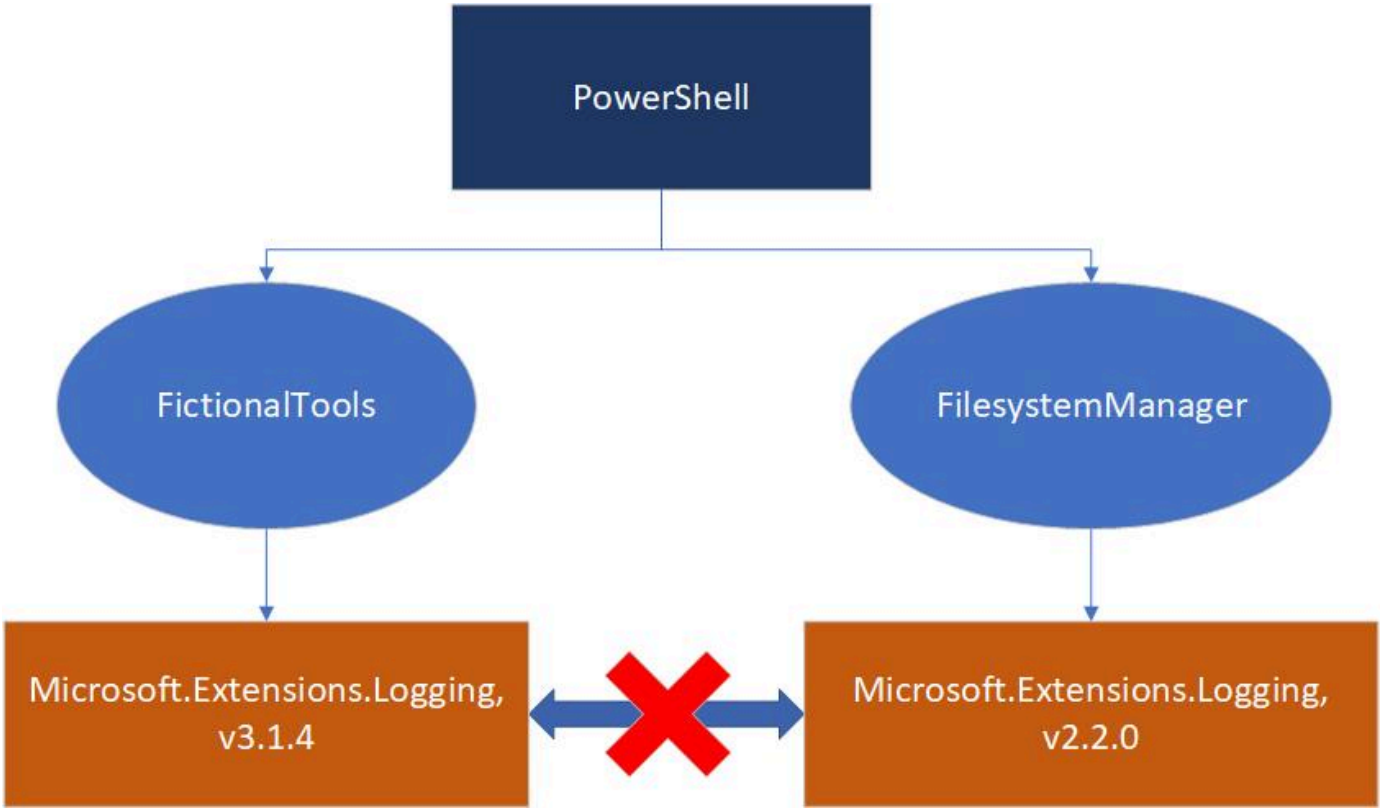
> ⓘ **Note**
>
> This is an example. PowerShell 7.0 currently ships with **Newtonsoft.Json 12.0.3**. Newer versions of PowerShell have newer versions of **Newtonsoft.Json**.

Because the module depends on a newer version of the assembly, it won't accept the version that PowerShell already has loaded. But because PowerShell has already loaded a version of the assembly, the module can't load its own version using the conventional load mechanism.

Conflicting with another module's dependencies

Another common scenario in PowerShell is that a module is loaded that depends on one version of an assembly, and then another module is loaded later that depends on a different version of that assembly.

This often looks like the following:



In this case, the `FictionalTools` module requires a newer version of `Microsoft.Extensions.Logging` than the `FilesystemManager` module.

Imagine these modules load their dependencies by placing the dependency assemblies in the same directory as the root module assembly. This allows .NET to implicitly load them by name. If we're running PowerShell 7.0 (on top of .NET Core 3.1), we can load and run `FictionalTools`, then load and run `FilesystemManager` without issue. However, in a new session, if we load and run `FilesystemManager`, then load `FictionalTools`, we get a `FileLoadException` from the `FictionalTools` command because it requires a newer version of `Microsoft.Extensions.Logging` than the one loaded. `FictionalTools` can't load the version needed because an assembly of the same name has already been loaded.

PowerShell and .NET

PowerShell runs on the .NET platform, which is responsible for resolving and loading assembly dependencies. We must understand how .NET operates here to understand dependency conflicts.

We must also confront the fact that different versions of PowerShell run on different .NET implementations. In general, PowerShell 5.1 and below run on .NET Framework, while PowerShell 6 and above run on .NET Core. These two implementations of .NET load and handle assemblies differently. This means that resolving dependency conflicts can vary depending on the underlying .NET platform.

Assembly Load Contexts

In .NET, an *Assembly Load Context* (ALC) is a runtime namespace into which assemblies are loaded. The assemblies' names must be unique. This concept allows assemblies to be uniquely resolved by name in each ALC.

Assembly reference loading in .NET

The semantics of assembly loading depend on both the .NET implementation (.NET Core vs .NET Framework) and the .NET API used to load a particular assembly. Rather than go into detail here, there are links in the Further reading section that go into great detail on how .NET assembly loading works in each .NET implementation.

In this article we'll refer to the following mechanisms:

- Implicit assembly loading (effectively `Assembly.Load(AssemblyName)`), when .NET implicitly tries to load an assembly by name from a static assembly reference in .NET code.
- `Assembly.LoadFrom()`, a plugin-oriented loading API that adds handlers to resolve dependencies of the loaded DLL. This method may not resolve dependencies the way we want.
- `Assembly.LoadFile()`, a basic loading API intended to load only the assembly asked for and does not handle any dependencies.

Differences in .NET Framework vs .NET Core

The way these APIs work has changed in subtle ways between .NET Core and .NET Framework, so it's worth reading through the included links. Importantly, Assembly Load Contexts and other assembly resolution mechanisms have changed between .NET Framework and .NET Core.

In particular, .NET Framework has the following features:

- The Global Assembly Cache, for machine-wide assembly resolution
- Application Domains, which work like in-process sandboxes for assembly isolation, but also present a serialization layer to contend with

- A limited assembly load context model that has a fixed set of assembly load contexts, each with their own behavior:
  - The default load context, where assemblies are loaded by default
  - The load-from context, for loading assemblies manually at runtime
  - The reflection-only context, for safely loading assemblies to read their metadata without running them
  - The mysterious void that assemblies loaded with `Assembly.LoadFile(string path)` and `Assembly.Load(byte[] asmBytes)` live in

For more information, see Best Practices for Assembly Loading.

.NET Core (and .NET 5+) has replaced this complexity with a simpler model:

- No Global Assembly Cache. Applications bring all their own dependencies. This removes an external factor for dependency resolution in applications, making dependency resolution more reproducible. PowerShell, as the plugin host, complicates this slightly for modules. Its dependencies in `$PSHOME` are shared with all modules.
- Only one Application Domain, and no ability to create new ones. The Application Domain concept is maintained in .NET to be the global state of the .NET process.
- A new, extensible Assembly Load Context (ALC) model. Assembly resolution can be namespaced by putting it in a new ALC. .NET processes begin with a single default ALC into which all assemblies are loaded (except for those loaded with `Assembly.LoadFile(string)` and `Assembly.Load(byte[])`). But the process can create and define its own custom ALCs with its own loading logic. When an assembly is loaded, the first ALC it's loaded into is responsible for resolving its dependencies. This creates opportunities to implement powerful .NET plugin loading mechanisms.

In both implementations, assemblies are loaded lazily. This means that they're loaded when a method requiring their type is run for the first time.

For example, here are two versions of the same code that load a dependency at different times.

The first always loads its dependency when `Program.GetRange()` is called, because the dependency reference is lexically present within the method:

C#

```
using Dependency.Library;

public static class Program
{
    public static List<int> GetRange(int limit)
```

```
    {
        var list = new List<int>();
        for (int i = 0; i < limit; i++)
        {
            if (i >= 20)
            {
                // Dependency.Library will be loaded when GetRange is run
                // because the dependency call occurs directly within the method
                DependencyApi.Use();
            }

            list.Add(i);
        }
        return list;
    }
}
```

The second loads its dependency only if the `limit` parameter is 20 or more, because of the internal indirection through a method:

C#

```
using Dependency.Library;

public static class Program
{
    public static List<int> GetNumbers(int limit)
    {
        var list = new List<int>();
        for (int i = 0; i < limit; i++)
        {
            if (i >= 20)
            {
                // Dependency.Library is only referenced within
                // the UseDependencyApi() method,
                // so will only be loaded when limit >= 20
                UseDependencyApi();
            }

            list.Add(i);
        }
        return list;
    }

    private static void UseDependencyApi()
    {
        // Once UseDependencyApi() is called, Dependency.Library is loaded
        DependencyApi.Use();
    }
}
```

This is a good practice since it minimizes the memory and filesystem I/O and uses the resources more efficiently. The unfortunate a side effect of this is that we won't know that the assembly fails to load until we reach the code path that tries to load the assembly.

It can also create a timing condition for assembly load conflicts. If two parts of the same program try to load different versions of the same assembly, the version loaded depends on which code path is run first.

For PowerShell, this means that the following factors can affect an assembly load conflict:

- Which module was loaded first?
- Was the code path that uses the dependency library run?
- Does PowerShell load a conflicting dependency at startup or only under certain code paths?

Quick fixes and their limitations

In some cases, it's possible to make small adjustments to your module and fix things with minimal effort. But these solutions tend to come with caveats. While they may apply to your module, they won't work for every module.

Change your dependency version

The simplest way to avoid dependency conflicts is to agree on a dependency. This may be possible when:

- Your conflict is with a direct dependency of your module and you control the version.
- Your conflict is with an indirect dependency, but you can configure your direct dependencies to use a workable indirect dependency version.
- You know the conflicting version and can rely on it not changing.

The **Newtonsoft.Json** package is a good example of this last scenario. This is a dependency of PowerShell 6 and above, and isn't used in Windows PowerShell. Meaning a simple way to resolve versioning conflicts is to target the lowest version of **Newtonsoft.Json** across the PowerShell versions you wish to target.

For example, PowerShell 6.2.6 and PowerShell 7.0.2 both currently use **Newtonsoft.Json** version 12.0.3. To create a module targeting Windows PowerShell, PowerShell 6, and PowerShell 7, you would target **Newtonsoft.Json 12.0.3** as a dependency and include it in your built module. When the module is loaded in PowerShell 6 or 7, PowerShell's own **Newtonsoft.Json** assembly is already loaded. Since it's the version required for your module, resolution succeeds. In Windows

PowerShell, the assembly isn't already present in PowerShell, so it's loaded from your module folder instead.

Generally, when targeting a concrete PowerShell package, like **Microsoft.PowerShell.Sdk** or **System.Management.Automation**, NuGet should be able to resolve the right dependency versions required. Targeting both Windows PowerShell and PowerShell 6+ becomes more difficult because you must choose between targeting multiple frameworks or **PowerShellStandard.Library**.

Circumstances where pinning to a common dependency version won't work include:

- The conflict is with an indirect dependency, and none of your dependencies can be configured to use a common version.
- The other dependency version is likely to change often, so settling on a common version is only a short-term fix.

Use the dependency out of process

This solution is more for module users than module authors. This is a solution to use when confronted with a module that won't work due to an existing dependency conflict.

Dependency conflicts occur because two versions of the same assembly are loaded into the same .NET process. A simple solution is to load them into different processes, as long as you can still use the functionality from both together.

In PowerShell, there are several ways to achieve this:

- Invoke PowerShell as a subprocess

  To run a PowerShell command out of the current process, start a new PowerShell process directly with the command call:

  PowerShell

  ```
  pwsh -c 'Invoke-ConflictingCommand'
  ```

  The main limitation here is that restructuring the result can be trickier or more error prone than other options.

- The PowerShell job system

The PowerShell job system also runs commands out of process, by sending commands to a new PowerShell process and returning the results:

```powershell
$result = Start-Job { Invoke-ConflictingCommand } | Receive-Job -Wait
```

In this case, you just need to be sure that any variables and state are passed in correctly.

The job system can also be slightly cumbersome when running small commands.

- PowerShell remoting

  When it's available, PowerShell remoting can be a useful way to run commands out of process. With remoting, you can create a fresh **PSSession** in a new process, call its commands over PowerShell remoting, then use the results locally with the other modules containing the conflicting dependencies.

  An example might look like this:

```powershell
# Create a local PowerShell session
# where the module with conflicting assemblies will be loaded
$s = New-PSSession

# Import the module with the conflicting dependency via remoting,
# exposing the commands locally
Import-Module -PSSession $s -Name ConflictingModule

# Run a command from the module with the conflicting dependencies
Invoke-ConflictingCommand
```

- Implicit remoting to Windows PowerShell

  Another option in PowerShell 7 is to use the `-UseWindowsPowerShell` flag on `Import-Module`. This imports the module through a local remoting session into Windows PowerShell:

```powershell
Import-Module -Name ConflictingModule -UseWindowsPowerShell
```

  Be aware that modules may not be compatible with or may work differently with Windows PowerShell.

When out-of-process invocation shouldn't be used

As a module author, out-of-process command invocation is difficult to bake into a module and may have edge cases that cause issues. In particular, remoting and jobs may not be available in all environments where your module needs to work. However, the general principle of moving the implementation out of process and allowing the PowerShell module to be a thinner client, may still be applicable.

As a module user, there are cases where out-of-process invocation won't work:

- When PowerShell remoting is unavailable because you don't have privileges to use it or it isn't enabled.
- When a particular .NET type is needed from output as input to a method or another command. Commands running over PowerShell remoting emit deserialized objects rather than strongly-typed .NET objects. This means that method calls and strongly typed APIs don't work with the output of commands imported over remoting.

More robust solutions

The previous solutions all had scenarios and modules that don't work. However, they also have the virtue of being relatively simple to implement correctly. The following solutions are more robust, but require more effort to implement correctly and can introduce subtle bugs if not written carefully.

Loading through .NET Core Assembly Load Contexts

Assembly Load Contexts (ALCs) were introduced in .NET Core 1.0 to specifically address the need to load multiple versions of the same assembly into the same runtime.

Within .NET, they offer the most robust solution to the problem of loading conflicting versions of an assembly. However, custom ALCs aren't available in .NET Framework. This means that this solution only works in PowerShell 6 and above.

Currently, the best example of using an ALC for dependency isolation in PowerShell is in PowerShell Editor Services, the language server for the PowerShell extension for Visual Studio Code. An ALC is used to prevent PowerShell Editor Services' own dependencies from clashing with those in PowerShell modules.

Implementing module dependency isolation with an ALC is conceptually difficult, but we will work through a minimal example. Imagine we've a simple module that's only intended to work in

PowerShell 7. The source code is organized as follows:

```
+ AlcModule.psd1
+ src/
    + TestAlcModuleCommand.cs
    + AlcModule.csproj
```

The cmdlet implementation looks like this:

C#

```
using Shared.Dependency;

namespace AlcModule
{
    [Cmdlet(VerbsDiagnostic.Test, "AlcModule")]
    public class TestAlcModuleCommand : Cmdlet
    {
        protected override void EndProcessing()
        {
            // Here's where our dependency gets used
            Dependency.Use();
            // Something trivial to make our cmdlet do *something*
            WriteObject("done!");
        }
    }
}
```

The (heavily simplified) manifest, looks like this:

PowerShell

```
@{
    Author = 'Me'
    ModuleVersion = '0.0.1'
    RootModule = 'AlcModule.dll'
    CmdletsToExport = @('Test-AlcModule')
    PowerShellVersion = '7.0'
}
```

And the `csproj` looks like this:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>netcoreapp3.1</TargetFramework>
```

```
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Shared.Dependency" Version="1.0.0" />
    <PackageReference Include="Microsoft.PowerShell.Sdk" Version="7.0.1"
PrivateAssets="all" />
  </ItemGroup>
</Project>
```

When we build this module, the generated output has the following layout:

```
AlcModule/
  + AlcModule.psd1
  + AlcModule.dll
  + Shared.Dependency.dll
```

In this example, our problem is in the `Shared.Dependency.dll` assembly, which is our imaginary conflicting dependency. This is the dependency that we need to put behind an ALC so that we can use the module-specific version.

We need to re-engineer the module so that:

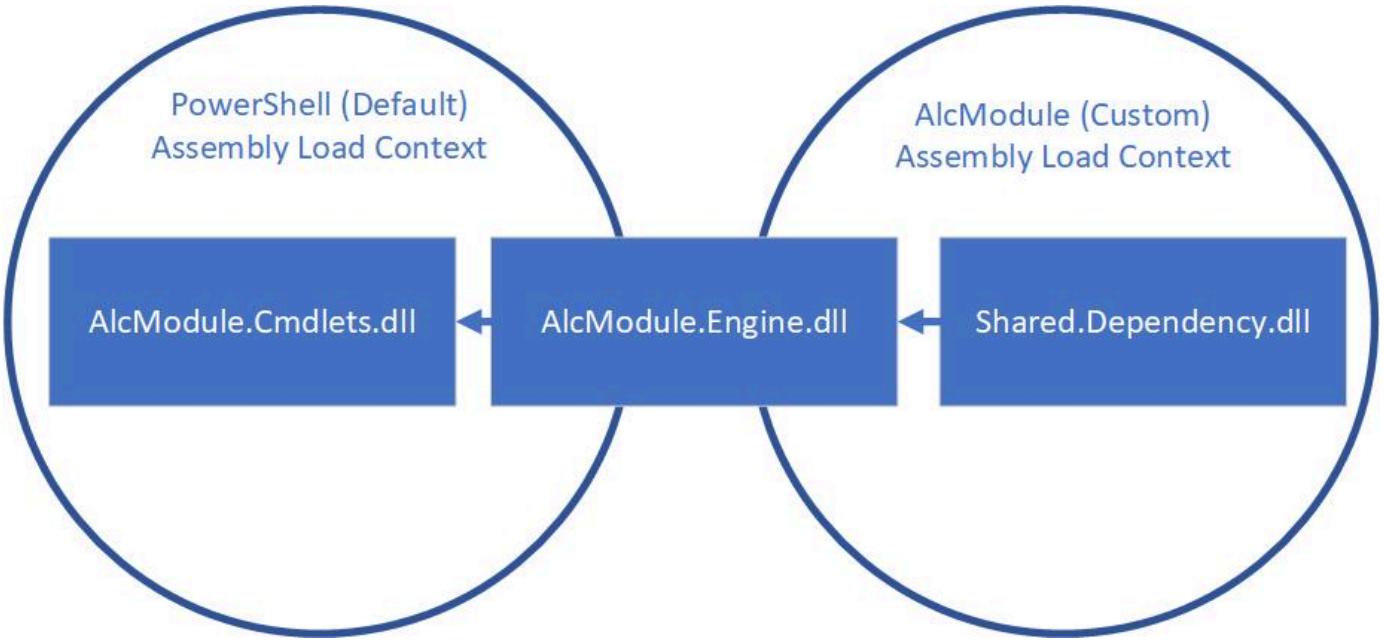
- Module dependencies are only loaded into our custom ALC, and not into PowerShell's ALC, so there can be no conflict. Moreover, as we add more dependencies to our project, we don't want to continuously add more code to keep loading working. Instead, we want reusable, generic dependency resolution logic.
- Loading the module still works as normal in PowerShell. Cmdlets and other types that the PowerShell module system needs are defined within PowerShell's own ALC.

To mediate these two requirements, we must break up our module into two assemblies:

- A cmdlets assembly, `AlcModule.Cmdlets.dll`, that contains definitions of all the types that PowerShell's module system needs to load our module correctly. Namely, any implementations of the `Cmdlet` base class and the class that implements `IModuleAssemblyInitializer`, which sets up the event handler for `AssemblyLoadContext.Default.Resolving` to properly load `AlcModule.Engine.dll` through our custom ALC. Since PowerShell 7 deliberately hides types defined in assemblies loaded in other ALCs, any types that are meant to be publicly exposed to PowerShell must also be defined here. Finally, our custom ALC definition needs to be defined in this assembly. Beyond that, as little code as possible should live in this assembly.
- An engine assembly, `AlcModule.Engine.dll`, that handles the actual implementation of the module. Types from this are available in the PowerShell ALC, but it's initially loaded through

our custom ALC. Its dependencies are only loaded into the custom ALC. Effectively, this becomes a *bridge* between the two ALCs.

Using this bridge concept, our new assembly situation looks like this:



To make sure the default ALC's dependency probing logic doesn't resolve the dependencies to be loaded into the custom ALC, we need to separate these two parts of the module in different directories. The new module layout has the following structure:

```
AlcModule/
  AlcModule.Cmdlets.dll
  AlcModule.psd1
  Dependencies/
  | + AlcModule.Engine.dll
  | + Shared.Dependency.dll
```

To see how the implementation changes, we'll start with the implementation of `AlcModule.Engine.dll`:

C#

```
using Shared.Dependency;

namespace AlcModule.Engine
{
    public class AlcEngine
    {
        public static void Use()
        {
            Dependency.Use();
```

```
        }
    }
}
```

This is a simple container for the dependency, `Shared.Dependency.dll`, but you should think of it as the .NET API for your functionality that the cmdlets in the other assembly wrap for PowerShell.

The cmdlet in `AlcModule.Cmdlets.dll` looks like this:

C#

```
// Reference our module's Engine implementation here
using AlcModule.Engine;

namespace AlcModule.Cmdlets
{
    [Cmdlet(VerbsDiagnostic.Test, "AlcModule")]
    public class TestAlcModuleCommand : Cmdlet
    {
        protected override void EndProcessing()
        {
            AlcEngine.Use();
            WriteObject("done!");
        }
    }
}
```

At this point, if we were to load **AlcModule** and run `Test-AlcModule`, we get a **FileNotFoundException** when the default ALC tries to load `Alc.Engine.dll` to run `EndProcessing()`. This is good, since it means the default ALC can't find the dependencies we want to hide.

Now we need to add code to `AlcModule.Cmdlets.dll` so that it knows how to resolve `AlcModule.Engine.dll`. First we must define our custom ALC to resolve assemblies from our module's `Dependencies` directory:

C#

```
namespace AlcModule.Cmdlets
{
    internal class AlcModuleAssemblyLoadContext : AssemblyLoadContext
    {
        private readonly string _dependencyDirPath;

        public AlcModuleAssemblyLoadContext(string dependencyDirPath)
        {
            _dependencyDirPath = dependencyDirPath;
        }
```

```
        protected override Assembly Load(AssemblyName assemblyName)
        {
            // We do the simple logic here of looking for an assembly of the given
name
            // in the configured dependency directory.
            string assemblyPath = Path.Combine(
                _dependencyDirPath,
                $"{assemblyName.Name}.dll");

            if (File.Exists(assemblyPath))
            {
                // The ALC must use inherited methods to load assemblies.
                // Assembly.Load*() won't work here.
                return LoadFromAssemblyPath(assemblyPath);
            }

            // For other assemblies, return null to allow other resolutions to
continue.
            return null;
        }
    }
}
```

Then we need to hook up our custom ALC to the default ALC's `Resolving` event, which is the ALC version of the `AssemblyResolve` event on Application Domains. This event is fired to find `AlcModule.Engine.dll` when `EndProcessing()` is called.

C#

```
namespace AlcModule.Cmdlets
{
    public class AlcModuleResolveEventHandler : IModuleAssemblyInitializer,
IModuleAssemblyCleanup
    {
        // Get the path of the dependency directory.
        // In this case we find it relative to the AlcModule.Cmdlets.dll location
        private static readonly string s_dependencyDirPath = Path.GetFullPath(
            Path.Combine(
                Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location),
                "Dependencies"));

        private static readonly AlcModuleAssemblyLoadContext s_dependencyAlc =
            new AlcModuleAssemblyLoadContext(s_dependencyDirPath);

        public void OnImport()
        {
            // Add the Resolving event handler here
            AssemblyLoadContext.Default.Resolving += ResolveAlcEngine;
        }

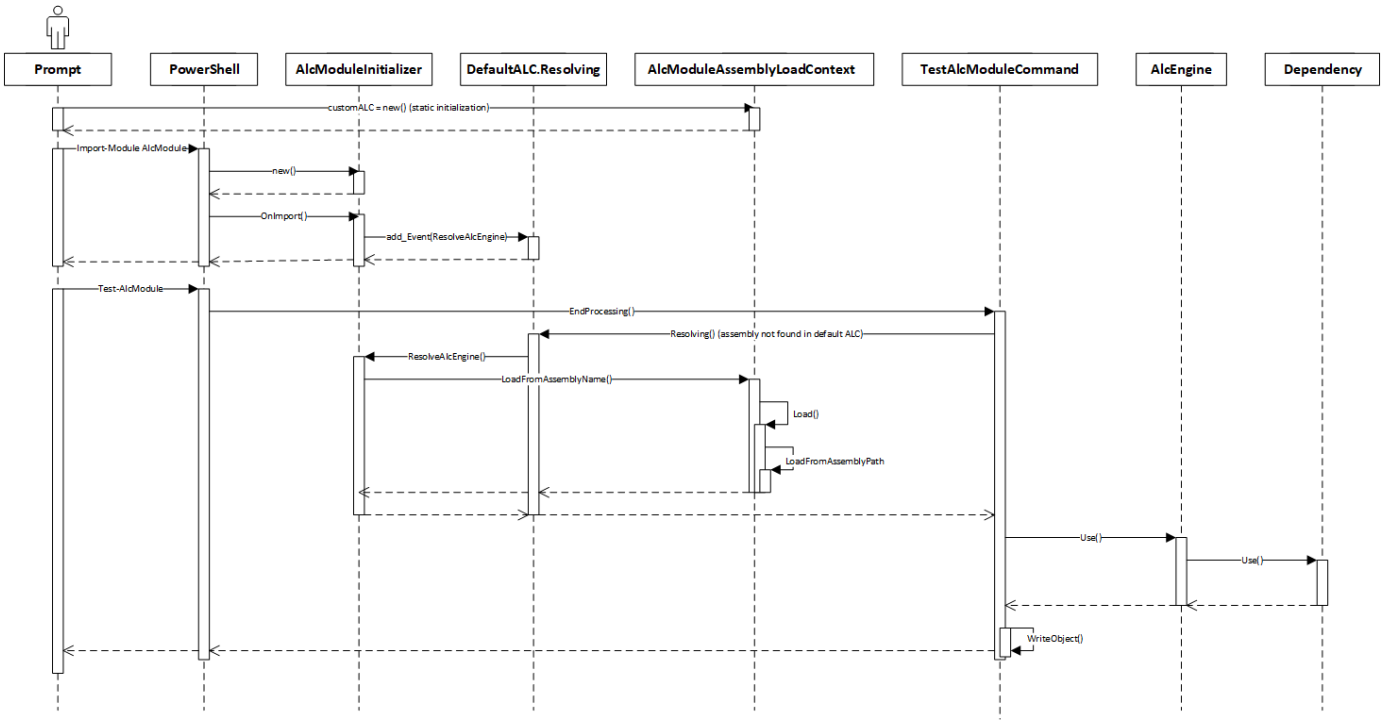
        public void OnRemove(PSModuleInfo psModuleInfo)
```

```
        {
            // Remove the Resolving event handler here
            AssemblyLoadContext.Default.Resolving -= ResolveAlcEngine;
        }

        private static Assembly ResolveAlcEngine(AssemblyLoadContext defaultAlc,
AssemblyName assemblyToResolve)
        {
            // We only want to resolve the Alc.Engine.dll assembly here.
            // Because this will be loaded into the custom ALC,
            // all of *its* dependencies will be resolved
            // by the logic we defined for that ALC's implementation.
            //
            // Note that we're safe in our assumption that the name is enough
            // to distinguish our assembly here,
            // since it's unique to our module.
            // There should be no other AlcModule.Engine.dll on the system.
            if (!assemblyToResolve.Name.Equals("AlcModule.Engine"))
            {
                return null;
            }

            // Allow our ALC to handle the directory discovery concept
            //
            // This is where Alc.Engine.dll is loaded into our custom ALC
            // and then passed through into PowerShell's ALC,
            // becoming the bridge between both
            return s_dependencyAlc.LoadFromAssemblyName(assemblyToResolve);
        }
    }
}
```

With the new implementation, take a look at the sequence of calls that occurs when the module is loaded and `Test-AlcModule` is run:



Some points of interest are:

- The `IModuleAssemblyInitializer` is run first when the module loads and sets the `Resolving` event.
- We don't load the dependencies until `Test-AlcModule` is run and its `EndProcessing()` method is called.
- When `EndProcessing()` is called, the default ALC fails to find `AlcModule.Engine.dll` and fires the `Resolving` event.
- Our event handler hooks up the custom ALC to the default ALC and loads `AlcModule.Engine.dll` only.
- When `AlcEngine.Use()` is called within `AlcModule.Engine.dll`, the custom ALC again kicks in to resolve `Shared.Dependency.dll`. Specifically, it always loads *our* `Shared.Dependency.dll` since it never conflicts with anything in the default ALC and only looks in our `Dependencies` directory.

Assembling the implementation, our new source code layout looks like this:

```
+ AlcModule.psd1
+ src/
  + AlcModule.Cmdlets/
  | + AlcModule.Cmdlets.csproj
  | + TestAlcModuleCommand.cs
  | + AlcModuleAssemblyLoadContext.cs
  | + AlcModuleInitializer.cs
  |
  + AlcModule.Engine/
```

```
  | + AlcModule.Engine.csproj
  | + AlcEngine.cs
```

AlcModule.Cmdlets.csproj looks like:

XML

```xml
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>netcoreapp3.1</TargetFramework>
  </PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="..\AlcModule.Engine\AlcModule.Engine.csproj" />
    <PackageReference Include="Microsoft.PowerShell.Sdk" Version="7.0.1"
PrivateAssets="all" />
  </ItemGroup>
</Project>
```

AlcModule.Engine.csproj looks like this:

XML

```xml
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>netcoreapp3.1</TargetFramework>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Shared.Dependency" Version="1.0.0" />
  </ItemGroup>
</Project>
```

So, when we build the module, our strategy is:

- Build `AlcModule.Engine`
- Build `AlcModule.Cmdlets`
- Copy everything from `AlcModule.Engine` into the `Dependencies` directory, and remember what we copied
- Copy everything from `AlcModule.Cmdlets` that wasn't in `AlcModule.Engine` into the base module directory

Since the module layout here is so crucial to dependency separation, here's a build script to use from the source root:

PowerShell

```powershell
param(
    # The .NET build configuration
```

```
    [ValidateSet('Debug', 'Release')]
    [string]
    $Configuration = 'Debug'
)

# Convenient reusable constants
$mod = "AlcModule"
$netcore = "netcoreapp3.1"
$copyExtensions = @('.dll', '.pdb')

# Source code locations
$src = "$PSScriptRoot/src"
$engineSrc = "$src/$mod.Engine"
$cmdletsSrc = "$src/$mod.Cmdlets"

# Generated output locations
$outDir = "$PSScriptRoot/out/$mod"
$outDeps = "$outDir/Dependencies"

# Build AlcModule.Engine
Push-Location $engineSrc
dotnet publish -c $Configuration
Pop-Location

# Build AlcModule.Cmdlets
Push-Location $cmdletsSrc
dotnet publish -c $Configuration
Pop-Location

# Ensure out directory exists and is clean
Remove-Item -Path $outDir -Recurse -ErrorAction Ignore
New-Item -Path $outDir -ItemType Directory
New-Item -Path $outDeps -ItemType Directory

# Copy manifest
Copy-Item -Path "$PSScriptRoot/$mod.psd1"

# Copy each Engine asset and remember it
$deps = [System.Collections.Generic.Hashtable[string]]::new()
Get-ChildItem -Path "$engineSrc/bin/$Configuration/$netcore/publish/" |
    Where-Object { $_.Extension -in $copyExtensions } |
    ForEach-Object { [void]$deps.Add($_.Name); Copy-Item -Path $_.FullName -
Destination $outDeps }

# Now copy each Cmdlets asset, not taking any found in Engine
Get-ChildItem -Path "$cmdletsSrc/bin/$Configuration/$netcore/publish/" |
    Where-Object { -not $deps.Contains($_.Name) -and $_.Extension -in $copyExtensions
} |
    ForEach-Object { Copy-Item -Path $_.FullName -Destination $outDir }
```

Finally, we've a general way to isolate our module's dependencies in an Assembly Load Context that remains robust over time as more dependencies are added.

For a more detailed example, go to this GitHub repository. This example demonstrates how to migrate a module to use an ALC, while keeping that module working in .NET Framework. It also shows how to use .NET Standard and PowerShell Standard to simplify the core implementation.

This solution is also used by the Bicep PowerShell module, and the blog post Resolving PowerShell Module Conflicts is another good read about this solution.

Assembly resolving handler for side-by-side loading

Although being robust, the solution described above requires the module assembly to not directly reference the dependency assemblies, but instead, reference a wrapper assembly that references the dependency assemblies. The wrapper assembly acts like a bridge, forwarding the calls from the module assembly to the dependency assemblies. This makes it usually a non-trivial amount of work to adopt this solution:

- For a new module, this would add additional complexity to the design and implementation
- For an existing module, this would require significant refactoring

There is a simplified solution to achieve side-by-side assembly loading, by hooking up a `Resolving` event with a custom `AssemblyLoadContext` instance. Using this method is easier for the module author but has two limitations. Check out the PowerShell-ALC-Samples repository for sample code and documentation that describes these limitations and detailed scenarios for this solution.

> **Important**
>
> Don't use `Assembly.LoadFile` for the dependency isolation purpose. Using `Assembly.LoadFile` creates a *Type Identity* issue when another module loads a different version of the same assembly into the default `AssemblyLoadContext`. While this API loads an assembly to a separate `AssemblyLoadContext` instance, the assemblies loaded are discoverable by PowerShell's type resolution code. Therefore, there could be duplicate types with the same fully qualifed type name available from two different ALCs.

Custom Application Domains

The final and most extreme option for assembly isolation is to use custom **Application Domains**. **Application Domains** are only available in .NET Framework. They're used to provide in-process isolation between parts of a .NET application. One of the uses is to isolate assembly loads from each other within the same process.

However, **Application Domains**are serialization boundaries. Objects in one application domain can't be referenced and used directly by objects in another application domain. You can work around this by implementing `MarshalByRefObject`. But when you don't control the types, as is often the case with dependencies, it's not possible to force an implementation here. The only solution is to make large architectural changes. The serialization boundary also has serious performance implications.

Because **Application Domains** have this serious limitation, are complicated to implement, and only work in .NET Framework, we won't give an example of how you might use them here. While they're worth mentioning as a possibility, they're not recommended.

If you're interested in trying to use a custom application domain, the following links might help:

- Conceptual documentation on Application Domains
- Examples for using Application Domains

Solutions for dependency conflicts that don't work for PowerShell

Finally, we'll address some possibilities that come up when researching .NET dependency conflicts in .NET that can look promising, but generally won't work for PowerShell.

These solutions have the common theme that they're changes to deployment configurations for an environment where you control the application and possibly the entire machine. These solutions are oriented toward scenarios like web servers and other applications deployed to server environments, where the environment is intended to host the application and is free to be configured by the deploying user. They also tend to be very much .NET Framework oriented, meaning they don't work with PowerShell 6 or higher.

If you know that your module is only used in Windows PowerShell 5.1 environments that you have total control over, some of these may be options. In general however, **modules shouldn't modify global machine state like this**. It can break configurations that cause problems in `powershell.exe`, other modules, or other dependent applications that cause your module to fail in unexpected ways.

Static binding redirect with app.config to force using the same dependency version

.NET Framework applications can take advantage of an `app.config` file to configure some application behaviors declaratively. It's possible to write an `app.config` entry that configures

assembly binding to redirect assembly loading to a particular version.

Two issues with this for PowerShell are:

- .NET Core doesn't support `app.config`, so this solution only applies to `powershell.exe`.
- `powershell.exe` is a shared application that lives in the `System32` directory. It's likely that your module won't be able to modify its contents on many systems. Even if it can, modifying the `app.config` could break an existing configuration or affect the loading of other modules.

Setting `codebase` with app.config

For the same reasons, trying to configure the `codebase` setting in `app.config` isn't going to work in PowerShell modules.

Installing dependencies to the Global Assembly Cache (GAC)

Another way to resolve dependency version conflicts in .NET Framework is to install dependencies to the GAC, so that different versions can be loaded side-by-side from the GAC.

Again, for PowerShell modules, the chief issues here are:

- The GAC only applies to .NET Framework, so this doesn't help in PowerShell 6 and above.
- Installing assemblies to the GAC is a modification of global machine state and may cause side-effects in other applications or to other modules. It may also be difficult to do correctly, even when your module has the required access privileges. Getting it wrong could cause serious, machine-wide issues in other .NET applications.

Further reading

There's plenty more to read on .NET assembly version dependency conflicts. Here are some nice jumping off points:

- .NET: Assemblies in .NET
- .NET Core: The managed assembly loading algorithm
- .NET Core: Understanding System.Runtime.Loader.AssemblyLoadContext
- .NET Core: Discussion about side-by-side assembly loading solutions
- .NET Framework: Redirecting assembly versions
- .NET Framework: Best practices for assembly loading
- .NET Framework: How the runtime locates assemblies

- .NET Framework: Resolve assembly loads
- Stack Overflow: Assembly binding redirect, how and why?
- PowerShell: Discussion about implementing AssemblyLoadContexts
- PowerShell: Assembly.LoadFile() doesn't load into default AssemblyLoadContext
- Rick Strahl: When does a .NET assembly dependency get loaded?
- Jon Skeet: Summary of versioning in .NET

Last updated on 09/14/2026

How to create a command-line predictor

PSReadLine 2.1.0 introduced the concept of a smart command-line predictor by implementing the Predictive IntelliSense feature. PSReadLine 2.2.2 expanded on that feature by adding a plugin model that allows you create your own command-line predictors.

Predictive IntelliSense enhances tab completion by providing suggestions, on the command line, as you type. The prediction suggestion appears as colored text following your cursor. This enables you to discover, edit, and execute full commands based on matching predictions from your command history or additional domain-specific plugins.

System requirements

To create and use a plugin predictor, you must be using the following versions of software:

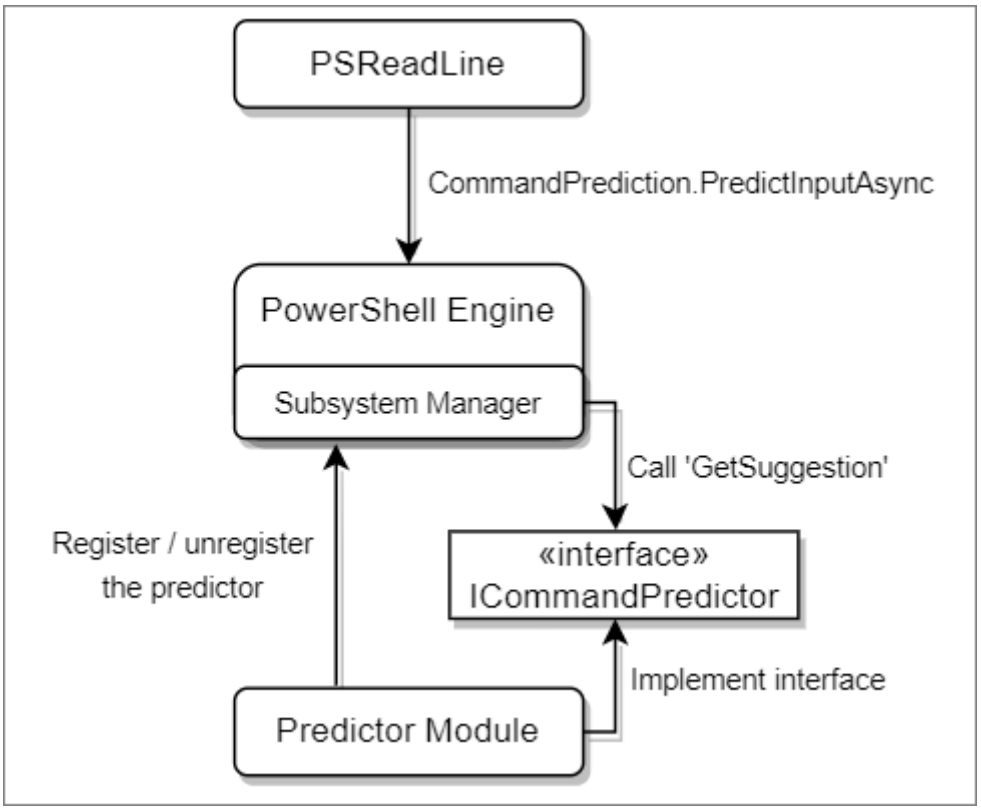
- PowerShell 7.2 (or higher) - provides the APIs necessary for creating a command predictor
- PSReadLine 2.2.2 (or higher) - allows you to configure PSReadLine to use the plugin

Overview of a predictor

A predictor is a PowerShell binary module. The module must implement the `System.Management.Automation.Subsystem.Prediction.ICommandPredictor` interface. This interface declares the methods used to query for prediction results and provide feedback.

A predictor module must register a `CommandPredictor` subsystem with PowerShell's `SubsystemManager` when loaded and unregister itself when unloaded.

The following diagram shows the architecture of a predictor in PowerShell.



Creating the code

To create a predictor, you must have the .NET 6 SDK installed for your platform. For more information on the SDK, see Download .NET 6.0.

Create a new PowerShell module project by following these steps:

1. Use the `dotnet` command-line tool to create a starter classlib project.

   PowerShell

   ```
   dotnet new classlib --name SamplePredictor
   ```

2. Edit the `SamplePredictor.csproj` to contain the following information:

   XML

   ```
   <Project Sdk="Microsoft.NET.Sdk">

     <PropertyGroup>
       <TargetFramework>net6.0</TargetFramework>
     </PropertyGroup>

     <ItemGroup>
       <PackageReference Include="Microsoft.PowerShell.SDK" Version="7.2.0" />
     </ItemGroup>
   ```

```
</Project>
```

3. Delete the default `Class1.cs` file created by `dotnet` and copy the following code to a `SamplePredictorClass.cs` file in your project folder.

C#

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Management.Automation;
using System.Management.Automation.Subsystem;
using System.Management.Automation.Subsystem.Prediction;

namespace PowerShell.Sample
{
    public class SamplePredictor : ICommandPredictor
    {
        private readonly Guid _guid;

        internal SamplePredictor(string guid)
        {
            _guid = new Guid(guid);
        }

        /// <summary>
        /// Gets the unique identifier for a subsystem implementation.
        /// </summary>
        public Guid Id => _guid;

        /// <summary>
        /// Gets the name of a subsystem implementation.
        /// </summary>
        public string Name => "SamplePredictor";

        /// <summary>
        /// Gets the description of a subsystem implementation.
        /// </summary>
        public string Description => "A sample predictor";

        /// <summary>
        /// Get the predictive suggestions. It indicates the start of a
suggestion rendering session.
        /// </summary>
        /// <param name="client">Represents the client that initiates the call.
</param>
        /// <param name="context">The <see cref="PredictionContext"/> object to
be used for prediction.</param>
        /// <param name="cancellationToken">The cancellation token to cancel
the prediction.</param>
        /// <returns>An instance of <see cref="SuggestionPackage"/>.</returns>
        public SuggestionPackage GetSuggestion(PredictionClient client,
```

```
PredictionContext context, CancellationToken cancellationToken)
        {
            string input = context.InputAst.Extent.Text;
            if (string.IsNullOrWhiteSpace(input))
            {
                return default;
            }

            return new SuggestionPackage(new List<PredictiveSuggestion>{
                new PredictiveSuggestion(string.Concat(input, " HELLO WORLD"))
            });
        }

        #region "interface methods for processing feedback"

        /// <summary>
        /// Gets a value indicating whether the predictor accepts a specific
kind of feedback.
        /// </summary>
        /// <param name="client">Represents the client that initiates the call.
</param>
        /// <param name="feedback">A specific type of feedback.</param>
        /// <returns>True or false, to indicate whether the specific feedback
is accepted.</returns>
        public bool CanAcceptFeedback(PredictionClient client,
PredictorFeedbackKind feedback) => false;

        /// <summary>
        /// One or more suggestions provided by the predictor were displayed to
the user.
        /// </summary>
        /// <param name="client">Represents the client that initiates the call.
</param>
        /// <param name="session">The mini-session where the displayed
suggestions came from.</param>
        /// <param name="countOrIndex">
        /// When the value is greater than 0, it's the number of displayed
suggestions from the list
        /// returned in <paramref name="session"/>, starting from the index 0.
When the value is
        /// less than or equal to 0, it means a single suggestion from the list
got displayed, and
        /// the index is the absolute value.
        /// </param>
        public void OnSuggestionDisplayed(PredictionClient client, uint
session, int countOrIndex) { }

        /// <summary>
        /// The suggestion provided by the predictor was accepted.
        /// </summary>
        /// <param name="client">Represents the client that initiates the call.
</param>
        /// <param name="session">Represents the mini-session where the
accepted suggestion came from.</param>
        /// <param name="acceptedSuggestion">The accepted suggestion text.
```

```
</param>
        public void OnSuggestionAccepted(PredictionClient client, uint session,
string acceptedSuggestion) { }

        /// <summary>
        /// A command line was accepted to execute.
        /// The predictor can start processing early as needed with the latest
history.
        /// </summary>
        /// <param name="client">Represents the client that initiates the call.
</param>
        /// <param name="history">History command lines provided as references
for prediction.</param>
        public void OnCommandLineAccepted(PredictionClient client,
IReadOnlyList<string> history) { }

        /// <summary>
        /// A command line was done execution.
        /// </summary>
        /// <param name="client">Represents the client that initiates the call.
</param>
        /// <param name="commandLine">The last accepted command line.</param>
        /// <param name="success">Shows whether the execution was successful.
</param>
        public void OnCommandLineExecuted(PredictionClient client, string
commandLine, bool success) { }

        #endregion;
    }

    /// <summary>
    /// Register the predictor on module loading and unregister it on module
un-loading.
    /// </summary>
    public class Init : IModuleAssemblyInitializer, IModuleAssemblyCleanup
    {
        private const string Identifier = "843b51d0-55c8-4c1a-8116-
f0728d419306";

        /// <summary>
        /// Gets called when assembly is loaded.
        /// </summary>
        public void OnImport()
        {
            var predictor = new SamplePredictor(Identifier);
            SubsystemManager.RegisterSubsystem(SubsystemKind.CommandPredictor,
predictor);
        }

        /// <summary>
        /// Gets called when the binary module is unloaded.
        /// </summary>
        public void OnRemove(PSModuleInfo psModuleInfo)
        {
```

```
SubsystemManager.UnregisterSubsystem(SubsystemKind.CommandPredictor, new
Guid(Identifier));
        }
    }
}
```

The following example code returns the string "HELLO WORLD" for the prediction result for all user input. Since the sample predictor doesn't process any feedback, the code doesn't implement the feedback methods from the interface. Change the prediction and feedback code to meet the needs of your predictor.

> **Note**
>
> The list view of **PSReadLine** doesn't support multiline suggestions. Each suggestion should be a single line. If your code has a multiline suggestion, you should split the lines into separate suggestions or join the lines with a semicolon (`;`).

4. Run `dotnet build` to produce the assembly. You can find the compiled assembly in the `bin/Debug/net6.0` location of your project folder.

> **Note**
>
> To ensure a responsive user experience, the ICommandPredictor interface has a 20ms time out for responses from the Predictors. Your predictor code must return results in less than 20ms to be displayed.

Using your predictor plugin

To try out your new predictor, open a new PowerShell 7.2 session and run the following commands:

PowerShell

```
Set-PSReadLineOption -PredictionSource HistoryAndPlugin
Import-Module .\bin\Debug\net6.0\SamplePredictor.dll
```

With the assembly is loaded in the session, you see the text "HELLO WORLD" appear as you type in the terminal. You can press `F2` to switch between the `Inline` view and the `List` view.

For more information about PSReadLine options, see Set-PSReadLineOption.

You can get a list of installed predictors, using the following command:

PowerShell

```
Get-PSSubsystem -Kind CommandPredictor
```

Output

```
Kind             SubsystemType      IsRegistered Implementations
----             -------------      ------------ ---------------
CommandPredictor  ICommandPredictor          True {SamplePredictor}
```

> **Note**
>
> `Get-PSSubsystem` is an experimental cmdlet that was introduced in PowerShell 7.1 You must enable the `PSSubsystemPluginModel` experimental feature to use this cmdlet. For more information, see Using Experimental Features.

Last updated on 12/08/2025

How to create a feedback provider

PowerShell 7.4 introduced the concept of feedback providers. A feedback provider is a PowerShell module that implements the `IFeedbackProvider` interface to provide command suggestions based on user command execution attempts. The provider is triggered when there's a success or failure execution. Feedback providers use information from the success or failure to provide feedback.

Prerequisites

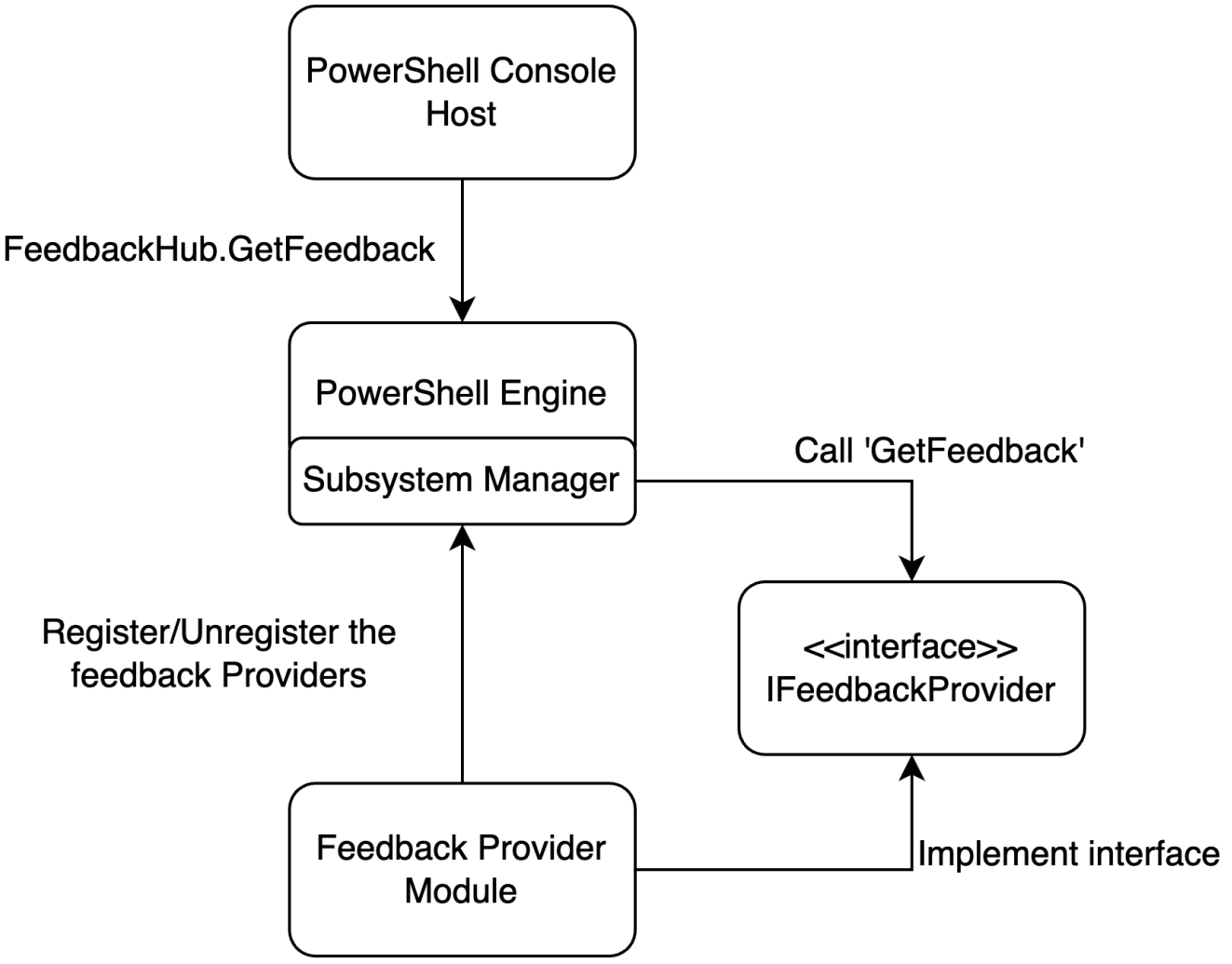
To create a feedback provider, you must satisfy the following prerequisites:

- Install PowerShell 7.4 or higher
  - You must enable the `PSFeedbackProvider` experimental feature to enable support for feedback providers and predictors. For more information, see Using Experimental Features.
- Install .NET 8 SDK - 8.0.0 or higher
  - See the Download .NET 8.0 page to get the latest version of the SDK.

Overview of a feedback provider

A feedback provider is a PowerShell binary module that implements the `System.Management.Automation.Subsystem.Feedback.IFeedbackProvider` interface. This interface declares the methods to get feedback based on the command line input. The feedback interface can provide suggestions based on the success or failure of the command invoked by the user. The suggestions can be anything that you want. For example, you might suggest ways to address an error or better practices, like avoiding the use of aliases. For more information, see the What are Feedback Providers? blog post.

The following diagram shows the architecture of a feedback provider:



The following examples walk you through the process of creating a simple feedback provider. Also, you can register the provider with the command predictor interface to add feedback suggestions to the command-line predictor experience. For more information about predictors, see Using predictors in PSReadLine and How to create a command line predictor.

Step 1 - Create a new class library project

Use the following command to create a new project in the project directory:

PowerShell

```
dotnet new classlib --name MyFeedbackProvider
```

Add a package reference for the `System.Management.Automation` package to your `.csproj` file. The following example shows the updated `.csproj` file:

XML

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
```

```
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="System.Management.Automation" Version="7.4.0-
preview.3">
        <ExcludeAssets>contentFiles</ExcludeAssets>
        <PrivateAssets>All</PrivateAssets>
    </PackageReference>
  </ItemGroup>
</Project>
```

> **Note**
>
> You should change the version of the `System.Management.Automation` assembly to match the version of the PowerShell preview that you are targeting. The minimum version is 7.4.0-preview.3.

Step 2 - Add the class definition for your provider

Change the name of the `Class1.cs` file to match the name of your provider. This example uses `myFeedbackProvider.cs`. This file contains the two main classes that define the feedback provider. The following example shows the basic template for the class definitions.

C#

```
using System.Management.Automation;
using System.Management.Automation.Subsystem;
using System.Management.Automation.Subsystem.Feedback;
using System.Management.Automation.Subsystem.Prediction;
using System.Management.Automation.Language;

namespace myFeedbackProvider;

public sealed class myFeedbackProvider : IFeedbackProvider, ICommandPredictor
{

}

public class Init : IModuleAssemblyInitializer, IModuleAssemblyCleanup
{

}
```

Step 3 - Implement the Init class

The `Init` class registers and unregisters the feedback provider with the subsystem manager. The `OnImport()` method runs when the binary module is being loaded. The `OnRemove()` method runs when the binary module is being removed. This example registers both the feedback provider and command predictor subsystem.

C#

```
public class Init : IModuleAssemblyInitializer, IModuleAssemblyCleanup
{
    private const string Id = "<ADD YOUR GUID HERE>";

    public void OnImport()
    {
        var feedback = new myFeedbackProvider(Id);
        SubsystemManager.RegisterSubsystem(SubsystemKind.FeedbackProvider,
feedback);
        SubsystemManager.RegisterSubsystem(SubsystemKind.CommandPredictor,
feedback);
    }

    public void OnRemove(PSModuleInfo psModuleInfo)
    {
        SubsystemManager.UnregisterSubsystem<ICommandPredictor>(new Guid(Id));
        SubsystemManager.UnregisterSubsystem<IFeedbackProvider>(new Guid(Id));
    }
}
```

Replace the `<ADD YOUR GUID HERE>` placeholder value with a unique Guid. You can generate a Guid using the `New-Guid` cmdlet.

PowerShell

```
New-Guid
```

The Guid is a unique identifier for your provider. The provider must have a unique Id to be registered with the subsystem.

Step 4 - Add class members and define the constructor

The following code implements the properties defined in the interfaces, adds needed class members, and creates the constructor for the `myFeedbackProvider` class.

C#

```
/// <summary>
/// Gets the global unique identifier for the subsystem implementation.
/// </summary>
private readonly Guid _guid;
public Guid Id => _guid;

/// <summary>
/// Gets the name of a subsystem implementation, this will be the name displayed
when triggered
/// </summary>
public string Name => "myFeedbackProvider";

/// <summary>
/// Gets the description of a subsystem implementation.
/// </summary>
public string Description => "This is very simple feedback provider";

/// <summary>
/// Default implementation. No function is required for a feedback provider.
/// </summary>
Dictionary<string, string>? ISubsystem.FunctionsToDefine => null;

/// <summary>
/// Gets the types of trigger for this feedback provider.
/// </summary>
/// <remarks>
/// The default implementation triggers a feedback provider by <see
cref="FeedbackTrigger.CommandNotFound"/> only.
/// </remarks>
public FeedbackTrigger Trigger => FeedbackTrigger.All;

/// <summary>
/// List of candidates from the feedback provider to be passed as predictor results
/// </summary>
private List<string>? _candidates;

/// <summary>
/// PowerShell session used to run PowerShell commands that help create suggestions.
/// </summary>
private PowerShell _powershell;

internal myFeedbackProvider(string guid)
{
    _guid = new Guid(guid); // Save guid
    _powershell = PowerShell.Create(); // Create PowerShell instance
}
```

Step 5 - Create the GetFeedback() method

The `GetFeedback` method takes two parameters, `context` and `token`. The `context` parameter receives the information about the trigger so you can decide how to respond with suggestions. The `token` parameter is used for cancellation. This function returns a `FeedbackItem` containing the suggestion.

C#

```
/// <summary>
/// Gets feedback based on the given commandline and error record.
/// </summary>
/// <param name="context">The context for the feedback call.</param>
/// <param name="token">The cancellation token to cancel the operation.</param>
/// <returns>The feedback item.</returns>
public FeedbackItem? GetFeedback(FeedbackContext context, CancellationToken token)
{
    // Target describes the different kinds of triggers to activate on,
    var target = context.Trigger;
    var commandLine = context.CommandLine;
    var ast = context.CommandLineAst;

    // defining the header and footer variables
    string header;
    string footer;

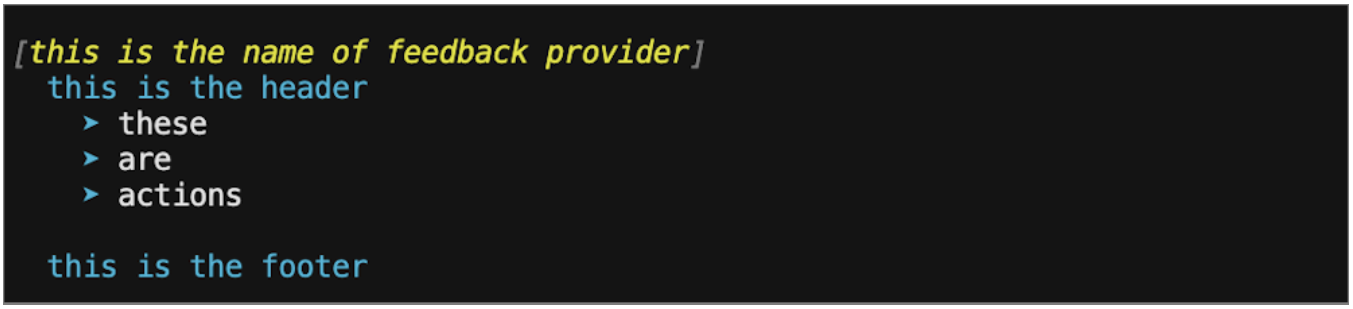
    // List of the actions
    List<string>? actions = new List<string>();

    // Trigger on success code goes here

    // Trigger on error code goes here

    return null;
}
```

The following image shows how these fields are used in the suggestions that are displayed to the user.



Create suggestions for a Success trigger

For a successful invocation, we want to expand any aliases used in the last execution. Using the `CommandLineAst`, we identify any aliased commands and create a suggestion to use the fully

qualified command name instead.

C#

```
// Trigger on success
if (target == FeedbackTrigger.Success)
{
    // Getting the commands from the AST and only finding those that are Commands
    var astCmds = ast.FindAll((cAst) => cAst is CommandAst, true);

    // Inspect each of the commands
    foreach(var command in astCmds)
    {

        // Get the command name
        var aliasedCmd = ((CommandAst) command).GetCommandName();

        // Check if its an alias or not, if so then add it to the list of actions
        if(TryGetAlias(aliasedCmd, out string commandString))
        {
            actions.Add($"{aliasedCmd} --> {commandString}");
        }
    }

    // If no alias was found return null
    if(actions.Count == 0)
    {
        return null;
    }

    // If aliases are found, set the header to a description and return a new
FeedbackItem.
    header = "You have used an aliased command:";
    // Copy actions to _candidates for the predictor
    _candidates = actions;

    return new FeedbackItem(header, actions);
}
```

Implement the TryGetAlias() method

The `TryGetAlias()` method is a private helper function that returns a boolean value to indicate whether the command is an alias. In the class constructor, we created a PowerShell instance that we can use to run PowerShell commands. The `TryGetAlias()` method uses this PowerShell instance to invoke the `GetCommand` method to determine if the command is an alias. The `AliasInfo` object returned by `GetCommand` contains full name of the aliased command.

C#

```
/// <summary>
/// Checks if a command is an alias.
/// </summary>
/// <param name="command">The command to check if alias</param>
/// <param name="targetCommand">The referenced command by the aliased
command</param>
/// <returns>True if an alias and false if not</returns>
private bool TryGetAlias(string command, out string targetCommand)
{
    // Create PowerShell runspace as a session state proxy to run GetCommand and
check
    // if its an alias
    AliasInfo? pwshAliasInfo =
        _powershell.Runspace.SessionStateProxy.InvokeCommand.GetCommand(command,
CommandTypes.Alias) as AliasInfo;

    // if its null then it is not an aliased command so just return false
    if(pwshAliasInfo is null)
    {
        targetCommand = String.Empty;
        return false;
    }

    // Set targetCommand to referenced command name
    targetCommand = pwshAliasInfo.ReferencedCommand.Name;
    return true;
}
```

Create suggestions for a Failure trigger

When a command execution fails, we want to suggest that the user `Get-Help` to get more information about how to use the command.

C#

```
// Trigger on error
if (target == FeedbackTrigger.Error)
{
    // Gets the command that caused the error.
    var erroredCommand = context.LastError?.InvocationInfo.MyCommand;
    if (erroredCommand is null)
    {
        return null;
    }

    header = $"You have triggered an error with the command {erroredCommand}. Try
using the following command to get help:";

    actions.Add($"Get-Help {erroredCommand}");
    footer = $"You can also check online documentation at
https://learn.microsoft.com/en-us/powershell/module/?term={erroredCommand}";
```

```
    // Copy actions to _candidates for the predictor
    _candidates = actions;
    return new FeedbackItem(header, actions, footer,
FeedbackDisplayLayout.Portrait);
}
```

Step 6 - Send suggestions to the command line predictor

Another way your feedback provider can enhance the user experience is to provide command suggestions to the **ICommandPredictor** interface. For more information about creating a command line predictor, see How to create a command line predictor.

The following code implements the methods necessary from the **ICommandPredictor** interface to add predictor behavior to your feedback provider.

- `CanAcceptFeedback()` - This method returns a Boolean value that indicates whether the predictor accepts a specific type of feedback.
- `GetSuggestion()` - This method returns a `SuggestionPackage` object that contains the suggestions to be displayed by the predictor.
- `OnCommandLineAccepted()` - This method is called when a command line is accepted to execute.

C#

```
/// <summary>
/// Gets a value indicating whether the predictor accepts a specific kind of
feedback.
/// </summary>
/// <param name="client">Represents the client that initiates the call.</param>
/// <param name="feedback">A specific type of feedback.</param>
/// <returns>True or false, to indicate whether the specific feedback is accepted.
</returns>
public bool CanAcceptFeedback(PredictionClient client, PredictorFeedbackKind
feedback)
{
    return feedback switch
    {
        PredictorFeedbackKind.CommandLineAccepted => true,
        _ => false,
    };
}

/// <summary>
/// Get the predictive suggestions. It indicates the start of a suggestion rendering
session.
```

```
/// </summary>
/// <param name="client">Represents the client that initiates the call.</param>
/// <param name="context">The <see cref="PredictionContext"/> object to be used for
prediction.</param>
/// <param name="cancellationToken">The cancellation token to cancel the prediction.
</param>
/// <returns>An instance of <see cref="SuggestionPackage"/>.</returns>
public SuggestionPackage GetSuggestion(
    PredictionClient client,
    PredictionContext context,
    CancellationToken cancellationToken)
{
    if (_candidates is not null)
    {
        string input = context.InputAst.Extent.Text;
        List<PredictiveSuggestion>? result = null;

        foreach (string c in _candidates)
        {
            if (c.StartsWith(input, StringComparison.OrdinalIgnoreCase))
            {
                result ??= new List<PredictiveSuggestion>(_candidates.Count);
                result.Add(new PredictiveSuggestion(c));
            }
        }

        if (result is not null)
        {
            return new SuggestionPackage(result);
        }
    }

    return default;
}

/// <summary>
/// A command line was accepted to execute.
/// The predictor can start processing early as needed with the latest history.
/// </summary>
/// <param name="client">Represents the client that initiates the call.</param>
/// <param name="history">History command lines provided as references for
prediction.</param>
public void OnCommandLineAccepted(PredictionClient client, IReadOnlyList<string>
history)
{
    // Reset the candidate state once the command is accepted.
    _candidates = null;
}
```

Step 7 - Build the feedback provider

Now you are ready to build and begin using your feedback provider! To build the project, run the following command:

PowerShell

```
dotnet build
```

This command create the PowerShell module as a DLL file in the following path of your project folder: `bin/Debug/net8.0/myFeedbackProvider`

You may run into the error `error NU1101: Unable to find package System.Management.Automation.` when building on Windows machines. To fix this add a `nuget.config` file to your project directory and add the following:

YAML

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <clear />
    <add key="nuget.org" value="https://api.nuget.org/v3/index.json" />
  </packageSources>
  <disabledPackageSources>
    <clear />
  </disabledPackageSources>
</configuration>
```

Using a feedback provider

To test your new feedback provider, import the compiled module into your PowerShell session. This can be done by importing the folder described after building has succeeded:

PowerShell

```
Import-Module ./bin/Debug/net8.0/myFeedbackProvider
```

Once you're satisfied with your module, you should create a module manifest, publish it to the PowerShell Gallery, and install it in your `$Env:PSModulePath`. For more information, see How to create a module manifest. You can add the `Import-Module` command to your `$PROFILE` script so the module is available in PowerShell session.

You can get a list of installed feedback providers, using the following command:

PowerShell

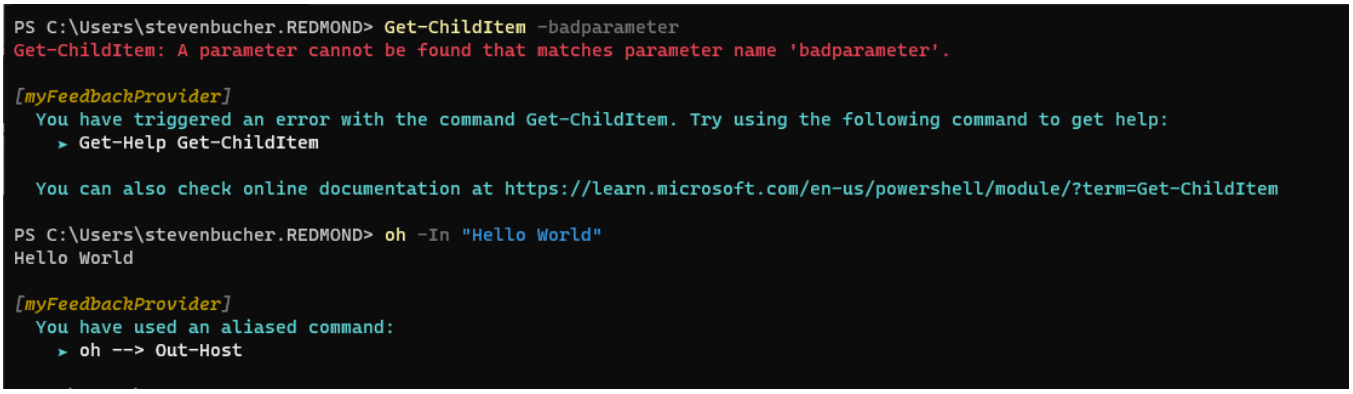
```
Get-PSSubsystem -Kind FeedbackProvider
```

Output

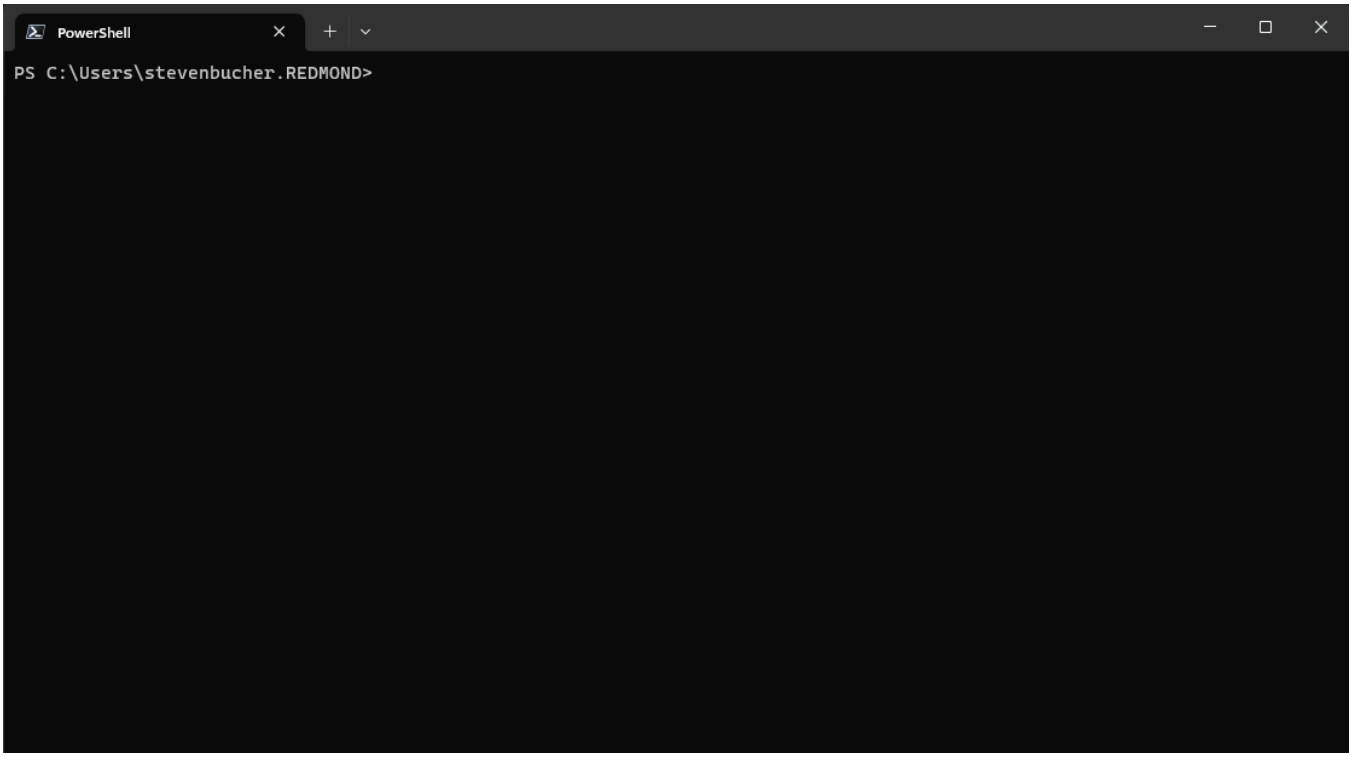
```
Kind              SubsystemType      IsRegistered Implementations
----              -------------      ------------ ---------------
FeedbackProvider  IFeedbackProvider          True {general}
```

> **Note**
>
> `Get-PSSubsystem` is an experimental cmdlet that was introduced in PowerShell 7.1 You must enable the `PSSubsystemPluginModel` experimental feature to use this cmdlet. For more information, see Using Experimental Features.

The following screenshot shows some example suggestions from the new provider.



The following is a GIF showing how the predictor integration works from the new provider.



Other feedback providers

We have created other feedback provider that can be used as a good reference for deeper examples.

command-not-found

The `command-not-found` feedback provider utilizes the `command-not-found` utility tool on Linux systems to provide suggestions when native commands are attempted to run but are missing. You can find the code in the GitHub Repository or can download for yourself on the PowerShell Gallery.

PowerShell Adapter

The `Microsoft.PowerShell.PowerShellAdapter` is a feedback provider that helps you convert text outputs from native commands into PowerShell objects. It detects "adapters" on your system and suggests you to use them when you use the native command. You can learn more about PowerShell Adapters from, PowerShell Adapter Feedback Provider blog post. You can also find the code in the GitHub Repository or can download for yourself on the PowerShell Gallery.

Appendix - Full implementation code

The following code combines the previous examples into the find full implementation of the provider class.

C#

```
using System.Management.Automation;
using System.Management.Automation.Subsystem;
using System.Management.Automation.Subsystem.Feedback;
using System.Management.Automation.Subsystem.Prediction;
using System.Management.Automation.Language;

namespace myFeedbackProvider;

public sealed class myFeedbackProvider : IFeedbackProvider, ICommandPredictor
{
    /// <summary>
    /// Gets the global unique identifier for the subsystem implementation.
    /// </summary>
    private readonly Guid _guid;
    public Guid Id => _guid;

    /// <summary>
```

```
    /// Gets the name of a subsystem implementation, this will be the name displayed
when triggered
    /// </summary>
    public string Name => "myFeedbackProvider";

    /// <summary>
    /// Gets the description of a subsystem implementation.
    /// </summary>
    public string Description => "This is very simple feedback provider";

    /// <summary>
    /// Default implementation. No function is required for a feedback provider.
    /// </summary>
    Dictionary<string, string>? ISubsystem.FunctionsToDefine => null;

    /// <summary>
    /// Gets the types of trigger for this feedback provider.
    /// </summary>
    /// <remarks>
    /// The default implementation triggers a feedback provider by <see
cref="FeedbackTrigger.CommandNotFound"/> only.
    /// </remarks>
    public FeedbackTrigger Trigger => FeedbackTrigger.All;

    /// <summary>
    /// List of candidates from the feedback provider to be passed as predictor
results
    /// </summary>
    private List<string>? _candidates;

    /// <summary>
    /// PowerShell session used to run PowerShell commands that help create
suggestions.
    /// </summary>
    private PowerShell _powershell;

    // Constructor
    internal myFeedbackProvider(string guid)
    {
        _guid = new Guid(guid); // Save guid
        _powershell = PowerShell.Create(); // Create PowerShell instance
    }

    #region IFeedbackProvider
    /// <summary>
    /// Gets feedback based on the given commandline and error record.
    /// </summary>
    /// <param name="context">The context for the feedback call.</param>
    /// <param name="token">The cancellation token to cancel the operation.</param>
    /// <returns>The feedback item.</returns>
    public FeedbackItem? GetFeedback(FeedbackContext context, CancellationToken
token)
    {
        // Target describes the different kinds of triggers to activate on,
        var target = context.Trigger;
```

```
        var commandLine = context.CommandLine;
        var ast = context.CommandLineAst;

        // defining the header and footer variables
        string header;
        string footer;

        // List of the actions
        List<string>? actions = new List<string>();

        // Trigger on success
        if (target == FeedbackTrigger.Success)
        {
            // Getting the commands from the AST and only finding those that are
Commands
            var astCmds = ast.FindAll((cAst) => cAst is CommandAst, true);

            // Inspect each of the commands
            foreach(var command in astCmds)
            {

                // Get the command name
                var aliasedCmd = ((CommandAst) command).GetCommandName();

                // Check if its an alias or not, if so then add it to the list of
actions
                if(TryGetAlias(aliasedCmd, out string commandString))
                {
                    actions.Add($"{aliasedCmd} --> {commandString}");
                }
            }

            // If no alias was found return null
            if(actions.Count == 0)
            {
                return null;
            }

            // If aliases are found, set the header to a description and return a
new FeedbackItem.
            header = "You have used an aliased command:";
            // Copy actions to _candidates for the predictor
            _candidates = actions;

            return new FeedbackItem(header, actions);
        }

        // Trigger on error
        if (target == FeedbackTrigger.Error)
        {
            // Gets the command that caused the error.
            var erroredCommand = context.LastError?.InvocationInfo.MyCommand;
            if (erroredCommand is null)
            {
                return null;
```

```
            }

            header = $"You have triggered an error with the command
{erroredCommand}. Try using the following command to get help:";

            actions.Add($"Get-Help {erroredCommand}");
            footer = $"You can also check online documentation at
https://learn.microsoft.com/en-us/powershell/module/?term={erroredCommand}";

            // Copy actions to _candidates for the predictor
            _candidates = actions;
            return new FeedbackItem(header, actions, footer,
FeedbackDisplayLayout.Portrait);
        }
        return null;
    }

    /// <summary>
    /// Checks if a command is an alias.
    /// </summary>
    /// <param name="command">The command to check if alias</param>
    /// <param name="targetCommand">The referenced command by the aliased
command</param>
    /// <returns>True if an alias and false if not</returns>
    private bool TryGetAlias(string command, out string targetCommand)
    {
        // Create PowerShell runspace as a session state proxy to run GetCommand and
check
        // if its an alias
        AliasInfo? pwshAliasInfo =
            _powershell.Runspace.SessionStateProxy.InvokeCommand.GetCommand(command,
CommandTypes.Alias) as AliasInfo;

        // if its null then it is not an aliased command so just return false
        if(pwshAliasInfo is null)
        {
            targetCommand = String.Empty;
            return false;
        }

        // Set targetCommand to referenced command name
        targetCommand = pwshAliasInfo.ReferencedCommand.Name;
        return true;
    }
    #endregion IFeedbackProvider

    #region ICommandPredictor

    /// <summary>
    /// Gets a value indicating whether the predictor accepts a specific kind of
feedback.
    /// </summary>
    /// <param name="client">Represents the client that initiates the call.</param>
    /// <param name="feedback">A specific type of feedback.</param>
    /// <returns>True or false, to indicate whether the specific feedback is
```

```
accepted.</returns>
    public bool CanAcceptFeedback(PredictionClient client, PredictorFeedbackKind
feedback)
    {
        return feedback switch
        {
            PredictorFeedbackKind.CommandLineAccepted => true,
            _ => false,
        };
    }

    /// <summary>
    /// Get the predictive suggestions. It indicates the start of a suggestion
rendering session.
    /// </summary>
    /// <param name="client">Represents the client that initiates the call.</param>
    /// <param name="context">The <see cref="PredictionContext"/> object to be used
for prediction.</param>
    /// <param name="cancellationToken">The cancellation token to cancel the
prediction.</param>
    /// <returns>An instance of <see cref="SuggestionPackage"/>.</returns>
    public SuggestionPackage GetSuggestion(
        PredictionClient client,
        PredictionContext context,
        CancellationToken cancellationToken)
    {
        if (_candidates is not null)
        {
            string input = context.InputAst.Extent.Text;
            List<PredictiveSuggestion>? result = null;

            foreach (string c in _candidates)
            {
                if (c.StartsWith(input, StringComparison.OrdinalIgnoreCase))
                {
                    result ??= new List<PredictiveSuggestion>(_candidates.Count);
                    result.Add(new PredictiveSuggestion(c));
                }
            }

            if (result is not null)
            {
                return new SuggestionPackage(result);
            }
        }

        return default;
    }

    /// <summary>
    /// A command line was accepted to execute.
    /// The predictor can start processing early as needed with the latest history.
    /// </summary>
    /// <param name="client">Represents the client that initiates the call.</param>
    /// <param name="history">History command lines provided as references for
```

```
prediction.</param>
    public void OnCommandLineAccepted(PredictionClient client, IReadOnlyList<string>
history)
    {
        // Reset the candidate state once the command is accepted.
        _candidates = null;
    }

    #endregion;
}

public class Init : IModuleAssemblyInitializer, IModuleAssemblyCleanup
{
    private const string Id = "<ADD YOUR GUID HERE>";

    public void OnImport()
    {
        var feedback = new myFeedbackProvider(Id);
        SubsystemManager.RegisterSubsystem(SubsystemKind.FeedbackProvider,
feedback);
        SubsystemManager.RegisterSubsystem(SubsystemKind.CommandPredictor,
feedback);
    }

    public void OnRemove(PSModuleInfo psModuleInfo)
    {
        SubsystemManager.UnregisterSubsystem<ICommandPredictor>(new Guid(Id));
        SubsystemManager.UnregisterSubsystem<IFeedbackProvider>(new Guid(Id));
    }
}
```

Last updated on 12/08/2025

PlatyPS overview

10/09/2025

PlatyPS is the primary tool for creating the PowerShell help displayed using `Get-Help`. PowerShell help files are written in Microsoft Assistance Markup Language (MAML) format. MAML defines an XML schema for the structure of help files.

There are two major versions of PlatyPS.

- **Microsoft.PowerShell.PlatyPS** v1.0.1 is the supported version of PlatyPS. This version is a complete rewrite in C# leveraging markdig for parsing Markdown. This release includes several improvements:
  - Provides a more accurate description of a PowerShell cmdlet and its parameters
  - Increased performance - processes 1000s of Markdown files in seconds
  - Creates an object model of the help file that you can manipulate in memory
  - Provides cmdlets that you can chain together to perform complex operations
  - Defines a new Markdown schema that includes all elements needed for `Get-Help`, plus information that was previously unavailable.
  - Provide automatic conversion of existing Markdown (using the old schema) to new objects, enabling you to export to new Markdown, YAML, or MAML.
- **platyPS** v0.14.2 is the original implementation of PlatyPS. This version is no longer supported.

Benefits of using PlatyPS

Prior to **PlatyPS**, the help files were hand written with limited help from existing tools and editors. **PlatyPS** simplifies the process by allowing you to write the help files in Markdown and then convert it to MAML.

Markdown is easy to learn, widely used in the open source community, and supported by many editors including Visual Studio Code. Markdown is also easy to convert to other formats such as HTML and PDF. You can use these Markdown files to create MAML help files and to create HTML pages for a website.

Get started with PlatyPS

Before getting started with PlatyPS, you should understand the types of help supported by PowerShell. For more information, see Types of help in PowerShell.

Creating help files with PlatyPS is a four-step process:

1. Create new or update existing Markdown help files.
2. Edit the Markdown help files to add descriptions and examples.
3. Test the Markdown help files to ensure they render correctly.
4. Convert and publish the help files.

Windows PowerShell Language Specification 3.0

Article • 01/08/2025

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389 That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

1. Introduction

PowerShell is a command-line *shell* and scripting language, designed especially for system administrators.

Most shells operate by executing a command or utility in a new process, and presenting the results to the user as text. These shells also have commands that are built into the shell and run in the shell process. Because there are few built-in commands, many utilities have been created to supplement them. PowerShell is very different. Instead of processing text, the shell processes objects. PowerShell also includes a large set of built-in commands with each having a consistent interface and these can work with user-written commands.

An *object* is a data entity that has *properties* (i.e., characteristics) and *methods* (i.e., actions that can be performed on the object). All objects of the same type have the same base set of properties and methods, but each *instance* of an object can have different property values.

A major advantage of using objects is that it is much easier to *pipeline* commands; that is, to write the output of one command to another command as input. (In a traditional command-line environment, the text output from one command needs to be manipulated to meet the input format of another.)

PowerShell includes a very rich scripting language that supports constructs for looping, conditions, flow-control, and variable assignment. This language has syntax features and keywords similar to those used in the C# programming language (§C.).

There are four kinds of commands in PowerShell: scripts, functions and methods, cmdlets, and native commands.

- A file of commands is called a *script*. By convention, a script has a filename extension of .ps1. The top-most level of a PowerShell program is a script, which, in turn, can invoke other commands.
- PowerShell supports modular programming via named procedures. A procedure written in PowerShell is called a *function*, while an external procedure made available by the execution environment (and typically written in some other language) is called a *method*.
- A *cmdlet* (pronounced "command-let") is a simple, single-task command-line tool. Although a cmdlet can be used on its own, the full power of cmdlets is realized when they are used in combination to perform complex tasks.
- A *native command* is part of the host environment.

Each time the PowerShell runtime environment begins execution, it begins what is called a *session*. Commands then execute within the context of that session.

This specification defines the PowerShell language, the built-in cmdlets, and the use of objects via the pipeline.

Unlike most shells, which accept and return text, Windows PowerShell is built on top of the .NET Framework common language runtime (CLR) and the .NET Framework, and accepts and returns .NET Framework objects.

2. Lexical Structure

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

2.1 Grammars

This specification shows the syntax of the PowerShell language using two grammars. The *lexical grammar* (§B.1) shows how Unicode characters are combined to form line terminators, comments, white space, and tokens. The *syntactic grammar* (§B.2) shows how the tokens resulting from the lexical grammar are combined to form PowerShell scripts.

For convenience, fragments of these grammars are replicated in appropriate places throughout this specification.

Any use of the characters 'a' through 'z' in the grammars is case insensitive. This means that letter case in variables, aliases, function names, keywords, statements, and operators is ignored. However, throughout this specification, such names are written in lowercase, except for some automatic and preference variables.

2.2 Lexical analysis

2.2.1 Scripts

Syntax:

> **Tip**
>
> The `~opt~` notation in the syntax definitions indicates that the lexical entity is optional in the syntax.

Syntax

```
input:
    input-elements~opt~   signature-block~opt~

input-elements:
    input-element
    input-elements   input-element

input-element:
    whitespace
    comment
    token

signature-block:
    signature-begin   signature   signature-end

signature-begin:
    new-line-character   # SIG # Begin signature block   new-line-character

signature:
    base64 encoded signature blob in multiple single-line-comments

signature-end:
    new-line-character   # SIG # End signature block   new-line-character
```

Description:

The input source stream to a PowerShell translator is the *input* in a script, which contains a sequence of Unicode characters. The lexical processing of this stream involves the reduction of those characters into a sequence of tokens, which go on to become the input of syntactic analysis.

A script is a group of PowerShell commands stored in a *script-file*. The script itself has no name, per se, and takes its name from its source file. The end of that file indicates the end of the script.

A script may optionally contain a digital signature. A host environment is not required to process any text that follows a signature or anything that looks like a signature. The creation

and use of digital signatures are not covered by this specification.

2.2.2 Line terminators

Syntax:

Syntax

```
new-line-character:
    Carriage return character (U+000D)
    Line feed character (U+000A)
    Carriage return character (U+000D) followed by line feed character (U+000A)

new-lines:
    new-line-character
    new-lines new-line-character
```

Description:

The presence of *new-line-characters* in the input source stream divides it into lines that can be used for such things as error reporting and the detection of the end of a single-line comment.

A line terminator can be treated as white space (§2.2.4).

2.2.3 Comments

Syntax:

Syntax

```
comment:
    single-line-comment
    requires-comment
    delimited-comment

single-line-comment:
    # input-characters~opt~

input-characters:
    input-character
    input-characters input-character

input-character:
    Any Unicode character except a new-line-character

requires-comment:
    #Requires whitespace command-arguments

dash:
```

```
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)

dashdash:
    dash dash

delimited-comment:
    < # delimited-comment-text~opt~ hashes >

delimited-comment-text:
    delimited-comment-section
    delimited-comment-text delimited-comment-section

delimited-comment-section:
    >
    hashes~opt~  not-greater-than-or-hash

hashes:
    #
    hashes #

not-greater-than-or-hash:
    Any Unicode character except > or #
```

Description:

Source code can be annotated by the use of *comments*.

A *single-line-comment* begins with the character `#` and ends with a *new-line-character*.

A *delimited-comment* begins with the character pair `<#` and ends with the character pair `#>`. It can occur as part of a source line, as a whole source line, or it can span any number of source lines.

A comment is treated as white space.

The productions above imply that

- Comments do not nest.
- The character sequences <# and #> have no special meaning in a single-line comment.
- The character # has no special meaning in a delimited comment.

The lexical grammar implies that comments cannot occur inside tokens.

(See §A for information about creating script files that contain special-valued comments that are used to generate documentation from script files.)

A *requires-comment* specifies the criteria that have to be met for its containing script to be allowed to run. The primary criterion is the version of PowerShell being used to run the script. The minimum version requirement is specified as follows:

`#Requires -Version N[.n]`

Where *N* is the (required) major version and *n* is the (optional) minor version.

A *requires-comment* can be present in any script file; however, it cannot be present inside a function or cmdlet. It must be the first item on a source line. A script can contain multiple *requires-comment*s.

A character sequence is only recognized as a comment if that sequence begins with `#` or `<#`. For example, hello#there is considered a single token whereas hello #there is considered the token hello followed by a single-line comment. As well as following white space, the comment start sequence can also be preceded by any expression-terminating or statement-terminating character (such as `)`, `}`, `]`, `'`, `"`, or `;`).

A *requires-comment* cannot be present inside a snap-in.

There are four other forms of a *requires-comment*:

Syntax

```
#Requires -Assembly AssemblyId
#Requires -Module ModuleName
#Requires -PSSnapin PSSnapin [ -Version *N* [.n] ]
#Requires -ShellId ShellId
```

2.2.4 White space

Syntax:

Syntax

```
whitespace:
    Any character with Unicode class Zs, Zl, or Zp
    Horizontal tab character (U+0009)
    Vertical tab character (U+000B)
    Form feed character (U+000C)
    ` (The backtick character U+0060) followed by new-line-character
```

Description:

White space consists of any sequence of one or more *whitespace* characters.

Except for the fact that white space may act as a separator for tokens, it is ignored.

Unlike some popular languages, PowerShell does not consider line-terminator characters (§2.2.2) to be white space. However, a line terminator can be treated as white space by preceding it immediately by a backtick character, `` ` `` (U+0060). This is necessary when the contents of a line are complete syntactically, yet the following line contains tokens intended to be associated with the previous line. For example,

PowerShell

```
$number = 10 # assigns 10 to $number; nothing is written to the pipeline
+ 20 # writes 20 to the pipeline
- 50 # writes -50 to the pipeline
$number # writes $number's value, 10, to the pipeline
```

In this example, the backtick indicates the source line is continued. The following expression is equivalent to `$number = 10 + 20 - 50`.

PowerShell

```
$number = 10 `
+ 20 `
- 50
$number # writes $number's value to the pipeline
-20
```

2.3 Tokens

Syntax:

Syntax

```
token:
    keyword
    variable
    command
    command-parameter
    command-argument-token
    integer-literal
    real-literal
    string-literal
    type-literal
    operator-or-punctuator
```

Description:

A *token* is the smallest lexical element within the PowerShell language.

Tokens can be separated by *new-lines*, comments, white space, or any combination thereof.

2.3.1 Keywords

Syntax:

Syntax

```
keyword: one of
    begin           break           catch       class
    continue        data            define      do
    dynamicparam    else            elseif      end
    exit            filter          finally     for
    foreach         from            function    if
    in              inlinescript    parallel    param
    process         return          switch      throw
    trap            try             until       using
    var             while           workflow
```

Description:

A *keyword* is a sequence of characters that has a special meaning when used in a context-dependent place. Most often, this is as the first token in a *statement*; however, there are other locations, as indicated by the grammar. (A token that looks like a keyword, but is not being used in a keyword context, is a *command-name* or a *command-argument*.)

The keywords `class`, `define`, `from`, `using`, and `var` are reserved for future use.

> ⓘ **Note**
>
> Editor's Note: The `class` and `using` keywords were introduced in PowerShell 5.0. See about_Classes and about_Using.

2.3.2 Variables

Syntax:

Syntax

```
variable:
    $$
    $?
    $^
    $   variable-scope~opt~  variable-characters
    @   variable-scope~opt~  variable-characters
    braced-variable


braced-variable:
    ${   variable-scope~opt~   braced-variable-characters   }

variable-scope:
    Global:
    Local:
    Private:
    Script:
    Using:
    Workflow:
    variable-namespace

variable-namespace:
    variable-characters   :

variable-characters:
    variable-character
    variable-characters   variable-character

variable-character:
    A Unicode character of classes Lu, Ll, Lt, Lm, Lo, or Nd
    _   (The underscore character U+005F)
    ?

braced-variable-characters:
    braced-variable-character
    braced-variable-characters   braced-variable-character

braced-variable-character:
    Any Unicode character except
        }   (The closing curly brace character U+007D)
        `   (The backtick character U+0060)
    escaped-character

escaped-character:
    `   (The backtick character U+0060) followed by any Unicode character
```

Description:

Variables are discussed in detail in (§5). The variable $? is discussed in §2.3.2.2. Scopes are discussed in §3.5.

The variables `$$` and `$^` are reserved for use in an interactive environment, which is outside the scope of this specification.

There are two ways of writing a variable name: A *braced variable name*, which begins with `$`, followed by a curly bracket-delimited set of one or more almost-arbitrary characters; and an *ordinary variable name*, which also begins with `$`, followed by a set of one or more characters from a more restrictive set than a braced variable name allows. Every ordinary variable name can be expressed using a corresponding braced variable name.

PowerShell

```
$totalCost
$Maximum_Count_26

$végösszeg # Hungarian
$итог # Russian
$総計 # Japanese (Kanji)

${Maximum_Count_26}
${Name with`twhite space and `{punctuation`}}
${E:\\File.txt}
```

There is no limit on the length of a variable name, all characters in a variable name are significant, and letter case is *not* distinct.

There are several different kinds of variables: user-defined (§2.3.2.1), automatic (§2.3.2.2), and preference (§2.3.2.3). They can all coexist in the same scope (§3.5).

Consider the following function definition and calls:

PowerShell

```
function Get-Power ([long]$Base, [int]$Exponent) { ... }

Get-Power 5 3 # $Base is 5, $Exponent is 3
Get-Power -Exponent 3 -Base 5 # " " "
```

Each argument is passed by position or name, one at a time. However, a set of arguments can be passed as a group with expansion into individual arguments being handled by the runtime environment. This automatic argument expansion is known as *splatting*. For example,

PowerShell

```
$values = 5,3 # put arguments into an array
Get-Power @values
```

```
$hash = @{ Exponent = 3; Base = 5 } # put arguments into a Hashtable
Get-Power @hash

function Get-Power2 { Get-Power @args } # arguments are in an array

Get-Power2 -Exponent 3 -Base 5 # named arguments splatted named in
@args
Get-Power2 5 3 # position arguments splatted positionally in @args
```

This is achieved by using `@` instead of `$` as the first character of the variable being passed. This notation can only be used in an argument to a command.

Names are partitioned into various namespaces each of which is stored on a virtual drive (§3.1). For example, variables are stored on `Variable:`, environment variables are stored on `Env:`, functions are stored on `Function:`, and aliases are stored on `Alias:`. All of these names can be accessed as variables using the *variable-namespace* production within *variable-scope*. For example,

PowerShell

```
function F { "Hello from F" }
$Function:F # invokes function F

Set-Alias A F
$Alias:A # invokes function F via A

$Count = 10
$Variable:Count # accesses variable Count
$Env:PATH # accesses environment variable PATH
```

Any use of a variable name with an explicit `Variable:` namespace is equivalent to the use of that same variable name without that qualification. For example, `$v` and `$Variable:v` are interchangeable.

As well as being defined in the language, variables can also be defined by the cmdlet New-Variable.

2.3.2.1 User-defined variables

Any variable name allowed by the grammar but not used by automatic or preference variables is available for user-defined variables.

User-defined variables are created and managed by user-defined script.

2.3.2.2 Automatic variables

Automatic variables store state information about the PowerShell environment. Their values can be read in user-written script but not written.

> ⓘ Note
>
> The table originally found in this document was removed to reduce duplication. For a complete list of automatic variables, see about_Automatic_Variables.

2.3.2.3 Preference variables

Preference variables store user preferences for the session. They are created and initialized by the PowerShell runtime environment. Their values can be read and written in user-written script.

> ⓘ Note
>
> The table originally found in this document was removed to reduce duplication. For a complete list of preference variables, see about_Preference_Variables.

2.3.3 Commands

Syntax:

Syntax

```
generic-token:
    generic-token-parts

generic-token-parts:
    generic-token-part
    generic-token-parts generic-token-part

generic-token-part:
    expandable-string-literal
    verbatim-here-string-literal
    variable
    generic-token-char

generic-token-char:
    Any Unicode character except
        {   }   (   )   ;   ,   |   &   $
        ` (The backtick character U+0060)
        double-quote-character
        single-quote-character
```

```
        whitespace
        new-line-character
        escaped-character

generic-token-with-subexpr-start:
    generic-token-parts $(
```

2.3.4 Parameters

Syntax:

Syntax

```
command-parameter:
    dash first-parameter-char parameter-chars colon~opt~

first-parameter-char:
    A Unicode character of classes Lu, Ll, Lt, Lm, or Lo
    _ (The underscore character U+005F)
    ?

parameter-chars:
    parameter-char
    parameter-chars parameter-char

parameter-char:
    Any Unicode character except
        { } ( ) ; , \| & . [
        colon
        whitespace
        new-line-character

colon:
    : (The colon character U+003A)

verbatim-command-argument-chars:
    verbatim-command-argument-part
    verbatim-command-argument-chars verbatim-command-argument-part

verbatim-command-argument-part:
    verbatim-command-string
    & non-ampersand-character
    Any Unicode character except
        |
        new-line-character

non-ampersand-character:
    Any Unicode character except &

verbatim-command-string:
    double-quote-character non-double-quote-chars
    double-quote-character
```

```
non-double-quote-chars:
    non-double-quote-char
    non-double-quote-chars non-double-quote-char

non-double-quote-char:
    Any Unicode character except
        double-quote-character
```

Description:

When a command is invoked, information may be passed to it via one or more *arguments* whose values are accessed from within the command through a set of corresponding *parameters*. The process of matching parameters to arguments is called *parameter binding*.

There are three kinds of argument:

- `[switch]` parameter (§8.10.5) -- This has the form *command-parameter* where *first-parameter-char* and *parameter-chars* together make up the switch name, which corresponds to the name of a parameter (without its leading `-`) in the command being invoked. If the trailing colon is omitted, the presence of this argument indicates that the corresponding parameter be set to `$true`. If the trailing colon is present, the argument immediately following must designate a value of type bool, and the corresponding parameter is set to that value. For example, the following invocations are equivalent:

  PowerShell

  ```
  Set-MyProcess -Strict
  Set-MyProcess -Strict: $true
  ```

- Parameter with argument (§8.10.2) -- This has the form *command-parameter* where *first-parameter-char* and *parameter-chars* together make up the parameter name, which corresponds to the name of a parameter (without its leading -) in the command being invoked. There must be no trailing colon. The argument immediately following designates an associated value. For example, given a command `Get-Power`, which has parameters `$Base` and `$Exponent`, the following invocations are equivalent:

  PowerShell

  ```
  Get-Power -Base 5 -Exponent 3
  Get-Power -Exponent 3 -Base 5
  ```

- Positional argument (§8.10.2) - Arguments and their corresponding parameters inside commands have positions with the first having position zero. The argument in position 0 is bound to the parameter in position 0; the argument in position 1 is bound to the parameter in position 1; and so on. For example, given a command `Get-Power`, that has parameters `$Base` and `$Exponent` in positions 0 and 1, respectively, the following invokes that command:

  PowerShell

  ```
  Get-Power 5 3
  ```

See §8.2 for details of the special parameters `--` and `--%`.

When a command is invoked, a parameter name may be abbreviated; any distinct leading part of the full name may be used, provided that is unambiguous with respect to the names of the other parameters accepted by the same command.

For information about parameter binding see §8.14.

2.3.5 Literals

Syntax:

Syntax

```
literal:
    integer-literal
    real-literal
    string-literal
```

2.3.5.1 Numeric literals

There are two kinds of numeric literals: integer (§2.3.5.1.1) and real (§2.3.5.1.2). Both can have multiplier suffixes (§2.3.5.1.3).

2.3.5.1.1 Integer literals

Syntax:

Syntax

```
integer-literal:
    decimal-integer-literal
```

```
    hexadecimal-integer-literal

decimal-integer-literal:
    decimal-digits numeric-type-suffix~opt~ numeric-multiplier~opt~

decimal-digits:
    decimal-digit
    decimal-digit decimal-digits

decimal-digit: one of
    0  1  2  3  4  5  6  7  8  9

numeric-type-suffix:
    long-type-suffix
    decimal-type-suffix

hexadecimal-integer-literal:
    0x hexadecimal-digits long-type-suffix~opt~
    numeric-multiplier~opt~

hexadecimal-digits:
    hexadecimal-digit
    hexadecimal-digit hexadecimal-digits

hexadecimal-digit: one of
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f

long-type-suffix:
    l

numeric-multiplier: one of
    kb mb gb tb pb
```

Description:

The type of an integer literal is determined by its value, the presence or absence of *long-type-suffix*, and the presence of a *numeric-multiplier* (§2.3.5.1.3).

For an integer literal with no *long-type-suffix*

- If its value can be represented by type int (§4.2.3), that is its type;
- Otherwise, if its value can be represented by type long (§4.2.3), that is its type.
- Otherwise, if its value can be represented by type decimal (§2.3.5.1.2), that is its type.
- Otherwise, it is represented by type double (§2.3.5.1.2).

For an integer literal with *long-type-suffix*

- If its value can be represented by type long (§4.2.3), that is its type;
- Otherwise, that literal is ill formed.

In the twos-complement representation of integer values, there is one more negative value than there is positive. For the int type, that extra value is -2147483648. For the long type, that extra value is -9223372036854775808. Even though the token 2147483648 would ordinarily be treated as a literal of type long, if it is preceded immediately by the unary - operator, that operator and literal are treated as a literal of type int having the smallest value. Similarly, even though the token 9223372036854775808 would ordinarily be treated as a real literal of type decimal, if it is immediately preceded by the unary - operator, that operator and literal are treated as a literal of type long having the smallest value.

Some examples of integer literals are 123 (int), 123L (long), and 200000000000 (long).

There is no such thing as an integer literal of type byte.

2.3.5.1.2 Real literals

Syntax:

Syntax

```
real-literal:
    decimal-digits . decimal-digits exponent-part~opt~ decimal-type-suffix~opt~
numeric-multiplier~opt~
    . decimal-digits exponent-part~opt~ decimal-type-suffix~opt~ numeric-
multiplier~opt~
    decimal-digits exponent-part decimal-type-suffix~opt~ numeric-multiplier~opt~

exponent-part:
    e sign~opt~  decimal-digits

sign: one of
    +
    dash

decimal-type-suffix:
    d
    l

numeric-multiplier: one of
    kb mb gb tb pb

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

A real literal may contain a *numeric-multiplier* (§2.3.5.1.3).

There are two kinds of real literal: *double* and *decimal*. These are indicated by the absence or presence, respectively, of *decimal-type-suffix*. (There is no such thing as a *float real literal*.)

A double real literal has type double (§4.2.4.1). A decimal real literal has type decimal (§4.2.4.2). Trailing zeros in the fraction part of a decimal real literal are significant.

If the value of *exponent-part*'s *decimal-digits* in a double real literal is less than the minimum supported, the value of that double real literal is 0. If the value of *exponent-part*'s *decimal-digits* in a decimal real literal is less than the minimum supported, that literal is ill formed. If the value of *exponent-part*'s *decimal-digits* in a double or decimal real literal is greater than the maximum supported, that literal is ill formed.

Some examples of double real literals are 1., 1.23, .45e35, 32.e+12, and 123.456E-231.

Some examples of decimal real literals are 1d (which has scale 0), 1.20d (which has scale 2), 1.23450e1d (i.e., 12.3450, which has scale 4), 1.2345e3d (i.e., 1234.5, which has scale 1), 1.2345e-1d (i.e., 0.12345, which has scale 5), and 1.2345e-3d (i.e., 0.0012345, which has scale 7).

> ⓘ **Note**
>
> Because a double real literal need not have a fraction or exponent part, the grouping parentheses in (123).M are needed to ensure that the property or method M is being selected for the integer object whose value is 123. Without those parentheses, the real literal would be ill-formed.

> ⓘ **Note**
>
> Although PowerShell does not provide literals for infinities and NaNs, double real literal-like equivalents can be obtained from the static read-only properties PositiveInfinity, NegativeInfinity, and NaN of the types float and double (§4.2.4.1).

The grammar permits what starts out as a double real literal to have an `l` or `L` type suffix. Such a token is really an integer literal whose value is represented by type long.

> ⓘ **Note**

This feature has been retained for backwards compatibility with earlier versions of PowerShell. However, programmers are discouraged from using integer literals of this form as they can easily obscure the literal's actual value. For example, 1.2L has value 1, 1.2345e1L has value 12, and 1.2345e-5L has value 0, none of which is immediately obvious.

2.3.5.1.3 Multiplier suffixes

Syntax:

Syntax

```
numeric-multiplier: *one of*
    kb mb gb tb pb
```

Description:

For convenience, integer and real literals can contain a *numeric-multiplier*, which indicates one of a set of commonly used powers of 10. *numeric-multiplier* can be written in any combination of upper- or lowercase letters.

Expand table

Multiplier	Meaning	Example
kb	kilobyte (1024)	1kb ≡ 1024
mb	megabyte (1024 x 1024)	1.30Dmb ≡ 1363148.80
gb	gigabyte (1024 x 1024 x 1024)	0x10Gb ≡ 17179869184
tb	terabyte (1024 x 1024 x 1024 x 1024)	1.4e23tb ≡ 1.5393162788864E+35
pb	petabyte (1024 x 1024 x 1024 x 1024 x 1024)	0x12Lpb ≡ 20266198323167232

2.3.5.2 String literals

Syntax:

Syntax

```
string-literal:
    expandable-string-literal
    expandable-here-string-literal
    verbatim-string-literal
```

```
    verbatim-here-string-literal

expandable-string-literal:
    double-quote-character expandable-string-characters~opt~  dollars~opt~ double-
quote-character

double-quote-character:
    " (U+0022)
    Left double quotation mark (U+201C)
    Right double quotation mark (U+201D)
    Double low-9 quotation mark (U+201E)

expandable-string-characters:
      expandable-string-part
      expandable-string-characters
      expandable-string-part

expandable-string-part:
    Any Unicode character except
        $
        double-quote-character
        ` (The backtick character U+0060)
    braced-variable
    $ Any Unicode character except
        (
        {
        double-quote-character
        ` (The backtick character U+0060)*
    $ escaped-character
    escaped-character
    double-quote-character double-quote-character

dollars:
    $
    dollars $

expandable-here-string-literal:
    @  double-quote-character  whitespace~opt~  new-line-character
        expandable-here-string-characters~opt~  new-line-character  double-quote-
character  @

expandable-here-string-characters:
    expandable-here-string-part
    expandable-here-string-characters  expandable-here-string-part

expandable-here-string-part:
    Any Unicode character except
        $
        new-line-character
    braced-variable
    $ Any Unicode character except
        (
        new-line-character
    $ new-line-character  Any Unicode character except double-quote-char
    $ new-line-character double-quote-char  Any Unicode character except @
```

```
    new-line-character  Any Unicode character except double-quote-char
    new-line-character double-quote-char  Any Unicode character except @

expandable-string-with-subexpr-start:
    double-quote-character  expandable-string-chars~opt~  $(

expandable-string-with-subexpr-end:
    double-quote-char

expandable-here-string-with-subexpr-start:
    @  double-quote-character  whitespace~opt~  new-line-character  expandable-
here-string-chars~opt~  $(

expandable-here-string-with-subexpr-end:
    new-line-character  double-quote-character  @

verbatim-string-literal:
    single-quote-character verbatim-string-characters~opt~ single-quote-char

single-quote-character:
    ' (U+0027)
    Left single quotation mark (U+2018)
    Right single quotation mark (U+2019)
    Single low-9 quotation mark (U+201A)
    Single high-reversed-9 quotation mark (U+201B)

verbatim-string-characters:
    verbatim-string-part
    verbatim-string-characters verbatim-string-part

verbatim-string-part:
    *Any Unicode character except* single-quote-character
    single-quote-character  single-quote-character

verbatim-here-string-literal:
    @ single-quote-character whitespace~opt~  new-line-character
        verbatim-here-string-characters~opt~  new-line-character
            single-quote-character *@*

verbatim-*here-string-characters:
    verbatim-here-string-part
    verbatim-here-string-characters  verbatim-here-string-part

verbatim-here-string-part:
    Any Unicode character except* new-line-character
    new-line-character  Any Unicode character except single-quote-character
    new-line-character  single-quote-character  Any Unicode character except @
```

Description:

There are four kinds of string literals:

- *verbatim-string-literal* (single-line single-quoted), which is a sequence of zero or more characters delimited by a pair of *single-quote-character*s. Examples are '' and 'red'.
- *expandable-string-literal* (single-line double-quoted), which is a sequence of zero or more characters delimited by a pair of *double-quote-character*s. Examples are "" and "red".
- *verbatim-here-string-literal* (multi-line single-quoted), which is a sequence of zero or more characters delimited by the character pairs @*single-quote-character* and *single-quote-character*@, respectively, all contained on two or more source lines. Examples are:

  PowerShell

  ```
  @'
  '@

  @'
  line 1
  '@

  @'
  line 1
  line 2
  '@
  ```

- *expandable-here-string-literal* (multi-line double-quoted), which is a sequence of zero or more characters delimited by the character pairs @*double-quote-character* and *double-quote-character*@, respectively, all contained on two or more source lines. Examples are:

  PowerShell

  ```
  @"
  "@

  @"
  line 1
  "@

  @"
  line 1
  line 2
  "@
  ```

For *verbatim-here-string-literal*s and *expandable-here-string-literal*s, except for white space (which is ignored) no characters may follow on the same source line as the opening delimiter-character pair, and no characters may precede on the same source line as the closing delimiter character pair.

The *body* of a *verbatim-here-string-literal* or an *expandable-here-string-literal* begins at the start of the first source line following the opening delimiter, and ends at the end of the last source line preceding the closing delimiter. The body may be empty. The line terminator on the last source line preceding the closing delimiter is not part of that literal's body.

A literal of any of these kinds has type string (§4.3.1).

The character used to delimit a *verbatim-string-literal* or *expandable-string-literal* can be contained in such a string literal by writing that character twice, in succession. For example, `'What''s the time?'` and `"I said, ""Hello""."`. However, a *single-quote-character* has no special meaning inside an *expandable-string-literal*, and a *double-quote-character* has no special meaning inside a *verbatim-string-literal*.

An *expandable-string-literal* and an *expandable-here-string-literal* may contain *escaped-characters* (§2.3.7). For example, when the following string literal is written to the pipeline, the result is as shown below:

PowerShell

```
"column1`tcolumn2`nsecond line, `"Hello`", ```Q`5`!"
```

Output

```
column1<horizontal-tab>column2<new-line>
second line, "Hello", `Q5!
```

If an *expandable-string-literal* or *expandable-here-string-literal* contains the name of a variable, unless that name is preceded immediately by an escape character, it is replaced by the string representation of that variable's value (§6.7). This is known as *variable substitution*.

> **Note**
>
> If the variable name is part of some larger expression, only the variable name is replaced. For example, if `$a` is an array containing the elements 100 and 200, `">$a.Length<"` results in `>100 200.Length<` while `">$($a.Length)<"` results in `>2<`. See sub-expression expansion below.

For example, the source code

PowerShell

```
$count = 10
"The value of `$count is $count"
```

results in the *expandable-string-literal*

Output

```
The value of $count is 10.
```

Consider the following:

PowerShell

```
$a = "red","blue"
"`$a[0] is $a[0], `$a[0] is $($a[0])" # second [0] is taken literally
```

The result is

Output

```
$a[0] is red blue[0], $a[0] is red
```

*expandable-string-literal*s and *expandable-here-string-literal*s also support a kind of substitution called *sub-expression expansion*, by treating text of the form `$( ... )` as a *sub-expression* (§7.1.6). Such text is replaced by the string representation of that expression's value (§6.8). Any white space used to separate tokens within *sub-expression*'s *statement-list* is ignored as far as the result string's construction is concerned.

The examples,

PowerShell

```
$count = 10
"$count + 5 is $($count + 5)"
"$count + 5 is `$($count + 5)"
"$count + 5 is `$(`$count + 5)"
```

result in the following *expandable-string-literal*s:

Output

```
10 + 5 is 15
10 + 5 is $(10 + 5)
```

```
10 + 5 is $($count + 5)
```

The following source,

PowerShell

```
$i = 5; $j = 10; $k = 15
"`$i, `$j, and `$k have the values $( $i; $j; $k )"
```

results in the following *expandable-string-literal*:

Output

```
$i, $j, and $k have the values 5 10 15
```

These four lines could have been written more succinctly as follows:

PowerShell

```
"`$i, `$j, and `$k have the values $(($i = 5); ($j = 10); ($k = 15))"
```

In the following example,

PowerShell

```
"First 10 squares: $(for ($i = 1; $i -le 10; ++$i) { "$i $($i*$i) " })"
```

the resulting *expandable-string-literal* is as follows:

Output

```
First 10 squares: 1 1 2 4 3 9 4 16 5 25 6 36 7 49 8 64 9 81 10 100
```

As shown, a *sub-expression* can contain string literals having both variable substitution and sub-expression expansion. Note also that the inner *expandable-string-literal*'s delimiters need not be escaped; the fact that they are inside a *sub-expression* means they cannot be terminators for the outer *expandable-string-literal*.

An *expandable-string-literal* or *expandable-here-string-literal* containing a variable substitution or sub-expression expansion is evaluated each time that literal is used; for example,

PowerShell

```
$a = 10
$s1 = "`$a = $($a; ++$a)"
"`$s1 = >$s1<"
$s2 = "`$a = $($a; ++$a)"
"`$s2 = >$s2<"
$s2 = $s1
"`$s2 = >$s2<"
```

which results in the following *expandable-string-literal*s:

Output

```
$s1 = >$a = 10<
$s2 = >$a = 11<
$s2 = >$a = 10<
```

The contents of a *verbatim-here-string-literal* are taken verbatim, including any leading or trailing white space within the body. As such, embedded *single-quote-character*s need not be doubled-up, and there is no substitution or expansion. For example,

PowerShell

```
$lit = @'
That's it!
2 * 3 = $(2*3)
'@
```

which results in the literal

Output

```
That's it!
2 * 3 = $(2*3)
```

The contents of an *expandable-here-string-literal* are subject to substitution and expansion, but any leading or trailing white space within the body but outside any *sub-expression*s is taken verbatim, and embedded *double-quote-character*s need not be doubled-up. For example,

PowerShell

```
$lit = @"
That's it!
2 * 3 = $(2*3)
"@
```

which results in the following literal when expanded:

```
PowerShell
That's it!
2 * 3 = 6
```

For both *verbatim-here-string-literal*s and *expandable-here-string-literal*s, each line terminator within the body is represented in the resulting literal in an implementation-defined manner. For example, in

```
PowerShell
$lit = @"
abc
xyz
"@
```

the second line of the body has two leading spaces, and the first and second lines of the body have line terminators; however, the terminator for the second line of the body is *not* part of that body. The resulting literal is equivalent to: `"abc<implementation-defined character sequence>xyz"`.

> ⓘ Note
>
> To aid readability of source, long string literals can be broken across multiple source lines without line terminators being inserted. This is done by writing each part as a separate literal and concatenating the parts with the + operator (§7.7.2). This operator allows its operands to designate any of the four kinds of string literal.

> ⓘ Note
>
> Although there is no such thing as a character literal per se, the same effect can be achieved by accessing the first character in a 1-character string, as follows: `[char]"A"` or `"A"[0]`.

For both *verbatim-here-string-literal*s and *expandable-here-string-literal*s, each line terminator within the body is represented exactly as it was provided.

2.3.5.3 Null literal

See the automatic variable `$null` (§2.3.2.2).

2.3.5.4 Boolean literals

See the automatic variables `$false` and `$true` (§2.3.2.2).

2.3.5.5 Array literals

PowerShell allows expressions of array type (§9) to be written using the unary comma operator (§7.2.1), *array-expression* (§7.1.7), the binary comma operator (§7.3), and the range operator (§7.4).

2.3.5.6 Hash literals

PowerShell allows expressions of type Hashtable (§10) to be written using a *hash-literal-expression* (§7.1.9)

2.3.5.7 Type names

Syntax:

Syntax

```
type-name:
    type-identifier
    type-name . type-identifier

type-identifier:
    type-characters

type-characters:
    type-character
    type-characters type-character

type-character:
    A Unicode character of classes Lu, Ll, Lt, Lm, Lo, or Nd
    _ (The underscore character U+005F)

array-type-name:
    type-name [

generic-type-name:
    type-name [
```

2.3.6 Operators and punctuators

Syntax:

Syntax

```
operator-or-punctuator: one of
    {   }   [   ]   (   )   @(   @{   $(   ;
    &&  ||  &   |   ,   ++  ..   ::   .
    !   *   /   %   +   -   --
    -and   -band   -bnot   -bor
    -bxor   -not   -or     -xor
    assignment-operator
    merging-redirection-operator
    file-redirection-operator
    comparison-operator
    format-operator

assignment-operator: one of
    =  -=  +=  *=  /=  %=

file-redirection-operator: one of
    >  >>  2>  2>>  3>  3>>  4>  4>>
    5>  5>>  6>  6>>  *>  *>>  <

merging-redirection-operator: one of
    *>&1  2>&1  3>&1  4>&1  5>&1  6>&1
    *>&2  1>&2  3>&2  4>&2  5>&2  6>&2

comparison-operator: *one of
    -as           -ccontains      -ceq
    -cge          -cgt            -cle
    -clike        -clt            -cmatch
    -cne          -cnotcontains   -cnotlike
    -cnotmatch    -contains       -creplace
    -csplit       -eq             -ge
    -gt           -icontains      -ieq
    -ige          -igt            -ile
    -ilike        -ilt            -imatch
    -in           -ine            -inotcontains
    -inotlike     -inotmatch      -ireplace
    -is           -isnot          -isplit
    -join         -le             -like
    -lt           -match          -ne
    -notcontains  -notin         -notlike
    -notmatch     -replace       -shl*
    -shr          -split

format-operator:
    -f
```

Description:

`&&` and `||` are reserved for future use.

> ⓘ **Note**

Editor's Note: The pipeline chain operators `&&` and `||` were introduced in PowerShell 7. See about_Pipeline_Chain_Operators.

The name following *dash* in an operator is reserved for that purpose only in an operator context.

An operator that begins with *dash* must not have any white space between that *dash* and the token that follows it.

2.3.7 Escaped characters

Syntax:

Syntax

```
escaped-character:
    ` (The backtick character U+0060) followed by any Unicode character
```

Description:

An *escaped character* is a way to assign a special interpretation to a character by giving it a prefix Backtick character (U+0060). The following table shows the meaning of each *escaped-character*:

Expand table

Escaped Character	Meaning
`` `a ``	Alert (U+0007)
`` `b ``	Backspace (U+0008)
`` `f ``	Form-feed (U+000C)
`` `n ``	New-line (U+000A)
`` `r ``	Carriage return (U+000D)
`` `t ``	Horizontal tab (U+0009)
`` `v ``	Vertical tab (U+0009)
`` `' ``	Single quote (U+0027)

Escaped Character	Meaning
`` `" ``	Double quote (U+0022)
`` `` ``	Backtick (U+0060)
`` `0 ``	NUL (U+0000)
`` `x ``	If `x` is a character other than those characters shown above, the backtick character is ignored and `x` is taken literally.

The implication of the final entry in the table above is that spaces that would otherwise separate tokens can be made part of a token instead. For example, a file name containing a space can be written as `Test` Data.txt` (as well as `'Test Data.txt'` or `"Test Data.txt"`).

Last updated on 04/08/2026

3. Basic concepts

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

3.1 Providers and drives

A *provider* allows access to data and components that would not otherwise be easily accessible at the command line. The data is presented in a consistent format that resembles a file system drive.

The data that a provider exposes appears on a *drive*, and the data is accessed via a *path* just like with a disk drive. Built-in cmdlets for each provider manage the data on the provider drive.

PowerShell includes the following set of built-in providers to access the different types of data stores:

Expand table

Provider	Drive Name	Description	Ref.
Alias	Alias:	PowerShell aliases	§3.1.1
Environment	Env:	Environment variables	§3.1.2
FileSystem	A:, B:, C:, ...	Disk drives, directories, and files	§3.1.3
Function	Function:	PowerShell functions	§3.1.4
Variable	Variable:	PowerShell variables	§3.1.5

Windows PowerShell:

Expand table

Provider	Drive Name	Description
Certificate	Cert:	x509 certificates for digital signatures
Registry	HKLM: (HKEY_LOCAL_MACHINE), HKCU: (HKEY_CURRENT_USER)	Windows registry
WSMan	WSMan:	WS-Management configuration information

The following cmdlets deal with providers and drives:

- Get-PSProvider: Gets information about one or more providers
- Get-PSDrive: Gets information about one or more drives

The type of an object that represents a provider is described in §4.5.1. The type of an object that represents a drive is described in §4.5.2.

3.1.1 Aliases

An *alias* is an alternate name for a command. A command can have multiple aliases, and the original name and all of its aliases can be used interchangeably. An alias can be reassigned. An alias is an item (§3.3).

An alias can be assigned to another alias; however, the new alias is not an alias of the original command.

The provider Alias is a flat namespace that contains only objects that represent the aliases. The variables have no child items.

PowerShell comes with a set of built-in aliases.

The following cmdlets deal with aliases:

- New-Alias: Creates an alias
- Set-Alias: Creates or changes one or more aliases
- Get-Alias: Gets information about one or more aliases
- Export-Alias: Exports one or more aliases to a file

When an alias is created for a command using `New-Alias`, parameters to that command cannot be included in that alias. However, direct assignment to a variable in the Alias: namespace does

permit parameters to be included.

> ⓘ **Note**
>
> It is a simple matter, however, to create a function that does nothing more than contain the invocation of that command with all desired parameters, and to assign an alias to that function.

The type of an object that represents an alias is described in §4.5.4.

Alias objects are stored on the drive Alias: (§3.1).

3.1.2 Environment variables

The PowerShell Environment provider allows operating system environment variables to be retrieved, added, changed, cleared, and deleted.

The provider Environment is a flat namespace that contains only objects that represent the environment variables. The variables have no child items.

An environment variable's name cannot include the equal sign (`=`).

Changes to the environment variables affect the current session only.

An environment variable is an item (§3.3).

The type of an object that represents an environment variable is described in §4.5.6.

Environment variable objects are stored on the drive Env: (§3.1).

3.1.3 File system

The PowerShell FileSystem provider allows directories and files to be created, opened, changed, and deleted.

The FileSystem provider is a hierarchical namespace that contains objects that represent the underlying file system.

Files are stored on drives with names like A:, B:, C:, and so on (§3.1). Directories and files are accessed using path notation (§3.4).

A directory or file is an item (§3.3).

3.1.4 Functions

The PowerShell Function provider allows functions (§8.10) and filters (§8.10.1) to be retrieved, added, changed, cleared, and deleted.

The provider Function is a flat namespace that contains only the function and filter objects. Neither functions nor filters have child items.

Changes to the functions affect the current session only.

A function is an item (§3.3).

The type of an object that represents a function is described in §4.5.10. The type of an object that represents a filter is described in §4.5.11.

Function objects are stored on drive Function: (§3.1).

3.1.5 Variables

Variables can be defined and manipulated directly in the PowerShell language.

The provider Variable is a flat namespace that contains only objects that represent the variables. The variables have no child items.

The following cmdlets also deal with variables:

- New-Variable: Creates a variable
- Set-Variable: Creates or changes the characteristics of one or more variables
- Get-Variable: Gets information about one or more variables
- Clear-Variable: Deletes the value of one or more variables
- Remove-Variable: Deletes one or more variables

As a variable is an item (§3.3), it can be manipulated by most Item-related cmdlets.

The type of an object that represents a variable is described in §4.5.3.

Variable objects are stored on drive Variable: (§3.1).

3.2 Working locations

The *current working location* is the default location to which commands point. This is the location used if an explicit path (§3.4) is not supplied when a command is invoked. This location includes the *current drive*.

A PowerShell host may have multiple drives, in which case, each drive has its own current location.

When a drive name is specified without a directory, the current location for that drive is implied.

The current working location can be saved on a stack, and then set to a new location. Later, that saved location can be restored from that stack and made the current working location. There are two kinds of location stacks: the *default working location stack*, and zero or more user-defined *named working location stacks*. When a session begins, the default working location stack is also the *current working location stack*. However, any named working location stack can be made the current working location stack.

The following cmdlets deal with locations:

- Set-Location: Establishes the current working location
- Get-Location: Determines the current working location for the specified drive(s), or the working locations for the specified stack(s)
- Push-Location: Saves the current working location on the top of a specified stack of locations
- Pop-Location: Restores the current working location from the top of a specified stack of locations

The object types that represents a working location and a stack of working locations are described in §4.5.5.

3.3 Items

An *item* is an alias (§3.1.1), a variable (§3.1.5), a function (§3.1.4), an environment variable (§3.1.2), or a file or directory in a file system (§3.1.3).

The following cmdlets deal with items:

- New-Item: Creates a new item
- Set-Item: Changes the value of one or more items
- Get-Item: Gets the items at the specified location
- Get-ChildItem: Gets the items and child items at the specified location
- Copy-Item: Copies one or more items from one location to another
- Move-Item: Moves one or more items from one location to another
- Rename-Item: Renames an item
- Invoke-Item: Performs the default action on one or more items
- Clear-Item: Deletes the contents of one or more items, but does not delete the items (see
- Remove-Item: Deletes the specified items

The following cmdlets deal with the content of items:

- Get-Content: Gets the content of the item
- Add-Content: Adds content to the specified items
- Set-Content: Writes or replaces the content in an item
- Clear-Content: Deletes the contents of an item

The type of an object that represents a directory is described in §4.5.17. The type of an object that represents a file is described in §4.5.18.

3.4 Path names

All items in a data store accessible through a PowerShell provider can be identified uniquely by their path names. A *path name* is a combination of the item name, the container and subcontainers in which the item is located, and the PowerShell drive through which the containers are accessed.

Path names are divided into one of two types: fully qualified and relative. A *fully qualified path name* consists of all elements that make up a path. The following syntax shows the elements in a fully qualified path name:

> **Tip**
>
> The `~opt~` notation in the syntax definitions indicates that the lexical entity is optional in the syntax.

Syntax

```
path:
    provider~opt~   drive~opt~   containers~opt~   item

provider:
    module~opt~   provider   ::

module:
    module-name   \

drive:
    drive-name   :

containers:
    container   \
    containers container   \
```

module-name refers to the parent module.

provider refers to the PowerShell provider through which the data store is accessed.

drive refers to the PowerShell drive that is supported by a particular PowerShell provider.

A *container* can contain other containers, which can contain other containers, and so on, with the final container holding an *item*. Containers must be specified in the hierarchical order in which they exist in the data store.

Here is an example of a path name:

```
E:\Accounting\InvoiceSystem\Production\MasterAccount\MasterFile.dat
```

If the final element in a path contains other elements, it is a *container element*; otherwise, it's a *leaf element*.

In some cases, a fully qualified path name is not needed; a relative path name will suffice. A *relative path name* is based on the current working location. PowerShell allows an item to be identified based on its location relative to the current working location. A relative path name involves the use of some special characters. The following table describes each of these characters and provides examples of relative path names and fully qualified path names. The examples in the table are based on the current working directory being set to C:\Windows:

Expand table

Symbol	Description	Relative path	Fully qualified path
`.`	Current working location	`.\System`	`C:\Windows\System`
`..`	Parent of the current working location	`..\Program Files`	`C:\Program Files`
`\`	Drive root of the current working location	`\Program Files`	`C:\Program Files`
none	No special characters	`System`	`C:\Windows\System`

To use a path name in a command, enter that name as a fully qualified or relative path name.

The following cmdlets deal with paths:

- Convert-Path: Converts a path from a PowerShell path to a PowerShell provider path
- Join-Path: Combines a path and a child path into a single path
- Resolve-Path: Resolves the wildcard characters in a path
- Split-Path: Returns the specified part of a path
- Test-Path: Determines whether the elements of a path exist or if a path is well formed

Some cmdlets (such as Add-Content and `Copy-Item` use file filters. A *file filter* is a mechanism for specifying the criteria for selecting from a set of paths.

The object type that represents a resolved path is described in §4.5.5. Paths are often manipulated as strings.

3.5 Scopes

3.5.1 Introduction

A name can denote a variable, a function, an alias, an environment variable, or a drive. The same name may denote different items at different places in a script. For each different item that a name denotes, that name is visible only within the region of script text called its *scope*. Different items denoted by the same name either have different scopes, or are in different name spaces.

Scopes may nest, in which case, an outer scope is referred to as a *parent scope,* and any nested scopes are *child scopes* of that parent. The scope of a name is the scope in which it is defined and all child scopes, unless it is made private. Within a child scope, a name defined there hides any items defined with the same name in parent scopes.

Unless dot source notation (§3.5.5) is used, each of the following creates a new scope:

- A script file
- A script block
- A function or filter

Consider the following example:

PowerShell

```
# Start of script
$x = 2; $y = 3
Get-Power $x $y

# Function defined in script
function Get-Power([int]$x, [int]$y) {
    if ($y -gt 0) {
        return $x * (Get-Power $x (--$y))
    } else {
        return 1
    }
}
# End of script
```

The scope of the variables `$x` and `$y` created in the script is the body of that script, including the function defined inside it. Function `Get-Power` defines two parameters with those same names. As each function has its own scope, these variables are different from those defined in

the parent scope, and they hide those from the parent scope. The function scope is nested inside the script scope.

Note that the function calls itself recursively. Each time it does so, it creates yet another nested scope, each with its own variables `$x` and `$y`.

Here is a more complex example, which also shows nested scopes and reuse of names:

PowerShell

```
# start of script scope
$x = 2              # top-level script-scope $x created
                    # $x is 2
F1                  # create nested scope with call to function F1
                    # $x is 2
F3                  # create nested scope with call to function F3
                    # $x is 2

function F1 {       # start of function scope
                    # $x is 2
    $x = $true      # function-scope $x created
                    # $x is $true

    & {             # create nested scope with script block
                    # $x is $true
        $x = 12.345 # scriptblock-scope $x created
                    # $x is 12.345
    }               # end of scriptblock scope, local $x goes away

                    # $x is $true
    F2              # create nested scope with call to function F2
                    # $x is $true
}                   # end of function scope, local $x goes away

function F2 {       # start of function scope
                    # $x is $true
    $x = "red"      # function-scope $x created
                    # $x is "red"
}                   # end of function scope, local $x goes away

function F3 {       # start of function scope
                    # $x is 2
    if ($x -gt 0) {
                    # $x is 2
        $x = "green"
                    # $x is "green"
    }               # end of block, but not end of any scope
                    # $x is still "green"
}                   # end of function scope, local $x goes away
# end of script scope
```

3.5.2 Scope names and numbers

PowerShell supports the following scopes:

- Global: This is the top-most level scope. All automatic and preference variables are defined in this scope. The global scope is the parent scope of all other scopes, and all other scopes are child scopes of the global scope.
- Local: This is the current scope at any execution point within a script, script block, or function. Any scope can be the local scope.
- Script: This scope exists for each script file that is executed. The script scope is the parent scope of all scopes created from within it. A script block does *not* have its own script scope; instead, its script scope is that of its nearest ancestor script file. Although there is no such thing as module scope, script scope provides the equivalent.

Names can be declared private, in which case, they are not visible outside of their parent scope, not even to child scopes. The concept of private is not a separate scope; it's an alias for local scope with the addition of hiding the name if used as a writable location.

Scopes can be referred to by a number, which describes the relative position of one scope to another. Scope 0 denotes the local scope, scope 1 denotes a 1-generation ancestor scope, scope 2 denotes a 2-generation ancestor scope, and so on. (Scope numbers are used by cmdlets that manipulate variables.)

3.5.3 Variable name scope

As shown by the following production, a variable name can be specified with any one of six different scopes:

Syntax

```
variable-scope:
    Global:
    Local:
    Private:
    Script:
    Using:
    Workflow:
    variable-namespace
```

The scope is optional. The following table shows the meaning of each in all possible contexts. It also shows the scope when no scope is specified explicitly:

Expand table

Scope Modifier	Within a Script File	Within a Script Block	Within a Function
Global	Global scope	Global scope	Global scope
Script	Nearest ancestor script file's scope or Global if there is no nearest ancestor script file	Nearest ancestor script file's scope or Global if there is no nearest ancestor script file	Nearest ancestor script file's scope or Global if there is no nearest ancestor script file
Private	Global/Script/Local scope	Local scope	Local scope
Local	Global/Script/Local scope	Local scope	Local scope
Using	Implementation defined	Implementation defined	Implementation defined
Workflow	Implementation defined	Implementation defined	Implementation defined
None	Global/Script/Local scope	Local scope	Local scope

Variable scope information can also be specified when using the family of cmdlets listed in (§3.1.5). In particular, refer to the parameter `Scope`, and the parameters `Option Private` and `Option AllScope` for more information.

The `Using:` scope modifier is used to access variables defined in another scope while running scripts via cmdlets like `Start-Job`, `Invoke-Command`, or within an *inlinescript-statement*. For example:

PowerShell

```
$a = 42
Invoke-Command --ComputerName RemoteServer { $Using:a } # returns 42
workflow foo
{
    $b = "Hello"
    inlinescript { $Using:b }
}
foo # returns "Hello"
```

The scope workflow is used with a *parallel-statement* or *sequence-statement* to access a variable defined in the workflow.

3.5.4 Function name scope

A function name may also have one of the four different scopes, and the visibility of that name is the same as for variables (§3.5.3).

3.5.5 Dot source notation

When a script file, script block, or function is executed from within another script file, script block, or function, the executed script file creates a new nested scope. For example,

PowerShell

```
Script1.ps1
& "Script1.ps1"
& { ... }
FunctionA
```

However, when *dot source notation* is used, no new scope is created before the command is executed, so additions/changes it would have made to its own local scope are made to the current scope instead. For example,

PowerShell

```
. Script2.ps1
. "Script2.ps1"
. { ... }
. FunctionA
```

3.5.6 Modules

Just like a top-level script file is at the root of a hierarchical nested scope tree, so too is each module (§3.14). However, by default, only those names exported by a module are available by name from within the importing context. The Global parameter of the cmdlet Import-Module allows exported names to have increased visibility.

3.6 ReadOnly and Constant Properties

Variables and aliases are described by objects that contain a number of properties. These properties are set and manipulated by two families of cmdlets (§3.1.5, §3.1.1). One such property is Options, which can be set to ReadOnly or Constant (using the Option parameter). A variable or alias marked ReadOnly can be removed, and its properties can changed provided the Force parameter is specified. However, a variable or alias marked Constant cannot be removed nor have its properties changed.

3.7 Method overloads and call resolution

3.7.1 Introduction

As stated in §1, an external procedure made available by the execution environment (and written in some language other than PowerShell) is called a *method*.

The name of a method along with the number and types of its parameters are collectively called that method's *signature*. (Note that the signature does not include the method's return type.) The execution environment may allow a type to have multiple methods with the same name provided each has a different signature. When multiple versions of some method are defined, that method is said to be *overloaded*. For example, the type Math (§4.3.8) contains a set of methods called `Abs`, which computes the absolute value of a specified number, where the specified number can have one of a number of types. The methods in that set have the following signatures:

PowerShell

```
Abs(decimal)
Abs(float)
Abs(double)
Abs(int)
Abs(long)
Abs(SByte)
Abs(Int16)
```

In this case, all of the methods have the same number of arguments; their signatures differ by argument type only.

Another example involves the type Array (§4.3.2), which contains a set of methods called Copy that copies a range of elements from one array to another, starting at the beginning of each array (by default) or at some designated element. The methods in that set have the following signatures:

PowerShell

```
Copy(Array, Array, int)
Copy(Array, Array, long)
Copy(Array, int, Array, int, int)
Copy(Array, long, Array, long, long)
```

In this case, the signatures differ by argument type and, in some cases, by argument number as well.

In most calls to overloaded methods, the number and type of the arguments passed exactly match one of the overloads, and the method selected is obvious. However, if that is not the case, there needs to be a way to resolve which overloaded version to call, if any. For example,

PowerShell

```
[Math]::Abs([byte]10) # no overload takes type byte
[array]::Copy($source, 3, $dest, 5L, 4) # both int and long indexes
```

Other examples include the type **string** (i.e.; **System.String**), which has numerous overloaded methods.

Although PowerShell has rules for resolving method calls that do not match an overloaded signature exactly, PowerShell does not itself provide a way to define overloaded methods.

> ⓘ **Note**
>
> Editor's Note: PowerShell 5.0 added the ability to define script-based classes. These classes can contain overloaded methods.

3.7.2 Method overload resolution

Given a method call (§7.1.3) having a list of argument expressions, and a set of *candidate methods* (i.e., those methods that could be called), the mechanism for selecting the *best method* is called *overload resolution*.

Given the set of applicable candidate methods (§3.7.3), the best method in that set is selected. If the set contains only one method, then that method is the best method. Otherwise, the best method is the one method that is better than all other methods with respect to the given argument list using the rules shown in §3.7.4. If there is not exactly one method that is better than all other methods, then the method invocation is ambiguous and an error is reported.

The best method must be accessible in the context in which it is called. For example, a PowerShell script cannot call a method that is private or protected.

The best method for a call to a static method must be a static method, and the best method for a call to an instance method must be an instance method.

3.7.3 Applicable method

A method is said to be *applicable* with respect to an argument list A when one of the following is true:

- The number of arguments in A is identical to the number of parameters that the method accepts.
- The method has M required parameters and N optional parameters, and the number of arguments in A is greater than or equal to M, but less than N.
- The method accepts a variable number of arguments and the number of arguments in A is greater than the number of parameters that the method accepts.

In addition to having an appropriate number of arguments, each argument in A must match the parameter-passing mode of the argument, and the argument type must match the parameter type, or there must be a conversion from the argument type to the parameter type.

If the argument type is ref (§4.3.6), the corresponding parameter must also be ref, and the argument type for conversion purposes is the type of the property Value from the ref argument.

If the argument type is `ref`, the corresponding parameter could be `out` instead of `ref`.

If the method accepts a variable number of arguments, the method may be applicable in either *normal form* or *expanded form*. If the number of arguments in A is identical to the number of parameters that the method accepts and the last parameter is an array, then the form depends on the rank of one of two possible conversions:

- The rank of the conversion from the type of the last argument in A to the array type for the last parameter.
- The rank of the conversion from the type of the last argument in A to the element type of the array type for the last parameter.

If the first conversion (to the array type) is better than the second conversion (to the element type of the array), then the method is applicable in normal form, otherwise it is applicable in expanded form.

If there are more arguments than parameters, the method may be applicable in expanded form only. To be applicable in expanded form, the last parameter must have array type. The method is replaced with an equivalent method that has the last parameter replaced with sufficient parameters to account for each unmatched argument in A. Each additional parameter type is the element type of the array type for the last parameter in the original method. The above rules for an applicable method are applied to this new method and argument list A.

3.7.4 Better method

Given an argument list A with a set of argument expressions `{ E~1~, E~2~, ..., E~N~ }` and two application methods `M~P~` and `M~Q~` with parameter types `{ P~1~, P~2~, ..., P~N~ }` and

`{ Q~1~, Q~2~, ..., Q~N~ }`, `M~P~` is defined to be a better method than `M~Q~` if the *cumulative ranking of conversions* for `M~P~` is better than that for `M~Q~`.

The cumulative ranking of conversions is calculated as follows. Each conversion is worth a different value depending on the number of parameters, with the conversion of `E~1~` worth N, `E~2~` worth N-1, down to `E~N~` worth 1. If the conversion from `E~X~` to `P~X~` is better than that from `E~X~` to `Q~X~`, the `M~P~` accumulates N-X+1; otherwise, `M~Q~` accumulates N-X+1. If `M~P~` and `M~Q~` have the same value, then the following tie breaking rules are used, applied in order:

- The cumulative ranking of conversions between parameter types (ignoring argument types) is computed in a manner similar to the previous ranking, so `P~1~` is compared against `Q~1~`, `P~2~` against `Q~2~`, ..., and `P~N~` against `Q~N~`. The comparison is skipped if the argument was `$null`, or if the parameter types are not numeric types. The comparison is also skipped if the argument conversion `E~X~` loses information when converted to `P~X~` but does not lose information when converted to `Q~X~`, or vice versa. If the parameter conversion types are compared, then if the conversion from `P~X~` to `Q~X~` is better than that from `Q~X~` to `P~X~`, the `M~P~` accumulates N-X+1; otherwise, `M~Q~` accumulates N-X+1. This tie breaking rule is intended to prefer the *most specific method* (i.e., the method with parameters having the smallest data types) if no information is lost in conversions, or to prefer the *most general method* (i.e., the method with the parameters with the largest data types) if conversions result in loss of information.
- If both methods use their expanded form, the method with more parameters is the better method.
- If one method uses the expanded form and the other uses normal form, the method using normal form is the better method.

3.7.5 Better conversion

The text below marked like this is specific to Windows PowerShell.

Conversions are ranked in the following manner, from lowest to highest:

- `T~1~[]` to `T~2~[]` where no assignable conversion between `T~1~` and `T~2~` exists
- T to string where T is any type
- `T~1~` to `T~2~` where `T~1~` or `T~2~` define a custom conversion in an implementation-defined manner
- `T~1~` to `T~2~` where `T~1~` implements IConvertible
- `T~1~` to `T~2~` where `T~1~` or `T~2~` implements the method `T~2~ op_Implicit(T1)`
- `T~1~` to `T~2~` where `T~1~` or `T~2~` implements the method `T~2~ op_Explicit(T1)`

- `T~1~` to `T~2~` where `T~2~` implements a constructor taking a single argument of type `T~1~`
- Either of the following conversions:
  - string to `T` where `T` implements a static method `T Parse(string)` or `T Parse(string, IFormatProvider)`
  - `T~1~` to `T~2~` where `T~2~` is any enum and `T~1~` is either string or a collection of objects that can be converted to string
- `T` to PSObject where `T` is any type
- Any of the following conversions: `Language`
  - T to bool where `T` is any numeric type
  - string to `T` where `T` is `regex`, `wmisearcher`, `wmi`, `wmiclass`, `adsi`, `adsisearcher`, or `type`
  - `T` to `bool`
  - T~1~ to `Nullable[T~2~]` where a conversion from `T~1~` to `T~2~` exists
  - `T` to void
  - `T~1~[]` to `T~2~[]` where an assignable conversion between `T~1~` and `T~2~` exists
  - `T~1~` to `T~2~[]` where `T~1~` is a collection
  - `IDictionary` to `Hashtable`
  - `T` to `ref`
  - `T` to `xml`
  - `scriptblock` to `delegate`
  - `T~1~` to `T~2~` where `T~1~` is an integer type and `T~2~` is an enum
- `$null` to `T` where `T` is any value type
- `$null` to `T` where `T` is any reference type
- Any of the following conversions:
  - byte to `T` where `T` is `SByte`
  - `UInt16` to `T` where `T` is `SByte`, `byte`, or `Int16`
  - `Int16` to `T` where `T` is `SByte` or `byte`
  - `UInt32` to `T` where `T` is `SByte`, `byte`, `Int16`, `UInt16`, or `int`
  - `int` to `T` where `T` is `SByte`, `byte`, `Int16`, or `UInt16`
  - `UInt64` to `T` where `T` is `SByte`, `byte`, `Int16`, `UInt16`, `int`, `UInt32`, or `long`
  - `long` to `T` where `T` is `SByte`, `byte`, `Int16`, `UInt16`, `int`, or `UInt32`
  - `float` to `T` where `T` is any integer type or `decimal`

    - `double` to `T` where `T` is any integer type or `decimal`
    - `decimal` to `T` where `T` is any integer type
- Any of the following conversions:
    - `SByte` to `T` where `T` is `byte`, `uint6`, `UInt32`, or `UInt64`
    - `Int16` to `T` where `T` is `UInt16`, `UInt32`, or `UInt64`
    - `int` to `T` where `T` is `UInt32` or `UInt64`
    - `long` to `UInt64`
    - `decimal` to `T` where `T` is `float` or `double`
- Any of the following conversions:
    - `T` to `string` where `T` is any numeric type
    - `T` to `char` where `T` is any numeric type
    - `string` to `T` where `T` is any numeric type
- Any of the following conversions, these conversion are considered an assignable conversions:
    - `byte` to `T` where `T` is `Int16`, `UInt16`, `int`, `UInt32`, `long`, `UInt64`, `single`, `double`, or `decimal`
    - `SByte` to `T` where `T` is `Int16`, `UInt16`, `int`, `UInt32`, `long`, `UInt64`, `single`, `double`, or `decimal`
    - `UInt16` to `T` where `T` is `int`, `UInt32`, `long`, or `UInt64`, `single`, `double`, or `decimal`
    - `Int16` to `T` where `T` is `int`, `UInt32`, `long`, or `UInt64`, `single`, `double`, or `decimal`
    - `UInt32` to `T` where `T` is `long`, or `UInt64`, `single`, `double`, or `decimal`
    - `int` to `T` where `T` is `long`, `UInt64`, `single`, `double`, or `decimal`
    - `single` to `double`
- `T~1~` to `T~2~` where `T~2~` is a base class or interface of `T~1~`. This conversion is considered an assignable conversion.
- `string` to `char[]`
- `T` to `T` -- This conversion is considered an assignable conversion.

For each conversion of the form `T~1~` to `T~2~[]` where `T~1~` is not an array and no other conversion applies, if there is a conversion from `T~1~` to `T~2~`, the rank of the conversion is worse than the conversion from `T~1~` to `T~2~`, but better than any conversion ranked less than the conversion from `T~1~` to `T~2~`

3.8 Name lookup

It is possible to have commands of different kinds all having the same name. The order in which name lookup is performed in such a case is alias, function, cmdlet, and external command.

3.9 Type name lookup

§7.1.10 contains the statement, "A *type-literal* is represented in an implementation by some unspecified *underlying type*. As a result, a type name is a synonym for its underlying type." Example of types are `int`, `double`, `long[]`, and `Hashtable`.

Type names are matched as follows: Compare a given type name with the list of built-in *type accelerators*, such as int, long, double. If a match is found, that is the type. Otherwise, presume the type name is fully qualified and see if such a type exists on the host system. If a match is found, that is the type. Otherwise, add the namespace prefix `System.`. If a match is found, that is the type. Otherwise, the type name is in error. This algorithm is applied for each type argument for generic types. However, there is no need to specify the arity (the number of arguments or operands taken by a function or operator).

3.10 Automatic memory management

Various operators and cmdlets result in the allocation of memory for reference-type objects, such as strings and arrays. The allocation and freeing of this memory is managed by the PowerShell runtime system. That is, PowerShell provides automatic *garbage collection*.

3.11 Execution order

A *side effect* is a change in the state of a command's execution environment. A change to the value of a variable (via the assignment operators or the pre- and post-increment and decrement operators) is a side effect, as is a change to the contents of a file.

Unless specified otherwise, statements are executed in lexical order.

Except as specified for some operators, the order of evaluation of terms in an expression and the order in which side effects take place are both unspecified.

An expression that invokes a command involves the expression that designates the command, and zero or more expressions that designate the arguments whose values are to be passed to that command. The order in which these expressions are evaluated relative to each other is unspecified.

3.12 Error handling

When a command fails, this is considered an *error*, and information about that error is recorded in an *error record*, whose type is unspecified (§4.5.15); however, this type supports subscripting.

An error falls into one of two categories. Either it terminates the operation (a *terminating error*) or it doesn't (a *non-terminating error*). With a terminating error, the error is recorded and the operation stops. With a non-terminating error, the error is recorded and the operation continues.

Non-terminating errors are written to the error stream. Although that information can be redirected to a file, the error objects are first converted to strings and important information in those objects would not be captured making diagnosis difficult if not impossible. Instead, the error text can be redirected (§7.12) and the error object saved in a variable, as in `$Error1 = command 2>&1`.

The automatic variable `$Error` contains a collection of error records that represent recent errors, and the most recent error is in `$Error[0]`. This collection is maintained in a buffer such that old records are discarded as new ones are added. The automatic variable `$MaximumErrorCount` controls the number of records that can be stored.

`$Error` contains all of the errors from all commands mixed in together in one collection. To collect the errors from a specific command, use the common parameter ErrorVariable, which allows a user-defined variable to be specified to hold the collection.

3.13 Pipelines

A *pipeline* is a series of one or more commands each separated by the pipe operator `|` (U+007C). Each command receives input from its predecessor and writes output to its successor. Unless the output at the end of the pipeline is discarded or redirected to a file, it is sent to the host environment, which may choose to write it to standard output. Commands in a pipeline may also receive input from arguments. For example, consider the following use of commands `Get-ChildItem`, `Sort-Object`, and `Process-File`, which create a list of file names in a given file system directory, sort a set of text records, and perform some processing on a text record, respectively:

PowerShell

```
Get-ChildItem
Get-ChildItem E:*.txt | Sort-Object -CaseSensitive | Process-File >results.txt
```

In the first case, `Get-ChildItem` creates a collection of names of the files in the current/default directory. That collection is sent to the host environment, which, by default, writes each element's value to standard output.

In the second case, `Get-ChildItem` creates a collection of names of the files in the directory specified, using the argument `E:*.txt`. That collection is written to the command `Sort-Object`,

which, by default, sorts them in ascending order, sensitive to case (by virtue of the **CaseSensitive** argument). The resulting collection is then written to command `Process-File`, which performs some (unknown) processing. The output from that command is then redirected to the file `results.txt`.

If a command writes a single object, its successor receives that object and then terminates after writing its own object(s) to its successor. If, however, a command writes multiple objects, they are delivered one at a time to the successor command, which executes once per object. This behavior is called *streaming*. In stream processing, objects are written along the pipeline as soon as they become available, not when the entire collection has been produced.

When processing a collection, a command can be written such that it can do special processing before the initial element and after the final element.

3.14 Modules

A *module* is a self-contained reusable unit that allows PowerShell code to be partitioned, organized, and abstracted. A module can contain commands (such as cmdlets and functions) and items (such as variables and aliases) that can be used as a single unit.

Once a module has been created, it must be *imported* into a session before the commands and items within it can be used. Once imported, commands and items behave as if they were defined locally. A module is imported explicitly with the `Import-Module` command. A module may also be imported automatically as determined in an implementation defined manner.

The type of an object that represents a module is described in §4.5.12.

Modules are discussed in detail in §11.

3.15 Wildcard expressions

A wildcard expression may contain zero or more of the following elements:

Expand table

Element	Description
Character other than *, ?, or [	Matches that one character
*	Matches zero or more characters. To match a * character, use [*].

Element	Description
?	Matches any one character. To match a ? character, use [?].
[*set*]	Matches any one character from *set*, which cannot be empty. If *set* begins with], that right square bracket is considered part of *set* and the next right square bracket terminates the set; otherwise, the first right square bracket terminates the set. If *set* begins or ends with -, that hyphen-minus is considered part of *set*; otherwise, it indicates a range of consecutive Unicode code points with the characters either side of the hyphen-minus being the inclusive range delimiters. For example, A-Z indicates the 26 uppercase English letters, and 0-9 indicates the 10 decimal digits.

> **Note**
>
> More information can be found in, The Open Group Base Specifications: Pattern Matching", IEEE Std 1003.1, 2004 Edition. . However, in PowerShell, the escape character is backtick, not backslash.

3.16 Regular expressions

A regular expression may contain zero or more of the following elements:

Expand table

Element	Description
Character other than ., [, ^, *, $, or \	Matches that one character
.	Matches any one character. To match a . character, use \..
[*set*] [^*set*]	The [*set*] form matches any one character from *set*. The [^*set*] form matches no characters from *set*. *set* cannot be empty. If *set* begins with] or ^], that right square bracket is considered part of *set* and the next right square bracket terminates the set; otherwise, the first right square bracket terminates the set. If *set* begins with - or ^-, or ends with -, that hyphen-minus is considered part of *set*; otherwise, it indicates a range of consecutive Unicode code points with the characters either side of the hyphen-minus being the inclusive range delimiters. For example, A-Z indicates the 26 uppercase English letters, and 0-9 indicates the 10 decimal digits.

Element	Description
*	Matches zero of more occurrences of the preceding element.
+	Matches one of more occurrences of the preceding element.
?	Matches zero of one occurrences of the preceding element.
^	Matches at the start of the string. To match a ^ character, use \^.
$	Matches at the end of the string. To match a $ character, use $.
\c	Escapes character *c*, so it isn't recognized as a regular expression element.

> **Note**
>
> More information can be found in, The Open Group Base Specifications: Regular Expressions, IEEE Std 1003.1, 2004 Edition. .

Windows PowerShell: Character classes available in Microsoft .NET Framework regular expressions are supported, as follows:

Expand table

Element	Description
`\p{name}`	Matches any character in the named character class specified by *name*. Supported names are Unicode groups and block ranges such as Ll, Nd, Z, IsGreek, and IsBoxDrawing.
`\P{name}`	Matches text not included in the groups and block ranges specified in *name*.
`\w`	Matches any word character. Equivalent to the Unicode character categories `[\p{Ll}\p{Lu}\p{Lt}\p{Lo}\p{Nd}\p{Pc}]`. If ECMAScript-compliant behavior is specified with the ECMAScript option, \w is equivalent to `[a-zA-Z_0-9]`.
`\W`	Matches any non-word character. Equivalent to the Unicode categories `[\^\p{Ll}\p{Lu}\p{Lt}\p{Lo}\p{Nd}\p{Pc}]`.
`\s`	Matches any white space character. Equivalent to the Unicode character categories `[\f\n\r\t\v\x85\p{Z}]`.
`\S`	Matches any non-white-space character. Equivalent to the Unicode character categories `[\^\f\n\r\t\v\x85\p{Z}]`.
`\d`	Matches any decimal digit. Equivalent to `\p{Nd}` for Unicode and `[0-9]` for non-Unicode behavior.

Element	Description
`\D`	Matches any non-digit. Equivalent to `\P{Nd}` for Unicode and `[\^0-9]` for non-Unicode behavior.

Quantifiers available in Microsoft .NET Framework regular expressions are supported, as follows:

Expand table

Element	Description
`*`	Specifies zero or more matches; for example, `\w*` or `(abc)*.` Equivalent to `{0,}`.
`+`	Matches repeating instances of the preceding characters.
`?`	Specifies zero or one matches; for example, `\w?` or `(abc)?`. Equivalent to `{0,1}`.
`{n}`	Specifies exactly *n* matches; for example, `(pizza){2}`.
`{n,}`	Specifies at least *n* matches; for example, `(abc){2,}`.
`{n,m}`	Specifies at least *n*, but no more than *m*, matches.

Last updated on 09/15/2023

4. Types

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

In PowerShell, each value has a type, and types fall into one of two main categories: **value types** and **reference types**. Consider the type `int`, which is typical of value types. A value of type `int` is completely self-contained; all the bits needed to represent that value are stored in that value, and every bit pattern in that value represents a valid value for its type. Now, consider the array type `int[]`, which is typical of reference types. A so-called value of an array type can hold either a reference to an object that actually contains the array elements, or the **null reference** whose value is `$null`. The important distinction between the two type categories is best demonstrated by the differences in their semantics during assignment. For example,

PowerShell

```
$i = 100 # $i designates an int value 100
$j = $i # $j designates an int value 100, which is a copy

$a = 10,20,30 # $a designates an object[], Length 3, value 10,20,30
$b = $a # $b designates exactly the same array as does $a, not a copy
$a[1] = 50 # element 1 (which has a value type) is changed from 20 to 50
$b[1] # $b refers to the same array as $a, so $b[1] is 50
```

As we can see, the assignment of a reference type value involves a **shallow copy**; that is, a copy of the reference to the object rather than its actual value. In contrast, a **deep copy** requires making a copy of the object as well.

A **numeric** type is one that allows representation of integer or fractional values, and that supports arithmetic operations on those values. The set of numerical types includes the integer (§4.2.3) and real number (§4.2.4) types, but does not include bool (§4.2.1) or char (§4.2.2). An implementation may provide other numeric types (such as signed byte, unsigned integer, and integers of other sizes).

A **collection** is a group of one or more related items, which need not have the same type. Examples of collection types are arrays, stacks, queues, lists, and hash tables. A program can *enumerate* (or *iterate*) over the elements in a collection, getting access to each element one at a time. Common ways to do this are with the `foreach` statement (§8.4.4) and the ForEach-Object cmdlet. The type of an object that represents an enumerator is described in §4.5.16.

In this chapter, there are tables that list the accessible members for a given type. For methods, the **Type** is written with the following form: *returnType/argumentTypeList*. If the argument type list is too long to fit in that column, it is shown in the **Purpose** column instead.

Other integer types are `SByte`, `Int16`, `UInt16`, `UInt32`, and `UInt64`, all in the namespace **System**.

Many collection classes are defined as part of the **System.Collections** or **System.Collections.Generic** namespaces. Most collection classes implement the interfaces `ICollection`, `IComparer`, `IEnumerable`, `IList`, `IDictionary`, and `IDictionaryEnumerator` and their generic equivalents.

You can also use shorthand names for some types. For more information, see about_Type_Accelerators.

4.1 Special types

4.1.1 The void type

This type cannot be instantiated. It provides a means to discard a value explicitly using the cast operator (§7.2.9).

4.1.2 The null type

The *null type* has one instance, the automatic variable $null (§2.3.2.2), also known as the null value. This value provides a means for expressing "nothingness" in reference contexts. The characteristics of this type are unspecified.

4.1.3 The object type

Every type in PowerShell except the null type (§4.1.2) is derived directly or indirectly from the type object, so object is the ultimate base type of all non-null types. A variable constrained (§5.3) to type object is really not constrained at all, as it can contain a value of any type.

4.2 Value types

4.2.1 Boolean

The Boolean type is `bool`. There are only two values of this type, **False** and **True**, represented by the automatic variables `$false` and `$true`, respectively (§2.3.2.2).

In PowerShell, `bool` maps to `System.Boolean`.

4.2.2 Character

A character value has type char, which is capable of storing any UTF-16-encoded 16-bit Unicode code point.

The type char has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
MaxValue	Static property (read-only)	char	The largest possible value of type char
MinValue	Static property (read-only)	char	The smallest possible value of type char
IsControl	Static method	bool/char	Tests if the character is a control character
IsDigit	Static method	bool/char	Tests if the character is a decimal digit
IsLetter	Static method	bool/char	Tests if the character is an alphabetic letter
IsLetterOrDigit	Static method	bool/char	Tests if the character is a decimal digit or alphabetic letter

Member	Member Kind	Type	Purpose
IsLower	Static method	bool/char	Tests if the character is a lowercase alphabetic letter
IsPunctuation	Static method	bool/char	Tests if the character is a punctuation mark
IsUpper	Static method	bool/char	Tests if the character is an uppercase alphabetic letter
IsWhiteSpace	Static method	bool/char	Tests if the character is a white space character.
ToLower	Static method	char/string	Converts the character to lowercase
ToUpper	Static method	char/string	Converts the character to uppercase

Windows PowerShell: char maps to System.Char.

4.2.3 Integer

There are two signed integer types, both of use two's-complement representation for negative values:

- Type `int`, which uses 32 bits giving it a range of -2147483648 to +2147483647, inclusive.
- Type `long`, which uses 64 bits giving it a range of -9223372036854775808 to +9223372036854775807, inclusive.

Type int has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
MaxValue	Static property (read-only)	int	The largest possible value of type int
MinValue	Static property (read-only)	int	The smallest possible value of type int

Type long has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
MaxValue	Static property (read-only)	long	The largest possible value of type long
MinValue	Static property (read-only)	long	The smallest possible value of type long

There is one unsigned integer type:

- Type `byte`, which uses 8 bits giving it a range of 0 to 255, inclusive.

Type `byte` has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
MaxValue	Static property (read-only)	byte	The largest possible value of type byte
MinValue	Static property (read-only)	byte	The smallest possible value of type byte

In PowerShell, `byte`, `int`, and `long` map to `System.Byte`, `System.Int32`, and `System.Int64`, respectively.

4.2.4 Real number

4.2.4.1 float and double

There are two real (or floating-point) types:

- Type `float` uses the 32-bit IEEE single-precision representation.
- Type `double` uses the 64-bit IEEE double-precision representation.

A third type name, `single`, is a synonym for type `float`; `float` is used throughout this specification.

Although the size and representation of the types `float` and `double` are defined by this specification, an implementation may use extended precision for intermediate results.

Type float has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
MaxValue	Static property (read-only)	float	The largest possible value of type float
MinValue	Static property (read-only)	float	The smallest possible value of type float
NaN	Static property (read-only)	float	The constant value Not-a-Number
NegativeInfinity	Static property (read-only)	float	The constant value negative infinity

Member	Member Kind	Type	Purpose
PositiveInfinity	Static property (read-only)	float	The constant value positive infinity

Type double has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
MaxValue	Static property (read-only)	double	The largest possible value of type double
MinValue	Static property (read-only)	double	The smallest possible value of type double
NaN	Static property (read-only)	double	The constant value Not-a-Number
NegativeInfinity	Static property (read-only)	double	The constant value negative infinity
PositiveInfinity	Static property (read-only)	double	The constant value positive infinity

In PowerShell, `float` and `double` map to `System.Single` and `System.Double`, respectively.

4.2.4.2 decimal

Type decimal uses a 128-bit representation. At a minimum it must support a scale **s** such that 0 <= *s* <= at least 28, and a value range -79228162514264337593543950335 to 79228162514264337593543950335. The actual representation of decimal is implementation defined.

Type decimal has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
MaxValue	Static property (read-only)	decimal	The largest possible value of type decimal
MinValue	Static property (read-only)	decimal	The smallest possible value of type decimal

> **Note**
>
> Decimal real numbers have a characteristic called *scale*, which represents the number of digits to the right of the decimal point. For example, the value 2.340 has a scale of 3 where trailing zeros are significant. When two decimal real numbers are added or subtracted, the scale of the result is the larger of the two scales. For example, 1.0 + 2.000 is 3.000, while

5.0 - 2.00 is 3.00. When two decimal real numbers are multiplied, the scale of the result is the sum of the two scales. For example, 1.0 * 2.000 is 2.0000. When two decimal real numbers are divided, the scale of the result is the scale of the first less the scale of the second. For example, 4.00000/2.000 is 2.00. However, a scale cannot be less than that needed to preserve the correct result. For example, 3.000/2.000, 3.00/2.000, 3.0/2.000, and 3/2 are all 1.5.

In PowerShell, `decimal` maps to `System.Decimal`. The representation of decimal is as follows:

- When considered as an array of four `int` values it contains the following elements:
  - Index 0 (bits 0-31) contains the low-order 32 bits of the decimal's coefficient.
  - Index 1 (bits 32-63) contains the middle 32 bits of the decimal's coefficient.
  - Index 2 (bits 64-95) contains the high-order 32 bits of the decimal's coefficient.
  - Index 3 (bits 96-127) contains the sign bit and scale, as follows:
    - bits 0--15 are zero
    - bits 16-23 contains the scale as a value 0--28
    - bits 24-30 are zero
    - bit 31 is the sign (0 for positive, 1 for negative)

4.2.5 The switch type

This type is used to constrain the type of a parameter in a command (§8.10.5). If an argument having the corresponding parameter name is present the parameter tests $true; otherwise, it tests `$false`.

In PowerShell, `switch` maps to `System.Management.Automation.SwitchParameter`.

4.2.6 Enumeration types

An enumeration type is one that defines a set of named constants representing all the possible values that can be assigned to an object of that enumeration type. In some cases, the set of values are such that only one value can be represented at a time. In other cases, the set of values are distinct powers of two, and by using the -bor operator (§7.8.5), multiple values can be encoded in the same object.

The PowerShell environment provides a number of enumeration types, as described in the following sections.

4.2.6.1 Action-Preference type

This implementation-defined type has the following mutually exclusive-valued accessible members:

Expand table

Member	Member Kind	Purpose
Continue	Enumeration constant	The PowerShell runtime will continue processing and notify the user that an action has occurred.
Inquire	Enumeration constant	The PowerShell runtime will stop processing and ask the user how it should proceed.
SilentlyContinue	Enumeration constant	The PowerShell runtime will continue processing without notifying the user that an action has occurred.
Stop	Enumeration constant	The PowerShell runtime will stop processing when an action occurs.

In PowerShell, this type is `System.Management.Automation.ActionPreference`.

4.2.6.2 Confirm-Impact type

This implementation-defined type has the following mutually exclusive-valued accessible members:

Expand table

Member	Member Kind	Purpose
High	Enumeration constant	The action performed has a high risk of losing data, such as reformatting a hard disk.
Low	Enumeration constant	The action performed has a low risk of losing data.
Medium	Enumeration constant	The action performed has a medium risk of losing data.
None	Enumeration constant	Do not confirm any actions (suppress all requests for confirmation).

In PowerShell, this type is `System.Management.Automation.ConfirmImpact`.

4.2.6.3 File-Attributes type

This implementation-defined type has the following accessible members, which can be combined:

Expand table

Member	Member Kind	Purpose
Archive	Enumeration constant	The file's archive status. Applications use this attribute to mark files for backup or removal.
Compressed	Enumeration constant	The file is compressed.
Device		Reserved for future use.
Directory	Enumeration constant	The file is a directory.
Encrypted	Enumeration constant	The file or directory is encrypted. For a file, this means that all data in the file is encrypted. For a directory, this means that encryption is the default for newly created files and directories.
Hidden	Enumeration constant	The file is hidden, and thus is not included in an ordinary directory listing.
Normal	Enumeration constant	The file is normal and has no other attributes set. This attribute is valid only if used alone.
NotContentIndexed	Enumeration constant	The file will not be indexed by the operating system's content indexing service.
Offline	Enumeration constant	The file is offline. The data of the file is not immediately available.
ReadOnly	Enumeration constant	The file is read-only.
ReparsePoint	Enumeration constant	The file contains a reparse point, which is a block of user-defined data associated with a file or a directory.
SparseFile	Enumeration constant	The file is a sparse file. Sparse files are typically large files whose data are mostly zeros.
System	Enumeration constant	The file is a system file. The file is part of the operating system or is used exclusively by the operating system.
Temporary	Enumeration constant	The file is temporary. File systems attempt to keep all of the data in memory for quicker access rather than flushing the data back to mass storage. A temporary file should be deleted by the application as soon as it is no longer needed.

In PowerShell, this type is System.IO.FileAttributes with attribute FlagsAttribute.

4.2.6.4 Regular-Expression-Option type

This implementation-defined type has the following accessible members, which can be combined:

Expand table

Member	Member Kind	Purpose
IgnoreCase	Enumeration constant	Specifies that the matching is case-insensitive.
None	Enumeration constant	Specifies that no options are set.

An implementation may provide other values.

In PowerShell, this type is `System.Text.RegularExpressions.RegexOptions` with attribute `FlagsAttribute`. The following extra values are defined: `Compiled`, `CultureInvariant`, `ECMAScript`, `ExplicitCapture`, `IgnorePatternWhitespace`, `Multiline`, `RightToLeft`, `Singleline`.

4.3 Reference types

4.3.1 Strings

A string value has type string and is an immutable sequence of zero or more characters of type char each containing a UTF-16-encoded 16-bit Unicode code point.

Type string has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Length	Instance Property	int (read-only)	Gets the number of characters in the string
ToLower	Instance Method	string	Creates a new string that contains the lowercase equivalent
ToUpper	Instance Method	string	Creates a new string that contains the uppercase equivalent

In PowerShell, `string` maps to `System.String`.

4.3.2 Arrays

All array types are derived from the type `array`. This type has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Length	Instance Property (read-only)	int	Number of elements in the array
Rank	Instance Property (read-only)	int	Number of dimensions in the array
Copy	Static Method	void/see Purpose column	Copies a range of elements from one array to another. There are four versions, where *source* is the source array, *destination* is the destination array, *count* is the number of elements to copy, and *sourceIndex* and *destinationIndex* are the starting locations in their respective arrays: Copy(*source*, *destination*, int *count*) Copy(*source*, *destination*, long *count*) Copy(*source*, *sourceIndex*, *destination*, *destinationIndex*, int *count*) Copy(*source*, *sourceIndex*, *destination*, *destinationIndex*, long *count*)
GetLength	Instance Method (read-only)	int/none	Number of elements in a given dimension GetLength(int *dimension*)

For more details on arrays, see §9.

In PowerShell, `array` maps to `System.Array`.

4.3.3 Hashtables

Type Hashtable has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Count	Instance Property	int	Gets the number of key/value pairs in the Hashtable
Keys	Instance Property	Implementation-defined	Gets a collection of all the keys
Values	Instance Property	Implementation-defined	Gets a collection of all the values
Remove	Instance Method	void/none	Removes the designated key/value

For more details on Hashtables, see §10.

In PowerShell, `Hashtable` maps to `System.Collections.Hashtable`. `Hashtable` elements are stored in an object of type `DictionaryEntry`, and the collections returned by Keys and Values have type `ICollection`.

4.3.4 The xml type

Type xml implements the W3C Document Object Model (DOM) Level 1 Core and the Core DOM Level 2. The DOM is an in-memory (cache) tree representation of an XML document and enables the navigation and editing of this document. This type supports the subscript operator [] (§7.1.4.4).

In PowerShell, `xml` maps to `System.Xml.XmlDocument`.

4.3.5 The regex type

Type `regex` provides machinery for supporting regular expression processing. It is used to constrain the type of a parameter (§5.3) whose corresponding argument might contain a regular expression.

In PowerShell, `regex` maps to `System.Text.RegularExpressions.Regex`.

4.3.6 The ref type

Ordinarily, arguments are passed to commands by value. In the case of an argument having some value type a copy of the value is passed. In the case of an argument having some reference type a copy of the reference is passed.

Type ref provides machinery to allow arguments to be passed to commands by reference, so the commands can modify the argument's value. Type ref has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Value	Instance property (read-write)	The type of the value being referenced.	Gets/sets the value being referenced.

Consider the following function definition and call:

PowerShell

```
function Doubler {
    param ([ref]$x) # parameter received by reference
    $x.Value *= 2.0 # note that 2.0 has type double
}

$number = 8 # designates a value of type int, value 8
Doubler([ref]$number) # argument received by reference
$number # designates a value of type double, value 8.0
```

Consider the case in which $number is type-constrained:

PowerShell

```
[int]$number = 8 # designates a value of type int, value 8
Doubler([ref]$number) # argument received by reference
$number # designates a value of type int, value 8
```

As shown, both the argument and its corresponding parameter must be declared `ref`.

In PowerShell, `ref` maps to `System.Management.Automation.PSReference`.

4.3.7 The scriptblock type

Type `scriptblock` represents a precompiled block of script text (§7.1.8) that can be used as a single unit. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Attributes	Instance property (read-only)	Collection of attributes	Gets the attributes of the script block.
File	Instance property (read-only)	string	Gets the name of the file in which the script block is defined.
Module	Instance property (read-only)	implementation defined ([§4.5.12] [§4.5.12])	Gets information about the module in which the script block is defined.
GetNewClosure	Instance method	scriptblock /none	Retrieves a script block that is bound to a module. Any local variables that are in the context of the caller will be copied into the module.
Invoke	Instance method	Collection of object/object[]	Invokes the script block with the specified arguments and returns the results.
InvokeReturnAsIs	Instance method	object/object[]	Invokes the script block with the specified arguments and returns any objects generated.
Create	Static method	scriptblock /string	Creates a new scriptblock object that contains the specified script.

In PowerShell, `scriptblock` maps to `System.Management.Automation.ScriptBlock`. `Invoke` returns a collection of `PSObject`.

4.3.8 The math type

Type `math` provides access to some constants and methods useful in mathematical computations. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
E	Static property (read-only)	double	Natural logarithmic base
PI	Static property (read-only)	double	Ratio of the circumference of a circle to its diameter
Abs	Static method	numeric/numeric	Absolute value (the return type is the same as

Member	Member Kind	Type	Purpose
			the type of the argument passed in)
Acos	Static method	double / double	Angle whose cosine is the specified number
Asin	Static method	double / double	Angle whose sine is the specified number
Atan	Static method	double / double	Angle whose tangent is the specified number
Atan2	Static method	double / double *y*, double *x*	Angle whose tangent is the quotient of two specified numbers *x* and *y*
Ceiling	Static method	decimal / decimal double / double	smallest integer greater than or equal to the specified number
Cos	Static method	double / double	Cosine of the specified angle
Cosh	Static method	double / double	Hyperbolic cosine of the specified angle
Exp	Static method	double / double	e raised to the specified power
Floor	Static method	decimal / decimal double / double	Largest integer less than or equal to the specified number
Log	Static method	double / double *number* double / double *number*, double *base*	Logarithm of number using base e or base *base*
Log10	Static method	double / double	Base-10 logarithm of a specified number
Max	Static method	numeric/numeric	Larger of two specified numbers (the return type is the same as the type of the arguments passed in)
Min	Static method	numeric/numeric, numeric	Smaller of two specified numbers (the return type is the same as the type of the arguments passed in)
Pow	Static method	double / double *x*, double *y*	A specified number *x* raised to the specified power *y*
Sin	Static method	double / double	Sine of the specified angle
Sinh	Static method	double / double	Hyperbolic sine of the specified angle
Sqrt	Static method	double / double	Square root of a specified number
Tan	Static method	double / double	Tangent of the specified angle
Tanh	Static method	double / double	Hyperbolic tangent of the specified angle

In PowerShell, `Math` maps to `System.Math`.

4.3.9 The ordered type

Type `ordered` is a pseudo type used only for conversions.

4.3.10 The pscustomobject type

Type `pscustomobject` is a pseudo type used only for conversions.

4.4 Generic types

A number of programming languages and environments provide types that can be *specialized*. Many of these types are referred to as *container types*, as instances of them are able to contain objects of some other type. Consider a type called Stack that can represent a stack of values, which can be pushed on and popped off. Typically, the user of a stack wants to store only one kind of object on that stack. However, if the language or environment does not support type specialization, multiple distinct variants of the type Stack must be implemented even though they all perform the same task, just with different type elements.

Type specialization allows a *generic type* to be implemented such that it can be constrained to handling some subset of types when it is used. For example,

- A generic stack type that is specialized to hold strings might be written as `Stack[string]`.
- A generic dictionary type that is specialized to hold int keys with associated string values might be written as `Dictionary[int,string]`.
- A stack of stack of strings might be written as `Stack[Stack[string]]`.

Although PowerShell does not define any built-in generic types, it can use such types if they are provided by the host environment. See the syntax in §7.1.10.

The complete name for the type `Stack[string]` suggested above is `System.Collections.Generic.Stack[string]`. The complete name for the type `Dictionary[int,string]` suggested above is `System.Collections.Generic.Dictionary[int,string]`.

4.5 Anonymous types

In some circumstances, an implementation of PowerShell creates objects of some type, and those objects have members accessible to script. However, the actual name of those types

need not be specified, so long as the accessible members are specified sufficiently for them to be used. That is, scripts can save objects of those types and access their members without actually knowing those types' names. The following subsections specify these types.

4.5.1 Provider description type

This type encapsulates the state of a provider. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Drives	Instance property (read-only)	Implementation defined (§4.5.2)	A collection of drive description objects
Name	Instance property (read-only)	string	The name of the provider

In PowerShell, this type is `System.Management.Automation.ProviderInfo`.

4.5.2 Drive description type

This type encapsulates the state of a drive. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
CurrentLocation	Instance property (read-write)	string	The current working location (§3.1.4) of the drive
Description	Instance property (read-write)	string	The description of the drive
Name	Instance property (read-only)	string	The name of the drive
Root	Instance property (read-only)	string	The name of the drive

In PowerShell, this type is `System.Management.Automation.PSDriveInfo`.

4.5.3 Variable description type

This type encapsulates the state of a variable. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Attributes	Instance property (read-only)	Implementation defined	A collection of attributes
Description	Instance property (read-write)	string	The description assigned to the variable via the New-Variable or Set-Variable cmdlets.
Module	Instance property (read-only)	Implementation defined (§4.5.12)	The module from which this variable was exported
ModuleName	Instance property (read-only)	string	The module in which this variable was defined
Name	Instance property (read-only)	string	The name assigned to the variable when it was created in the PowerShell language or via the `New-Variable` and `Set-Variable` cmdlets.
Options	Instance property (read-write)	string	The options assigned to the variable via the `New-Variable` and `Set-Variable` cmdlets.
Value	Instance property (read-write)	object	The value assigned to the variable when it was assigned in the PowerShell language or via the `New-Variable` and `Set-Variable` cmdlets.

In PowerShell, this type is `System.Management.Automation.PSVariable`.

Windows PowerShell: The type of the attribute collection is System.Management.Automation.PSVariableAttributeCollection.

4.5.4 Alias description type

This type encapsulates the state of an alias. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
CommandType	Instance property (read-only)	Implementation defined	Should compare equal with "Alias".
Definition	Instance property (read-	string	The command or alias to which the alias was assigned via the New-Alias or

Member	Member Kind	Type	Purpose
	only)		Set-Alias cmdlets.
Description	Instance property (read-write)	string	The description assigned to the alias via the `New-Alias` or `Set-Alias` cmdlets.
Module	Instance property (read-only)	Implementation defined (§4.5.12)	The module from which this alias was exported
ModuleName	Instance property (read-only)	string	The module in which this alias was defined
Name	Instance property (read-only)	string	The name assigned to the alias when it was created via the `New-Alias` or `Set-Alias` cmdlets.
Options	Instance property (read-write)	string	The options assigned to the alias via the New-Alias `New-Alias` or `Set-Alias` cmdlets.
OutputType	Instance property (read-only)	Implementation defined collection	Specifies the types of the values output by the command to which the alias refers.
Parameters	Instance property (read-only)	Implementation defined collection	The parameters of the command.
ParameterSets	Instance property (read-only)	Implementation defined collection	Information about the parameter sets associated with the command.
ReferencedCommand	Instance property (read-only)	Implementation defined	Information about the command that is immediately referenced by this alias.
ResolvedCommand	Instance property (read-only)	Implementation defined	Information about the command to which the alias eventually resolves.

In PowerShell, this type is `System.Management.Automation.AliasInfo`.

4.5.5 Working location description type

This type encapsulates the state of a working location. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Drive	Instance property (read-only)	Implementation defined (§4.5.2)	A drive description object
Path	Instance property (read-only)	string	The working location
Provider	Instance property (read-only)	Implementation defined (§4.5.1)	The provider
ProviderPath	Instance property (read-only)	string	The current path of the provider

A stack of working locations is a collection of working location objects, as described above.

In PowerShell, a current working location is represented by an object of type `System.Management.Automation.PathInfo`. A stack of working locations is represented by an object of type `System.Management.Automation.PathInfoStack`, which is a collection of `PathInfo` objects.

4.5.6 Environment variable description type

This type encapsulates the state of an environment variable. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Name	Instance property (read-write)	string	The name of the environment variable
Value	Instance property (read-write)	string	The value of the environment variable

In PowerShell, this type is `System.Collections.DictionaryEntry`. The name of the variable is the dictionary key. The value of the environment variable is the dictionary value. **Name** is an `AliasProperty` that equates to **Key**.

4.5.7 Application description type

This type encapsulates the state of an application. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
CommandType	Instance property (read-only)	Implementation defined	Should compare equal with "Application".
Definition	Instance property (read-only)	string	A description of the application.
Extension	Instance property (read-write)	string	The extension of the application file.
Module	Instance property (read-only)	Implementation defined (§4.5.12)	The module that defines this command.
ModuleName	Instance property (read-only)	string	The name of the module that defines the command.
Name	Instance property (read-only)	string	The name of the command.
OutputType	Instance property (read-only)	Implementation defined collection	Specifies the types of the values output by the command.
Parameters	Instance property (read-only)	Implementation defined collection	The parameters of the command.
ParameterSets	Instance property (read-only)	Implementation defined collection	Information about the parameter sets associated with the command.
Path	Instance property (read-only)	string	Gets the path of the application file.

In PowerShell, this type is `System.Management.Automation.ApplicationInfo`.

4.5.8 Cmdlet description type

This type encapsulates the state of a cmdlet. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
CommandType	Instance property (read-only)	Implementation defined	Should compare equal with "Cmdlet".

Member	Member Kind	Type	Purpose
DefaultParameterSet	Instance property (read-only)	Implementation defined	The default parameter set that is used if PowerShell cannot determine which parameter set to use based on the supplied arguments.
Definition	Instance property (read-only)	string	A description of the cmdlet.
HelpFile	Instance property (read-write)	string	The path to the Help file for the cmdlet.
ImplementingType	Instance property (read-write)	Implementation defined	The type that implements the cmdlet.
Module	Instance property (read-only)	Implementation defined (§4.5.12)	The module that defines this cmdlet.
ModuleName	Instance property (read-only)	string	The name of the module that defines the cmdlet.
Name	Instance property (read-only)	string	The name of the cmdlet.
Noun	Instance property (read-only)	string	The noun name of the cmdlet.
OutputType	Instance property (read-only)	Implementation defined collection	Specifies the types of the values output by the cmdlet.
Parameters	Instance property (read-only)	Implementation defined collection	The parameters of the cmdlet.
ParameterSets	Instance property (read-only)	Implementation defined collection	Information about the parameter sets associated with the cmdlet.
Verb	Instance property (read-only)	string	The verb name of the cmdlet.

Member	Member Kind	Type	Purpose
PSSnapIn	Instance property (read-only)	Implementation defined	Windows PowerShell: Information about the Windows PowerShell snap-in that is used to register the cmdlet.

In PowerShell, this type is `System.Management.Automation.CmdletInfo`.

4.5.9 External script description type

This type encapsulates the state of an external script (one that is directly executable by PowerShell, but is not built-in). It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
CommandType	Instance property (read-only)	Implementation defined	Should compare equal with "ExternalScript".
Definition	Instance property (read-only)	string	A definition of the script.
Module	Instance property (read-only)	Implementation defined (§4.5.12)	The module that defines this script.
ModuleName	Instance property (read-only)	string	The name of the module that defines the script.
Name	Instance property (read-only)	string	The name of the script.
OriginalEncoding	Instance property (read-only)	Implementation defined	The original encoding used to convert the characters of the script to bytes.
OutputType	Instance property (read-only)	Implementation defined collection	Specifies the types of the values output by the script.
Parameters	Instance property (read-only)	Implementation defined collection	The parameters of the script.

Member	Member Kind	Type	Purpose
ParameterSets	Instance property (read-only)	Implementation defined collection	Information about the parameter sets associated with the script.
Path	Instance property (read-only)	string	The path to the script file.
ScriptBlock	Instance property (read-only)	scriptblock	The external script.
ScriptContents	Instance property (read-only)	string	The original contents of the script.

In PowerShell, this type is `System.Management.Automation.ExternalScriptInfo`.

4.5.10 Function description type

This type encapsulates the state of a function. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
CmdletBinding	Instance property (read-only)	bool	Indicates whether the function uses the same parameter binding that compiled cmdlets use (see §12.3.5).
CommandType	Instance property (read-only)	Implementation defined	Can be compared for equality with "Function" or "Filter" to see which of those this object represents.
DefaultParameterSet	Instance property (read-only)	string	Specifies the parameter set to use if that cannot be determined from the arguments (see §12.3.5).
Definition	Instance property (read-only)	string	A string version of ScriptBlock

Member	Member Kind	Type	Purpose
Description	Instance property (read-write)	string	The description of the function.
Module	Instance property (read-only)	Implementation defined (§4.5.12)	The module from which this function was exported
ModuleName	Instance property (read-only)	string	The module in which this function was defined
Name	Instance property (read-only)	string	The name of the function
Options	Instance property (read-write)	Implementation defined	The scope options for the function (§3.5.4).
OutputType	Instance property (read-only)	Implementation defined collection	Specifies the types of the values output, in order (see §12.3.6).
Parameters	Instance property (read-only)	Implementation defined collection	Specifies the parameter names, in order. If the function acts like a cmdlet (see CmdletBinding above) the common parameters are included at the end of the collection.
ParameterSets	Instance property (read-only)	Implementation defined collection	Information about the parameter sets associated with the command. For each parameter, the result shows the parameter name and type, and indicates if the parameter is mandatory, by position or a `[switch]` parameter. If the function acts like a cmdlet (see CmdletBinding above) the common parameters are included at the end of the collection.
ScriptBlock	Instance property	scriptblock (§4.3.6)	The body of the function

Member	Member Kind	Type	Purpose
	(read-only)		

In PowerShell, this type is `System.Management.Automation.FunctionInfo`.

- `CommandType` has type `System.Management.Automation.CommandTypes`.
- `Options` has type `System.Management.Automation.ScopedItemOptions`.
- `OutputType` has type `System.Collections.ObjectModel.ReadOnlyCollection``1[[System.Management.Automation.PSTypeName,System.Management.Automation]]`.
- `Parameters` has type `System.Collections.Generic.Dictionary``2[[System.String,mscorlib],[System.Management.Automation.ParameterMetadata,System.Management.Automation]]`.
- `ParameterSets` has type `System.Collections.ObjectModel.ReadOnlyCollection``1[[System.Management.Automation.CommandParameterSetInfo,System.Management.Automation]]`.
- Visibility has type `System.Management.Automation.SessionStateEntryVisibility`.
- PowerShell also has a property called **Visibility**.

4.5.11 Filter description type

This type encapsulates the state of a filter. It has the same set of accessible members as the function description type (§4.5.10).

In PowerShell, this type is `System.Management.Automation.FilterInfo`. It has the same set of properties as `System.Management.Automation.FunctionInfo` (§4.5.11).

4.5.12 Module description type

This type encapsulates the state of a module. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Description	Instance property (read-write)	string	The description of the module (set by the manifest)
ModuleType	Instance property	Implementation	The type of the module (Manifest,

Member	Member Kind	Type	Purpose
	(read-only)	defined	Script, or Binary)
Name	Instance property (read-only)	string	The name of the module
Path	Instance property (read-only)	string	The module's path

In PowerShell, this type is `System.Management.Automation.PSModuleInfo`. The type of `ModuleType` is `System.Management.Automation.ModuleType`.

4.5.13 Custom object description type

This type encapsulates the state of a custom object. It has no accessible members.

In PowerShell, this type is `System.Management.Automation.PSCustomObject`. The cmdlets `Import-Module` and `New-Object` can generate an object of this type.

4.5.14 Command description type

The automatic variable `$PSCmdlet` is an object that represents the cmdlet or function being executed. The type of this object is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
ParameterSetName	Instance property (read-only)	string	Name of the current parameter set (see ParameterSetName)
ShouldContinue	Instance method	Overloaded /bool	Requests confirmation of an operation from the user.
ShouldProcess	Instance method	Overloaded /bool	Requests confirmation from the user before an operation is performed.

In PowerShell, this type is System.Management.Automation.PSScriptCmdlet.

4.5.15 Error record description type

The automatic variable `$Error` contains a collection of error records that represent recent errors (§3.12). Although the type of this collection is unspecified, it does support subscripting to get access to individual error records.

In PowerShell, the collection type is `System.Collections.ArrayList`. The type of an individual error record in the collection is `System.Management.Automation.ErrorRecord`. This type has the following public properties:

- CategoryInfo - Gets information about the category of the error.
- ErrorDetails - Gets and sets more detailed error information, such as a replacement error message.
- Exception - Gets the exception that is associated with this error record.
- FullyQualifiedErrorId - Gets the fully qualified error identifier for this error record.
- InvocationInfo - Gets information about the command that was invoked when the error occurred.
- PipelineIterationInfo - Gets the status of the pipeline when this error record was created
- TargetObject - Gets the object that was being processed when the error occurred.

4.5.16 Enumerator description type

A number of variables are enumerators for collections (§4). The automatic variable `$foreach` is the enumerator created for any `foreach` statement. The automatic variable `$input` is the enumerator for a collection delivered to a function from the pipeline. The automatic variable `$switch` is the enumerator created for any `switch` statement.

The type of an enumerator is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Current	Instance property (read-only)	object	Gets the current element in the collection. If the enumerator is not currently positioned at an element of the collection, the behavior is implementation defined.
MoveNext	Instance method	None/bool	Advances the enumerator to the next element of the collection. Returns $true if the enumerator was successfully advanced to the next element; $false if the enumerator has passed the end of the collection.

In PowerShell, these members are defined in the interface `System.IEnumerator`, which is implemented by the types identified below. If the enumerator is not currently positioned at an element of the collection, an exception of type `InvalidOperationException` is raised. For `$foreach`, this type is `System.Array+SZArrayEnumerator`. For `$input`, this type is `System.Collections.ArrayList+ArrayListEnumeratorSimple`. For `$switch`, this type is `System.Array+SZArrayEnumerator`.

4.5.17 Directory description type

The cmdlet New-Item can create items of various kinds including FileSystem directories. The type of a directory description object is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Attributes	Instance property (read-write)	Implementation defined (§4.2.6.3)	Gets or sets one or more of the attributes of the directory object.
CreationTime	Instance property (read-write)	Implementation defined (§4.5.19)	Gets and sets the creation time of the directory object.
Extension	Instance property (read- only)	string	Gets the extension part of the directory name.
FullName	Instance property (read-only)	string	Gets the full path of the directory.
LastWriteTime	Instance property (read-write)	Implementation defined (§4.5.19)	Gets and sets the time when the directory was last written to.
Name	Instance property (read- only)	string	Gets the name of the directory.

In PowerShell, this type is `System.IO.DirectoryInfo`. The type of the **Attributes** property is `System.IO.FileAttributes`.

4.5.18 File description type

The cmdlet `New-Item` can create items of various kinds including FileSystem files. The type of a file description object is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Attributes	Instance property (read-write)	Implementation defined (§4.2.6.3)	Gets or sets one or more of the attributes of the file object.
BaseName	Instance property (read- only)	string	Gets the name of the file excluding the extension.
CreationTime	Instance property (read-write)	Implementation defined (§4.5.19)	Gets and sets the creation time of the file object.
Extension	Instance property (read- only)	string	Gets the extension part of the file name.
FullName	Instance property (read-only)	string	Gets the full path of the file.
LastWriteTime	Instance property (read-write)	Implementation defined (§4.5.19)	Gets and sets the time when the file was last written to.
Length	Instance property (read- only)	long	Gets the size of the file, in bytes.
Name	Instance property (read- only)	string	Gets the name of the file.
VersionInfo	Instance property (read- only)	Implementation defined	Windows PowerShell: This ScriptProperty returns a System.Diagnostics.FileVersionInfo for the file.

In PowerShell, this type is `System.IO.FileInfo`.

4.5.19 Date-Time description type

The type of a date-time description object is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Day	Instance property (read-only)	int	Gets the day component of the month represented by this instance.
Hour	Instance property (read-only)	int	Gets the hour component of the date represented by this instance.
Minute	Instance property (read-only)	int	Gets the minute component of the date represented by this instance.
Month	Instance property (read-only)	int	Gets the month component of the date represented by this instance.
Second	Instance property (read-only)	int	Gets the seconds component of the date represented by this instance.
Year	Instance property (read-only)	int	Gets the year component of the date represented by this instance.

An object of this type can be created by cmdlet Get-Date.

In PowerShell, this type is `System.DateTime`.

4.5.20 Group-Info description type

The type of a **group-info** description object is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Count	Instance property (read-only)	int	Gets the number of elements in the group.
Group	Instance property (read-only)	Implementation-defined collection	Gets the elements of the group.
Name	Instance property (read-only)	string	Gets the name of the group.
Values	Instance property (read-only)	Implementation-defined collection	Gets the values of the elements of the group.

An object of this type can be created by cmdlet Group-Object.

In PowerShell, this type is `Microsoft.PowerShell.Commands.GroupInfo`.

4.5.21 Generic-Measure-Info description type

The type of a **generic-measure-info** description object is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Average	Instance property (read-only)	double	Gets the average of the values of the properties that are measured.
Count	Instance property (read-only)	int	Gets the number of objects with the specified properties.
Maximum	Instance property (read-only)	double	Gets the maximum value of the specified properties.
Minimum	Instance property (read-only)	double	Gets the minimum value of the specified properties.
Property	Instance property (read-only)	string	Gets the property to be measured.
Sum	Instance property (read-only)	double	Gets the sum of the values of the specified properties.

An object of this type can be created by cmdlet Measure-Object.

In PowerShell, this type is `Microsoft.PowerShell.Commands.GenericMeasureInfo`.

4.5.22 Text-Measure-Info description type

The type of a **text-info** description object is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Characters	Instance property (read-only)	int	Gets the number of characters in the target object.
Lines	Instance property (read-only)	int	Gets the number of lines in the target object.
Property	Instance property (read-only)	string	Gets the property to be measured.
Words	Instance property (read-only)	int	Gets the number of words in the target object.

An object of this type can be created by cmdlet `Measure-Object`.

In PowerShell, this type is `Microsoft.PowerShell.Commands.TextMeasureInfo`.

4.5.23 Credential type

A credential object can then be used in various security operations. The type of a credential object is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Password	Instance property (read-only)	Implementation defined	Gets the password.
UserName	Instance property (read-only)	string	Gets the username.

An object of this type can be created by cmdlet Get-Credential.

In PowerShell, this type is `System.Management.Automation.PSCredential`.

4.5.24 Method designator type

The type of a method designator is implementation defined; it has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Invoke	Instance method	object/variable number and type	Takes a variable number of arguments, and indirectly calls the method referred to by the parent method designator, passing in the arguments.

An object of this type can be created by an *invocation-expression* (§7.1.3).

In PowerShell, this type is System.Management.Automation.PSMethod.

4.5.25 Member definition type

This type encapsulates the definition of a member. It has the following accessible members:

Expand table

Member	Member Kind	Type	Purpose
Definition	Instance property (read-only)	string	Gets the definition of the member.
MemberType	Instance property (read-only)	Implementation defined	Gets the PowerShell type of the member.
Name	Instance property (read-only)	string	Gets the name of the member.
TypeName	Instance property (read-only)	string	Gets the type name of the member.

In PowerShell, this type is `Microsoft.PowerShell.Commands.MemberDefinition`.

4.6 Type extension and adaptation

A PowerShell implementation includes a family of core types (which are documented in this chapter) that each contain their own set of *base members*. Those members can be methods or properties, and they can be instance or static members. For example, the base members of the type string (§4.3.1) are the instance property Length and the instance methods ToLower and ToUpper.

When an object is created, it contains all the instance properties of that object's type, and the instance methods of that type can be called on that object. An object may be customized via the addition of instance members at runtime. The result is called a *custom object*. Any members added to an instance exist only for the life of that instance; other instances of the same core type are unaffected.

The base member set of a type can be augmented by the addition of the following kinds of members:

- *adapted members*, via the *Extended Type System* (ETS), most details of which are unspecified.
- *extended members*, via the cmdlet Add-Member.

In PowerShell, extended members can also be added via `types.ps1xml` files. Adapted and extended members are collectively called **synthetic** *members*.

The ETS adds the following members to all PowerShell objects: **psbase**, **psadapted**, **psextended**, and **pstypenames**. See the **Force** and **View** parameters in the cmdlet Get-Member for more information on these members.

An instance member may hide an extended and/or adapted member of the same name, and an extended member may hide an adapted member. In such cases, the member sets **psadapted** and **psextended** can be used to access those hidden members.

If a `types.ps1xml` specifies a member called **Supports**, `obj.psextended` provides access to just that member and not to a member added via `Add-Member`.

There are three ways create a custom object having a new member M:

1. This approach can be used to add one or more NoteProperty members.

   PowerShell

   ```
   $x = New-Object PSObject -Property @{M = 123}
   ```

2. This approach can be used to add NoteProperty or ScriptMethod members.

   PowerShell

   ```
   $x = New-Module -AsCustomObject {$M = 123 ; Export-ModuleMember --Variable M}
   ```

3. This approach can be used to add any kind of member.

   PowerShell

   ```
   $x = New-Object PSObject
   Add-Member -InputObject $x -Name M -MemberType NoteProperty -Value 123
   ```

`PSObject` is the base type of all PowerShell types.

Last updated on 04/08/2026

5. Variables

Article • 01/08/2025

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389 That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

A variable represents a storage location for a value, and that value has a type. Traditional procedural programming languages are statically typed; that is, the runtime type of a variable is that with which it was declared at compile time. Object-oriented languages add the idea of inheritance, which allows the runtime type of a variable to be that with which it was declared at compile time or some type derived from that type. Being a dynamically typed language, PowerShell's variables do not have types, per se. In fact, variables are not defined; they simply come into being when they are first assigned a value. And while a variable may be constrained (§5.3) to holding a value of a given type, type information in an assignment cannot always be verified statically.

At different times, a variable may be associated with values of different types either through assignment (§7.11) or the use of the `++` and `--` operators (§7.1.5, §7.2.6). When the value associated with a variable is changed, that value's type may change. For example,

PowerShell

```
$i = "abc"        # $i holds a value of type string
$i = 2147483647   # $i holds a value of type int
```

```
++$i              # $i now holds a value of type double because
                  # 2147483648 is too big to fit in type int
```

Any use of a variable that has not been created results in the value $null. To see if a variable has been defined, use the Test-Path cmdlet.

5.1 Writable location

A *writable location* is an expression that designates a resource to which a command has both read and write access. A writable location may be a variable (§5), an array element (§9), an associated value in a Hashtable accessed via a subscript (§10), a property (§7.1.2), or storage managed by a provider (§3.1).

5.2 Variable categories

PowerShell defines the following categories of variables: static variables, instance variables, array elements, Hashtable key/value pairs, parameters, ordinary variables, and variables on provider drives. The subsections that follow describe each of these categories.

In the following example

PowerShell

```
function F ($p1, $p2) {
    $radius = 2.45
    $circumference = 2 * ([Math]::PI) * $radius

    $date = Get-Date -Date "2010-2-1 10:12:14 pm"
    $month = $date.Month

    $values = 10, 55, 93, 102
    $value = $values[2]

    $h1 = @{ FirstName = "James"; LastName = "Anderson" }
    $h1.FirstName = "Smith"

    $Alias:A = "Help"
    $Env:MyPath = "E:\Temp"
    ${E:output.txt} = 123
    $Function:F = { "Hello there" }
    $Variable:v = 10
}
```

- `[Math::PI]` is a static variable

- `$date.Month` is an instance variable
- `$values[2]` is an array element
- `$h1.FirstName` is a `Hashtable` key whose corresponding value is $h1['FirstName']`
- `$p1` and `$p2` are parameters
- `$radius`, `$circumference`, `$date`, `$month`, `$values`, `$value`, and `$h1` are ordinary variables
- `$Alias:A`, `$Env:MyPath`, `${E:output.txt}`, and `$Function:F` are variables on the corresponding provider drives.
- `$Variable:v` is actually an ordinary variable written with its fully qualified provider drive.

5.2.1 Static variables

A data member of an object that belongs to the object's type rather than to that particular instance of the type is called a *static variable*. See §4.2.3, §4.2.4.1, and §4.3.8 for some examples.

PowerShell provides no way to create new types that contain static variables; however, objects of such types may be provided by the host environment.

Memory for creating and deleting objects containing static variables is managed by the host environment and the garbage collection system.

See §7.1.2 for information about accessing a static variable.

A static data member can be a field or a property.

5.2.2 Instance variables

A data member of an object that belongs to a particular instance of the object's type rather than to the type itself is called an *instance variable*. See §4.3.1, §4.3.2, and §4.3.3 for some examples.

A PowerShell host environment might provide a way to create new types that contain instance variables or to add new instance variables to existing types.

Memory for creating and deleting objects containing static variables is managed by the host environment and the garbage collection system.

See §7.1.2 for information about accessing an instance variable.

An instance data member can be a field or a property.

5.2.3 Array elements

An array can be created via a unary comma operator (§7.2.1), *sub-expression* (§7.1.6), *array-expression* (§7.1.7), binary comma operator (§7.3), range operator (§7.4), or New-Object cmdlet.

Memory for creating and deleting arrays is managed by the host environment and the garbage collection system.

Arrays and array elements are discussed in §9.

5.2.4 Hashtable key/value pairs

A Hashtable is created via a hash literal (§2.3.5.6) or the New-Object cmdlet. A new key/value pair can be added via the `[]` operator (§7.1.4.3).

Memory for creating and deleting Hashtables is managed by the host environment and the garbage collection system.

Hashtables are discussed in §10.

5.2.5 Parameters

A parameter is created when its parent command is invoked, and it is initialized with the value of the argument provided in the invocation or by the host environment. A parameter ceases to exist when its parent command terminates.

Parameters are discussed in §8.10.

5.2.6 Ordinary variables

An *ordinary variable* is defined by an *assignment-expression* (§7.11) or a *foreach-statement* (§8.4.4). Some ordinary variables are predefined by the host environment while others are transient, coming and going as needed at runtime.

The lifetime of an ordinary variable is that part of program execution during which storage is guaranteed to be reserved for it. This lifetime begins at entry into the scope with which it is associated, and ends no sooner than the end of the execution of that scope. If the parent scope is entered recursively or iteratively, a new instance of the local variable is created each time.

The storage referred to by an ordinary variable is reclaimed independently of the lifetime of that variable.

An ordinary variable can be named explicitly with a **Variable:** namespace prefix (§5.2.7).

5.2.7 Variables on provider drives

The concept of providers and drives is introduced in §3.1, with each provider being able to provide its own namespace drive(s). This allows resources on those drives to be accessed as though they were ordinary variables (§5.2.6). In fact, an ordinary variable is stored on the file system provider drive Variable: (§3.1.5) and can be accessed by its ordinary name or its fully qualified namespace name.

Some namespace variable types are constrained implicitly (§5.3).

5.3 Constrained variables

By default, a variable may designate a value of any type. However, a variable may be *constrained* to designating values of a given type by specifying that type as a type literal before its name in an assignment or a parameter. For example,

PowerShell

```
[int]$i = 10    # constrains $i to designating ints only
$i = "Hello"    # error, no conversion to int
$i = "0x10"     # ok, conversion to int
$i = $true      # ok, conversion to int

function F ([int]$p1, [switch]$p2, [regex]$p3) { ... }
```

Any variable belonging to the namespace **Env:**, **Alias:**, or to the file system namespace (§2.3.2, §3.1) is constrained implicitly to the type `string`. Any variable belonging to the namespace **Function:** (§2.3.2, §3.1) is constrained implicitly to the type `scriptblock`.

6. Conversions

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

A *type conversion* is performed when a value of one type is used in a context that requires a different type. If such a conversion happens automatically it is known as *implicit conversion*. (A common example of this is with some operators that need to convert one or more of the values designated by their operands.) Implicit conversion is permitted provided the sense of the source value is preserved, such as no loss of precision of a number when it is converted.

The cast operator (§7.2.9) allows for *explicit conversion*.

Conversions are discussed below, with supplementary information being provided as necessary in the description of each operator in §6.19.

Explicit conversion of a value to the type it already has causes no change to that value or its representation.

The rules for handing conversion when the value of an expression is being bound to a parameter are covered in §6.17.

6.1 Conversion to void

A value of any type can be discarded explicitly by casting it to type void. There is no result.

6.2 Conversion to bool

The rules for converting any value to type bool are as follows:

- A numeric or char value of zero is converted to False; a numeric or char value of non-zero is converted to True.
- A value of null type is converted to False.
- A string of length 0 is converted to False; a string of length > 0 is converted to True.
- A `[switch]` parameter with value `$true` is converted to True, and one with value `$false` is converted to False.
- All other non-null reference type values are converted to True.

If the type implements IList:

- If the object's Length > 2, the value is converted to True.
- If the object's Length is 1 and that first element is not itself an IList, then if that element's value is true, the value is converted to True.
- Otherwise, if the first element's Count >= 1, the value is converted to True.
- Otherwise, the value is converted to False.

6.3 Conversion to char

The rules for converting any value to type char are as follows:

- The conversion of a value of type bool, decimal, float, or double is in error.
- A value of null type is converted to the null (U+0000) character.
- An integer type value whose value can be represented in type char has that value; otherwise, the conversion is in error.
- The conversion of a string value having a length other than 1 is in error.
- A string value having a length 1 is converted to a char having that one character's value.
- A numeric type value whose value after rounding of any fractional part can be represented in the destination type has that rounded value; otherwise, the conversion is in error.
- For other reference type values, if the reference type supports such a conversion, that conversion is used; otherwise, the conversion is in error.

6.4 Conversion to integer

The rules for converting any value to type byte, int, or long are as follows:

- The bool value False is converted to zero; the bool value True is converted to 1.
- A char type value whose value can be represented in the destination type has that value; otherwise, the conversion is in error.
- A numeric type value whose value after rounding of any fractional part can be represented in the destination type has that rounded value; otherwise, the conversion is in error.
- A value of null type is converted to zero.
- A string that represents a number is converted as described in §6.16. If after truncation of the fractional part the result can be represented in the destination type the string is well formed and it has the destination type; otherwise, the conversion is in error. If the string does not represent a number, the conversion is in error.
- For other reference type values, if the reference type supports such a conversion, that conversion is used; otherwise, the conversion is in error.

6.5 Conversion to float and double

The rules for converting any value to type float or double are as follows:

- The bool value False is converted to zero; the bool value True is converted to 1.
- A char value is represented exactly.
- A numeric type value is represented exactly, if possible; however, for int, long, and decimal conversions to float, and for long and decimal conversions to double, some of the least significant bits of the integer value may be lost.
- A value of null type is converted to zero.
- A string that represents a number is converted as described in §6.16; otherwise, the conversion is in error.
- For other reference type values, if the reference type supports such a conversion, that conversion is used; otherwise, the conversion is in error.

6.6 Conversion to decimal

The rules for converting any value to type decimal are as follows:

- The bool value False is converted to zero; the bool value True is converted to 1.
- A char type value is represented exactly.
- A numeric type value is represented exactly; however, if that value is too large or too small to fit in the destination type, the conversion is in error.
- A value of null type is converted to zero.

- A string that represents a number is converted as described in §6.16; otherwise, the conversion is in error.
- For other reference type values, if the reference type supports such a conversion, that conversion is used; otherwise, the conversion is in error.
- The scale of the result of a successful conversion is such that the fractional part has no trailing zeros.

6.7 Conversion to object

The value of any type except the null type (4.1.2) can be converted to type object. The value retains its type and representation.

6.8 Conversion to string

The rules for converting any value to type string are as follows:

- The bool value `$false` is converted to "False"; the bool value `$true` is converted to "True".
- A char type value is converted to a 1-character string containing that char.
- A numeric type value is converted to a string having the form of a corresponding numeric literal. However, the result has no leading or trailing spaces, no leading plus sign, integers have base 10, and there is no type suffix. For a decimal conversion, the scale is preserved. For values of -∞, +∞, and NaN, the resulting strings are "-Infinity", "Infinity", and "NaN", respectively.
- A value of null type is converted to the empty string.
- For a 1-dimensional array, the result is a string containing the value of each element in that array, from start to end, converted to string, with elements being separated by the current Output Field Separator (§2.3.2.2). For an array having elements that are themselves arrays, only the top-level elements are converted. The string used to represent the value of an element that is an array, is implementation defined. For a multi-dimensional array, it is flattened (§9.12) and then treated as a 1-dimensional array.
- A value of null type is converted to the empty string.
- A scriptblock type value is converted to a string containing the text of that block without the delimiting { and } characters.
- For an enumeration type value, the result is a string containing the name of each enumeration constant encoded in that value, separated by commas.
- For other reference type values, if the reference type supports such a conversion, that conversion is used; otherwise, the conversion is in error.

The string used to represent the value of an element that is an array has the form `System.Type[]`, `System.Type[,]`, and so on. For other reference types, the method `ToString` is called. For other enumerable types, the source value is treated like a 1-dimensional array.

6.9 Conversion to array

The rules for converting any value to an array type are as follows:

- The target type may not be a multidimensional array.
- A value of null type is retained as is.
- For a scalar value other than `$null` or a value of type hashtable, a new 1-element array is created whose value is the scalar after conversion to the target element type.
- For a 1-dimensional array value, a new array of the target type is created, and each element is copied with conversion from the source array to the corresponding element in the target array.
- For a multi-dimensional array value, that array is first flattened (§9.12), and then treated as a 1-dimensional array value.
- A string value is converted to an array of char having the same length with successive characters from the string occupying corresponding positions in the array.

For other enumerable types, a new 1-element array is created whose value is the corresponding element after conversion to the target element type, if such a conversion exists. Otherwise, the conversion is in error.

6.10 Conversion to xml

The object is converted to type string and then into an XML Document object of type `xml`.

6.11 Conversion to regex

An expression that designates a value of type string may be converted to type `regex`.

6.12 Conversion to scriptblock

The rules for converting any value to type `scriptblock` are as follows:

- A string value is treated as the name of a command optionally following by arguments to a call to that command.

6.13 Conversion to enumeration types

The rules for converting any value to an enumeration type are as follows:

- A value of type string that contains one of the named values (with regard for case) for an enumeration type is converted to that named value.
- A value of type string that contains a comma-separated list of named values (with regard for case) for an enumeration type is converted to the bitwise-OR of all those named values.

6.14 Conversion to other reference types

The rules for converting any value to a reference type other than an array type or string are as follows:

- A value of null type is retained as is.
- Otherwise, the behavior is implementation defined.

A number of pieces of machinery come in to play here; these include the possible use of single argument constructors or default constructors if the value is a hashtable, implicit and explicit conversion operators, and Parse methods for the target type; the use of Convert.ConvertTo; and the ETS conversion mechanism.

6.15 Usual arithmetic conversions

If neither operand designates a value having numeric type, then

- If the left operand designates a value of type bool, the conversion is in error.
- Otherwise, all operands designating the value `$null` are converted to zero of type int and the process continues with the numeric conversions listed below.
- Otherwise, if the left operand designates a value of type char and the right operand designates a value of type bool, the conversion is in error.
- Otherwise, if the left operand designates a value of type string but does not represent a number (§6.16), the conversion is in error.
- Otherwise, if the right operand designates a value of type string but does not represent a number (§6.16), the conversion is in error.
- Otherwise, all operands designating values of type string are converted to numbers (§6.16), and the process continues with the numeric conversions listed below.
- Otherwise, the conversion is in error.

Numeric conversions:

- If one operand designates a value of type decimal, the value designated by the other operand is converted to that type, if necessary. The result has type decimal.
- Otherwise, if one operand designates a value of type double, the value designated by the other operand is converted to that type, if necessary. The result has type double.
- Otherwise, if one operand designates a value of type float, the values designated by both operands are converted to type double, if necessary. The result has type double.
- Otherwise, if one operand designates a value of type long, the value designated by the other operand value is converted to that type, if necessary. The result has the type first in the sequence long and double that can represent its value.
- Otherwise, the values designated by both operands are converted to type int, if necessary. The result has the first in the sequence int, long, double that can represent its value without truncation.

6.16 Conversion from string to numeric type

Depending on its contents, a string can be converted explicitly or implicitly to a numeric value. Specifically,

- An empty string is converted to the value zero.
- Leading and trailing spaces are ignored; however, a string may not consist of spaces only.
- A string containing only white space and/or line terminators is converted to the value zero.
- One leading + or - sign is permitted.
- An integer number may have a hexadecimal prefix (0x or 0X).
- An optionally signed exponent is permitted.
- Type suffixes and multipliers are not permitted.
- The case-distinct strings "-Infinity", "Infinity", and "NaN" are recognized as the values -∞, +∞, and NaN, respectively.

6.17 Conversion during parameter binding

For information about parameter binding see §8.14.

When the value of an expression is being bound to a parameter, there are extra conversion considerations, as described below:

- If the parameter type is switch (§4.2.5, §8.10.5) and the parameter has no argument, the value of the parameter in the called command is set to `$true`. If the parameter type is other than switch, a parameter having no argument is in error.
- If the parameter type is switch and the argument value is `$null`, the parameter value is set to `$false`.
- If the parameter type is object or is the same as the type of the argument, the argument's value is passed without conversion.
- If the parameter type is not object or scriptblock, an argument having type scriptblock is evaluated and its result is passed as the argument's value. (This is known as *delayed script block binding*.) If the parameter type is object or scriptblock, an argument having type scriptblock is passed as is.
- If the parameter type is a collection of type T2, and the argument is a scalar of type T1, that scalar is converted to a collection of type T2 containing one element. If necessary, the scalar value is converted to type T2 using the conversion rules of this section.
- If the parameter type is a scalar type other than object and the argument is a collection, the argument is in error.
- If the expected parameter type is a collection of type T2, and the argument is a collection of type T1, the argument is converted to a collection of type T2 having the same length as the argument collection. If necessary, the argument collection element values are converted to type T2 using the conversion rules of this section.
- If the steps above and the conversions specified earlier in this chapter do not suffice, the rules in §6.18 are applied. If those fail, the parameter binding fails.

6.18 .NET Conversion

For an implicit conversion, PowerShell's built-in conversions are tried first. If they cannot resolve the conversion, the .NET custom converters below are tried, in order, from top to bottom. If a conversion is found, but it throws an exception, the conversion has failed.

- **PSTypeConverter**: There are two ways of associating the implementation of the **PSTypeConverter** class with its target class: through the type configuration file (types.ps1xml) or by applying the `System.ComponentModel.TypeConverterAttribute` attribute to the target class. Refer to the PowerShell SDK documentation for more information.

- **TypeConverter**: This CLR type provides a unified way of converting types of values to other types, as well as for accessing standard values and sub-properties. The most common type of converter is one that converts to and from a text representation. The

type converter for a class is bound to the class with a `System.ComponentModel.TypeConverterAttribute`. Unless this attribute is overridden, all classes that inherit from this class use the same type converter as the base class. Refer to the PowerShell SDK and the Microsoft .NET framework documentation for more information.

- **Parse Method**: If the source type is string and the destination type has a method called `Parse`, that method is called to perform the conversion.
- **Constructors**: If the destination type has a constructor taking a single argument whose type is that of the source type, that constructor is called to perform the conversion.
- **Implicit Cast Operator**: If the source type has an implicit cast operator that converts to the destination type, that operator is called to perform the conversion.
- **Explicit Cast Operator**: If the source type has an explicit cast operator that converts to the destination type, that operator is called to perform the conversion. If the destination type has an explicit cast operator that converts from the source type, that operator is called to perform the conversion.
- **IConvertable**: `System.Convert.ChangeType` is called to perform the conversion.

6.19 Conversion to ordered

The rules for converting any value to the pseudo-type ordered are as follows:

- If the value is a hash literal (§2.3.5.6), the result is an object with an implementation defined type that behaves like a hashtable and the order of the keys matches the order specified in the hash literal.
- Otherwise, the behavior is implementation defined.

Only hash literals (§2.3.5.6) can be converted to ordered. The result is an instance of `System.Collections.Specialized.OrderedDictionary`.

6.20 Conversion to pscustomobject

The rules for converting any value to the pseudo-type pscustomobject are as follows:

- A value of type hashtable is converted to a PowerShell object. Each key in the hashtable becomes a NoteProperty with the corresponding value.
- Otherwise, the behavior is implementation defined.

The conversion is always allowed but does not change the type of the value.

Last updated on 04/08/2026

7. Expressions

Article • 04/25/2024

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389 That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

Syntax:

Syntax

```
expression:
    primary-expression
    bitwise-expression
    logical-expression
    comparison-expression
    additive-expression
    multiplicative-expression

dash: one of
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)

dashdash:
    dash dash
```

Description:

An *expression* is a sequence of operators and operands that designates a method, a function, a writable location, or a value; specifies the computation of a value; produces one or more side effects; or performs some combination thereof. For example,

- The literal 123 is an expression that designates the int value 123.
- The expression `1,2,3,4` designates the 4-element array object having the values shown.
- The expression `10.4 * $a` specifies a computation.
- The expression `$a++` produces a side effect.
- The expression `$a[$i--] = $b[++$j]` performs a combination of these things.

Except as specified for some operators, the order of evaluation of terms in an expression and the order in which side effects take place are both unspecified. Examples of unspecified behavior include the following: `$i++ + $i`, `$i + --$i`, and `$w[$j++] = $v[$j]`.

An implementation of PowerShell may provide support for user-defined types, and those types may have operations defined on them. All details of such types and operations are implementation defined.

A *top-level expression* is one that is not part of some larger expression. If a top-level expression contains a side-effect operator the value of that expression is not written to the pipeline; otherwise, it is. See §7.1.1 for a detailed discussion of this.

Ordinarily, an expression that designates a collection ([§4§4]) is enumerated into its constituent elements when the value of that expression is used. However, this is not the case when the expression is a cmdlet invocation. For example,

PowerShell

```
$x = 10,20,30
$a = $($x; 99)                      # $a.Length is 4

$x = New-Object 'int[]' 3
$a = $($x; 99)                      # equivalent, $a.Length is 4

$a = $(New-Object 'int[]' 3; 99)    # $a.Length is 2
```

In the first two uses of the `$(...)` operator, the expression designating the collection is the variable `$x`, which is enumerated resulting in three `int` values, plus the `int` 99. However, in the third case, the expression is a direct call to a cmdlet, so the result is not enumerated, and `$a` is an array of two elements, `int[3]` and `int`.

If an operation is not defined by PowerShell, the type of the value designated by the left operand is inspected to see if it has a corresponding `op_<operation>` method.

7.1 Primary expressions

Syntax:

Syntax

```
primary-expression:
    value
    member-access
    element-access
    invocation-expression
    post-increment-expression
    post-decrement-expression

value:
    parenthesized-expression
    sub-expression
    array-expression
    script-block-expression
    hash-literal-expression
    literal
    type-literal
    variable
```

7.1.1 Grouping parentheses

Syntax:

> **Tip**
>
> The `~opt~` notation in the syntax definitions indicates that the lexical entity is optional in the syntax.

Syntax

```
parenthesized-expression:
    ( new-lines~opt~ pipeline new-lines~opt~ )
```

Description:

A parenthesized expression is a *primary-expression* whose type and value are the same as those of the expression without the parentheses. If the expression designates a

variable then the parenthesized expression designates that same variable. For example, `$x.m` and `($x).m` are equivalent.

Grouping parentheses may be used in an expression to document the default precedence and associativity within that expression. They can also be used to override that default precedence and associativity. For example,

PowerShell

```
4 + 6 * 2    # 16
4 + (6 * 2)  # 16 document default precedence
(4 + 6) * 2  # 20 override default precedence
```

Ordinarily, grouping parentheses at the top-most level are redundant. However, that is not always the case. Consider the following example:

PowerShell

```
2,4,6       # Length 3; values 2,4,6
(2,4),6     # Length 2; values [Object[]],int
```

In the second case, the parentheses change the semantics, resulting in an array whose two elements are an array of 2 ints and the scalar int 6.

Here's another exception:

PowerShell

```
23.5/2.4          # pipeline gets 9.79166666666667
$a = 1234 * 3.5   # value not written to pipeline
$a                # pipeline gets 4319
```

In the first and third cases, the value of the result is written to the pipeline. However, although the expression in the second case is evaluated, the result is not written to the pipeline due to the presence of the side-effect operator `=` at the top level. (Removal of the `$a =` part allows the value to be written, as `*` is not a side-effect operator.)

To stop a value of any expression not containing top-level side effects from being written to the pipeline, discard it explicitly, as follows:

PowerShell

```
# None of these value are written to pipeline
[void](23.5/2.4)
[void]$a
```

```
$null = $a
$a > $null
```

To write to the pipeline the value of any expression containing top-level side effects, enclose that expression in parentheses, as follows:

PowerShell

```
($a = 1234 * 3.5) # pipeline gets 4319
```

As such, the grouping parentheses in this case are not redundant.

In the following example, we have variable substitution (§2.3.5.2) taking place in a string literal:

PowerShell

```
">$($a = -23)<"    # value not written to pipeline, get ><
">$(($a = -23))<"  # pipeline gets >-23<
```

In the first case, the parentheses represent a *sub-expression*'s delimiters *not* grouping parentheses, and as the top-level expression contains a side-effect operator, the expression's value is not written to the pipeline. Of course, the `>` and `<` characters are still written.) If grouping parenthesis are added -- as shown in the second case -- writing is enabled.

The following examples each contain top-level side-effect operators:

PowerShell

```
$a = $b = 0      # value not written to pipeline
$a = ($b = 0)    # value not written to pipeline
($a = ($b = 0))  # pipeline gets 0

++$a             # value not written to pipeline
(++$b)           # pipeline gets 1

$a--             # value not written to pipeline
($b--)           # pipeline gets 1
```

The use of grouping parentheses around an expression containing no top-level side effects makes those parentheses redundant. For example;

PowerShell

```
$a      # pipeline gets 0
($a)    # no side effect, so () redundant
```

Consider the following example that has two side effects, neither of which is at the top level:

PowerShell

```
12.6 + ($a = 10 - ++$b) # pipeline gets 21.6.
```

The result is written to the pipeline, as the top-level expression has no side effects.

7.1.2 Member access

Syntax:

Syntax

```
member-access:
    primary-expression . new-line~opt~ member-name
    primary-expression :: new-line~opt~ member-name
```

Note that no whitespace is allowed after *primary-expression*.

Description:

The operator `.` is used to select an instance member from an object, or a key from a `Hashtable`. The left operand must designate an object, and the right operand must designate an accessible instance member.

Either the right operand designates an accessible instance member within the type of the object designated by the left operand or, if the left operand designates an array, the right operand designates accessible instance members within each element of the array.

Whitespace is not permitted before the `.` operator.

This operator is left associative.

The operator `::` is used to select a static member from a given type. The left operand must designate a type, and the right-hand operand must designate an accessible static member within that type.

Whitespace is not permitted before the `::` operator.

This operator is left associative.

If the right-hand operand designates a writable location within the type of the object designated by the left operand, then the whole expression designates a writable location.

Examples:

PowerShell

```
$a = 10, 20, 30
$a.Length                    # get instance property

(10, 20, 30).Length

$property = "Length"
$a.$property                 # property name is a variable

$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123
}
$h1.FirstName                # designates the key FirstName
$h1.Keys                     # gets the collection of keys

[int]::MinValue              # get static property
[double]::PositiveInfinity   # get static property
$property = "MinValue"
[long]::$property            # property name is a variable

foreach ($t in [byte], [int], [long]) {
    $t::MaxValue             # get static property
}

$a = @{ID = 1 }, @{ID = 2 }, @{ID = 3 }
$a.ID                        # get ID from each element in the array
```

7.1.3 Invocation expressions

Syntax:

Syntax

```
invocation-expression:
    primary-expression . new-line~opt~ member-name argument-list
    primary-expression :: new-line~opt~ member-name argument-list

argument-list:
    ( argument-expression-list~opt~ new-lines~opt~ )
```

Note that no whitespace is allowed after *primary-expression*.

Description:

An *invocation-expression* calls the method designated by `primary-expression.member-name` or `primary-expression::member-name`. The parentheses in *argument-list* contain a possibly empty, comma-separated list of expressions that designate the *arguments* whose values are passed to the method. Before the method is called, the arguments are evaluated and converted according to the rules of §6, if necessary, to match the types expected by the method. The order of evaluation of `primary-expression.member-name`, `primary-expression::member-name`, and the arguments is unspecified.

This operator is left associative.

The type of the result of an *invocation-expression* is a *method-designator* (§4.5.24).

Examples:

PowerShell

```
[Math]::Sqrt(2.0)             # call method with argument 2.0
[char]::IsUpper("a")          # call method
$b = "abc#$%XYZabc"
$b.ToUpper()                  # call instance method

[Math]::Sqrt(2)               # convert 2 to 2.0 and call method
[Math]::Sqrt(2D)              # convert 2D to 2.0 and call method
[Math]::Sqrt($true)           # convert $true to 1.0 and call method
[Math]::Sqrt("20")            # convert "20" to 20 and call method

$a = [Math]::Sqrt             # get method descriptor for Sqrt
$a.Invoke(2.0)                # call Sqrt via the descriptor
$a = [Math]::("Sq"+"rt")      # get method descriptor for Sqrt
$a.Invoke(2.0)                # call Sqrt via the descriptor
$a = [char]::ToLower          # get method descriptor for ToLower
$a.Invoke("X")                # call ToLower via the descriptor
```

7.1.4 Element access

Syntax:

Syntax

```
element-access:
    primary-expression [ new-lines~opt~ expression new-lines~opt~ ]
```

Description:

There must not be any whitespace between *primary-expression* and the left square bracket (`[`).

7.1.4.1 Subscripting an array

Description:

Arrays are discussed in detail in §9. If *expression* is a 1-dimensional array, see §7.1.4.5.

When *primary-expression* designates a 1-dimensional array *A*, the operator `[]` returns the element located at `A[0 + expression]` after the value of *expression* has been converted to `int`. The result has the element type of the array being subscripted. If *expression* is negative, `A[expression]` designates the element located at `A[A.Length + expression]`.

When *primary-expression* designates a 2-dimensional array *B*, the operator `[]` returns the element located at `B[0 + row,0 + column]` after the value of the *row* and *column* components of *expression* (which are specified as a comma-separated list) have been converted to `int`. The result has the element type of the array being subscripted. Unlike for a 1-dimensional array, negative positions have no special meaning.

When *primary-expression* designates an array of three or more dimensions, the rules for 2-dimensional arrays apply and the dimension positions are specified as a comma-separated list of values.

If a read access on a non-existing element is attempted, the result is `$null`. It is an error to write to a non-existing element.

For a multidimensional-array subscript expression, the order of evaluation of the dimension position expressions is unspecified. For example, given a 3-dimensional array `$a`, the behavior of `$a[$i++,$i,++$i]` is unspecified.

If *expression* is an array, see §7.1.4.5.

This operator is left associative.

Examples:

PowerShell

```
$a = [int[]](10,20,30) # [int[]], Length 3
$a[1] # returns int 20
$a[20] # no such position, returns $null
$a[-1] # returns int 30, i.e., $a[$a.Length-1]
$a[2] = 5 # changes int 30 to int 5
```

```
$a[20] = 5 # implementation-defined behavior

$a = New-Object 'double[,]' 3,2
$a[0,0] = 10.5 # changes 0.0 to 10.5
$a[0,0]++ # changes 10.5 to 10.6

$list = ("red",$true,10),20,(1.2, "yes")
$list[2][1] # returns string "yes"

$a = @{ A = 10 },@{ B = $true },@{ C = 123.45 }
$a[1]["B"] # $a[1] is a Hashtable, where B is a key

$a = "red","green"
$a[1][4] # returns string "n" from string in $a[1]
```

If a write access to a non-existing element is attempted, an **IndexOutOfRange** exception is raised.

7.1.4.2 Subscripting a string

Description:

When *primary-expression* designates a string *S*, the operator `[]` returns the character located in the zero-based position indicated by *expression*, as a char. If *expression* is greater than or equal to that string's length, the result is `$null`. If *expression* is negative, `S[expression]` designates the element located at `S[S.Length + expression]`.

Examples:

PowerShell

```
$s = "Hello"   # string, Length 5, positions 0-4
$c = $s[1]     # returns "e" as a string
$c = $s[20]    # no such position, returns $null
$c = $s[-1]    # returns "o", i.e., $s[$s.Length-1]
```

7.1.4.3 Subscripting a Hashtable

Description:

When *primary-expression* designates a Hashtable, the operator `[]` returns the value(s) associated with the key(s) designated by *expression*. The type of *expression* is not restricted.

When *expression* is a single key name, the result is the associated value and has that type, unless no such key exists, in which case, the result is `$null`. If `$null` is used as the

key the behavior is implementation defined. If *expression* is an array of key names, see §7.1.4.5.

If *expression* is an array, see §7.1.4.5.

Examples:

PowerShell

```
$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123 }
$h1['FirstName']     # the value associated with key FirstName
$h1['BirthDate']     # no such key, returns $null

$h1 = @{ 10 = "James"; 20.5 = "Anderson"; $true = 123 }
$h1[10]              # returns value "James" using key 10
$h1[20.5]            # returns value "Anderson" using key 20.5
$h1[$true]           # returns value 123 using key $true
```

When *expression* is a single key name, if `$null` is used as the only value to subscript a Hashtable, a **NullArrayIndex** exception is raised.

7.1.4.4 Subscripting an XML document

Description:

When *primary-expression* designates an object of type xml, *expression* is converted to string, if necessary, and the operator `[]` returns the first child element having the name specified by *expression*. The type of *expression* must be string. The type of the result is implementation defined. The result can be subscripted to return its first child element. If no child element exists with the name specified by *expression*, the result is `$null`. The result does not designate a writable location.

Examples:

PowerShell

```
$x = [xml]@"
<Name>
<FirstName>Mary</FirstName>
<LastName>King</LastName>
</Name>
"@

$x['Name']                # refers to the element Name
$x['Name']['FirstName']   # refers to the element FirstName within Name
$x['FirstName']           # No such child element at the top level, result
is $null
```

The type of the result is `System.Xml.XmlElement` or `System.String`.

7.1.4.5 Generating array slices

When *primary-expression* designates an object of a type that is enumerable (§4) or a Hashtable, and *expression* is a 1-dimensional array, the result is an array slice (§9.9) containing the elements of *primary-expression* designated by the elements of *expression*.

In the case of a Hashtable, the array slice contains the associated values to the keys provided, unless no such key exists, in which case, the corresponding element is `$null`. If `$null` is used as any key name the behavior is implementation defined.

Examples:

PowerShell

```
$a = [int[]](30,40,50,60,70,80,90)
$a[1,3,5]                 # slice has Length 3, value 40,60,80
$a[,5]                    # slice with Length 1
$a[@()]                   # slice with Length 0
$a[-1..-3]                # slice with Length 3, value 90,80,70
$a = New-Object 'int[,]' 3,2
$a[0,0] = 10; $a[0,1] = 20; $a[1,0] = 30
$a[1,1] = 40; $a[2,0] = 50; $a[2,1] = 60
$a[(0,1),(1,0)]           # slice with Length 2, value 20,30, parens needed
$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123 }
$h1['FirstName']          # the value associated with key FirstName
$h1['BirthDate']          # no such key, returns $null
$h1['FirstName','IDNum']  # returns [Object[]], Length 2 (James/123)
$h1['FirstName','xxx']    # returns [Object[]], Length 2 (James/$null)
$h1[$null,'IDNum']        # returns [Object[]], Length 2 ($null/123)
```

Windows PowerShell: When *expression* is a collection of two or more key names, if `$null` is used as any key name that key is ignored and has no corresponding element in the resulting array.

7.1.5 Postfix increment and decrement operators

Syntax:

Syntax

```
post-increment-expression:
    primary-expression ++
```

```
post-decrement-expression:
    primary-expression dashdash
```

Description:

The *primary-expression* must designate a writable location having a value of numeric type (§4) or the value `$null`. If the value designated by the operand is `$null`, that value is converted to type int and value zero before the operator is evaluated. The type of the value designated by *primary-expression* may change when the result is stored. See §7.11 for a discussion of type change via assignment.

The result produced by the postfix `++` operator is the value designated by the operand. After that result is obtained, the value designated by the operand is incremented by 1 of the appropriate type. The type of the result of expression `E++` is the same as for the result of the expression `E + 1` (§7.7).

The result produced by the postfix `--` operator is the value designated by the operand. After that result is obtained, the value designated by the operand is decremented by 1 of the appropriate type. The type of the result of expression `E--` is the same as for the result of the expression `E - 1` (§7.7).

These operators are left associative.

Examples:

PowerShell

```
$i = 0                  # $i = 0
$i++                    # $i is incremented by 1
$j = $i--               # $j takes on the value of $i before the decrement

$a = 1,2,3
$b = 9,8,7
$i = 0
$j = 1
$b[$j--] = $a[$i++]     # $b[1] takes on the value of $a[0], then $j is
                        # decremented, $i incremented

$i = 2147483647         # $i holds a value of type int
$i++                    # $i now holds a value of type double because
                        # 2147483648 is too big to fit in type int

[int]$k = 0             # $k is constrained to int
$k = [int]::MaxValue    # $k is set to 2147483647
$k++                    # 2147483648 is too big to fit, imp-def behavior
```

```
$x = $null              # target is unconstrained, $null goes to [int]0
$x++                    # value treated as int, 0->1
```

7.1.6 $(...) operator

Syntax:

Syntax

```
sub-expression:
    $( new-lines~opt~ statement-list~opt~ new-lines~opt~ )
```

Description:

If *statement-list* is omitted, the result is `$null`. Otherwise, *statement-list* is evaluated. Any objects written to the pipeline as part of the evaluation are collected in an unconstrained 1-dimensional array, in order. If the array of collected objects is empty, the result is `$null`. If the array of collected objects contains a single element, the result is that element; otherwise, the result is the unconstrained 1-dimensional array of collected results.

Examples:

PowerShell

```
$j = 20
$($i = 10) # pipeline gets nothing
$(($i = 10)) # pipeline gets int 10
$($i = 10; $j) # pipeline gets int 20
$(($i = 10); $j) # pipeline gets [Object[]](10,20)
$(($i = 10); ++$j) # pipeline gets int 10
$(($i = 10); (++$j)) # pipeline gets [Object[]](10,22)
$($i = 10; ++$j) # pipeline gets nothing
$(2,4,6) # pipeline gets [Object[]](2,4,6)
```

7.1.7 @(...) operator

Syntax:

Syntax

```
array-expression:
    @( new-lines~opt~ statement-list~opt~ new-lines~opt~ )
```

Description:

If *statement-list* is omitted, the result is an unconstrained 1-dimensional array of length zero. Otherwise, *statement-list* is evaluated, and any objects written to the pipeline as part of the evaluation are collected in an unconstrained 1-dimensional array, in order. The result is the (possibly empty) unconstrained 1-dimensional array.

Examples:

PowerShell

```
$j = 20
@($i = 10)              # 10 not written to pipeline, result is array of 0
@(($i = 10))            # pipeline gets 10, result is array of 1
@($i = 10; $j)          # 10 not written to pipeline, result is array of 1
@(($i = 10); $j)        # pipeline gets 10, result is array of 2
@(($i = 10); ++$j)      # pipeline gets 10, result is array of 1
@(($i = 10); (++$j))    # pipeline gets both values, result is array of 2
@($i = 10; ++$j)        # pipeline gets nothing, result is array of 0

$a = @(2,4,6)           # result is array of 3
@($a)                   # result is the same array of 3
@(@($a))                # result is the same array of 3
```

7.1.8 Script block expression

Syntax:

Syntax

```
script-block-expression:
    { new-lines~opt~ script-block new-lines~opt~ }

script-block:
    param-block~opt~ statement-terminators~opt~ script-block-body~opt~

script-block-body:
    named-block-list
    statement-list
```

Description:

param-block is described in §8.10.9. *named-block-list* is described in §8.10.7.

A script block is an unnamed block of statements that can be used as a single unit. Script blocks can be used to invoke a block of code as if it was a single command, or they can be assigned to variables that can be executed.

The *named-block-list* or *statement-list* is executed and the type and value(s) of the result are the type and value(s) of the results of those statement sets.

A *script-block-expression* has type scriptblock (§4.3.7).

If *param-block* is omitted, any arguments passed to the script block are available via `$args` (§8.10.1).

During parameter binding, a script block can be passed either as a script block object or as the result after the script block has been evaluated. See §6.17 for further information.

7.1.9 Hash literal expression

Syntax:

Syntax

```
hash-literal-expression:
    @{ new-lines~opt~ hash-literal-body~opt~ new-lines~opt~ }

hash-literal-body:
    hash-entry
    hash-literal-body statement-terminators hash-entry

hash-entry:
    key-expression = new-lines~opt~ statement

key-expression:
    simple-name
    unary-expression

statement-terminators:
    statement-terminator
    statement-terminators statement-terminator

statement-terminator:
    ;
    new-line-character
```

Description:

A *hash-literal-expression* is used to create a Hashtable (§10) of zero or more elements each of which is a key/value pair.

The key may have any type except the null type. The associated values may have any type, including the null type, and each of those values may be any expression that designates the desired value, including `$null`.

The ordering of the key/value pairs is not significant.

Examples:

PowerShell

```
$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123 }
$last = "Anderson"; $IDNum = 120
$h2 = @{ FirstName = "James"; LastName = $last; IDNum = $IDNum + 3 }
$h3 = @{ }
$h4 = @{ 10 = "James"; 20.5 = "Anderson"; $true = 123 }
```

which creates two Hashtables, `$h1` and `$h2`, each containing three key/value pairs, and a third, `$h3`, that is empty. Hashtable `$h4` has keys of various types.

7.1.10 Type literal expression

Syntax:

Syntax

```
type-literal:
    [ type-spec ]

type-spec:
    array-type-name new-lines~opt~ dimension~opt~ ]
    generic-type-name new-lines~opt~ generic-type-arguments ]
    type-name

dimension:
    ,
    dimension ,

generic-type-arguments:
    type-spec new-lines~opt~
    generic-type-arguments , new-lines~opt~ type-spec

array-type-name:
    type-name [

generic-type-name:
    type-name [
```

Description:

A *type-literal* is represented in an implementation by some unspecified *underlying type*. As a result, a type name is a synonym for its underlying type.

Type literals are used in a number of contexts:

- Specifying an explicit conversion (§6, §7.2.9)
- Creating a type-constrained array (§9.4)
- Accessing the static members of an object (§7.1.2)
- Specifying a type constraint on a variable (§5.3) or a function parameter (§8.10.2)

Examples:

PowerShell

```
[int].IsPrimitive        # $true
[Object[]].FullName      # "System.Object[]"
[int[,,]].GetArrayRank() # 3
```

A generic stack type (§4.4) that is specialized to hold strings might be written as `[Stack[string]]`, and a generic dictionary type that is specialized to hold `int` keys with associated string values might be written as `[Dictionary[int,string]]`.

The type of a *type-literal* is `System.Type`. The complete name for the type `Stack[string]` suggested above is `System.Collections.Generic.Stack[int]`. The complete name for the type `Dictionary[int,string]` suggested above is `System.Collections.Generic.Dictionary[int,string]`.

7.2 Unary operators

Syntax:

Syntax

```
unary-expression:
    primary-expression
    expression-with-unary-operator

expression-with-unary-operator:
    , new-lines~opt~ unary-expression
    -not new-lines~opt~ unary-expression
    ! new-lines~opt~ unary-expression
    -bnot new-lines~opt~ unary-expression
    + new-lines~opt~ unary-expression
    dash new-lines~opt~ unary-expression
    pre-increment-expression
    pre-decrement-expression
    cast-expression
    -split new-lines~opt~ unary-expression
    -join new-lines~opt~ unary-expression
```

```
pre-increment-expression:
    ++ new-lines~opt~ unary-expression

pre-decrement-expression:
    dashdash new-lines~opt~ unary-expression

dash: one of
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)

cast-expression:
    type-literal unary-expression

dashdash:
    dash dash
```

7.2.1 Unary comma operator

Description:

The comma operator (`,`) creates an unconstrained 1-dimensional array having one element, whose type and value are that of *unary-expression*.

This operator is right associative.

Examples:

PowerShell

```
$a = ,10         # create an unconstrained array of 1 element, $a[0],
                 # which has type int

$a = ,(10,"red") # create an unconstrained array of 1 element,
$a[0],
                 # which is an unconstrained array of 2 elements,
                 # $a[0][0] an int, and $a[0][1] a string

$a = ,,10        # create an unconstrained array of 1 element, which is
                 # an unconstrained array of 1 element, which is an int
                 # $a[0][0] is the int. Contrast this with @(@(10))
```

7.2.2 Logical NOT

Syntax:

Syntax

```
logical-not-operator:
    dash not

dash: one of
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The operator `-not` converts the value designated by *unary-expression* to type bool (§6.2), if necessary, and produces a result of that type. If *unary-expression*'s value is True, the result is False, and vice versa. The operator `!` is an alternate spelling for `-not`.

This operator is right associative.

Examples:

PowerShell

```
-not $true          # False
-not -not $false    # False
-not 0              # True
-not 1.23           # False
!"xyz"              # False
```

7.2.3 Bitwise NOT

Syntax:

Syntax

```
bitwise-not-operator:
    dash bnot

dash: one of
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The operator `-bnot` converts the value designated by *unary-expression* to an integer type (§6.4), if necessary. If the converted value can be represented in type **int** then that is

the result type. Else, if the converted value can be represented in type **long** then that is the result type. Otherwise, the expression is ill-formed. The resulting value is the ones-complement of the converted value.

This operator is right associative.

Examples:

```
PowerShell

-bnot $true          # int with value 0xFFFFFFFE
-bnot 10             # int with value 0xFFFFFFF5
-bnot 2147483648.1   # long with value 0xFFFFFFFF7FFFFFFF
-bnot $null          # int with value 0xFFFFFFFF
-bnot "0xabc"        # int with value 0xFFFFF543
```

7.2.4 Unary plus

Description:

An expression of the form `+ unary-expression` is treated as if it were written as `0 + unary-expression` (§7.7). The integer literal `0` has type `int`.

This operator is right associative.

Examples:

```
PowerShell

+123L          # type long, value 123
+0.12340D      # type decimal, value 0.12340
+"0xabc"       # type int, value 2748
```

7.2.5 Unary minus

Description:

An expression of the form `- unary-expression` is treated as if it were written as `0 - unary-expression` (§7.7). The integer literal 0 has type `int`. The minus operator can be any one of the *dash* characters listed in §7.2.

This operator is right associative.

Examples:

PowerShell

```
-$true     # type int, value -1
-123L      # type long, value -123
-0.12340D  # type decimal, value -0.12340
```

7.2.6 Prefix increment and decrement operators

Description:

The *unary-expression* must designate a writable location having a value of numeric type (§4) or the value `$null`. If the value designated by its *unary-expression* is `$null`, *unary-expression*'s value is converted to type int and value zero before the operator is evaluated.

> ⓘ **Note**
>
> The type of the value designated by *unary-expression* may change when the result is stored. See §7.11 for a discussion of type change via assignment.

For the prefix increment operator `++` , the value of *unary-expression* is incremented by `1` of the appropriate type. The result is the new value after incrementing has taken place. The expression `++E` is equivalent to `E += 1` (§7.11.2).

For the prefix decrement operator `--` , the value of *unary-expression* is decremented by `1` of the appropriate type. The result is the new value after decrementing has taken place. The expression `--E` is equivalent to `E -= 1` (§7.11.2). The prefix decrement operator can be any of the patterns matching the *dashdash* pattern in §7.2.

These operators are right associative.

Examples:

PowerShell

```
$i = 0                 # $i = 0
++$i                   # $i is incremented by 1
$j = --$i              # $i is decremented then $j takes on the value of $i

$a = 1,2,3
$b = 9,8,7
$i = 0;
$j = 1
$b[--$j] = $a[++$i]    # $j is # decremented, $i incremented, then $b[0]
                       # takes on the value of $a[1]
```

```
$i = 2147483647       # $i holds a value of type int
++$i                  # $i now holds a value of type double because
                      # 2147483648 is too big to fit in type int

[int]$k = 0           # $k is constrained to int
$k = [int]::MinValue  # $k is set to -2147483648
--$k                  # -2147483649 is too small to fit, imp-def behavior

$x = $null            # target is unconstrained, $null goes to [int]0
--$x                  # value treated as int, 0 becomes -1
```

7.2.7 The unary -join operator

Syntax:

Syntax

```
join-operator:
    dash join

dash: one of
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The unary `-join` operator produces a string that is the concatenation of the value of one or more objects designated by *unary-expression*. (A separator can be inserted by using the binary version of this operator (§7.8.4.4).)

unary-expression can be a scalar value or a collection.

Examples:

PowerShell

```
-join (10, 20, 30)             # result is "102030"
-join (123, $false, 19.34e17)  # result is "123False1.934E+18"
-join 12345                    # result is "12345"
-join $null                    # result is ""
```

7.2.8 The unary -split operator

Syntax:

Syntax

```
split-operator:
    dash split

dash: one of
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The unary `-split` operator splits one or more strings designated by *unary-expression*, returning their subparts in a constrained 1-dimensional array of string. It treats any contiguous group of whitespace characters as the delimiter between successive subparts. An explicit delimiter string can be specified by using the binary version of this operator (§7.8.4.5) or its two variants (§7.8).

The delimiter text is not included in the resulting strings. Leading and trailing whitespace in the input string is ignored. An input string that is empty or contains whitespace only results in an array of one string, which is empty.

unary-expression can designate a scalar value or an array of strings.

Examples:

PowerShell

```
-split " red`tblue`ngreen " # 3 strings: "red", "blue", "green"
-split ("yes no", "up down") # 4 strings: "yes", "no", "up", "down"
-split " " # 1 (empty) string
```

7.2.9 Cast operator

Description:

This operator converts explicitly (§6) the value designated by *unary-expression* to the type designated by *type-literal* (§7.1.10). If *type-literal* is other than void, the type of the result is the named type, and the value is the value after conversion. If *type-literal* is void, no object is written to the pipeline and there is no result.

When an expression of any type is cast to that same type, the resulting type and value is the *unary-expression*'s type and value.

This operator is right associative.

Examples:

PowerShell

```
[bool]-10        # a bool with value True
[int]-10.70D     # a decimal with value -10
[int]10.7        # an int with value 11
[long]"+2.3e+3"  # a long with value 2300
[char[]]"Hello"  # an array of 5 char with values H, e, l, l, and o.
```

7.3 Binary comma operator

Syntax:

Syntax

```
array-literal-expression:
    unary-expression , new-lines~opt~ array-literal-expression
```

Description:

The binary comma operator creates a 1-dimensional array whose elements are the values designated by its operands, in lexical order. The array has unconstrained type.

Examples:

PowerShell

```
2,4,6                   # Length 3; values 2,4,6
(2,4),6                 # Length 2; values [Object[]],int
(2,4,6),12,(2..4)       # Length 3; [Object[]],int,[Object[]]
2,4,6,"red",$null,$true  # Length 6
```

The addition of grouping parentheses to certain binary comma expressions does not document the default precedence; instead, it changes the result.

7.4 Range operator

Syntax:

Syntax

```
range-expression:
    unary-expression .. new-lines~opt~ unary-expression
```

Description:

A *range-expression* creates an unconstrained 1-dimensional array whose elements are the values of the **int** sequence specified by the range bounds. The values designated by the operands are converted to **int**, if necessary (§6.4). The operand designating the lower value after conversion is the *lower bound*, while the operand designating the higher value after conversion is the *upper bound*. Both bounds may be the same, in which case, the resulting array has length `1`. If the left operand designates the lower bound, the sequence is in ascending order. If the left operand designates the upper bound, the sequence is in descending order.

Conceptually, this operator is a shortcut for the corresponding binary comma operator sequence. For example, the range `5..8` can also be generated using `5,6,7,8`. However, if an ascending or descending sequence is needed without having an array, an implementation may avoid generating an actual array. For example, in `foreach ($i in 1..5) { ... }`, no array need be created.

A *range-expression* can be used to specify an array slice (§9.9).

Examples:

PowerShell

```
1..10        # ascending range 1..10
-495..-500   # descending range -495..-500
16..16       # sequence of 1

$x = 1.5
$x..5.40D    # ascending range 2..5

$true..3     # ascending range 1..3
-2..$null    # ascending range -2..0
0xf..0xa     # descending range 15..10
```

7.5 Format operator

Syntax:

Syntax

```
format-expression:
    format-specification-string format-operator new-lines~opt~ range-
expression

format-operator:
    dash f

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

A *format-expression* formats one or more values designated by *range-expression* according to a *format-specification-string* designated by *format-expression*. The positions of the values designated by *range-expression* are numbered starting at zero and increasing in lexical order. The result has type `string`.

A format specification string may contain zero or more format specifications each having the following form:

`{N [ ,M ][ : FormatString ]}`

N represents a (required) *range-expression* value position, *M* represents the (optional) minimum display width, and *FormatString* indicates the (optional) format. If the width of a formatted value exceeds the specified width, the width is increased accordingly. Values whose positions are not referenced in *FormatString* are ignored after being evaluated for any side effects. If *N* refers to a non-existent position, the behavior is implementation defined. Value of type `$null` and void are formatted as empty strings. Arrays are formatted as for *sub-expression* (§7.1.6). To include the characters `{` and `}` in a format specification without their being interpreted as format delimiters, write them as `{{` and `}}`, respectively.

For a complete definition of format specifications, see the type `System.IFormattable` in Ecma Technical Report TR/84.

Examples:

PowerShell

```
"__{0,3}__" -f 5                    # __  5__
"__{0,-3}__" -f 5                   # __5  __
"__{0,3:000}__" -f 5                # __005__
"__{0,5:0.00}__" -f 5.0             # __ 5.00__
```

```
"__{0:C}__" -f 1234567.888                # __$1,234,567.89__
"__{0:C}__" -f -1234.56                   # __($1,234.56)__
"__{0,12:e2}__" -f 123.456e2              # __ 1.23e+004__
"__{0,-12:p}__" -f -0.252                 # __-25.20 % __

$i = 5; $j = 3
"__{0} + {1} <= {2}__" -f $i,$j,($i+$j)   # __5 + 3 <= 8__

$format = "__0x{0:X8}__"
$format -f 65535                          # __0x0000FFFF__
```

In a format specification, if *N* refers to a non-existent position, a **FormatError** is raised.

7.6 Multiplicative operators

Syntax:

Syntax

```
multiplicative-expression:
    multiplicative-expression * new-lines~opt~ format-expression
    multiplicative-expression / new-lines~opt~ format-expression
    multiplicative-expression % new-lines~opt~ format-expression
```

7.6.1 Multiplication

Description:

The result of the multiplication operator `*` is the product of the values designated by the two operands after the usual arithmetic conversions (§6.15) have been applied.

This operator is left associative.

Examples:

PowerShell

```
12 * -10L       # long result -120
-10.300D * 12   # decimal result -123.600
10.6 * 12       # double result 127.2
12 * "0xabc"    # int result 32976
```

7.6.2 String replication

Description:

When the left operand designates a string the binary `*` operator creates a new string that contains the one designated by the left operand replicated the number of times designated by the value of the right operand as converted to integer type (§6.4).

This operator is left associative.

Examples:

```PowerShell
"red" * "3"       # string replicated 3 times
"red" * 4         # string replicated 4 times
"red" * 0         # results in an empty string
"red" * 2.3450D   # string replicated twice
"red" * 2.7       # string replicated 3 times
```

7.6.3 Array replication

Description:

When the left operand designates an array the binary `*` operator creates a new unconstrained 1-dimensional array that contains the value designated by the left operand replicated the number of times designated by the value of the right operand as converted to integer type (§6.4). A replication count of zero results in an array of length 1. If the left operand designates a multidimensional array, it is flattened (§9.12) before being used.

This operator is left associative.

Examples:

```PowerShell
$a = [int[]](10,20)              # [int[]], Length 2*1
$a * "3"                         # [Object[]], Length 2*3
$a * 4                           # [Object[]], Length 2*4
$a * 0                           # [Object[]], Length 2*0
$a * 2.3450D                     # [Object[]], Length 2*2
$a * 2.7                         # [Object[]], Length 2*3
(New-Object 'float[,]' 2,3) * 2  # [Object[]], Length 2*2
```

7.6.4 Division

Description:

The result of the division operator `/` is the quotient when the value designated by the left operand is divided by the value designated by the right operand after the usual arithmetic conversions (§6.15) have been applied.

If an attempt is made to perform integer or decimal division by zero, an implementation-defined terminating error is raised.

This operator is left associative.

Examples:

```PowerShell
10/-10      # int result -1
12/-10      # double result -1.2
12/-10D     # decimal result 1.2
12/10.6     # double result 1.13207547169811
12/"0xabc"  # double result 0.00436681222707424
```

If an attempt is made to perform integer or decimal division by zero, a **RuntimeException** exception is raised.

7.6.5 Remainder

Description:

The result of the remainder operator `%` is the remainder when the value designated by the left operand is divided by the value designated by the right operand after the usual arithmetic conversions (§6.15) have been applied.

If an attempt is made to perform integer or decimal division by zero, an implementation-defined terminating error is raised.

Examples:

```PowerShell
10 % 3          # int result 1
10.0 % 0.3      # double result 0.1
10.00D % "0x4"  # decimal result 2.00
```

If an attempt is made to perform integer or decimal division by zero, a **RuntimeException** exception is raised.

7.7 Additive operators

Syntax:

Syntax

```
additive-expression:
    primary-expression + new-lines~opt~ expression
    primary-expression dash new-lines~opt~ expression

dash: one of
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

7.7.1 Addition

Description:

The result of the addition operator `+` is the sum of the values designated by the two operands after the usual arithmetic conversions (§6.15) have been applied.

This operator is left associative.

Examples:

PowerShell

```
12 + -10L       # long result 2
-10.300D + 12   # decimal result 1.700
10.6 + 12       # double result 22.6
12 + "0xabc"    # int result 2760
```

7.7.2 String concatenation

Description:

When the left operand designates a string the binary `+` operator creates a new string that contains the value designated by the left operand followed immediately by the value(s) designated by the right operand as converted to type string (§6.8).

This operator is left associative.

Examples:

PowerShell

```
"red" + "blue"      # "redblue"
"red" + "123"       # "red123"
"red" + 123         # "red123"
"red" + 123.456e+5  # "red12345600"
"red" + (20,30,40)  # "red20 30 40"
```

7.7.3 Array concatenation

Description:

When the left operand designates an array the binary `+` operator creates a new unconstrained 1-dimensional array that contains the elements designated by the left operand followed immediately by the value(s) designated by the right operand. Multidimensional arrays present in either operand are flattened (§9.12) before being used.

This operator is left associative.

Examples:

PowerShell

```
$a = [int[]](10,20)               # [int[]], Length 2
$a + "red"                        # [Object[]], Length 3
$a + 12.5,$true                   # [Object[]], Length 4
$a + (New-Object 'float[,]' 2,3)  # [Object[]], Length 8
(New-Object 'float[,]' 2,3) + $a  # [Object[]], Length 8
```

7.7.4 Hashtable concatenation

Description:

When both operands designate Hashtables the binary `+` operator creates a new Hashtable that contains the elements designated by the left operand followed immediately by the elements designated by the right operand.

If the Hashtables contain the same key, an implementation-defined terminating error is raised.

This operator is left associative.

Examples:

PowerShell

```
$h1 = @{ FirstName = "James"; LastName = "Anderson" }
$h2 = @{ Dept = "Personnel" }
$h3 = $h1 + $h2      # new Hashtable, Count = 3
```

If the Hashtables contain the same key, an exception of type **BadOperatorArgument** is raised.

7.7.5 Subtraction

Description:

The result of the subtraction operator `-` is the difference when the value designated by the right operand is subtracted from the value designated by the left operand after the usual arithmetic conversions (§6.15) have been applied. The subtraction operator can be any one of the *dash* characters listed in §7.7.

This operator is left associative.

Examples:

PowerShell

```
12 - -10L      # long result 22
-10.300D - 12  # decimal result -22.300
10.6 - 12      # double result -1.4
12 - "0xabc"   # int result -2736
```

7.8 Comparison operators

Syntax:

Syntax

```
comparison-expression:
    primary-expression comparison-operator new-lines~opt~ expression

comparison-operator:
    equality-operator
    relational-operator
    containment-operator
    type-operator
    like-operator
    match-operator
```

Description:

The type of the value designated by the left operand determines how the value designated by the right operand is converted (§6), if necessary, before the comparison is done.

Some comparison operators have two variants, one that is case sensitive (`-c<operator>`), and one that isn't case sensitive (`-i<operator>`). The `-<operator>` version is equivalent to `-i<operator>`. Case sensitivity is meaningful only with comparisons of values of type string. In non-string comparison contexts, the two variants behave the same.

These operators are left associative.

7.8.1 Equality and relational operators

Syntax:

Syntax

```
equality-operator: one of
    dash eq     dash ceq    dash ieq
    dash ne     dash cne    dash ine

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)

relational-operator: one of
    dash lt     dash clt    dash ilt
    dash le     dash cle    dash ile
    dash gt     dash cgt    dash igt
    dash ge     dash cge    dash ige
```

Description:

There are two equality operators: equality (`-eq`) and inequality (`-ne`); and four relational operators: less-than (`-lt`), less-than-or-equal-to (`-le`), greater-than (`-gt`), and greater-than-or-equal-to (`-ge`). Each of these has two variants (§7.8).

For two strings to compare equal, they must have the same length and contents, and letter case, if appropriate.

If the value designated by the left operand is not a collection, the result has type `bool`. Otherwise, the result is a possibly empty unconstrained 1-dimensional array containing

the elements of the collection that test True when compared to the value designated by the right operand.

Examples:

PowerShell

```
10 -eq "010"           # True, int comparison
"010" -eq 10           # False, string comparison
"RED" -eq "Red"        # True, case-insensitive comparison
"RED" -ceq "Red"       # False, case-sensitive comparison
"ab" -lt "abc"         # True

10,20,30,20,10 -ne 20  # 10,30,10, Length 3
10,20,30,20,10 -eq 40  # Length 0
10,20,30,20,10 -ne 40  # 10,20,30,20,10, Length 5
10,20,30,20,10 -gt 25  # 30, Length 1
0,1,30 -ne $true       # 0,30, Length 2
0,"00" -eq "0"         # 0 (int), Length 1
```

7.8.2 Containment operators

Syntax:

Syntax

```
containment-operator: one of
    dash contains       dash ccontains      dash icontains
    dash notcontains    dash cnotcontains   dash inotcontains
    dash in             dash cin            dash iin
    dash notin          dash cnotin         dash inotin

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

There are four containment operators: contains (`-contains`), does-not-contain (`-notcontains`), in (`-in`) and not-in (`-notin`). Each of these has two variants (§7.8).

The containment operators return a result of type bool that indicates whether a value occurs (or does not occur) at least once in the elements of an array. With `-contains` and `-notcontains`, the value is designated by the right operand and the array is designated

by the left operand. With -in and `-notin`, the operands are reversed. The value is designated by the left operand and the array is designated by the right operand.

For the purposes of these operators, if the array operand has a scalar value, the scalar value is treated as an array of one element.

Examples:

PowerShell

```
10,20,30,20,10 -contains 20     # True
10,20,30,20,10 -contains 42.9   # False
10,20,30 -contains "10"         # True
"10",20,30 -contains 10         # True
"010",20,30 -contains 10        # False
10,20,30,20,10 -notcontains 15  # True
"Red",20,30 -ccontains "RED"    # False
```

7.8.3 Type testing and conversion operators

Syntax:

Syntax

```
type-operator: one of
    dash is
    dash as

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The type operator `-is` tests whether the value designated by the left operand has the type, or is derived from a type that has the type, designated by the right operand. The right operand must designate a type or a value that can be converted to a type (such as a string that names a type). The type of the result is `bool`. The type operator `-isnot` returns the logical negation of the corresponding `-is` form.

The type operator `-as` attempts to convert the value designated by the left operand to the type designated by the right operand. The right operand must designate a type or a value that can be converted to a type (such as a string that names a type). If the

conversion fails, `$null` is returned; otherwise, the converted value is returned and the return type of that result is the runtime type of the converted value.

Examples:

PowerShell

```
$a = 10            # value 10 has type int
$a -is [int]       # True

$t = [int]
$a -isnot $t       # False
$a -is "int"       # True
$a -isnot [double] # True

$x = [int[]](10,20)
$x -is [int[]]     # True

$a = "abcd"        # string is derived from object
$a -is [Object]    # True

$x = [double]
foreach ($t in [int],$x,[decimal],"string") {
    $b = (10.60D -as $t) * 2  # results in int 22, double 21.2
}                             # decimal 21.20, and string "10.6010.60"
```

7.8.4 Pattern matching and text manipulation operators

7.8.4.1 The -like and -notlike operators

Syntax:

Syntax

```
like-operator: one of
    dash like       dash clike      dash ilike
    dash notlike    dash cnotlike   dash inotlike

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

If the left operand does not designate a collection, the result has type `bool`. Otherwise, the result is a possibly empty unconstrained 1-dimensional array containing the

elements of the collection that test True when compared to the value designated by the right operand. The right operand may designate a string that contains wildcard expressions (§3.15). These operators have two variants (§7.8).

Examples:

PowerShell

```
"Hello" -like "h*"                    # True, starts with h
"Hello" -clike "h*"                   # False, does not start with lowercase
h
"Hello" -like "*l*"                   # True, has an l in it somewhere
"Hello" -like "??l"                   # False, no length match

"-abc" -like "[-xz]*"                 # True, - is not a range separator
"#$%\^&" -notlike "*[A-Za-z]"         # True, does not end with alphabetic
character
"He" -like "h[aeiou]?*"               # False, need at least 3 characters
"When" -like "*[?]"                   # False, ? is not a wildcard character
"When?" -like "*[?]"                  # True, ? is not a wildcard character

"abc","abbcde","abcgh" -like "abc*"   # object[2], values
"abc" and "abcgh"
```

7.8.4.2 The -match and -notmatch operators

Syntax:

Syntax

```
match-operator: one of
    dash match      dash cmatch     dash imatch
    dash notmatch   dash cnotmatch  dash inotmatch

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

If the left operand does not designate a collection, the result has type `bool` and if that result is `$true`, the elements of the Hashtable `$Matches` are set to the strings that match (or do-not-match) the value designated by the right operand. Otherwise, the result is a possibly empty unconstrained 1-dimensional array containing the elements of the collection that test True when compared to the value designated by the right operand,

and `$Matches` is not set. The right operand may designate a string that contains regular expressions (§3.16), in which case, it is referred to as a *pattern*. These operators have two variants (§7.8).

These operators support submatches (§7.8.4.6).

Examples:

PowerShell

```
"Hello" -match ".l"                    # True, $Matches key/value is 0/"el"
"Hello" -match '\^h.*o$'               # True, $Matches key/value is
0/"Hello"
"Hello" -cmatch '\^h.*o$'              # False, $Matches not set
"abc\^ef" -match ".\\\^e"              # True, $Matches key/value is
0/"c\^e"

"abc" -notmatch "[A-Za-z]"             # False
"abc" -match "[\^A-Za-z]"              # False
"He" -match "h[aeiou]."                # False, need at least 3 characters
"abc","abbcde","abcgh" -match "abc.*"  # Length is 2, values "abc", "abcgh"
```

7.8.4.3 The -replace operator

Syntax:

Syntax

```
binary-replace-operator: one of
    dash replace    dash creplace   dash ireplace

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The `-replace` operator allows text replacement in one or more strings designated by the left operand using the values designated by the right operand. This operator has two variants (§7.8). The right operand has one of the following forms:

- The string to be located, which may contain regular expressions (§3.16). In this case, the replacement string is implicitly "".
- An array of 2 objects containing the string to be located, followed by the replacement string.

If the left operand designates a string, the result has type string. If the left operand designates a 1-dimensional array of string, the result is an unconstrained 1-dimensional array, whose length is the same as for left operand's array, containing the input strings after replacement has completed.

This operator supports submatches (§7.8.4.6).

Examples:

PowerShell

```
"Analogous","an apple" -replace "a","*"      # "*n*logous","*n *pple"
"Analogous" -creplace "[aeiou]","?"          # "An?l?g??s"
"Analogous","an apple" -replace '\^a',"%%A"  # "%%Analogous","%%An apple"
"Analogous" -replace "[aeiou]",'$&$&'        # "AAnaaloogoouus"
```

7.8.4.4 The binary -join operator

Syntax:

Syntax

```
binary-join-operator: one of
    dash join

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The binary `-join` operator produces a string that is the concatenation of the value of one or more objects designated by the left operand after having been converted to string (§6.7), if necessary. The string designated by the right operand is used to separate the (possibly empty) values in the resulting string.

The left operand can be a scalar value or a collection.

Examples:

PowerShell

```
(10, 20, 30) -join "\|"    # result is "10\|20\|30"
12345 -join ","            # result is "12345", no separator needed
```

```
($null,$null) -join "<->"  # result is "<->", two zero-length values
```

7.8.4.5 The binary -split operator

Syntax:

Syntax

```
binary-split-operator: one of
    dash split      dash csplit     dash isplit

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The binary `-split` operator splits one or more strings designated by the left operand, returning their subparts in a constrained 1-dimensional array of string. This operator has two variants (§7.8). The left operand can designate a scalar value or an array of strings. The right operand has one of the following forms:

- A *delimiter string*
- An array of 2 objects containing a delimiter string followed by a numeric *split count*
- An array of 3 objects containing a delimiter string, a numeric split count, and an *options string*
- A script block
- An array of 2 objects containing a script block followed by a numeric split count

The delimiter string may contain regular expressions (§3.16). It is used to locate subparts with the input strings. The delimiter is not included in the resulting strings. If the left operand designates an empty string, that results in an empty string element. If the delimiter string is an empty string, it is found at every character position in the input strings.

By default, all subparts of the input strings are placed into the result as separate elements; however, the split count can be used to modify this behavior. If that count is negative, zero, or greater than or equal to the number of subparts in an input string, each subpart goes into a separate element. If that count is less than the number of subparts in the input string, there are count elements in the result, with the final element containing all of the subparts beyond the first **count - 1** subparts.

An options string contains zero or more *option names* with each adjacent pair separated by a comma. Leading, trailing, and embedded whitespace is ignored. Option names may be in any order and are case-sensitive.

If an options string contains the option name **SimpleMatch**, it may also contain the option name **IgnoreCase**. If an options string contains the option name **RegexMatch** or it does not contain either **RegexMatch** or **SimpleMatch**, it may contain any option name except **SimpleMatch**. However, it must not contain both **Multiline** and **Singleline**.

Here is the set of option names:

Expand table

Option	Description
CultureInvariant	Ignores cultural differences in language when evaluating the delimiter.
ExplicitCapture	Ignores non-named match groups so that only explicit capture groups are returned in the result list.
IgnoreCase	Force case-insensitive matching, even if `-csplit` is used.
IgnorePatternWhitespace	Ignores unescaped whitespace and comments marked with the number sign (`#`).
Multiline	This mode recognizes the start and end of lines and strings. The default mode is **Singleline**.
RegexMatch	Use regular expression matching to evaluate the delimiter. This is the default.
SimpleMatch	Use simple string comparison when evaluating the delimiter.
Singleline	This mode recognizes only the start and end of strings. It is the default mode.

The script block (§7.1.8) specifies the rules for determining the delimiter, and must evaluate to type bool.

Examples:

PowerShell

```
"one,forty two,," -split ","              # 5 strings: "one" "forty two" ""
""

"abc","de" -split ""                      # 9 strings: "" "a" "b" "c" "" ""
"d" "e" ""
```

```
"ab,cd","1,5,7,8" -split ",", 2           # 4 strings: "ab" "cd" "1" "5,7,8"

"10X20x30" -csplit "X", 0, "SimpleMatch"  # 2 strings: "10" "20x30"

"analogous" -split "[AEIOU]", 0, "RegexMatch, IgnoreCase"
                                          # 6 strings: "" "n" "l" "g" "" "s"

"analogous" -split { $_ -eq "a" -or $_ -eq "o" }, 4
                                          # 4 strings: "" "n" "l" "gous"
```

7.8.4.6 Submatches

The pattern being matched by `-match`, `-notmatch`, and `-replace` may contain subparts (called *submatches*) delimited by parentheses. Consider the following example:

```
"red" -match "red"
```

The result is `$true` and key 0 of `$Matches` contains "red", that part of the string designated by the left operand that exactly matched the pattern designated by the right operand.

In the following example, the whole pattern is a submatch:

```
"red" -match "(red)"
```

As before, key 0 contains "red"; however, key 1 also contains "red", which is that part of the string designated by the left operand that exactly matched the submatch.

Consider the following, more complex, pattern:

```
"red" -match "((r)e)(d)"
```

This pattern allows submatches of "re", "r", "d", or "red".

Again, key 0 contains "red". Key 1 contains "re", key 2 contains "r", and key 3 contains "d". The key/value pairs are in matching order from left-to-right in the pattern, with longer string matches preceding shorter ones.

In the case of `-replace`, the replacement text can access the submatches via names of the form `$n`, where the first match is `$1`, the second is `$3`, and so on. For example,

PowerShell

```
"Monday morning" -replace '(Monday|Tuesday)
(morning|afternoon|evening)','the $2 of $1'
```

The resulting string is "the morning of Monday".

Instead of having keys in `$Matches` be zero-based indexes, submatches can be named using the form `?<*name*>`. For example, `"((r)e)(d)"` can be written with three named submatches, `m1`, `m2`, and `m3`, as follows: `"(?<m1>(?<m2>r)e)(?<m3>d)"`.

7.8.5 Shift operators

Syntax:

Syntax

```
shift-operator: one of
    dash shl
    dash shr

dash:
    - (U+002D)
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)
```

Description:

The shift left (`-shl`) operator and shift right (`-shr`) operator convert the value designed by the left operand to an integer type and the value designated by the right operand to int, if necessary, using the usual arithmetic conversions (§6.15).

The shift left operator shifts the left operand left by a number of bits computed as described below. The low-order empty bit positions are set to zero.

The shift right operator shifts the left operand right by a number of bits computed as described below. The low-order bits of the left operand are discarded, the remaining bits shifted right. When the left operand is a signed value, the high-order empty bit positions are set to zero if the left operand is non-negative and set to one if the left operand is negative. When the left operand is an unsigned value, the high-order empty bit positions are set to zero.

When the left operand has type int, the shift count is given by the low-order five bits of the right operand. When the right operand has type long, the shift count is given by the low-order six bits of the right operand.

Examples:

PowerShell

```
0x0408 -shl 1             # int with value 0x0810
0x0408 -shr 3             # int with value 0x0081
0x100000000 -shr 0xfff81  # long with value 0x80000000
```

7.9 Bitwise operators

Syntax:

Syntax

```
bitwise-expression:
    unary-expression -band new-lines~opt~ unary-expression
    unary-expression -bor new-lines~opt~ unary-expression
    unary-expression -bxor new-lines~opt~ unary-expression
```

Description:

The bitwise AND operator `-band`, the bitwise OR operator `-bor`, and the bitwise XOR operator `-bxor` convert the values designated by their operands to integer types, if necessary, using the usual arithmetic conversions (§6.15). After conversion, if both values have type int that is the type of the result. Otherwise, if both values have type long, that is the type of the result. If one value has type **int** and the other has type long, the type of the result is long. Otherwise, the expression is ill formed. The result is the bitwise AND, bitwise OR, or bitwise XOR, respectively, of the possibly converted operand values.

These operators are left associative. They are commutative if neither operand contains a side effect.

Examples:

PowerShell

```
0x0F0F -band 0xFE    # int with value 0xE
0x0F0F -band 0xFEL   # long with value 0xE
0x0F0F -band 14.6    # long with value 0xF

0x0F0F -bor 0xFE     # int with value 0xFFF
0x0F0F -bor 0xFEL    # long with value 0xFFF
0x0F0F -bor 14.40D   # long with value 0xF0F

0x0F0F -bxor 0xFE    # int with value 0xFF1
0x0F0F -bxor 0xFEL   # long with value 0xFF1
0x0F0F -bxor 14.40D  # long with value 0xF01
0x0F0F -bxor 14.6    # long with value 0xF00
```

7.10 Logical operators

Syntax:

Syntax

```
logical-expression:
    unary-expression -and new-lines~opt~ unary-expression
    unary-expression -or new-lines~opt~ unary-expression
    unary-expression -xor new-lines~opt~ unary-expression
```

Description:

The logical AND operator `-and` converts the values designated by its operands to `bool`, if necessary (§6.2). The result is the logical AND of the possibly converted operand values, and has type `bool`. If the left operand evaluates to False the right operand is not evaluated.

The logical OR operator `-or` converts the values designated by its operands to `bool`, if necessary (§6.2). The result is the logical OR of the possibly converted operand values, and has type `bool`. If the left operand evaluates to True the right operand is not evaluated.

The logical XOR operator `-xor` converts the values designated by its operands to `bool` (§6.2). The result is the logical XOR of the possibly converted operand values, and has type `bool`.

These operators are left associative.

Examples:

PowerShell

```
$j = 10
$k = 20
($j -gt 5) -and (++$k -lt 15)   # True -and False -> False
($j -gt 5) -and ($k -le 21)     # True -and True -> True
($j++ -gt 5) -and ($j -le 10)   # True -and False -> False
($j -eq 5) -and (++$k -gt 15)   # False -and True -> False

$j = 10
$k = 20
($j++ -gt 5) -or (++$k -lt 15)  # True -or False -> True
($j -eq 10) -or ($k -gt 15)     # False -or True -> True
($j -eq 10) -or (++$k -le 20)   # False -or False -> False

$j = 10
```

```
$k = 20
($j++ -gt 5) -xor (++$k -lt 15) # True -xor False -> True
($j -eq 10) -xor ($k -gt 15)    # False -xor True -> True
($j -gt 10) -xor (++$k -le 25)  # True -xor True -> False
```

7.11 Assignment operators

Syntax:

Syntax

```
assignment-expression:
    expression assignment-operator statement

assignment-operator: *one of
    =   dash =   +=   *=   /=   %=
```

Description:

An assignment operator stores a value in the writable location designated by *expression*. For a discussion of *assignment-operator* `=` see §7.11.1. For a discussion of all other assignment operators see §7.11.2.

An assignment expression has the value designated by *expression* after the assignment has taken place; however, that assignment expression does not itself designate a writable location. If *expression* is type-constrained (§5.3), the type used in that constraint is the type of the result; otherwise, the type of the result is the type after the usual arithmetic conversions (§6.15) have been applied.

This operator is right associative.

7.11.1 Simple assignment

Description:

In *simple assignment* (`=`), the value designated by *statement* replaces the value stored in the writable location designated by *expression*. However, if *expression* designates a non-existent key in a Hashtable, that key is added to the Hashtable with an associated value of the value designated by *statement*.

As shown by the grammar, *expression* may designate a comma-separated list of writable locations. This is known as *multiple assignment*. *statement* designates a list of one or more comma-separated values. The commas in either operand list are part of the multiple-assignment syntax and do *not* represent the binary comma operator. Values are

taken from the list designated by *statement*, in lexical order, and stored in the corresponding writable location designated by *expression*. If the list designated by *statement* has fewer values than there are *expression* writable locations, the excess locations take on the value `$null`. If the list designated by *statement* has more values than there are *expression* writable locations, all but the right-most *expression* location take on the corresponding *statement* value and the right-most *expression* location becomes an unconstrained 1-dimensional array with all the remaining *statement* values as elements.

For statements that have values (§8.1.2), *statement* can be a statement.

Examples:

PowerShell

```
$a = 20; $b = $a + 12L              # $b has type long, value 22
$hypot = [Math]::Sqrt(3*3 + 4*4)    # type double, value 5
$a = $b = $c = 10.20D               # all have type decimal, value 10.20
$a = (10,20,30),(1,2)               # type [Object[]], Length 2
[int]$x = 10.6                      # type int, value 11
[long]$x = "0xabc"                  # type long, value 0xabc
$a = [float]                        # value type literal [float]
$i,$j,$k = 10,"red",$true           # $i is 10, $j is "red", $k is True
$i,$j = 10,"red",$true              # $i is 10, $j is [Object[]], Length 2
$i,$j = (10,"red"),$true            # $i is [Object[]], Length 2, $j is True
$i,$j,$k = 10                       # $i is 10, $j is $null, $k is $null

$h = @{}
[int] $h.Lower, [int] $h.Upper = -split "10 100"

$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123 }
$h1.Dept = "Finance"                # adds element Finance
$h1["City"] = "New York"            # adds element City

[int]$Variable:v = 123.456          # v takes on the value 123
${E:output.txt} = "a"               # write text to the given file
$Env:MyPath = "X:\data\file.txt"    # define the environment variable
$Function:F = { param ($a, $b) "Hello there, $a, $b" }
F 10 "red"                          # define and invoke a function
function Demo { "Hi there from inside Demo" }
$Alias:A = "Demo"                   # create alias for function Demo
A                                   # invoke function Demo via the alias
```

7.11.2 Compound assignment

Description:

A *compound assignment* has the form `E1 op= E2`, and is equivalent to the simple assignment expression `E1 = E1 op (E2)` except that in the compound assignment case the expression *E1* is evaluated only once. If *expression* is type-constrained (§5.3), the type used in that constraint is the type of the result; otherwise, the type of the result is determined by *op*. For `*=`, see §7.6.1, §7.6.2, §7.6.3; for `/=`, see §7.6.4; for `%=`, see §7.6.5; for `+=`, see §7.7.1, §7.7.2, §7.7.3; for `-=`, see §7.7.5.

> ⓘ **Note**
>
> An operand designating an unconstrained value of numeric type may have its type changed by an assignment operator when the result is stored.

Examples:

PowerShell

```
$a = 1234; $a *= (3 + 2)  # type is int, value is 1234 * (3 + 2)
$b = 10,20,30             # $b[1] has type int, value 20
$b[1] /= 6                # $b[1] has type double, value 3.33...

$i = 0
$b = 10,20,30
$b[++$i] += 2             # side effect evaluated only once

[int]$Variable:v = 10     # v takes on the value 10
$Variable:v -= 3          # 3 is subtracted from v

${E:output.txt} = "a"     # write text to the given file
${E:output.txt} += "b"    # append text to the file giving ab
${E:output.txt} *= 4      # replicate ab 4 times giving abababab
```

7.12 Redirection operators

Syntax:

Syntax

```
pipeline:
    expression redirections~opt~ pipeline-tail~opt~
    command verbatim-command-argument~opt~ pipeline-tail~opt~

redirections:
    redirection
    redirections redirection

redirection:
```

```
    merging-redirection-operator
    file-redirection-operator redirected-file-name

redirected-file-name:
    command-argument
    primary-expression

file-redirection-operator: one of
    >   >>   2>   2>>   3>   3>>   4>   4>>
    5>  5>>  6>   6>>   >    >>    <

merging-redirection-operator: one of
    >&1   2>&1   3>&1   4>&1   5>&1   6>&1
    >&2   1>&2   3>&2   4>&2   5>&2   6>&2
```

Description:

The redirection operator `>` takes the standard output from the pipeline and redirects it to the location designated by *redirected-file-name*, overwriting that location's current contents.

The redirection operator `>>` takes the standard output from the pipeline and redirects it to the location designated by *redirected-file-name*, appending to that location's current contents, if any. If that location does not exist, it is created.

The redirection operator with the form `n>` takes the output of stream *n* from the pipeline and redirects it to the location designated by *redirected-file-name*, overwriting that location's current contents.

The redirection operator with the form `n>>` takes the output of stream *n* from the pipeline and redirects it to the location designated by *redirected-file-name*, appending to that location's current contents, if any. If that location does not exist, it is created.

The redirection operator with the form `m>&n` writes output from stream *m* to the same location as stream *n*.

The following are the valid streams:

Expand table

Stream	Description
1	Standard output stream
2	Error output stream
3	Warning output stream

Stream	Description
4	Verbose output stream
5	Debug output stream
*	Standard output, error output, warning output, verbose output, and debug output streams

The redirection operators `1>&2`, `6>`, `6>>` and `<` are reserved for future use.

If on output the value of *redirected-file-name* is `$null`, the output is discarded.

Ordinarily, the value of an expression containing a top-level side effect is not written to the pipeline unless that expression is enclosed in a pair of parentheses. However, if such an expression is the left operand of an operator that redirects standard output, the value is written.

Examples:

PowerShell

```
$i = 200                        # pipeline gets nothing
$i                              # pipeline gets result
$i > output1.txt                # result redirected to named file
++$i >> output1.txt             # result appended to named file
type file1.txt 2> error1.txt    # error output redirected to named file
type file2.txt 2>> error1.txt   # error output appended to named file
dir -Verbose 4> verbose1.txt    # verbose output redirected to named file

# Send all output to output2.txt
dir -Verbose -Debug -WarningAction Continue *> output2.txt

# error output redirected to named file, verbose output redirected
# to the same location as error output
dir -Verbose 4>&2 2> error2.txt
```

8. Statements

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

8.1 Statement blocks and lists

Syntax:

> **Tip**
>
> The `~opt~` notation in the syntax definitions indicates that the lexical entity is optional in the syntax.

Syntax

```
statement-block:
    new-lines~opt~ { statement-list~opt~ new-lines~opt~ }

statement-list:
    statement
    statement-list statement

statement:
    if-statement
    label~opt~ labeled-statement
    function-statement
```

```
    flow-control-statement statement-terminator
    trap-statement
    try-statement
    data-statement
    inlinescript-statement
    parallel-statement
    sequence-statement
    pipeline statement-terminator

statement-terminator:
    ;
    new-line-character
```

Description:

A *statement* specifies some sort of action that is to be performed. Unless indicated otherwise within this clause, statements are executed in lexical order.

A *statement-block* allows a set of statements to be grouped into a single syntactic unit.

8.1.1 Labeled statements

Syntax:

Syntax

```
labeled-statement:
    switch-statement
    foreach-statement
    for-statement
    while-statement
    do-statement
```

Description:

An iteration statement (§8.4) or a switch statement (§8.6) may optionally be preceded immediately by one statement label, *label*. A statement label is used as the optional target of a break (§8.5.1) or continue (§8.5.2) statement. However, a label does not alter the flow of control.

White space is not permitted between the colon (`:`) and the token that follows it.

Examples:

PowerShell

```
:go_here while ($j -le 100) {
    # ...
}

:labelA
for ($i = 1; $i -le 5; ++$i) {
    :labelB
    for ($j = 1; $j -le 3; ++$j) {
        :labelC
        for ($k = 1; $k -le 2; ++$k) {
            # ...
        }
    }
}
```

8.1.2 Statement values

The value of a statement is the cumulative set of values that it writes to the pipeline. If the statement writes a single scalar value, that is the value of the statement. If the statement writes multiple values, the value of the statement is that set of values stored in elements of an unconstrained 1-dimensional array, in the order in which they were written. Consider the following example:

```
$v = for ($i = 10; $i -le 5; ++$i) { }
```

There are no iterations of the loop and nothing is written to the pipeline. The value of the statement is `$null`.

```
$v = for ($i = 1; $i -le 5; ++$i) { }
```

Although the loop iterates five times nothing is written to the pipeline. The value of the statement is $null.

```
$v = for ($i = 1; $i -le 5; ++$i) { $i }
```

The loop iterates five times each time writing to the pipeline the `int` value `$i`. The value of the statement is `object[]` of Length 5.

```
$v = for ($i = 1; $i -le 5; ) { ++$i }
```

Although the loop iterates five times nothing is written to the pipeline. The value of the statement is `$null`.

```
$v = for ($i = 1; $i -le 5; ) { (++$i) }
```

The loop iterates five times with each value being written to the pipeline. The value of the statement is `object[]` of Length 5.

`$i = 1; $v = while ($i++ -lt 2) { $i }`

The loop iterates once. The value of the statement is the `int` with value 2.

Here are some other examples:

PowerShell

```
# if $count is not currently defined then define it with int value 10
$count = if ($count -eq $null) { 10 } else { $count }

$i = 1
$v = while ($i -le 5) {
    $i                  # $i is written to the pipeline
    if ($i -band 1) {

        "odd"           # conditionally written to the pipeline

    }

    ++$i                # not written to the pipeline

}
# $v is object[], Length 8, value 1,"odd",2,3,"odd",4,5,"odd"
```

8.2 Pipeline statements

Syntax:

Syntax

```
pipeline:
    assignment-expression
    expression redirections~opt~ pipeline-tail~opt~
    command verbatim-command-argument~opt~ pipeline-tail~opt~

assignment-expression:
    expression assignment-operator statement

pipeline-tail:
    | new-lines~opt~ command
    | new-lines~opt~ command pipeline-tail

command:
    command-name command-elements~opt~
    command-invocation-operator command-module~opt~ command-name-expr command-
```

```
elements~opt~

command-invocation-operator: one of
    &   .

command-module:
    primary-expression

command-name:
    generic-token
    generic-token-with-subexpr

generic-token-with-subexpr:
    No whitespace is allowed between ) and command-name.
    generic-token-with-subexpr-start statement-list~opt~ )

command-namecommand-name-expr:
    command-name

primary-expressioncommand-elements:
    command-element
    command-elements command-element

command-element:
    command-parameter
    command-argument
    redirection

command-argument:
    command-name-expr

verbatim-command-argument:
    --% verbatim-command-argument-chars
```

Description:

redirections is discussed in §7.12; *assignment-expression* is discussed in §7.11; and the *command-invocation-operator* dot (`.`) is discussed in §3.5.5. For a discussion of argument-to-parameter mapping in command invocations, see §8.14.

The first command in a *pipeline* is an expression or a command invocation. Typically, a command invocation begins with a *command-name*, which is usually a bare identifier. *command-elements* represents the argument list to the command. A newline or n unescaped semicolon terminates a pipeline.

A command invocation consists of the command's name followed by zero or more arguments. The rules governing arguments are as follows:

- An argument that is not an expression, but which contains arbitrary text without unescaped white space, is treated as though it were double quoted. Letter case is preserved.
- Variable substitution and sub-expression expansion (§2.3.5.2) takes place inside *expandable-string-literal*s and *expandable-here-string-literal*s.
- Text inside quotes allows leading, trailing, and embedded white space to be included in the argument's value. [*Note*: The presence of whitespace in a quoted argument does not turn a single argument into multiple arguments. *end note*]
- Putting parentheses around an argument causes that expression to be evaluated with the result being passed instead of the text of the original expression.
- To pass an argument that looks like a `[switch]` parameter (§2.3.4) but is not intended as such, enclose that argument in quotes.
- When specifying an argument that matches a parameter having the `[switch]` type constraint (§8.10.5), the presence of the argument name on its own causes that parameter to be set to `$true`. However, the parameter's value can be set explicitly by appending a suffix to the argument. For example, given a type constrained parameter *p*, an argument of `-p:$true` sets p to True, while `-p:$false` sets p to False.
- An argument of `--` indicates that all arguments following it are to be passed in their actual form as though double quotes were placed around them.
- An argument of `--%` indicates that all arguments following it are to be passed with minimal parsing and processing. This argument is called the verbatim parameter. Arguments after the verbatim parameter are not PowerShell expressions even if they are syntactically valid PowerShell expressions.

If the command type is Application, the parameter `--%` is not passed to the command. The arguments after `--%` have any environment variables (strings surrounded by `%`) expanded. For example:

PowerShell

```
echoargs.exe --% "%path%" # %path% is replaced with the value $Env:path
```

The order of evaluation of arguments is unspecified.

For information about parameter binding see §8.14. For information about name lookup see §3.8.

Once argument processing has been completed, the command is invoked. If the invoked command terminates normally (§8.5.4), control reverts to the point in the script or function immediately following the command invocation. For a description of the behavior on abnormal termination see `break` (§8.5.1), `continue` (§8.5.2), `throw` (§8.5.3), `exit` (§8.5.5), `try` (§8.7), and `trap` (§8.8).

Ordinarily, a command is invoked by using its name followed by any arguments. However, the command-invocation operator, &, can be used. If the command name contains unescaped white space, it must be quoted and invoked with this operator. As a script block has no name, it too must be invoked with this operator. For example, the following invocations of a command call `Get-Factorial` are equivalent:

PowerShell

```
Get-Factorial 5
& Get-Factorial 5
& "Get-Factorial" 5
```

Direct and indirect recursive function calls are permitted. For example,

PowerShell

```
function Get-Power([int]$x, [int]$y) {
    if ($y -gt 0) { return $x * (Get-Power $x (--$y)) }
    else { return 1 }
}
```

Examples:

PowerShell

```
New-Object 'int[,]' 3,2
New-Object -ArgumentList 3,2 -TypeName 'int[,]'

dir E:\PowerShell\Scripts\*statement*.ps1 | ForEach-Object {$_.Length}

dir E:\PowerShell\Scripts\*.ps1 |
    Select-String -List "catch" |
    Format-Table Path, LineNumber -AutoSize
```

8.3 The if statement

Syntax:

Syntax

```
if-statement:
    if new-lines~opt~ ( new-lines~opt~ pipeline new-lines~opt~ ) statement-block
        elseif-clauses~opt~ else-clause~opt~

elseif-clauses:
    elseif-clause
    elseif-clauses elseif-clause

elseif-clause:
    new-lines~opt~ elseif new-lines~opt~ ( new-lines~opt~ pipeline new-lines~opt~ )
statement-block

else-clause:
    new-lines~opt~ else statement-block
```

Description:

The *pipeline* controlling expressions must have type bool or be implicitly convertible to that type. The *else-clause* is optional. There may be zero or more *elseif-clause*s.

If the top-level *pipeline* tests True, then its *statement-block* is executed and execution of the statement terminates. Otherwise, if an *elseif-clause* is present, if its *pipeline* tests True, then its *statement-block* is executed and execution of the statement terminates. Otherwise, if an *else-clause* is present, its *statement-block* is executed.

Examples:

PowerShell

```
$grade = 92
if ($grade -ge 90) { "Grade A" }
elseif ($grade -ge 80) { "Grade B" }
elseif ($grade -ge 70) { "Grade C" }
elseif ($grade -ge 60) { "Grade D" }
else { "Grade F" }
```

8.4 Iteration statements

8.4.1 The while statement

Syntax:

Syntax

```
while-statement:
    while new-lines~opt~ ( new-lines~opt~ while-condition new-lines~opt~ )
statement-block

while-condition:
    new-lines~opt~ pipeline
```

Description:

The controlling expression *while-condition* must have type bool or be implicitly convertible to that type. The loop body, which consists of *statement-block*, is executed repeatedly until the controlling expression tests False. The controlling expression is evaluated before each execution of the loop body.

Examples:

PowerShell

```
$i = 1
while ($i -le 5) {                    # loop 5 times
    "{0,1}`t{1,2}" -f $i, ($i*$i)
    ++$i
}
```

8.4.2 The do statement

Syntax:

Syntax

```
do-statement:
    do statement-block new-lines~opt~ while new-lines~opt~ ( while-condition new-
lines~opt~ )
    do statement-block new-lines~opt~ until new-lines~opt~ ( while-condition new-
lines~opt~ )

while-condition:
    new-lines~opt~ pipeline
```

Description:

The controlling expression *while-condition* must have type bool or be implicitly convertible to that type. In the while form, the loop body, which consists of *statement-block*, is executed repeatedly while the controlling expression tests True. In the until form, the loop body is

executed repeatedly until the controlling expression tests True. The controlling expression is evaluated after each execution of the loop body.

Examples:

PowerShell

```
$i = 1
do {
    "{0,1}`t{1,2}" -f $i, ($i * $i)
}
while (++$i -le 5)                # loop 5 times

$i = 1
do {
    "{0,1}`t{1,2}" -f $i, ($i * $i)
}
until (++$i -gt 5)                # loop 5 times
```

8.4.3 The for statement

Syntax:

Syntax

```
for-statement:
    for new-lines~opt~ (
        new-lines~opt~ for-initializer~opt~ statement-terminator
        new-lines~opt~ for-condition~opt~ statement-terminator
        new-lines~opt~ for-iterator~opt~
        new-lines~opt~ ) statement-block

    for new-lines~opt~ (
        new-lines~opt~ for-initializer~opt~ statement-terminator
        new-lines~opt~ for-condition~opt~
        new-lines~opt~ ) statement-block

    for new-lines~opt~ (
        new-lines~opt~ for-initializer~opt~
        new-lines~opt~ ) statement-block

for-initializer:
    pipeline

for-condition:
    pipeline

for-iterator:
    pipeline
```

Description:

The controlling expression *for-condition* must have type bool or be implicitly convertible to that type. The loop body, which consists of *statement-block*, is executed repeatedly while the controlling expression tests True. The controlling expression is evaluated before each execution of the loop body.

Expression *for-initializer* is evaluated before the first evaluation of the controlling expression. Expression *for-initializer* is evaluated for its side effects only; any value it produces is discarded and is not written to the pipeline.

Expression *for-iterator* is evaluated after each execution of the loop body. Expression *for-iterator* is evaluated for its side effects only; any value it produces is discarded and is not written to the pipeline.

If expression *for-condition* is omitted, the controlling expression tests True.

Examples:

PowerShell

```
for ($i = 5; $i -ge 1; --$i) { # loop 5 times
    "{0,1}`t{1,2}" -f $i, ($i * $i)
}

$i = 5
for (; $i -ge 1; ) { # equivalent behavior
    "{0,1}`t{1,2}" -f $i, ($i * $i)
    --$i
}
```

8.4.4 The foreach statement

Syntax:

Syntax

```
foreach-statement:
    foreach new-lines~opt~ foreach-parameter~opt~ new-lines~opt~
        ( new-lines~opt~ variable new-lines~opt~ *in* new-lines~opt~ pipeline
        new-lines~opt~ ) statement-block

foreach-parameter:
    -parallel
```

Description:

The loop body, which consists of *statement-block*, is executed for each element designated by the variable *variable* in the collection designated by *pipeline*. The scope of *variable* is not limited to the foreach statement. As such, it retains its final value after the loop body has finished executing. If *pipeline* designates a scalar (excluding the value $null) instead of a collection, that scalar is treated as a collection of one element. If *pipeline* designates the value `$null`, *pipeline* is treated as a collection of zero elements.

If the *foreach-parameter* `-parallel` is specified, the behavior is implementation defined.

The *foreach-parameter* `-parallel` is only allowed in a workflow (§8.10.2).

Every foreach statement has its own enumerator, `$foreach` (§2.3.2.2, §4.5.16), which exists only while that loop is executing.

The objects produced by *pipeline* are collected before *statement-block* begins to execute. However, with the ForEach-Object cmdlet, *statement-block* is executed on each object as it is produced.

Examples:

PowerShell

```
$a = 10, 53, 16, -43
foreach ($e in $a) {
    ...
}
$e # the int value -43

foreach ($e in -5..5) {
    ...
}

foreach ($t in [byte], [int], [long]) {
    $t::MaxValue # get static property
}

foreach ($f in Get-ChildItem *.txt) {
    ...
}

$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123 }
foreach ($e in $h1.Keys) {
    "Key is " + $e + ", Value is " + $h1[$e]
}
```

8.5 Flow control statements

Syntax:

Syntax

```
flow-control-statement:
    break label-expression~opt~
    continue label-expression~opt~
    throw pipeline~opt~
    return pipeline~opt~
    exit pipeline~opt~

label-expression:
    simple-name
    unary-expression
```

Description:

A flow-control statement causes an unconditional transfer of control to some other location.

8.5.1 The break statement

Description:

A break statement with a *label-expression* is referred to as a *labeled break statement*. A break statement without a *label-expression* is referred to as an *unlabeled break statement*.

Outside a trap statement, an unlabeled break statement directly within an iteration statement (§8.4) terminates execution of that smallest enclosing iteration statement. An unlabeled break statement directly within a switch statement (§8.6) terminates pattern matching for the current switch's *switch-condition*. See (§8.8) for details of using break from within a trap statement.

An iteration statement or a switch statement may optionally be preceded immediately by one statement label (§8.1.1).Such a statement label may be used as the target of a labeled break statement, in which case, that statement terminates execution of the targeted enclosing iteration statement.

A labeled break need not be resolved in any local scope; the search for a matching label may continue up the calling stack even across script and function-call boundaries. If no matching label is found, the current command invocation is terminated.

The name of the label designated by *label-expression* need not have a constant value.

If *label-expression* is a *unary-expression*, it is converted to a string.

Examples:

PowerShell

```
$i = 1
while ($true) { # infinite loop
    if ($i * $i -gt 100) {
        break # break out of current while loop
    }
    ++$i
}

$lab = "go_here"
:go_here
for ($i = 1; ; ++$i) {
    if ($i * $i -gt 50) {
        break $lab # use a string value as target
    }
}

:labelA
for ($i = 1; $i -le 2; $i++) {

    :labelB
    for ($j = 1; $j -le 2; $j++) {

        :labelC
        for ($k = 1; $k -le 3; $k++) {
            if (...) { break labelA }
        }
    }
}
```

8.5.2 The continue statement

Description:

A `continue` statement with a *label-expression* is referred to as a *labeled continue statement*. A continue statement without a *label-expression* is referred to as an *unlabeled continue statement*.

The use of `continue` from within a trap statement is discussed in §8.8.

An unlabeled `continue` statement within a loop terminates execution of the current loop and transfers control to the closing brace of the smallest enclosing iteration statement (§8.4). An unlabeled `continue` statement within a switch terminates execution of the current `switch` iteration and transfers control to the smallest enclosing `switch`'s *switch-condition* (§8.6).

An iteration statement or a `switch` statement (§8.6) may optionally be preceded immediately by one statement label (§8.1.1). Such a statement label may be used as the target of an enclosed labeled `continue` statement, in which case, that statement terminates execution of the current loop or `switch` iteration, and transfers control to the targeted enclosing iteration or `switch` statement label.

A labeled `continue` need not be resolved in any local scope; the search for a matching label may `continue` up the calling stack even across script and function-call boundaries. If no matching label is found, the current command invocation is terminated.

The name of the label designated by *label-expression* need not have a constant value.

If *label-expression* is a *unary-expression*, it is converted to a string.

Examples:

PowerShell

```
$i = 1
while (...) {
    ...
    if (...) {
        continue # start next iteration of current loop
    }
    ...
}

$lab = "go_here"
:go_here
for (...; ...; ...) {
    if (...) {
        continue $lab # start next iteration of labeled loop
    }
}

:labelA
for ($i = 1; $i -le 2; $i++) {

    :labelB
    for ($j = 1; $j -le 2; $j++) {

        :labelC
        for ($k = 1; $k -le 3; $k++) {
            if (...) { continue labelB }
        }
    }
}
```

8.5.3 The throw statement

Description:

An exception is a way of handling a system- or application-level error condition. The throw statement raises an exception. (See §8.7 for a discussion of exception handling.)

If *pipeline* is omitted and the throw statement is not in a *catch-clause*, the behavior is implementation defined. If *pipeline* is present and the throw statement is in a *catch-clause*, the exception that was caught by that *catch-clause* is re-thrown after any *finally-clause* associated with the *catch-clause* is executed.

If *pipeline* is present, the type of the exception thrown is implementation defined.

When an exception is thrown, control is transferred to the first catch clause in an enclosing try statement that can handle the exception. The location at which the exception is thrown initially is called the *throw point*. Once an exception is thrown the steps described in §8.7 are followed repeatedly until a catch clause that matches the exception is found or none can be found.

Examples:

PowerShell

```
throw
throw 100
throw "No such record in file"
```

If *pipeline* is omitted and the throw statement is not from within a *catch-clause*, the text "ScriptHalted" is written to the pipeline, and the type of the exception raised is `System.Management.Automation.RuntimeException`.

If *pipeline* is present, the exception raised is wrapped in an object of type `System.Management.Automation.RuntimeException`, which includes information about the exception as a `System.Management.Automation.ErrorRecord` object (accessible via `$_`).

Example 1: `throw 123` results in an exception of type **RuntimeException**. From within the catch block, `$_.TargetObject` contains the object wrapped inside, in this case, a `System.Int32` with value 123.

Example 2: `throw "xxx"` results in an exception of type **RuntimeException**. From within the catch block, `$_.TargetObject` contains the object wrapped inside, in this case, a `System.String` with value "xxx".

Example 3: `throw 10,20` results in an exception of type **RuntimeException**. From within the catch block, `$_.TargetObject` contains the object wrapped inside, in this case, a `System.Object[]`, an unconstrained array of two elements with the `System`.Int32` values 10 and 20.

8.5.4 The return statement

Description:

The `return` statement writes to the pipeline the value(s) designated by *pipeline*, if any, and returns control to the function or script's caller. A function or script may have zero or more `return` statements.

If execution reaches the closing brace of a function an implied `return` without *pipeline* is assumed.

The `return` statement is a bit of "syntactic sugar" to allow programmers to express themselves as they can in other languages; however, the value returned from a function or script is actually all of the values written to the pipeline by that function or script plus any value(s) specified by *pipeline*. If only a scalar value is written to the pipeline, its type is the type of the value returned; otherwise, the return type is an unconstrained 1-dimensional array containing all the values written to the pipeline.

Examples:

PowerShell

```powershell
function Get-Factorial ($v) {
    if ($v -eq 1) {
        return 1 # return is not optional
    }

    return $v * (Get-Factorial ($v - 1)) # return is optional
}
```

The caller to `Get-Factorial` gets back an `int`.

PowerShell

```powershell
function Test {
    "text1" # "text1" is written to the pipeline
    # ...
    "text2" # "text2" is written to the pipeline
    # ...
```

```
    return 123 # 123 is written to the pipeline
}
```

The caller to `Test` gets back an unconstrained 1-dimensional array of three elements.

8.5.5 The exit statement

Description:

The exit statement terminates the current script and returns control and an *exit code* to the host environment or the calling script. If *pipeline* is provided, the value it designates is converted to int, if necessary. If no such conversion exists, or if *pipeline* is omitted, the int value zero is returned.

Examples:

PowerShell

```
exit $count # terminate the script with some accumulated count
```

8.6 The switch statement

Syntax:

Syntax

```
switch-statement:
    switch new-lines~opt~ switch-parameters~opt~ switch-condition switch-body

switch-parameters:
    switch-parameter
    switch-parameters switch-parameter

switch-parameter:
    -Regex
    -Wildcard
    -Exact
    -CaseSensitive
    -Parallel

switch-condition:
    ( new-lines~opt~ pipeline new-lines~opt~ )
    -File new-lines~opt~ switch-filename

switch-filename:
    command-argument
    primary-expression
```

```
switch-body:
    new-lines~opt~ { new-lines~opt~ switch-clauses }

switch-clauses:
    switch-clause
    switch-clauses switch-clause

switch-clause:
    switch-clause-condition statement-block statement-terimators~opt~

switch-clause-condition:
    command-argument
    primary-expression
```

Description:

If *switch-condition* designates a single value, control is passed to one or more matching pattern statement blocks. If no patterns match, some default action can be taken.

A switch must contain one or more *switch-clause*s, each starting with a pattern (a *non-default switch clause*), or the keyword `default` (a *default switch clause*). A switch must contain zero or one `default` switch clauses, and zero or more non-default switch clauses. Switch clauses may be written in any order.

Multiple patterns may have the same value. A pattern need not be a literal, and a switch may have patterns with different types.

If the value of *switch-condition* matches a pattern value, that pattern's *statement-block* is executed. If multiple pattern values match the value of *switch-condition*, each matching pattern's *statement-block* is executed, in lexical order, unless any of those *statement-block*s contains a `break` statement (§8.5.1).

If the value of *switch-condition* does not match any pattern value, if a `default` switch clause exists, its *statement-block* is executed; otherwise, pattern matching for that *switch-condition* is terminated.

Switches may be nested, with each switch having its own set of switch clauses. In such instances, a switch clause belongs to the innermost switch currently in scope.

On entry to each *statement-block*, `$_` is automatically assigned the value of the *switch-condition* that caused control to go to that *statement-block*. `$_` is also available in that *statement-block*'s *switch-clause-condition*.

Matching of non-strings is done by testing for equality (§7.8.1).

If the matching involves strings, by default, the comparison is case-insensitive. The presence of the *switch-parameter* `-CaseSensitive` makes the comparison case-sensitive.

A pattern may contain wildcard characters (§3.15), in which case, wildcard string comparisons are performed, but only if the *switch-parameter* `-Wildcard` is present. By default, the comparison is case-insensitive.

A pattern may contain a regular expression (§3.16), in which case, regular expression string comparisons are performed, but only if the *switch-parameter* `-Regex` is present. By default, the comparison is case-insensitive. If `-Regex` is present and a pattern is matched, `$Matches` is defined in the *switch-clause statement-block* for that pattern.

A *switch-parameter* may be abbreviated; any distinct leading part of a parameter may be used. For example, `-Regex`, `-Rege`, `-Reg`, `-Re`, and `-R` are equivalent.

If conflicting *switch-parameter*s are specified, the lexically final one prevails. The presence of `-Exact` disables `-Regex` and `-Wildcard`; it has no affect on `-Case`, however.

If the *switch-parameter* `-Parallel` is specified, the behavior is implementation defined.

The *switch-parameter* `-Parallel` is only allowed in a workflow (§8.10.2).

If a pattern is a *script-block-expression*, that block is evaluated and the result is converted to bool, if necessary. If the result has the value `$true`, the corresponding *statement-block* is executed; otherwise, it is not.

If *switch-condition* designates multiple values, the switch is applied to each value in lexical order using the rules described above for a *switch-condition* that designates a single value. Every switch statement has its own enumerator, `$switch` (§2.3.2.2, §4.5.16), which exists only while that switch is executing.

A switch statement may have a label, and it may contain labeled and unlabeled break (§8.5.1) and continue (§8.5.2) statements.

If *switch-condition* is `-File` *switch-filename*, instead of iterating over the values in an expression, the switch iterates over the values in the file designated by *switch-filename*.The file is read a line at a time with each line comprising a value. Line terminator characters are not included in the values.

Examples:

PowerShell

```
$s = "ABC def`nghi`tjkl`fmno @#$"
$charCount = 0; $pageCount = 0; $lineCount = 0; $otherCount = 0
for ($i = 0; $i -lt $s.Length; ++$i) {
    ++$charCount
    switch ($s[$i]) {
        "`n" { ++$lineCount }
        "`f" { ++$pageCount }
        "`t" { }
        " " { }
        default { ++$otherCount }
    }
}

switch -Wildcard ("abc") {
    a* { "a*, $_" }
    ?B? { "?B? , $_" }
    default { "default, $_" }
}

switch -Regex -CaseSensitive ("abc") {
    ^a* { "a*" }
    ^A* { "A*" }
}

switch (0, 1, 19, 20, 21) {
    { $_ -lt 20 } { "-lt 20" }
    { $_ -band 1 } { "Odd" }
    { $_ -eq 19 } { "-eq 19" }
    default { "default" }
}
```

8.7 The try/finally statement

Syntax:

Syntax

```
try-statement:
    try statement-block catch-clauses
    try statement-block finally-clause
    try statement-block catch-clauses finally-clause

catch-clauses:
    catch-clause
    catch-clauses catch-clause

catch-clause:
    new-lines~opt~ catch catch-type-list~opt~
    statement-block

catch-type-list:
```

```
    new-lines~opt~ type-literal
    catch-type-list new-lines~opt~ , new-lines~opt~

type-literalfinally-clause:
    new-lines~opt~ finally statement-block
```

Description:

The try statement provides a mechanism for catching exceptions that occur during execution of a block. The try statement also provides the ability to specify a block of code that is always executed when control leaves the try statement. The process of raising an exception via the throw statement is described in §8.5.3.

A *try block* is the *statement-block* associated with the try statement. A *catch block* is the *statement-block* associated with a *catch-clause*. A *finally block* is the *statement-block* associated with a *finally-clause*.

A *catch-clause* without a *catch-type-list* is called a *general catch clause*.

Each *catch-clause* is an *exception handler*, and a *catch-clause* whose *catch-type-list* contains the type of the raised exception is a *matching catch clause*. A general catch clause matches all exception types.

Although *catch-clauses* and *finally-clause* are optional, at least one of them must be present.

The processing of a thrown exception consists of evaluating the following steps repeatedly until a catch clause that matches the exception is found.

- In the current scope, each try statement that encloses the throw point is examined. For each try statement *S*, starting with the innermost try statement and ending with the outermost try statement, the following steps are evaluated:
    - If the `try` block of *S* encloses the throw point and if *S* has one or more catch clauses, the catch clauses are examined in lexical order to locate a suitable handler for the exception. The first catch clause that specifies the exception type or a base type of the exception type is considered a match. A general catch clause is considered a match for any exception type. If a matching catch clause is located, the exception processing is completed by transferring control to the block of that catch clause. Within a matching catch clause, the variable `$_` contains a description of the current exception.
    - Otherwise, if the `try` block or a `catch` block of *S* encloses the throw point and if *S* has a `finally` block, control is transferred to the finally block. If the `finally` block throws

another exception, processing of the current exception is terminated. Otherwise, when control reaches the end of the `finally` block, processing of the current exception is continued.

- If an exception handler was not located in the current scope, the steps above are then repeated for the enclosing scope with a throw point corresponding to the statement from which the current scope was invoked.

- If the exception processing ends up terminating all scopes, indicating that no handler exists for the exception, then the behavior is unspecified.

To prevent unreachable catch clauses in a try block, a catch clause may not specify an exception type that is equal to or derived from a type that was specified in an earlier catch clause within that same try block.

The statements of a `finally` block are always executed when control leaves a `try` statement. This is true whether the control transfer occurs as a result of normal execution, as a result of executing a `break`, `continue`, or `return` statement, or as a result of an exception being thrown out of the `try` statement.

If an exception is thrown during execution of a `finally` block, the exception is thrown out to the next enclosing `try` statement. If another exception was in the process of being handled, that exception is lost. The process of generating an exception is further discussed in the description of the `throw` statement.

`try` statements can co-exist with `trap` statements; see §8.8 for details.

Examples:

PowerShell

```
$a = New-Object 'int[]' 10
$i = 20 # out-of-bounds subscript

while ($true) {
    try {
        $a[$i] = 10
        "Assignment completed without error"
        break
    }

    catch [IndexOutOfRangeException] {
        "Handling out-of-bounds index, >$_<`n"
        $i = 5
    }
```

```
    catch {
        "Caught unexpected exception"
    }

    finally {
        # ...
    }
}
```

Each exception thrown is raised as a `System.Management.Automation.RuntimeException`. If there are type-specific *catch-clause*s in the `try` block, the **InnerException** property of the exception is inspected to try and find a match, such as with the type `System.IndexOutOfRangeException` above.

8.8 The trap statement

Syntax:

Syntax

```
trap-statement:
    *trap* new-lines~opt~ type-literal~opt~ new-lines~opt~ statement-block
```

Description:

A `trap` statement with and without *type-literal* is analogous to a `catch` block (§8.7) with and without *catch-type-list*, respectively, except that a `trap` statement can trap only one type at a time.

Multiple `trap` statements can be defined in the same *statement-block*, and their order of definition is irrelevant. If two `trap` statements with the same *type-literal* are defined in the same scope, the lexically first one is used to process an exception of matching type.

Unlike a `catch` block, a `trap` statement matches an exception type exactly; no derived type matching is performed.

When an exception occurs, if no matching `trap` statement is present in the current scope, a matching trap statement is searched for in the enclosing scope, which may involve looking in the calling script, function, or filter, and then in its caller, and so on. If the lookup ends up terminating all scopes, indicating that no handler exists for the exception, then the behavior is unspecified.

A `trap` statement's *statement-body* only executes to process the corresponding exception; otherwise, execution passes over it.

If a `trap`'s *statement-body* exits normally, by default, an error object is written to the error stream, the exception is considered handled, and execution continues with the statement immediately following the one in the scope containing the `trap` statement that made the exception visible. The cause of the exception might be in a command called by the command containing the `trap` statement.

If the final statement executed in a `trap`'s *statement-body* is continue (§8.5.2), the writing of the error object to the error stream is suppressed, and execution continues with the statement immediately following the one in the scope containing the trap statement that made the exception visible. If the final statement executed in a `trap`'s *statement-body* is break (§8.5.1), the writing of the error object to the error stream is suppressed, and the exception is re-thrown.

Within a `trap` statement the variable `$_` contains a description of the current error.

Consider the case in which an exception raised from within a `try` block does not have a matching `catch` block, but a matching `trap` statement exists at a higher block level. After the `try` block's finally clause is executed, the `trap` statement gets control even if any parent scope has a matching `catch` block. If a `trap` statement is defined within the `try` block itself, and that `try` block has a matching `catch` block, the `trap` statement gets control.

Examples:

In the following example, the error object is written and execution continues with the statement immediately following the one that caused the trap; that is, "Done" is written to the pipeline.

PowerShell

```
$j = 0; $v = 10/$j; "Done"
trap { $j = 2 }
```

In the following example, the writing of the error object is suppressed and execution continues with the statement immediately following the one that caused the trap; that is, "Done" is written to the pipeline.

PowerShell

```
$j = 0; $v = 10/$j; "Done"
trap { $j = 2; continue }
```

In the following example, the writing of the error object is suppressed and the exception is re-thrown.

PowerShell

```
$j = 0; $v = 10/$j; "Done"
trap { $j = 2; break }
```

In the following example, the trap and exception-generating statements are in the same scope. After the exception is caught and handled, execution resumes with writing 1 to the pipeline.

PowerShell

```
&{trap{}; throw '\...'; 1}
```

In the following example, the trap and exception-generating statements are in different scopes. After the exception is caught and handled, execution resumes with writing 2 (not 1) to the pipeline.

PowerShell

```
trap{} &{throw '\...'; 1}; 2
```

8.9 The data statement

Syntax:

Syntax

```
data-statement:
    data new-lines~opt~ data-name data-commands-allowed~opt~ statement-block

data-name:
    simple-name

data-commands-allowed:
    new-lines~opt~ -SupportedCommand data-commands-list

data-commands-list:
    new-lines~opt~ data-command
    data-commands-list , new-lines~opt~ data-command
```

```
data-command:
    command-name-expr
```

Description:

A data statement creates a *data section*, keeping that section's data separate from the code. This separation supports facilities like separate string resource files for text, such as error messages and Help strings. It also helps support internationalization by making it easier to isolate, locate, and process strings that will be translated into different languages.

A script or function can have zero or more data sections.

The *statement-block* of a data section is limited to containing the following PowerShell features only:

- All operators except `-match`
- The `if` statement
- The following automatic variables: `$PSCulture`, `$PSUICulture`, `$true`, `$false`, and `$null`.
- Comments
- Pipelines
- Statements separated by semicolons (`;`)
- Literals
- Calls to the ConvertFrom-StringData cmdlet
- Any other cmdlets identified via the **SupportedCommand** parameter

If the `ConvertFrom-StringData` cmdlet is used, the key/value pairs can be expressed using any form of string literal. However, *expandable-string-literal*s and *expandable-here-string-literal*s must not contain any variable substitutions or sub-expression expansions.

Examples:

The **SupportedCommand** parameter indicates that the given cmdlets or functions generate data only. For example, the following data section includes a user-written cmdlet, `ConvertTo-Xml`, which formats data in an XML file:

PowerShell

```powershell
data -SupportedCommand ConvertTo-Xml {
    Format-Xml -Strings string1, string2, string3
}
```

Consider the following example, in which the data section contains a `ConvertFrom-StringData` command that converts the strings into a hash table, whose value is assigned to `$messages`.

PowerShell

```
$messages = data {
    ConvertFrom-StringData -StringData @'
    Greeting = Hello
    Yes = yes
    No = no
'@
}
```

The keys and values of the hash table are accessed using `$messages.Greeting`, `$messages.Yes`, and `$messages.No`, respectively.

Now, this can be saved as an English-language resource. German- and Spanish-language resources can be created in separate files, with the following data sections:

PowerShell

```
$messages = data {
    ConvertFrom-StringData -StringData @"
    Greeting = Guten Tag
    Yes = ja
    No = nein
"@
}

$messagesS = data {
    ConvertFrom-StringData -StringData @"
    Greeting = Buenos días
    Yes = sí
    No = no
"@
}
```

If *dataname* is present, it names the variable (without using a leading `$`) into which the value of the data statement is to be stored. Specifically, `$name = data { ... }` is equivalent to `data name { ... }`.

8.10 Function definitions

Syntax:

Syntax

```
function-statement:
    function new-lines~opt~ function-name function-parameter-declaration~opt~ {
script-block }
    filter new-lines~opt~ function-name function-parameter-declaration~opt~ {
script-block }
    workflow new-lines~opt~ function-name function-parameter-declaration~opt~ {
script-block }

function-name:
    command-argument

command-argument:
    command-name-expr

function-parameter-declaration:
    new-lines~opt~ ( parameter-list new-lines~opt~ )

parameter-list:
    script-parameter
    parameter-list new-lines~opt~ , script-parameter

script-parameter:
    new-lines~opt~ attribute-list~opt~ new-lines~opt~ variable script-parameter-
default~opt~

script-block:
    param-block~opt~ statement-terminators~opt~ script-block-body~opt~

param-block:
    new-lines~opt~ attribute-list~opt~ new-lines~opt~ param new-lines~opt~
        ( parameter-list~opt~ new-lines~opt~ )

parameter-list:
    script-parameter
    parameter-list new-lines~opt~ , script-parameter

script-parameter-default:
    new-lines~opt~ = new-lines~opt~ expression

script-block-body:
    named-block-list
    statement-list

named-block-list:
    named-block
    named-block-list named-block

named-block:
    block-name statement-block statement-terminators~opt~

block-name: one of
    dynamicparam   begin   process   end
```

Description:

A *function definition* specifies the name of the function, filter, or workflow being defined and the names of its parameters, if any. It also contains zero or more statements that are executed to achieve that function's purpose.

Each function is an instance of the class `System.Management.Automation.FunctionInfo`.

8.10.1 Filter functions

Whereas an ordinary function runs once in a pipeline and accesses the input collection via `$input`, a *filter* is a special kind of function that executes once for each object in the input collection. The object currently being processed is available via the variable `$_`.

A filter with no named blocks (§8.10.7) is equivalent to a function with a process block, but without any begin block or end block.

Consider the following filter function definition and calls:

PowerShell

```
filter Get-Square2 { # make the function a filter
    $_ * $_ # access current object from the collection
}

-3..3 | Get-Square2 # collection has 7 elements
6, 10, -3 | Get-Square2 # collection has 3 elements
```

Each filter is an instance of the class `System.Management.Automation.FilterInfo` (§4.5.11).

8.10.2 Workflow functions

A workflow function is like an ordinary function with implementation defined semantics. A workflow function is translated to a sequence of Windows Workflow Foundation activities and executed in the Windows Workflow Foundation engine.

8.10.3 Argument processing

Consider the following definition for a function called `Get-Power`:

PowerShell

```
function Get-Power ([long]$Base, [int]$Exponent) {
    $result = 1
```

```
    for ($i = 1; $i -le $Exponent; ++$i) {
        $result *= $Base
    }
    return $result
}
```

This function has two parameters, `$Base` and `$Exponent`. It also contains a set of statements that, for non-negative exponent values, computes `$Base^$Exponent^` and returns the result to `Get-Power`'s caller.

When a script, function, or filter begins execution, each parameter is initialized to its corresponding argument's value. If there is no corresponding argument and a default value (§8.10.4) is supplied, that value is used; otherwise, the value `$null` is used. As such, each parameter is a new variable just as if it was initialized by assignment at the start of the *script-block*.

If a *script-parameter* contains a type constraint (such as `[long]` and `[int]` above), the value of the corresponding argument is converted to that type, if necessary; otherwise, no conversion occurs.

When a script, function, or filter begins execution, variable `$args` is defined inside it as an unconstrained 1-dimensional array, which contains all arguments not bound by name or position, in lexical order.

Consider the following function definition and calls:

PowerShell

```
function F ($a, $b, $c, $d) { ... }

F -b 3 -d 5 2 4       # $a is 2, $b is 3, $c is 4, $d is 5, $args Length 0
F -a 2 -d 3 4 5       # $a is 2, $b is 4, $c is 5, $d is 3, $args Length 0
F 2 3 4 5 -c 7 -a 1   # $a is 1, $b is 2, $c is 7, $d is 3, $args Length 2
```

For more information about parameter binding see §8.14.

8.10.4 Parameter initializers

The declaration of a parameter *p* may contain an initializer, in which case, that initializer's value is used to initialize *p* provided *p* is not bound to any arguments in the call.

Consider the following function definition and calls:

PowerShell

```
function Find-Str ([string]$Str, [int]$StartPos = 0) { ... }

Find-Str "abcabc" # 2nd argument omitted, 0 used for $StartPos
Find-Str "abcabc" 2 # 2nd argument present, so it is used for $StartPos
```

8.10.5 The [switch] type constraint

When a `[switch]` parameter is passed, the corresponding parameter in the command must be constrained by the type switch. Type switch has two values, True and False.

Consider the following function definition and calls:

PowerShell

```
function Process ([switch]$Trace, $P1, $P2) { ... }

Process 10 20                # $Trace is False, $P1 is 10, $P2 is 20
Process 10 -Trace 20         # $Trace is True, $P1 is 10, $P2 is 20
Process 10 20 -Trace         # $Trace is True, $P1 is 10, $P2 is 20
Process 10 20 -Trace:$false  # $Trace is False, $P1 is 10, $P2 is 20
Process 10 20 -Trace:$true   # $Trace is True, $P1 is 10, $P2 is 20
```

8.10.6 Pipelines and functions

When a script, function, or filter is used in a pipeline, a collection of values is delivered to that script or function. The script, function, or filter gets access to that collection via the enumerator $input (§2.3.2.2, §4.5.16), which is defined on entry to that script, function, or filter.

Consider the following function definition and calls:

PowerShell

```
function Get-Square1 {
    foreach ($i in $input) {   # iterate over the collection
        $i * $i
    }
}

-3..3 | Get-Square1            # collection has 7 elements
6, 10, -3 | Get-Square1        # collection has 3 elements
```

8.10.7 Named blocks

The statements within a *script-block* can belong to one large unnamed block, or they can be distributed into one or more named blocks. Named blocks allow custom processing of collections coming from pipelines; named blocks can be defined in any order.

The statements in a *begin block* (i.e.; one marked with the keyword begin) are executed once, before the first pipeline object is delivered.

The statements in a *process block* (i.e.; one marked with the keyword process) are executed for each pipeline object delivered. (`$_` provides access to the current object being processed from the input collection coming from the pipeline.) This means that if a collection of zero elements is sent via the pipeline, the process block is not executed at all. However, if the script or function is called outside a pipeline context, this block is executed exactly once, and `$_` is set to `$null`, as there is no current collection object.

The statements in an *end block* (i.e.; one marked with the keyword end) are executed once, after the last pipeline object has been delivered.

8.10.8 dynamicparam block

The subsections of §8.10 thus far deal with *static parameters*, which are defined as part of the source code. It is also possible to define *dynamic parameters* via a *dynamicparam block*, another form of named block (§8.10.7), which is marked with the keyword `dynamicparam`. Much of this machinery is implementation defined.

Dynamic parameters are parameters of a cmdlet, function, filter, or script that are available under certain conditions only. One such case is the **Encoding** parameter of the `Set-Item` cmdlet.

In the *statement-block*, use an if statement to specify the conditions under which the parameter is available in the function. Use the New-Object cmdlet to create an object of an implementation-defined type to represent the parameter, and specify its name. Also, use `New-Object` to create an object of a different implementation-defined type to represent the implementation-defined attributes of the parameter.

The following example shows a function with standard parameters called Name and Path, and an optional dynamic parameter named **DP1**. The **DP1** parameter is in the PSet1 parameter set and has a type of `Int32`. The **DP1** parameter is available in the Sample function only when the value of the Path parameter contains "HKLM:", indicating that it is being used in the `HKEY_LOCAL_MACHINE` registry drive.

PowerShell

```
function Sample {
    param ([string]$Name, [string]$Path)
    dynamicparam {
        if ($Path -match "*HKLM*:") {
            $dynParam1 = New-Object
System.Management.Automation.RuntimeDefinedParameter("dp1", [int32],
$attributeCollection)

            $attributes = New-Object
System.Management.Automation.ParameterAttribute
            $attributes.ParameterSetName = 'pset1'
            $attributes.Mandatory = $false

            $attributeCollection = New-Object -Type
System.Collections.ObjectModel.Collection``1[System.Attribute]
            $attributeCollection.Add($attributes)

            $paramDictionary = New-Object
System.Management.Automation.RuntimeDefinedParameterDictionary
            $paramDictionary.Add("dp1", $dynParam1)
            return $paramDictionary
        }
    }
}
```

The type used to create an object to represent a dynamic parameter is `System.Management.Automation.RuntimeDefinedParameter`.

The type used to create an object to represent the attributes of the parameter is `System.Management.Automation.ParameterAttribute`.

The implementation-defined attributes of the parameter include **Mandatory**, **Position**, and **ValueFromPipeline**.

8.10.9 param block

A *param-block* provides an alternate way of declaring parameters. For example, the following sets of parameter declarations are equivalent:

PowerShell

```
function FindStr1 ([string]$Str, [int]$StartPos = 0) { ... }
function FindStr2 {
    param ([string]$Str, [int]$StartPos = 0) ...
}
```

A *param-block* allows an *attribute-list* on the *param-block* whereas a *function-parameter-declaration* does not.

A script may have a *param-block* but not a *function-parameter-declaration*. A function or filter definition may have a *function-parameter-declaration* or a *param-block*, but not both.

Consider the following example:

PowerShell

```
param ( [Parameter(Mandatory = $true, ValueFromPipeline=$true)]
        [string[]] $ComputerName )
```

The one parameter, `$ComputerName`, has type `string[]`, it is required, and it takes input from the pipeline.

See §12.3.7 for a discussion of the **Parameter** attribute and for more examples.

8.11 The parallel statement

Syntax:

Syntax

```
parallel-statement:
    *parallel* statement-block
```

The parallel statement contains zero or more statements that are executed in an implementation defined manner.

A parallel statement is only allowed in a workflow (§8.10.2).

8.12 The sequence statement

Syntax:

Syntax

```
sequence-statement:
    *sequence* statement-block
```

The sequence statement contains zero or more statements that are executed in an implementation defined manner.

A sequence statement is only allowed in a workflow (§8.10.2).

8.13 The inlinescript statement

Syntax:

Syntax

```
inlinescript-statement:
    inlinescript statement-block
```

The inlinescript statement contains zero or more statements that are executed in an implementation defined manner.

An inlinescript statement is only allowed in a workflow (§8.10.2).

8.14 Parameter binding

When a script, function, filter, or cmdlet is invoked, each argument can be bound to the corresponding parameter by position, with the first parameter having position zero.

Consider the following definition fragment for a function called `Get-Power`, and the calls to it:

PowerShell

```powershell
function Get-Power ([long]$Base, [int]$Exponent) { ... }

Get-Power 5 3       # argument 5 is bound to parameter $Base in position 0
                    # argument 3 is bound to parameter $Exponent in position 1
                    # no conversion is needed, and the result is 5 to the power 3

Get-Power 4.7 3.2   # double argument 4.7 is rounded to int 5, double argument
                    # 3.2 is rounded to int 3, and result is 5 to the power 3

Get-Power 5         # $Exponent has value $null, which is converted to int 0

Get-Power           # both parameters have value $null, which is converted to int 0
```

When a script, function, filter, or cmdlet is invoked, an argument can be bound to the corresponding parameter by name. This is done by using a *parameter with argument*, which is an argument that is the parameter's name with a leading dash (-), followed by the associated value for that argument. The parameter name used can have any case-insensitive spelling and can use any prefix that uniquely designates the corresponding parameter. When choosing parameter names, avoid using the names of the common parameters.

Consider the following calls to function `Get-Power`:

PowerShell

```
Get-Power -Base 5 -Exponent 3   # -Base designates $Base, so 5 is
                                # bound to that, -Exponent designates
                                # $Exponent, so 3 is bound to that

Get-Power -Exp 3 -Bas 5         # $Base takes on 5 and $Exponent takes on 3

Get-Power -E 3 -B 5             # $Base takes on 5 and $Exponent takes on 3
```

On the other hand, calls to the following function

PowerShell

```
function Get-Hypot ([double]$Side1, [double]$Side2) {
    return [Math]::Sqrt($Side1 * $Side1 + $Side2 * $Side2)
}
```

must use parameters `-Side1` and `-Side2`, as there is no prefix that uniquely designates the parameter.

The same parameter name cannot be used multiple times with or without different associated argument values.

Parameters can have attributes (§12). For information about the individual attributes see the sections within §12.3. For information about parameter sets see §12.3.7.

A script, function, filter, or cmdlet can receive arguments via the invocation command line, from the pipeline, or from both. Here are the steps, in order, for resolving parameter binding:

1. Bind all named parameters, then
2. Bind positional parameters, then
3. Bind from the pipeline by value (§12.3.7) with exact match, then
4. Bind from the pipeline by value (§12.3.7) with conversion, then
5. Bind from the pipeline by name (§12.3.7) with exact match, then
6. Bind from the pipeline by name (§12.3.7) with conversion

Several of these steps involve conversion, as described in §6. However, the set of conversions used in binding is not exactly the same as that used in language conversions. Specifically,

- Although the value `$null` can be cast to bool, `$null` cannot be bound to `bool`.

- When the value `$null` is passed to a `[switch]` parameter for a cmdlet, it is treated as if `$true` was passed. However, when passed to a `[switch]` parameter for a function, it is treated as if `$false` was passed.
- Parameters of type bool or switch can only bind to numeric or bool arguments.
- If the parameter type is not a collection, but the argument is some sort of collection, no conversion is attempted unless the parameter type is object or PsObject. (The main point of this restriction is to disallow converting a collection to a string parameter.) Otherwise, the usual conversions are attempted.

If the parameter type is `IList` or `ICollection<T>`, only those conversions via Constructor, op_Implicit, and op_Explicit are attempted. If no such conversions exist, a special conversion for parameters of "collection" type is used, which includes `IList`, `ICollection<T>`, and arrays.

Positional parameters prefer to be bound without type conversion, if possible. For example,

PowerShell

```powershell
function Test {
    [CmdletBinding(DefaultParameterSetName = "SetB")]
    param([Parameter(Position = 0, ParameterSetName = "SetA")]
        [decimal]$Dec,
        [Parameter(Position = 0, ParameterSetName = "SetB")]
        [int]$In
    )
    $PSCmdlet.ParameterSetName
}

Test 42d   # outputs "SetA"
Test 42    # outputs "SetB"
```

Last updated on 04/08/2026

9. Arrays

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

9.1 Introduction

PowerShell supports arrays of one or more dimensions with each dimension having zero or more *elements*. Within a dimension, elements are numbered in ascending integer order starting at zero. Any individual element can be accessed via the array subscript operator `[]` (§7.1.4). The number of dimensions in an array is called its *rank*.

An element can contain a value of any type including an array type. An array having one or more elements whose values are of any array type is called a *jagged array*. A *multidimensional array* has multiple dimensions, in which case, the number of elements in each row of a dimension is the same. An element of a jagged array may contain a multidimensional array, and vice versa.

Multidimensional arrays are stored in row-major order. The number of elements in an array is called that array's *length*, which is fixed when the array is created. As such, the elements in a 1-dimensional array *A* having length *N* can be accessed (i.e., *subscripted*) using the expressions `A[0], A[1], ..., A[N-1]`. The elements in a 2-dimensional array *B* having *M* rows, with each row having *N* columns, can be accessed using the expressions `B[0,0], B[0,1], ..., B[0,N-1],`

`B[1,0], B[1,1], ..., B[1,N-1], ..., B[M-1,0], B[M-1,1], ..., B[M-1,N-1]`. And so on for arrays with three or more dimensions.

By default, an array is *polymorphic*; i.e., its elements do not need to all have the same type. For example,

PowerShell

```
$items = 10,"blue",12.54e3,16.30D # 1-D array of length 4
$items[1] = -2.345
$items[2] = "green"

$a = New-Object 'object[,]' 2,2 # 2-D array of length 4
$a[0,0] = 10
$a[0,1] = $false
$a[1,0] = "red"
$a[1,1] = $null
```

A 1-dimensional array has type `type[]`, a 2-dimensional array has type `type[,]`, a 3-dimensional array has type `type[,,]`, and so on, where *type* is object for an unconstrained type array, or the constrained type for a constrained array (§9.4).

All array types are derived from the type Array (§4.3.2).

9.2 Array creation

An array is created via an *array creation expression*, which has the following forms: unary comma operator (§7.2.1) ,*array-expression* (§7.1.7), binary comma operator (§7.3), range operator (§7.4), or New-Object cmdlet.

Here are some examples of array creation and usage:

PowerShell

```
$values = 10, 20, 30
for ($i = 0; $i -lt $values.Length; ++$i) {
    "`$values[$i] = $($values[$i])"
}

$x = , 10                          # x refers to an array of length 1
$x = @(10)                         # x refers to an array of length 1
$x = @()                           # x refers to an array of length 0

$a = New-Object 'object[,]' 2, 2   # create a 2x2 array of anything
$a[0, 0] = 10                      # set to an int value
$a[0, 1] = $false                  # set to a boolean value
$a[1, 0] = "red"                   # set to a string value
```

```
$a[1, 1] = 10.50D                # set to a decimal value
foreach ($e in $a) {             # enumerate over the whole array
    $e
}
```

The following is written to the pipeline:

Output

```
$values[0] = 10
$values[1] = 20
$values[2] = 30

10
False
red
10.50
```

The default initial value of any element not explicitly initialized is the default value for that element's type (that is, `$false`, zero, or `$null`).

9.3 Array concatenation

Arrays of arbitrary type and length can be concatenated via the `+` and `+=` operators, both of which result in the creation of a new unconstrained 1-dimensional array. The existing arrays are unchanged. See §7.7.3 for more information, and §9.4 for a discussion of adding to an array of constrained type.

9.4 Constraining element types

A 1-dimensional array can be created so that it is type-constrained by prefixing the array-creation expression with an array type cast. For example,

PowerShell

```
$a = [int[]](1,2,3,4)   # constrained to int
$a[1] = "abc"           # implementation-defined behavior
$a += 1.23              # new array is unconstrained
```

The syntax for creating a multidimensional array requires the specification of a type, and that type becomes the constraint type for that array. However, by specifying type `object[]`, there really is no constraint as a value of any type can be assigned to an element of an array of that type.

Concatenating two arrays (§7.7.3) always results in a new array that is unconstrained even if both arrays are constrained by the same type. For example,

PowerShell

```
$a = [int[]](1,2,3)    # constrained to int
$b = [int[]](10,20)    # constrained to int
$c = $a + $b           # constraint not preserved
$c = [int[]]($a + $b)  # result explicitly constrained to int
```

9.5 Arrays as reference types

As array types are reference types, a variable designating an array can be made to refer to any array of any rank, length, and element type. For example,

PowerShell

```
$a = 10,20                    # $a refers to an array of length 2
$a = 10,20,30                 # $a refers to a different array, of length 3
$a = "red",10.6               # $a refers to a different array, of length 2
$a = New-Object 'int[,]' 2,3  # $a refers to an array of rank 2
```

Assignment of an array involves a shallow copy; that is, the variable assigned to refers to the same array, no copy of the array is made. For example,

PowerShell

```
$a = 10,20,30
">$a<"
$b = $a         # make $b refer to the same array as $a
">$b<"

$a[0] = 6       # change value of [0] via $a
">$a<"
">$b<"          # change is reflected in $b

$b += 40        # make $b refer to a new array
$a[0] = 8       # change value of [0] via $a
">$a<"
">$b<"          # change is not reflected in $b
```

The following is written to the pipeline:

Output

```
>10 20 30<
>10 20 30<
>6 20 30<
>6 20 30<
>8 20 30<
>6 20 30 40<
```

9.6 Arrays as array elements

Any element of an array can itself be an array. For example,

PowerShell

```powershell
$colors = "red", "blue", "green"
$list = $colors, (,7), (1.2, "yes") # parens in (,7) are redundant; they
                                    # are intended to aid readability
"`$list refers to an array of length $($list.Length)"
">$($list[1][0])<"
">$($list[2][1])<"
```

The following is written to the pipeline:

Output

```
$list refers to an array of length 3
>7<
>yes<
```

`$list[1]` refers to an array of 1 element, the integer 7, which is accessed via `$list[1][0]`, as shown. Compare this with the following subtly different case:

PowerShell

```powershell
$list = $colors, 7, (1.2, "yes") # 7 has no prefix comma
">$($list[1])<"
```

Here, `$list[1]` refers to a scalar, the integer 7, which is accessed via `$list[1]`.

Consider the following example,

PowerShell

```powershell
$x = [string[]]("red","green")
$y = 12.5, $true, "blue"
$a = New-Object 'object[,]' 2,2
$a[0,0] = $x               # element is an array of 2 strings
```

```
$a[0,1] = 20                # element is an int
$a[1,0] = $y                # element is an array of 3 objects
$a[1,1] = [int[]](92,93)    # element is an array of 2 ints
```

9.7 Negative subscripting

This is discussed in §7.1.4.1.

9.8 Bounds checking

This is discussed in §7.1.4.1.

9.9 Array slices

An *array slice* is an unconstrained 1-dimensional array whose elements are copies of zero or more elements from a collection. An array slice is created via the subscript operator `[]` (§7.1.4.5).

9.10 Copying an array

A contiguous set of elements can be copied from one array to another using the method `[array]::Copy`. For example,

PowerShell

```
$a = [int[]](10,20,30)
$b = [int[]](0,1,2,3,4,5)
[array]::Copy($a, $b, 2)        # $a[0]->$b[0],
$a[1]->$b[1]
[array]::Copy($a, 1, $b, 3, 2)  # $a[1]->$b[3],
$a[2]->$b[4]
```

9.11 Enumerating over an array

Although it is possible to loop through an array accessing each of its elements via the subscript operator, we can enumerate over that array's elements using the foreach statement. For a multidimensional array, the elements are processed in row-major order. For example,

PowerShell

```
$a = 10, 53, 16, -43
foreach ($elem in $a) {
```

```
    # do something with element via $elem
}

foreach ($elem in -5..5) {
    # do something with element via $elem
}

$a = New-Object 'int[,]' 3, 2
foreach ($elem in $a) {
    # do something with element via $elem
}
```

9.12 Multidimensional array flattening

Some operations on a multidimensional array (such as replication (§7.6.3) and concatenation (§7.7.3)) require that array to be *flattened*; that is, to be turned into a 1-dimensional array of unconstrained type. The resulting array takes on all the elements in row-major order.

Consider the following example:

PowerShell

```
$a = "red",$true
$b = (New-Object 'int[,]' 2,2)
$b[0,0] = 10
$b[0,1] = 20
$b[1,0] = 30
$b[1,1] = 40
$c = $a + $b
```

The array designated by `$c` contains the elements "red", `$true`, 10, 20, 30, and 40.

Last updated on 03/24/2025

10. Hashtables

Article • 01/08/2025

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389 That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

Syntax:

> **Tip**
>
> The `~opt~` notation in the syntax definitions indicates that the lexical entity is optional in the syntax.

Syntax

```
hash-literal-expression:
    @{ new-lines~opt~ hash-literal-body~opt~ new-lines~opt~ }

hash-literal-body:
    hash-entry
    hash-literal-body statement-terminators hash-entry

hash-entry:
    key-expression = new-lines~opt~ statement

key-expression:
    simple-name
    unary-expression
```

```
statement-terminator:
    ;
    new-line-character
```

10.1 Introduction

The type Hashtable represents a collection of *key/value pair* objects that supports efficient retrieval of a value when indexed by the key. Each key/value pair is an *element*, which is stored in some implementation-defined object type.

An element's key cannot be the null value. There are no restrictions on the type of a key or value. Duplicate keys are not supported.

Given a key/value pair object, the key and associated value can be obtained by using the instance properties Key and Value, respectively.

Given one or more keys, the corresponding value(s) can be accessed via the Hashtable subscript operator `[]` (§7.1.4.3).

All Hashtables have type `Hashtable` (§4.3.3).

The order of the keys in the collection returned by Keys is unspecified; however, it is the same order as the associated values in the collection returned by Values.

Here are some examples involving Hashtables:

PowerShell

```
$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123 }
$h1.FirstName # designates the key FirstName
$h1["LastName"] # designates the associated value for key LastName
$h1.Keys # gets the collection of keys
```

`Hashtable` elements are stored in an object of type **DictionaryEntry**, and the collections returned by Keys and Values have type **ICollection**.

10.2 Hashtable creation

A `Hashtable` is created via a hash literal (§7.1.9) or the New-Object cmdlet. It can be created with zero or more elements. The Count property returns the current element count.

10.3 Adding and removing Hashtable elements

An element can be added to a `Hashtable` by assigning (§7.11.1) a value to a non-existent key name or to a subscript (§7.1.4.3) that uses a non-existent key name. Removal of an element requires the use of the Remove method. For example,

PowerShell

```
$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123 }
$h1.Dept = "Finance" # adds element Finance
$h1["Salaried"] = $false # adds element Salaried
$h1.Remove("Salaried") # removes element Salaried
```

10.4 Hashtable concatenation

Hashtables can be concatenated via the `+` and `+=` operators, both of which result in the creation of a new `Hashtable`. The existing Hashtables are unchanged. See §7.7.4 for more information.

10.5 Hashtables as reference types

As `Hashtable` is a reference type, assignment of a `Hashtable` involves a shallow copy; that is, the variable assigned to refers to the same `Hashtable`; no copy of the `Hashtable` is made. For example,

PowerShell

```
$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123 }
$h2 = $h1
$h1.FirstName = "John" # change key's value in $h1
$h2.FirstName # change is reflected in $h2
```

10.6 Enumerating over a Hashtable

To process every pair in a `Hashtable`, use the **Keys** property to retrieve the list of keys as an array, and then enumerate over the elements of that array getting the associated value via the **Value** property or a subscript, as follows

PowerShell

```
$h1 = @{ FirstName = "James"; LastName = "Anderson"; IDNum = 123}
foreach ($e in $h1.Keys) {
   "Key is " + $e + ", Value is " + $h1[$e]
}
```

11. Modules

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

11.1 Introduction

As stated in §3.14, a module is a self-contained reusable unit that allows PowerShell code to be partitioned, organized, and abstracted. A module can contain one or more *module members*, which are commands (such as cmdlets and functions) and items (such as variables and aliases). The names of these members can be kept private to the module or they may be *exported* to the session into which the module is *imported*.

There are three different *module types*: manifest, script, and binary. A *manifest module* is a file that contains information about a module, and controls certain aspects of that module's use. A *script module* is a PowerShell script file with a file extension of `.psm1` instead of `.ps1`. A *binary module* contains class types that define cmdlets and providers. Unlike script modules, binary modules are written in compiled languages. Binary modules are not covered by this specification.

A binary module is a .NET assembly (i.e.; a DLL) that was compiled against the PowerShell libraries.

Modules may *nest*; that is, one module may import another module. A module that has associated nested modules is a *root module*.

When a PowerShell session is created, by default, no modules are imported.

When modules are imported, the search path used to locate them is defined by the environment variable **PSModulePath**.

The following cmdlets deal with modules:

- Get-Module: Identifies the modules that have been, or can be imported
- Import-Module: Adds one or more modules to the current session (see §11.4)
- Export-ModuleMember: Identifies the module members that are to be exported
- Remove-Module: Removes one or more modules from the current session (see §11.5)
- New-Module: Creates a dynamic module (see §11.7)

11.2 Writing a script module

A script module is a script file. Consider the following script module:

PowerShell

```
function Convert-CentigradeToFahrenheit ([double]$tempC) {
    return ($tempC * (9.0 / 5.0)) + 32.0
}
New-Alias c2f Convert-CentigradeToFahrenheit

function Convert-FahrenheitToCentigrade ([double]$tempF) {
    return ($tempF - 32.0) * (5.0 / 9.0)
}
New-Alias f2c Convert-FahrenheitToCentigrade

Export-ModuleMember -Function Convert-CentigradeToFahrenheit
Export-ModuleMember -Function Convert-FahrenheitToCentigrade
Export-ModuleMember -Alias c2f, f2c
```

This module contains two functions, each of which has an alias. By default, all function names, and only function names are exported. However, once the cmdlet `Export-ModuleMember` has been used to export anything, then only those things exported explicitly will be exported. A series of commands and items can be exported in one call or a number of calls to this cmdlet; such calls are cumulative for the current session.

11.3 Installing a script module

A script module is defined in a script file, and modules can be stored in any directory. The environment variable PSModulePath points to a set of directories to be searched when module-related cmdlets look for modules whose names do not include a fully qualified path. Additional lookup paths can be provided; for example,

`$Env:PSModulePath = $Env:PSModulePath + ";<additional-path>"`

Any additional paths added affect the current session only.

Alternatively, a fully qualified path can be specified when a module is imported.

11.4 Importing a script module

Before the resources in a module can be used, that module must be imported into the current session, using the cmdlet `Import-Module`. `Import-Module` can restrict the resources that it actually imports.

When a module is imported, its script file is executed. That process can be configured by defining one or more parameters in the script file, and passing in corresponding arguments via the ArgumentList parameter of `Import-Module`.

Consider the following script that uses these functions and aliases defined in §11.2:

`Import-Module` "E:\Scripts\Modules\PSTest_Temperature" -Verbose

PowerShell

```
"0 degrees C is " + (Convert-CentigradeToFahrenheit 0) + " degrees F"
"100 degrees C is " + (c2f 100) + " degrees F"
"32 degrees F is " + (Convert-FahrenheitToCentigrade 32) + " degrees C"
"212 degrees F is " + (f2c 212) + " degrees C"
```

Importing a module causes a name conflict when commands or items in the module have the same names as commands or items in the session. A name conflict results in a name being hidden or replaced. The Prefix parameter of `Import-Module` can be used to avoid naming conflicts. Also, the **Alias**, **Cmdlet**, **Function**, and **Variable** parameters can limit the selection of commands to be imported, thereby reducing the chances of name conflict.

Even if a command is hidden, it can be run by qualifying its name with the name of the module in which it originated. For example, `& M\F 100` invokes the function *F* in module *M*, and passes it the argument 100.

When the session includes commands of the same kind with the same name, such as two cmdlets with the same name, by default it runs the most recently added command.

See §3.5.6 for a discussion of scope as it relates to modules.

11.5 Removing a script module

One or more modules can be removed from a session via the cmdlet `Remove-Module`.

Removing a module does not uninstall the module.

In a script module, it is possible to specify code that is to be executed prior to that module's removal, as follows:

```
$MyInvocation.MyCommand.ScriptBlock.Module.OnRemove = { *on-removal-code* }
```

11.6 Module manifests

As stated in §11.1, a manifest module is a file that contains information about a module, and controls certain aspects of that module's use.

A module need not have a corresponding manifest, but if it does, that manifest has the same name as the module it describes, but with a `.psd1` file extension.

A manifest contains a limited subset of PowerShell script, which returns a Hashtable containing a set of keys. These keys and their values specify the *manifest elements* for that module. That is, they describe the contents and attributes of the module, define any prerequisites, and determine how the components are processed.

Essentially, a manifest is a data file; however, it can contain references to data types, the if statement, and the arithmetic and comparison operators. (Assignments, function definitions and loops are not permitted.) A manifest also has read access to environment variables and it can contain calls to the cmdlet `Join-Path`, so paths can be constructed.

> ⓘ **Note**
>
> Editor's Note: The original document contains a list of keys allowed in a module manifest file. That list is outdated and incomplete. For a complete list of keys in a module manifest, see New-ModuleManifest.

The only key that is required is **ModuleVersion**.

Here is an example of a simple manifest:

PowerShell

```
@{
ModuleVersion = '1.0'
Author = 'John Doe'
RequiredModules = @()
FunctionsToExport = 'Set*','Get*','Process*'
}
```

The key **GUID** has a `string` value. This specifies a Globally Unique IDentifier (GUID) for the module. The **GUID** can be used to distinguish among modules having the same name. To create a new GUID, call the method `[guid]::NewGuid()`.

11.7 Dynamic modules

A *dynamic module* is a module that is created in memory at runtime by the cmdlet `New-Module`; it is not loaded from disk. Consider the following example:

PowerShell

```
$sb = {
    function Convert-CentigradeToFahrenheit ([double]$tempC) {
        return ($tempC * (9.0 / 5.0)) + 32.0
    }

    New-Alias c2f Convert-CentigradeToFahrenheit

    function Convert-FahrenheitToCentigrade ([double]$tempF) {
        return ($tempF - 32.0) * (5.0 / 9.0)
    }

    New-Alias f2c Convert-FahrenheitToCentigrade

    Export-ModuleMember -Function Convert-CentigradeToFahrenheit
    Export-ModuleMember -Function Convert-FahrenheitToCentigrade
    Export-ModuleMember -Alias c2f, f2c
}

New-Module -Name MyDynMod -ScriptBlock $sb
Convert-CentigradeToFahrenheit 100
c2f 100
```

The script block `$sb` defines the contents of the module, in this case, two functions and two aliases to those functions. As with an on-disk module, only functions are exported by default, so `Export-ModuleMember` cmdlets calls exist to export both the functions and the aliases.

Once `New-Module` runs, the four names exported are available for use in the session, as is shown by the calls to the `Convert-CentigradeToFahrenheit` and c2f.

Like all modules, the members of dynamic modules run in a private module scope that is a child of the global scope. `Get-Module` cannot get a dynamic module, but `Get-Command` can get the exported members.

To make a dynamic module available to `Get-Module`, pipe a `New-Module` command to `Import-Module`, or pipe the module object that `New-Module` returns, to `Import-Module`. This action adds the dynamic module to the `Get-Module` list, but it does not save the module to disk or make it persistent.

11.8 Closures

A dynamic module can be used to create a *closure*, a function with attached data. Consider the following example:

PowerShell

```
function Get-NextID ([int]$StartValue = 1) {
    $nextID = $StartValue
    {
        ($Script:nextID++)
    }.GetNewClosure()
}

$v1 = Get-NextID      # get a scriptblock with $StartValue of 0
& $v1                 # invoke Get-NextID getting back 1
& $v1                 # invoke Get-NextID getting back 2

$v2 = Get-NextID 100  # get a scriptblock with $StartValue of 100
& $v2                 # invoke Get-NextID getting back 100
& $v2                 # invoke Get-NextID getting back 101
```

The intent here is that `Get-NextID` return the next ID in a sequence whose start value can be specified. However, multiple sequences must be supported, each with its own `$StartValue` and `$nextID` context. This is achieved by the call to the method `[scriptblock]::GetNewClosure` (§4.3.7).

Each time a new closure is created by `GetNewClosure`, a new dynamic module is created, and the variables in the caller's scope (in this case, the script block containing the increment) are copied into this new module. To ensure that the nextId defined inside the parent function (but outside the script block) is incremented, the explicit Script: scope prefix is needed.

Of course, the script block need not be a named function; for example:

PowerShell

```
$v3 = & {       # get a scriptblock with $StartValue of 200
    param ([int]$StartValue = 1)
    $nextID = $StartValue
    {
        ($Script:nextID++)
    }.GetNewClosure()
} 200

& $v3           # invoke script getting back 200
& $v3           # invoke script getting back 201
```

Last updated on 03/24/2025

12. Attributes

Article • 03/24/2025

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389 That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

An *attribute* object associates predefined system information with a *target element*, which can be a param block or a parameter (§8.10). Each attribute object has an *attribute type*.

Information provided by an attribute is also known as *metadata*. Metadata can be examined by the command or the execution environment to control how the command processes data or before run time by external tools to control how the command itself is processed or maintained.

Multiple attributes can be applied to the same target element.

12.1 Attribute specification

> **Tip**
>
> The `~opt~` notation in the syntax definitions indicates that the lexical entity is optional in the syntax.

Syntax

```
attribute-list:
    attribute
    attribute-list new-lines~opt~ attribute

attribute:
    [ new-lines~opt~ attribute-name ( attribute-arguments new-lines~opt~ )
new-lines~opt~ ]
    type-literal

attribute-name:
    type-spec

attribute-arguments:
    attribute-argument
    attribute-argument new-lines~opt~ ,
    attribute-arguments

attribute-argument:
    new-lines~opt~ expression
    new-lines~opt~ simple-name
    new-lines~opt~ simple-name = new-lines~opt~ expression
```

An attribute consists of an *attribute-name* and an optional list of positional and named arguments. The positional arguments (if any) precede the named arguments. A named argument consists of a *simple-name*, optionally followed by an equal sign and followed by an *expression*. If the expression is omitted, the value `$true` is assumed.

The *attribute-name* is a reserved attribute type (§12.3) or some implementation-defined attribute type.

12.2 Attribute instances

An attribute instance is an object of an attribute type. The instance represents an attribute at run-time.

To create an object of some attribute type *A*, use the notation `A()`. An attribute is declared by enclosing its instance inside `[]`, as in `[A()]`. Some attribute types have positional and named parameters (§8.14), just like functions and cmdlets. For example,

```
[A(10,IgnoreCase=$true)]
```

shows an instance of type *A* being created using a positional parameter whose argument value is 10, and a named parameter, **IgnoreCase**, whose argument value is `$true`.

12.3 Reserved attributes

The attributes described in the following sections can be used to augment or modify the behavior of PowerShell functions, filters, scripts, and cmdlets.

12.3.1 The Alias attribute

This attribute is used in a *script-parameter* to specify an alternate name for a parameter. A parameter may have multiple aliases, and each alias name must be unique within a *parameter-list*. One possible use is to have different names for a parameter in different parameter sets (see **ParameterSetName**).

The attribute argument has type string[].

Consider a function call `Test1` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [Parameter(Mandatory = $true)]
    [Alias("CN")]
    [Alias("Name", "System")]
    [string[]] $ComputerName
)

Test1 "Mars", "Saturn"                # pass argument by position
Test1 -ComputerName "Mars", "Saturn"  # pass argument by name
Test1 -CN "Mars", "Saturn"            # pass argument using first alias
Test1 -Name "Mars", "Saturn"          # pass argument using second alias
Test1 -Sys "Mars", "Saturn"           # pass argument using third alias
```

Consider a function call `Test2` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [Parameter(Mandatory = $true, ValueFromPipelineByPropertyName = $true)]
    [Alias('PSPath')]
    [string] $LiteralPath
)

Get-ChildItem "E:\*.txt" | Test2 -LiteralPath { $_ ; "`n`t";
    $_.FullName + ".bak" }
Get-ChildItem "E:\*.txt" | Test2
```

Cmdlet `Get-ChildItem` (alias `dir`) adds to the object it returns a new **NoteProperty** of type `string`, called **PSPath**.

12.3.2 The AllowEmptyCollection attribute

This attribute is used in a *script-parameter* to allow an empty collection as the argument of a mandatory parameter.

Consider a function call `Test` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [Parameter(Mandatory = $true)]
    [AllowEmptyCollection()]
    [string[]] $ComputerName
)

Test "Red", "Green" # $ComputerName has Length 2
Test "Red" # $ComputerName has Length 1
Test -Comp @() # $ComputerName has Length 0
```

12.3.3 The AllowEmptyString attribute

This attribute is used in a *script-parameter* to allow an empty string as the argument of a mandatory parameter.

Consider a function call `Test` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [Parameter(Mandatory = $true)]
    [AllowEmptyString()]
    [string] $ComputerName
)

Test "Red" # $ComputerName is "Red"
Test "" # empty string is permitted
Test -Comp "" # empty string is permitted
```

12.3.4 The AllowNull attribute

This attribute is used in a *script-parameter* to allow $null as the argument of a mandatory parameter for which no implicit conversion is available.

Consider a function call Test that has the following param block, and which is called as shown:

PowerShell

```
param (
    [Parameter(Mandatory = $true)]
    [AllowNull()]
    [int[]] $Values
)

Test 10, 20, 30     # $values has Length 3, values 10, 20, 30
Test 10, $null, 30  # $values has Length 3, values 10, 0, 30
Test -Val $null     # $values has value $null
```

Note that the second case above does not need this attribute; there is already an implicit conversion from `$null` to int.

12.3.5 The CmdletBinding attribute

This attribute is used in the *attribute-list* of *param-block* of a function to indicate that function acts similar to a cmdlet. Specifically, it allows functions to access a number of methods and properties through the $PSCmdlet variable by using begin, process, and end named blocks (§8.10.7).

When this attribute is present, positional arguments that have no matching positional parameters cause parameter binding to fail and $args is not defined. (Without this attribute $args would take on any unmatched positional argument values.)

The following arguments are used to define the characteristics of the parameter:

Expand table

Parameter Name	Purpose
SupportsShouldProcess (named)	Type: bool; Default value: $false Specifies whether the function supports calls to the ShouldProcess method, which is used to prompt the user for feedback before the function makes a change to the system. A value of $true indicates that it does. A value of $false indicates that it doesn't.
ConfirmImpact (named)	Type: string; Default value: "Medium"

Parameter Name	Purpose
	Specifies the impact level of the action performed. The call to the ShouldProcess method displays a confirmation prompt only when the ConfirmImpact argument is greater than or equal to the value of the $ConfirmPreference preference variable. The possible values of this argument are: None: Suppress all requests for confirmation. Low: The action performed has a low risk of losing data. Medium: The action performed has a medium risk of losing data. High: The action performed has a high risk of losing data. The value of $ConfirmPreference can be set so that only cmdlets with an equal or higher impact level can request confirmation before they perform their operation. For example, if $ConfirmPreference is set to Medium, cmdlets with a Medium or High impact level can request confirmation. Requests from cmdlets with a low impact level are suppressed.
DefaultParameterSetName (named)	Type: string; Default value: "__AllParameterSets" Specifies the parameter set to use if that cannot be determined from the arguments. See the named argument ParameterSetName in the attribute Parameter ([§12.3.7][§12.3.7]).
PositionalBinding (named)	Type: bool; Default value: $true Specifies whether positional binding is supported or not. The value of this argument is ignored if any parameters specify non-default values for either the named argument Position or the named argument ParameterSetName in the attribute Parameter ([§12.3.7][§12.3.7]). Otherwise, if the argument is $false then no parameters are positional, otherwise parameters are assigned a position based on the order the parameters are specified.

Here's is an example of the framework for using this attribute:

PowerShell

```
[CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = "Low")]
param ( ... )

begin { ... }
Get-process { ... }
end { ... }
```

12.3.6 The OutputType attribute

This attribute is used in the *attribute-list* of *param-block* to specify the types returned. The following arguments are used to define the characteristics of the parameter:

Expand table

Parameter Name	Purpose
Type (position 0)	Type: string[] or array of type literals A list of the types of the values that are returned.
ParameterSetName (named)	Type: string[] Specifies the parameter sets that return the types indicated by the corresponding elements of the Type parameter.

Here are several examples of this attribute's use:

PowerShell

```
[OutputType([int])] param ( ... )
[OutputType("double")] param ( ... )
[OutputType("string","string")] param ( ... )
```

12.3.7 The Parameter attribute

This attribute is used in a *script-parameter*. The following named arguments are used to define the characteristics of the parameter:

Expand table

Parameter	Purpose
HelpMessage (named)	Type: string This argument specifies a message that is intended to contain a short description of the parameter. This message is used in an implementation-defined manner when the function or cmdlet is run yet a mandatory parameter having a HelpMessage does not have a corresponding argument. The following example shows a parameter declaration that provides a description of the parameter.

Parameter	Purpose
	param ([Parameter(Mandatory = $true, HelpMessage = "An array of computer names.")] [string[]] $ComputerName) Windows PowerShell: If a required parameter is not provided the runtime prompts the user for a parameter value. The prompt dialog box includes the HelpMessage text.
Mandatory (named)	Type: bool; Default value: $false This argument specifies whether the parameter is required within the given parameter set (see ParameterSetName argument below). A value of $true indicates that it is. A value of $false indicates that it isn't. param ([Parameter(Mandatory = $true)] [string[]] $ComputerName) Windows PowerShell: If a required parameter is not provided the runtime prompts the user for a parameter value. The prompt dialog box includes the HelpMessage text, if any.
ParameterSetName (named)	Type: string; Default value: "__AllParameterSets" It is possible to write a single function or cmdlet that can perform different actions for different scenarios. It does this by exposing different groups of parameters depending on the action it wants to take. Such parameter groupings are called *parameter sets*. The argument ParameterSetName specifies the parameter set to which a parameter belongs. This behavior means that each parameter set must have one unique parameter that is not a member of any other parameter set. For parameters that belong to multiple parameter sets, add a Parameter attribute for each parameter set. This allows the parameter to be defined differently for each parameter set. A parameter set that contains multiple positional parameters must define unique positions for each parameter. No two positional parameters can specify the same position. If no parameter set is specified for a parameter, the parameter belongs to all parameter sets.

Parameter	Purpose

	When multiple parameter sets are defined, the named argument DefaultParameterSetName of the attribute CmdletBinding ([§12.3.5][§12.3.5]) is used to specify the default parameter set. The runtime uses the default parameter set if it cannot determine the parameter set to use based on the information provided by the command, or raises an exception if no default parameter set has been specified.
	
	The following example shows a function Test with a parameter declaration of two parameters that belong to two different parameter sets, and a third parameter that belongs to both sets:
	
	param ([Parameter(Mandatory = $true,
	ParameterSetName = "Computer")]
	[string[]] $ComputerName,
	
	[Parameter(Mandatory = $true,
	ParameterSetName = "User")]
	[string[]] $UserName,
	
	[Parameter(Mandatory = $true,
	ParameterSetName = "Computer")]
	[Parameter(ParameterSetName = "User")]
	[int] $SharedParam = 5)
	
	if ($PSCmdlet.ParameterSetName -eq "Computer")
	{
	# handle "Computer" parameter set
	}
	
	elseif ($PSCmdlet.ParameterSetName -eq "User")
	{
	# handle "User" parameter set
	}
	…
	}
	
	Test -ComputerName "Mars","Venus" -SharedParam 10
	Test -UserName "Mary","Jack"
	Test -UserName "Mary","Jack" -SharedParam 20

Position (named)	Type: int This argument specifies the position of the parameter in the argument list. If this argument is not specified, the parameter name or its alias must be specified explicitly when the parameter is set. If none of the parameters of a

Parameter	Purpose
	function has positions, positions are assigned to each parameter based on the order in which they are received. The following example shows the declaration of a parameter whose value must be specified as the first argument when the function is called. param ([Parameter(Position = 0)] [string[]] $ComputerName)
ValueFromPipeline (named)	Type: bool; Default value: $false This argument specifies whether the parameter accepts input from a pipeline object. A value of $true indicates that it does. A value of $false indicates that it does not. Specify $true if the function or cmdlet accesses the complete object, not just a property of the object. Only one parameter in a parameter set can declare ValueFromPipeline as $true. The following example shows the parameter declaration of a mandatory parameter, $ComputerName, that accepts the input object that is passed to the function from the pipeline. param ([Parameter(Mandatory = $true, ValueFromPipeline=$true)] [string[]] $ComputerName) For an example of using this parameter in conjunction with the Alias attribute see [§12.3.1][§12.3.1].
ValueFromPipelineByPropertyName (named)	Type: bool; Default value: $false This argument specifies whether the parameter takes its value from a property of a pipeline object that has either the same name or the same alias as this parameter. A value of $true indicates that it does. A value of $false indicates that it does not. Specify $true if the following conditions are true: the parameter accesses a property of the piped object, and the property has the same name as the parameter, or the property has the same alias as the parameter. A parameter having ValueFromPipelineByPropertyName set to $true need not have a parameter in the same set with ValueFromPipeline set to $true.

Parameter	Purpose

	If a function has a parameter $ComputerName, and the piped object has a ComputerName property, the value of the ComputerName property is assigned to the $ComputerName parameter of the Function:
	
	param ([Parameter(Mandatory = $true, ValueFromPipelineByPropertyName = $true)] [string[]] $ComputerName)
	
	Multiple parameters in a parameter set can define the ValueFromPipelineByPropertyName as $true. Although, a single input object cannot be bound to multiple parameters, different properties in that input object may be bound to different parameters.
	
	When binding a parameter with a property of an input object, the runtime environment first looks for a property with the same name as the parameter. If such a property does not exist, the runtime environment looks for aliases to that parameter, in their declaration order, picking the first such alias for which a property exists.
	
	function Process-Date
	{
	param(
	[Parameter(ValueFromPipelineByPropertyName=$true)] [int]$Year,
	
	[Parameter(ValueFromPipelineByPropertyName=$true)] [int]$Month,
	
	[Parameter(ValueFromPipelineByPropertyName=$true)] [int]$Day
	)
	
	process { ... }
	}
	
	Get-Date | Process-Date

ValueFromRemainingArguments (named)	Type: bool; Default value: $false This argument specifies whether the parameter accepts all of the remaining arguments that are not bound to the parameters of the function. A value of $true indicates that it does. A value of $false indicates that it does not. The following example shows a parameter $Others that accepts all the remaining arguments of the input object that is passed to the function Test:

Parameter	Purpose
	param ([Parameter(Mandatory = $true)][int] $p1, [Parameter(Mandatory = $true)][int] $p2, [Parameter(ValueFromRemainingArguments = $true)] [string[]] $Others) Test 10 20 # $Others has Length 0 Test 10 20 30 40 # $Others has Length 2, value 30,40

An implementation may define other attributes as well.

The following attributes are provided as well:

- **HelpMessageBaseName**: Specifies the location where resource identifiers reside. For example, this parameter could specify a resource assembly that contains Help messages that are to be localized.
- **HelpMessageResourceId**: Specifies the resource identifier for a Help message.

12.3.8 The PSDefaultValue attribute

This attribute is used in a *script-parameter* to provide additional information about the parameter. The attribute is used in an implementation defined manner. The following arguments are used to define the characteristics of the parameter:

Expand table

Parameter Name	Purpose
Help (named)	Type: string This argument specifies a message that is intended to contain a short description of the default value of a parameter. This message is used in an implementation-defined manner. Windows PowerShell: The message is used as part of the description of the parameter for the help topic displayed by the [Get-Help] (xref:Microsoft.PowerShell.Core.Get-Help) cmdlet.
Value (named)	Type: object This argument specifies a value that is intended to be the default value of a parameter. The value is used in an implementation-defined manner. Windows PowerShell: The value is used as part of the description of the parameter for the help topic displayed by the [Get-Help](xref:Microsoft.PowerShell.Core.Get-Help)cmdlet when the Help property is not specified.

12.3.9 The SupportsWildcards attribute

This attribute is used in a *script-parameter* to provide additional information about the parameter. The attribute is used in an implementation defined manner.

This attribute is used as part of the description of the parameter for the help topic displayed by the Get-Help cmdlet.

12.3.10 The ValidateCount attribute

This attribute is used in a *script-parameter* to specify the minimum and maximum number of argument values that the parameter can accept. The following arguments are used to define the characteristics of the parameter:

Expand table

Parameter Name	Purpose
MinLength (position 0)	Type: int This argument specifies the minimum number of argument values allowed.
MaxLength (position 1)	Type: int This argument specifies the maximum number of argument values allowed.

In the absence of this attribute, the parameter's corresponding argument value list can be of any length.

Consider a function call Test that has the following param block, and which is called as shown:

PowerShell

```
param (
    [ValidateCount(2, 5)]
    [int[]] $Values
)

Temp 10, 20, 30
Temp 10                         # too few argument values
Temp 10, 20, 30, 40, 50, 60     # too many argument values

[ValidateCount(3, 4)]$Array = 1..3
```

```
$Array = 10                     # too few argument values
$Array = 1..100                 # too many argument values
```

12.3.11 The ValidateLength attribute

This attribute is used in a *script-parameter* or *variable* to specify the minimum and maximum length of the parameter's argument, which must have type string. The following arguments are used to define the characteristics of the parameter:

Expand table

Parameter Name	Purpose
MinLength (position 0)	Type: int This argument specifies the minimum number of characters allowed.
MaxLength (position 1)	Type: int This argument specifies the maximum number of characters allowed.

In the absence of this attribute, the parameter's corresponding argument can be of any length.

Consider a function call Test that has the following param block, and which is called as shown:

PowerShell

```
param ( [Parameter(Mandatory = $true)]
[ValidateLength(3,6)]
[string[]] $ComputerName )

Test "Thor","Mars"     # length is ok
Test "Io","Mars"       # "Io" is too short
Test "Thor","Jupiter"  # "Jupiter" is too long
```

12.3.12 The ValidateNotNull attribute

This attribute is used in a *script-parameter* or *variable* to specify that the argument of the parameter cannot be `$null` or be a collection containing a null-valued element.

Consider a function call `Test` that has the following param block, and which is called as `shown:

```
```

PowerShell

```
param (
    [ValidateNotNull()]
    [string[]] $Names
)

Test "Jack", "Jill"     # ok
Test "Jane", $null      # $null array element value not allowed
Test $null              # null array not allowed

[ValidateNotNull()]$Name = "Jack" # ok
$Name = $null           # null value not allowed
```

12.3.13 The ValidateNotNullOrEmpty attribute

This attribute is used in a *script-parameter* or *variable* to specify that the argument if the parameter cannot be $null, an empty string, or an empty array, or be a collection containing a $null-valued or empty string element.

Consider a function call `Test` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [ValidateNotNullOrEmpty()]
    [string[]] $Names
)

Test "Jack", "Jill"    # ok
Test "Mary", ""        # empty string not allowed
Test "Jane", $null     # $null array element value not allowed
Test $null             # null array not allowed
Test @()               # empty array not allowed

[ValidateNotNullOrEmpty()]$Name = "Jack" # ok
$Name = ""             # empty string not allowed
$Name = $null          # null value not allowed
```

12.3.14 The ValidatePattern attribute

This attribute is used in a *script-parameter* or *variable* to specify a regular expression for matching the pattern of the parameter's argument. The following arguments are used to define the characteristics of the parameter:

Expand table

Parameter Name	Purpose
RegexString (position 0)	Type: String A regular expression that is used to validate the parameter's argument
Options (named)	Type: Regular-Expression-Option See [§4.2.6.4][§4.2.6.4] for the allowed values.

If the argument is a collection, each element in the collection must match the pattern.

Consider a function call `Test` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [ValidatePattern('\^[A-Z][1-5][0-9]$')]
    [string] $Code,

    [ValidatePattern('\^(0x|0X)([A-F]|[a-f]|[0-9])([A-F]|[a-f]|[0-9])$')]
    [string] $HexNum,

    [ValidatePattern('\^[+|-]?[1-9]$')]
    [int] $Minimum
)

Test -C A12 # matches pattern
Test -C A63 # does not match pattern

Test -H 0x4f # matches pattern
Test -H "0XB2" # matches pattern
Test -H 0xK3 # does not match pattern

Test -M -4 # matches pattern
Test -M "+7" # matches pattern
Test -M -12 # matches pattern, but is too long

[ValidatePattern('\^[a-z][a-z0-9]\*$')]$ident = "abc"
$ident = "123" # does not match pattern
```

12.3.15 The ValidateRange attribute

This attribute is used in a *script-parameter* or *variable* to specify the minimum and maximum values of the parameter's argument. The following arguments are used to define the characteristics of the parameter:

Expand table

Parameter Name	Purpose
MinRange (position 0)	Type: object This argument specifies the minimum value allowed.
MaxRange (position 1)	Type: object This argument specifies the maximum value allowed.

In the absence of this attribute, there is no range restriction.

Consider a function call `Test1` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [Parameter(Mandatory = $true)]
    [ValidateRange(1, 10)]
    [int] $StartValue
)

Test1 2
Test1 -St 7
Test1 -3 # value is too small
Test1 12 # value is too large
```

Consider a function call Test2 that has the following param block and calls:

PowerShell

```
param (
    [Parameter(Mandatory = $true)]
    [ValidateRange("b", "f")]
    [string] $Name
)

Test2 "Bravo" # ok
Test2 "Alpha" # value compares less than the minimum
Test2 "Hotel" # value compares greater than the maximum
```

Consider a function call `Test3` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [Parameter(Mandatory = $true)]
    [ValidateRange(0.002, 0.003)]
    [double] $Distance
)

Test3 0.002
Test3 0.0019    # value is too small
Test3 "0.005"   # value is too large

[ValidateRange(13, 19)]$teenager = 15
$teenager = 20  # value is too large
```

12.3.16 The ValidateScript attribute

This attribute is used in a *script-parameter* or *variable* to specify a script that is to be used to validate the parameter's argument.

The argument in position 1 is a *script-block-expression*.

Consider a function call `Test` that has the following param block, and which is called as shown:

PowerShell

```
param (
    [Parameter(Mandatory = $true)]
    [ValidateScript( { ($_ -ge 1 -and $_ -le 3) -or ($_ -ge 20) })]
    [int] $Count
)

Test 2 # ok, valid value
Test 25 # ok, valid value
Test 5 # invalid value
Test 0 # invalid value

[ValidateScript({$_.Length --gt 7})]$password = "password" # ok
$password = "abc123" # invalid value
```

12.3.17 The ValidateSet attribute

This attribute is used in a *script-parameter* or *variable* to specify a set of valid values for the argument of the parameter. The following arguments are used to define the characteristics of the parameter:

Expand table

Parameter Name	Purpose
ValidValues (position 0)	Type: string[] The set of valid values.
IgnoreCase (named)	Type: bool; Default value: $true Specifies whether case should be ignored for parameters of type string.

If the parameter has an array type, every element of the corresponding argument array must match an element of the value set.

Consider a function call `Test` that has the following param block, and which is called as shown:

PowerShell

```
param ( [ValidateSet("Red", "Green", "Blue")]
    [string] $Color,

    [ValidateSet("up", "down", "left", "right", IgnoreCase =
        $false)]
    [string] $Direction

)

Test -Col "RED"    # case is ignored, is a member of the set
Test -Col "white"  # case is ignored, is not a member of the set

Test -Dir "up"     # case is not ignored, is a member of the set
Test -Dir "Up"     # case is not ignored, is not a member of the set

[ValidateSet(("Red", "Green", "Blue")]$color = "RED" # ok, case is ignored
$color = "Purple"  # case is ignored, is not a member of the set
```

13. Cmdlets

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

A cmdlet is a single-feature command that manipulates objects in PowerShell. Cmdlets can be recognized by their name format, a verb and noun separated by a dash (`-`), such as `Get-Help`, `Get-Process`, and `Start-Service`. A *verb pattern* is a verb expressed using wildcards, as in `W*`. A *noun pattern* is a noun expressed using wildcards, as in *event*.

Cmdlets should be simple and be designed to be used in combination with other cmdlets. For example, **Get** cmdlets should only retrieve data, **Set** cmdlets should only establish or change data, **Format** cmdlets should only format data, and **Out** cmdlets should only direct the output to a specified destination.

For each cmdlet, provide a help file that can be accessed by typing:

```
Get-Help *cmdlet-name* -Detailed
```

The detailed view of the cmdlet help file should include a description of the cmdlet, the command syntax, descriptions of the parameters, and an example that demonstrate the use of that cmdlet.

Cmdlets are used similarly to operating system commands and utilities. PowerShell commands are not case-sensitive.

> **Note**
>
> Editor's note: The original document contains a list of cmdlet with descriptions, syntax diagrams, parameter definitions, and examples. This information is incomplete and out dated. For current information about cmdlet, consult the **Reference** section of the PowerShell documentation.

13.1 Common parameters

The *common parameters* are a set of cmdlet parameters that can be used with any cmdlet. They are implemented by the PowerShell runtime environment itself, not by the cmdlet developer, and they are automatically available to any cmdlet or function that uses the **Parameter** attribute (§12.3.7) or **CmdletBinding** attribute (§12.3.5).

Although the common parameters are accepted by any cmdlet, they might not have any semantics for that cmdlet. For example, if a cmdlet does not generate any verbose output, using the **Verbose** common parameter has no effect.

Several common parameters override system defaults or preferences that can be set via preference variables (§2.3.2.3). Unlike the preference variables, the common parameters affect only the commands in which they are used.

> **Note**
>
> Editor's note: The original document contains a list of the Common Parameters. This information is incomplete and out dated. For current information see about_CommonParameters.

Last updated on 03/24/2025

A. Comment-Based Help

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

PowerShell provides a mechanism for programmers to document their scripts using special comment directives. Comments using such syntax are called *help comments*. The cmdlet Get-Help generates documentation from these directives.

A.1 Introduction

A help comment contains a *help directive* of the form *.name* followed on one or more subsequent lines by the help content text. The help comment can be made up of a series of *single-line-comment*s or a *delimited-comment* (§2.2.3). The set of comments comprising the documentation for a single entity is called a *help topic*.

For example,

```PowerShell
# <help-directive-1>
# <help-content-1>
...

# <help-directive-n>
# <help-content-n>
```

or

PowerShell

```
<#
<help-directive-1>
<help-content-1>
...

<help-directive-n>
<help-content-n>
#>
```

All of the lines in a help topic must be contiguous. If a help topic follows a comment that is not part of that topic, there must be at least one blank line between the two.

The directives can appear in any order, and some of the directives may appear multiple times.

Directive names are not case-sensitive.

When documenting a function, help topics may appear in one of three locations:

- Immediately before the function definition with no more than one blank line between the last line of the function help and the line containing the function statement.
- Inside the function's body immediately following the opening curly bracket.
- Inside the function's body immediately preceding the closing curly bracket.

When documenting a script file, help topics may appear in one of two locations:

- At the beginning of the script file, optionally preceded by comments and blank lines only. If the first item in the script after the help is a function definition, there must be at least two blank lines between the end of the script help and that function declaration. Otherwise, the help will be interpreted as applying to the function instead of the script file.
- At the end of the script file.

A.2 Help directives

A.2.1 .DESCRIPTION

Syntax:

Syntax

```
.DESCRIPTION
```

Description:

This directive allows for a detailed description of the function or script. (The `.SYNOPSIS` directive (§A.2.11) is intended for a brief description.) This directive can be used only once in each topic.

Examples:

PowerShell

```powershell
<#
.DESCRIPTION
Computes Base to the power Exponent. Supports non-negative integer
powers only.
#>
```

A.2.2 .EXAMPLE

Syntax:

Syntax

```
.EXAMPLE
```

Description:

This directive allows an example of command usage to be shown.

If this directive occurs multiple times, each associated help content block is displayed as a separate example.

Examples:

PowerShell

```powershell
<#
.EXAMPLE
Get-Power 3 4
81

.EXAMPLE
Get-Power -Base 3 -Exponent 4
```

```
81
#>
```

A.2.3 .EXTERNALHELP

Syntax:

Syntax

```
.EXTERNALHELP <XMLHelpFilePath>
```

Description:

This directive specifies the path to an XML-based help file for the script or function.

Although comment-based help is easier to implement, XML-based Help is required if more precise control is needed over help content or if help topics are to be translated into multiple languages. The details of XML-based help are not defined by this specification.

Examples:

PowerShell

```
<#
.EXTERNALHELP C:\MyScripts\Update-Month-Help.xml
#>
```

A.2.4 .FORWARDHELPCATEGORY

Syntax:

Syntax

```
.FORWARDHELPCATEGORY <Category>
```

Description:

Specifies the help category of the item in **ForwardHelpTargetName** (§A.2.5). Valid values are **Alias**, **All**, **Cmdlet**, **ExternalScript**, **FAQ**, **Filter**, **Function**, **General**, **Glossary**, **HelpFile**, **Provider**, and **ScriptCommand**. Use this directive to avoid conflicts when there are commands with the same name.

Examples:

See §A.2.5.

A.2.5 .FORWARDHELPTARGETNAME

Syntax:

Syntax

```
.FORWARDHELPTARGETNAME <Command-Name>
```

Description:

Redirects to the help topic specified by `<Command-Name>`.

Examples:

PowerShell

```
function Help {
<#
.FORWARDHELPTARGETNAME Get-Help
.FORWARDHELPCATEGORY Cmdlet
#>
    ...
}
```

The command `Get-Help help` is treated as if it were `Get-Help Get-Help` instead.

A.2.6 .INPUTS

Syntax:

Syntax

```
.INPUTS
```

Description:

The pipeline can be used to pipe one or more objects to a script or function. This directive is used to describe such objects and their types.

If this directive occurs multiple times, each associated help content block is collected in the one documentation entry, in the directives' lexical order.

Examples:

PowerShell

```
<#
.INPUTS
None. You cannot pipe objects to Get-Power.

.INPUTS
For the Value parameter, one or more objects of any kind can be written
to the pipeline. However, the object is converted to a string before it
is added to the item.
#>
function Process-Thing {
    param ( ...
        [Parameter(ValueFromPipeline=$true)]
        [Object[]]$Value,
        ...
    )
    ...
}
```

A.2.7 .LINK

Syntax:

Syntax

```
.LINK
```

Description:

This directive specifies the name of a related topic.

If this directive occurs multiple times, each associated help content block is collected in the one documentation entry, in the directives' lexical order.

The Link directive content can also include a URI to an online version of the same help topic. The online version is opens when Get-Help is invoked with the Online parameter. The URI must begin with "http" or "https".

Examples:

PowerShell

```
<#
.LINK
Online version: http://www.acmecorp.com/widget.html

.LINK
```

```
Set-ProcedureName
#>
```

A.2.8 .NOTES

Syntax:

Syntax

```
.NOTES
```

Description:

This directive allows additional information about the function or script to be provided. This directive can be used only once in each topic.

Examples:

PowerShell

```
<#
.NOTES
*arbitrary text goes here*
#>
```

A.2.9 .OUTPUTS

Syntax:

Syntax

```
.OUTPUTS
```

Description:

This directive is used to describe the objects output by a command.

If this directive occurs multiple times, each associated help content block is collected in the one documentation entry, in the directives' lexical order.

Examples:

PowerShell

```
<#
.OUTPUTS
double - Get-Power returns Base to the power Exponent.

.OUTPUTS
None unless the -PassThru switch parameter is used.
#>
```

A.2.10 .PARAMETER

Syntax:

Syntax

```
.PARAMETER <Parameter-Name>
```

Description:

This directive allows for a detailed description of the given parameter. This directive can be used once for each parameter. Parameter directives can appear in any order in the comment block; however, the order in which their corresponding parameters are actually defined in the source determines the order in which the parameters and their descriptions appear in the resulting documentation.

An alternate format involves placing a parameter description comment immediately before the declaration of the corresponding parameter variable's name. If the source contains both a parameter description comment and a Parameter directive, the description associated with the Parameter directive is used.

Examples:

PowerShell

```
<#
.PARAMETER Base
The integer value to be raised to the Exponent-th power.

.PARAMETER Exponent
The integer exponent to which Base is to be raised.
#>

function Get-Power {
    param ([long]$Base, [int]$Exponent)
    ...
}
```

```
function Get-Power {
    param ([long]
        # The integer value to be raised to the Exponent-th power.
        $Base,
        [int]
        # The integer exponent to which Base is to be raised.
        $Exponent
    )
    ...
}
```

A.2.11 .SYNOPSIS

Syntax:

PowerShell

```
.SYNOPSIS
```

Description:

This directive allows for a brief description of the function or script. (The `.DESCRIPTION` directive (§A.2.1) is intended for a detailed description.) This directive can be used only once in each topic.

Examples:

PowerShell

```
<#
.SYNOPSIS
Computes Base to the power Exponent.
#>
```

Last updated on 03/24/2025

B. Grammar

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389
> That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

This appendix contains summaries of the lexical and syntactic grammars found in the main document.

> **Tip**
>
> The `~opt~` notation in the syntax definitions indicates that the lexical entity is optional in the syntax.

B.1 Lexical grammar

Syntax

```
input:
    input-elements~opt~ signature-block~opt~

input-elements:
    input-element
    input-elements input-element

input-element:
    whitespace
    comment
```

```
    token

signature-block:
    signature-begin signature signature-end

signature-begin:
    new-line-character # SIG # Begin signature block new-line-character

signature:
    base64 encoded signature blob in multiple single-line-comments

signature-end:
    new-line-character # SIG # End signature block new-line-character
```

B.1.1 Line terminators

Syntax

```
new-line-character:
    Carriage return character (U+000D)
    Line feed character (U+000A)
    Carriage return character (U+000D) followed by line feed character (U+000A)

new-lines:
    new-line-character
    new-lines new-line-character
```

B.1.2 Comments

Syntax

```
comment:
    single-line-comment
    requires-comment
    delimited-comment

single-line-comment:
    # input-characters~opt~

input-characters:
    input-character
    input-characters input-character

input-character:
    Any Unicode character except a new-line-character

requires-comment:
    #Requires whitespace command-arguments

dash:
    - (U+002D)
```

```
    EnDash character (U+2013)
    EmDash character (U+2014)
    Horizontal bar character (U+2015)

dashdash:
    dash dash

delimited-comment:
    <# delimited-comment-text~opt~ hashes >

delimited-comment-text:
    delimited-comment-section
    delimited-comment-text delimited-comment-section

delimited-comment-section:
    >
    hashes~opt~ not-greater-than-or-hash

hashes:
    #
    hashes #

not-greater-than-or-hash:
    Any Unicode character except > or #
```

B.1.3 White space

Syntax

```
whitespace:
    Any character with Unicode class Zs, Zl, or Zp
    Horizontal tab character (U+0009)
    Vertical tab character (U+000B)
    Form feed character (U+000C)
    ` (The backtick character U+0060) followed by new-line-character
```

B.1.4 Tokens

Syntax

```
token:
    keyword
    variable
    command
    command-parameter
    command-argument-token
    integer-literal
    real-literal
    string-literal
```

```
    type-literal
    operator-or-punctuator
```

B.1.5 Keywords

Syntax

```
keyword: one of
    begin           break           catch       class
    continue        data            define      do
    dynamicparam    else            elseif      end
    exit            filter          finally     for
    foreach         from            function    if
    in              inlinescript    parallel    param
    process         return          switch      throw
    trap            try             until       using
    var             while           workflow
```

B.1.6 Variables

Syntax

```
variable:
    $$
    $?
    $^
    $   variable-scope~opt~  variable-characters
    @   variable-scope~opt~  variable-characters
    braced-variable


braced-variable:
    ${   variable-scope~opt~   braced-variable-characters   }

variable-scope:
    Global:
    Local:
    Private:
    Script:
    Using:
    Workflow:
    variable-namespace

variable-namespace:
    variable-characters   :

variable-characters:
    variable-character
    variable-characters   variable-character

variable-character:
```

```
    A Unicode character of classes Lu, Ll, Lt, Lm, Lo, or Nd
    _   (The underscore character U+005F)
    ?

braced-variable-characters:
    braced-variable-character
    braced-variable-characters   braced-variable-character

braced-variable-character:
    Any Unicode character except
        }   (The closing curly brace character U+007D)
        `   (The backtick character U+0060)
    escaped-character

escaped-character:
    `   (The backtick character U+0060) followed by any Unicode character
```

B.1.7 Commands

Syntax

```
generic-token:
    generic-token-parts

generic-token-parts:
    generic-token-part
    generic-token-parts generic-token-part

generic-token-part:
    expandable-string-literal
    verbatim-here-string-literal
    variable
    generic-token-char

generic-token-char:
    Any Unicode character except
        {   }   (   )   ;   ,   |   &   $
        ` (The backtick character U+0060)
        double-quote-character
        single-quote-character
        whitespace
        new-line-character
        escaped-character

generic-token-with-subexpr-start:
    generic-token-parts $(
```

B.1.8 Parameters

Syntax

```
command-parameter:
    dash first-parameter-char parameter-chars colon~opt~

first-parameter-char:
    A Unicode character of classes Lu, Ll, Lt, Lm, or Lo
    _ (The underscore character U+005F)
    ?

parameter-chars:
    parameter-char
    parameter-chars parameter-char

parameter-char:
    Any Unicode character except
        { } ( ) ; , | & . [
        colon
        whitespace
        new-line-character

colon:
    : (The colon character U+003A)

verbatim-command-argument-chars:
    verbatim-command-argument-part
    verbatim-command-argument-chars verbatim-command-argument-part

verbatim-command-argument-part:
    verbatim-command-string
    & non-ampersand-character
    Any Unicode character except
        |
        new-line-character

non-ampersand-character:
    Any Unicode character except &

verbatim-command-string:
    double-quote-character non-double-quote-chars
    double-quote-character

non-double-quote-chars:
    non-double-quote-char
    non-double-quote-chars non-double-quote-char

non-double-quote-char:
    Any Unicode character except
        double-quote-character
```

B.1.9 Literals

Syntax

```
literal:
    integer-literal
    real-literal
    string-literal
```

B.1.9.1 Integer Literals

Syntax

```
integer-literal:
    decimal-integer-literal
    hexadecimal-integer-literal

decimal-integer-literal:
    decimal-digits numeric-type-suffix~opt~ numeric-multiplier~opt~

decimal-digits:
    decimal-digit
    decimal-digit decimal-digits

decimal-digit: one of
    0  1  2  3  4  5  6  7  8  9

numeric-type-suffix:
    long-type-suffix
    decimal-type-suffix

hexadecimal-integer-literal:
    0x hexadecimal-digits long-type-suffix~opt~
    numeric-multiplier~opt~

hexadecimal-digits:
    hexadecimal-digit
    hexadecimal-digit decimal-digits

hexadecimal-digit: one of
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f

long-type-suffix:
    l

numeric-multiplier: one of
    kb mb gb tb pb
```

B.1.9.2 Real Literals

Syntax

```
real-literal:
    decimal-digits . decimal-digits exponent-part~opt~ decimal-type-suffix~opt~
numeric-multiplier~opt~
```

```
    . decimal-digits exponent-part~opt~ decimal-type-suffix~opt~ numeric-
multiplier~opt~
    decimal-digits exponent-part decimal-type-suffix~opt~ numeric-multiplier~opt~

exponent-part:
    e sign~opt~  decimal-digits

sign: one of
    +
    dash

decimal-type-suffix:
    d
    l
```

B.1.9.3 String Literals

Syntax

```
string-literal:
    expandable-string-literal
    expandable-here-string-literal
    verbatim-string-literal
    verbatim-here-string-literal

expandable-string-literal:
    double-quote-character expandable-string-characters~opt~  dollars~opt~ double-
quote-character

double-quote-character:
    " (U+0022)
    Left double quotation mark (U+201C)
    Right double quotation mark (U+201D)
    Double low-9 quotation mark (U+201E)

expandable-string-characters:
      expandable-string-part
      expandable-string-characters
      expandable-string-part

expandable-string-part:
    Any Unicode character except
        $
        double-quote-character
        ` (The backtick character U+0060)
    braced-variable
    $ Any Unicode character except
        (
        {
        double-quote-character
        ` (The backtick character U+0060)*
    $ escaped-character
    escaped-character
```

```
    double-quote-character double-quote-character

dollars:
    $
    dollars $

expandable-here-string-literal:
    @  double-quote-character  whitespace~opt~  new-line-character
        expandable-here-string-characters~opt~  new-line-character  double-quote-
character  @

expandable-here-string-characters:
    expandable-here-string-part
    expandable-here-string-characters  expandable-here-string-part

expandable-here-string-part:
    Any Unicode character except
        $
        new-line-character
    braced-variable
    $ Any Unicode character except
        (
        new-line-character
    $ new-line-character  Any Unicode character except double-quote-char
    $ new-line-character double-quote-char  Any Unicode character except @
    new-line-character  Any Unicode character except double-quote-char
    new-line-character double-quote-char  Any Unicode character except @

expandable-string-with-subexpr-start:
    double-quote-character  expandable-string-chars~opt~  $(

expandable-string-with-subexpr-end:
    double-quote-char

expandable-here-string-with-subexpr-start:
    @  double-quote-character whitespace~opt~ new-line-character expandable-here-
string-chars~opt~  $(

expandable-here-string-with-subexpr-end:
    new-line-character  double-quote-character  @

verbatim-string-literal:
    single-quote-character verbatim-string-characters~opt~ single-quote-char

single-quote-character:
    ' (U+0027)
    Left single quotation mark (U+2018)
    Right single quotation mark (U+2019)
    Single low-9 quotation mark (U+201A)
    Single high-reversed-9 quotation mark (U+201B)

verbatim-string-characters:
    verbatim-string-part
    verbatim-string-characters verbatim-string-part
```

```
verbatim-string-part:
    *Any Unicode character except* single-quote-character
    single-quote-character  single-quote-character

verbatim-here-string-literal:
    @ single-quote-character whitespace~opt~  new-line-character
        verbatim-here-string-characters~opt~  new-line-character
            single-quote-character *@*

verbatim-*here-string-characters:
    verbatim-here-string-part
    verbatim-here-string-characters  verbatim-here-string-part

verbatim-here-string-part:
    Any Unicode character except* new-line-character
    new-line-character  Any Unicode character except single-quote-character
    new-line-character  single-quote-character  Any Unicode character except @
```

B.1.10 Simple Names

Syntax

```
simple-name:
    simple-name-first-char simple-name-chars

simple-name-first-char:
    A Unicode character of classes Lu, Ll, Lt, Lm, or Lo
    _ (The underscore character U+005F)

simple-name-chars:
    simple-name-char
    simple-name-chars simple-name-char

simple-name-char:
    A Unicode character of classes Lu, Ll, Lt, Lm, Lo, or Nd
    _ (The underscore character U+005F)
```

B.1.11 Type Names

Syntax

```
type-name:
    type-identifier
    type-name . type-identifier

type-identifier:
    type-characters

type-characters:
    type-character
    type-characters type-character
```

```
type-character:
    A Unicode character of classes Lu, Ll, Lt, Lm, Lo, or Nd
    _ (The underscore character U+005F)

array-type-name:
    type-name [

generic-type-name:
    type-name [
```

B.1.12 Operators and punctuators

Syntax

```
operator-or-punctuator: one of
    {   }   [   ]   (   )   @(   @{   $(   ;
    &&  ||  &   |   ,   ++  ..   ::   .
    !   *   /   %   +
    dash       dashdash
    dash and   dash band   dash bnot   dash bor
    dash bxor  dash not    dash or     dash xor
    assignment-operator
    merging-redirection-operator
    file-redirection-operator
    comparison-operator
    format-operator

assignment-operator: one of
    =    dash =    +=    *=    /=    %=

file-redirection-operator: one of
    >  >>  2>  2>>  3>  3>>  4>  4>>
    5>  5>>  6>  6>>  *>  *>>  <

merging-redirection-operator: one of
    *>&1  2>&1  3>&1  4>&1  5>&1  6>&1
    *>&2  1>&2  3>&2  4>&2  5>&2  6>&2

comparison-operator: one of
    dash as           dash ccontains      dash ceq
    dash cge          dash cgt            dash cle
    dash clike        dash clt            dash cmatch
    dash cne          dash cnotcontains   dash cnotlike
    dash cnotmatch    dash contains       dash creplace
    dash csplit       dash eq             dash ge
    dash gt           dash icontains      dash ieq
    dash ige          dash igt            dash ile
    dash ilike        dash ilt            dash imatch
    dash in           dash ine            dash inotcontains
    dash inotlike     dash inotmatch      dash ireplace
    dash is           dash isnot          dash isplit
    dash join         dash le             dash like
```

```
    dash lt           dash match          dash ne
    dash notcontains  dash notin         dash notlike
    dash notmatch     dash replace       dash shl*
    dash shr          dash split

format-operator:
    dash f
```

B.2 Syntactic grammar

B.2.1 Basic concepts

Syntax

```
script-file:
    script-block

module-file:
    script-block

interactive-input:
    script-block

data-file:
    statement-list
```

B.2.2 Statements

Syntax

```
script-block:
    param-block~opt~ statement-terminators~opt~ script-block-body~opt~

param-block:
    new-lines~opt~ attribute-list~opt~ new-lines~opt~ param new-lines~opt~
        ( parameter-list~opt~ new-lines~opt~ )

parameter-list:
    script-parameter
    parameter-list new-lines~opt~ , script-parameter

script-parameter:
    new-lines~opt~ attribute-list~opt~ new-lines~opt~ variable script-parameter-
default~opt~

script-parameter-default:
    new-lines~opt~ = new-lines~opt~ expression

script-block-body:
    named-block-list
```

```
    statement-list

named-block-list:
    named-block
    named-block-list named-block

named-block:
    block-name statement-block statement-terminators~opt~

block-name: one of
    dynamicparam begin process end

statement-block:
    new-lines~opt~ { statement-list~opt~ new-lines~opt~ }

statement-list:
    statement
    statement-list statement

statement:
    if-statement
    label~opt~ labeled-statement
    function-statement
    flow-control-statement statement-terminator
    trap-statement
    try-statement
    data-statement
    inlinescript-statement
    parallel-statement
    sequence-statement
    pipeline statement-terminator

statement-terminator:
    ;
    new-line-character

statement-terminators:
    statement-terminator
    statement-terminators statement-terminator

if-statement:
    if new-lines~opt~ ( new-lines~opt~ pipeline new-lines~opt~ ) statement-block
        elseif-clauses~opt~ else-clause~opt~

elseif-clauses:
    elseif-clause
    elseif-clauses elseif-clause

elseif-clause:
    new-lines~opt~ elseif new-lines~opt~ ( new-lines~opt~ pipeline new-lines~opt~ )
statement-block

else-clause:
    new-lines~opt~ else statement-block
```

```
labeled-statement:
    switch-statement
    foreach-statement
    for-statement
    while-statement
    do-statement

switch-statement:
    switch new-lines~opt~ switch-parameters~opt~ switch-condition switch-body

switch-parameters:
    switch-parameter
    switch-parameters switch-parameter

switch-parameter:
    -Regex
    -Wildcard
    -Exact
    -CaseSensitive
    -Parallel

switch-condition:
    ( new-lines~opt~ pipeline new-lines~opt~ )
    -file new-lines~opt~ switch-filename

switch-filename:
    command-argument
    primary-expression

switch-body:
    new-lines~opt~ { new-lines~opt~ switch-clauses }

switch-clauses:
    switch-clause
    switch-clauses switch-clause

switch-clause:
    switch-clause-condition statement-block statement-terimators~opt~

switch-clause-condition:
    command-argument
    primary-expression

foreach-statement:
    foreach new-lines~opt~ foreach-parameter~opt~ new-lines~opt~
        ( new-lines~opt~ variable new-lines~opt~ in new-lines~opt~ pipeline
        new-lines~opt~ ) statement-block

foreach-parameter:
    -parallel

for-statement:
    for new-lines~opt~ (
        new-lines~opt~ for-initializer~opt~ statement-terminator
        new-lines~opt~ for-condition~opt~ statement-terminator
```

```
        new-lines~opt~ for-iterator~opt~
        new-lines~opt~ ) statement-block
    for new-lines~opt~ (
        new-lines~opt~ for-initializer~opt~ statement-terminator
        new-lines~opt~ for-condition~opt~
        new-lines~opt~ ) statement-block
    for new-lines~opt~ (
        new-lines~opt~ for-initializer~opt~
        new-lines~opt~ ) statement-block

for-initializer:
    pipeline

for-condition:
    pipeline

for-iterator:
    pipeline

while-statement:
    while new-lines~opt~ ( new-lines~opt~ while-condition new-lines~opt~ )
statement-block

do-statement:
    do statement-block new-lines~opt~ while new-lines~opt~ ( while-condition new-
lines~opt~ )
    do statement-block new-lines~opt~ until new-lines~opt~ ( while-condition new-
lines~opt~ )

while-condition:
    new-lines~opt~ pipeline

function-statement:
    function new-lines~opt~ function-name function-parameter-declaration~opt~ {
script-block }
    filter new-lines~opt~ function-name function-parameter-declaration~opt~ {
script-block }
    workflow new-lines~opt~ function-name function-parameter-declaration~opt~ {
script-block }

function-name:
    command-argument

function-parameter-declaration:
    new-lines~opt~ ( parameter-list new-lines~opt~ )

flow-control-statement:
    break label-expression~opt~
    continue label-expression~opt~
    throw pipeline~opt~
    return pipeline~opt~
    exit pipeline~opt~

label-expression:
    simple-name
```

```
    unary-expression

trap-statement:
    trap new-lines~opt~ type-literal~opt~ new-lines~opt~ statement-block

try-statement:
    try statement-block catch-clauses
    try statement-block finally-clause
    try statement-block catch-clauses finally-clause

catch-clauses:
    catch-clause
    catch-clauses catch-clause

catch-clause:
    new-lines~opt~ catch catch-type-list~opt~ statement-block

catch-type-list:
    new-lines~opt~ type-literal
    catch-type-list new-lines~opt~ , new-lines~opt~ type-literal

finally-clause:
    new-lines~opt~ finally statement-block

data-statement:
    data new-lines~opt~ data-name data-commands-allowed~opt~
    statement-block

data-name:
    simple-name

data-commands-allowed:
    new-lines~opt~ -SupportedCommand data-commands-list

data-commands-list:
    new-lines~opt~ data-command
    data-commands-list , new-lines~opt~ data-command

data-command:
    command-name-expr

inlinescript-statement:
    inlinescript statement-block

parallel-statement:
    parallel statement-block

sequence-statement:
    sequence statement-block

pipeline:
    assignment-expression
    expression redirections~opt~ pipeline-tail~opt~
    command verbatim-command-argument~opt~ pipeline-tail~opt~
```

```
assignment-expression:
    expression assignment-operator statement

pipeline-tail:
    | new-lines~opt~ command
    | new-lines~opt~ command pipeline-tail

command:
    command-name command-elements~opt~
    command-invocation-operator command-module~opt~ command-name-expr command-
elements~opt~

command-invocation-operator: one of
    &   .

command-module:
    primary-expression

command-name:
    generic-token
    generic-token-with-subexpr

generic-token-with-subexpr:
    No whitespace is allowed between ) and command-name.
    generic-token-with-subexpr-start statement-list~opt~ ) command-name

command-name-expr:
    command-name
    primary-expression

command-elements:
    command-element
    command-elements command-element

command-element:
    command-parameter
    command-argument
    redirection

command-argument:
    command-name-expr

verbatim-command-argument:
    --% verbatim-command-argument-chars

redirections:
    redirection
    redirections redirection

redirection:
    merging-redirection-operator
    file-redirection-operator redirected-file-name

redirected-file-name:
```

```
    command-argument
    primary-expression
```

B.2.3 Expressions

Syntax

```
expression:
    logical-expression

logical-expression:
    bitwise-expression
    logical-expression -and new-lines~opt~ bitwise-expression
    logical-expression -or new-lines~opt~ bitwise-expression
    logical-expression -xor new-lines~opt~ bitwise-expression

bitwise-expression:
    comparison-expression
    bitwise-expression -band new-lines~opt~ comparison-expression
    bitwise-expression -bor new-lines~opt~ comparison-expression
    bitwise-expression -bxor new-lines~opt~ comparison-expression

comparison-expression:
    additive-expression
    comparison-expression comparison-operator new-lines~opt~
    additive-expression

additive-expression:
    multiplicative-expression
    additive-expression + new-lines~opt~ multiplicative-expression
    additive-expression dash new-lines~opt~ multiplicative-expression

multiplicative-expression:
    format-expression
    multiplicative-expression \ new-lines~opt~ format-expression
    multiplicative-expression / new-lines~opt~ format-expression
    multiplicative-expression % new-lines~opt~ format-expression

format-expression:
    range-expression
    format-expression format-operator new-lines~opt~ range-expression

range-expression:
    array-literal-expression
    range-expression .. new-lines~opt~ array-literal-expression

array-literal-expression:
    unary-expression
    unary-expression , new-lines~opt~ array-literal-expression

unary-expression:
    primary-expression
    expression-with-unary-operator
```

```
expression-with-unary-operator:
    , new-lines~opt~ unary-expression
    -not new-lines~opt~ unary-expression
    ! new-lines~opt~ unary-expression
    -bnot new-lines~opt~ unary-expression
    + new-lines~opt~ unary-expression
    dash new-lines~opt~ unary-expression
    pre-increment-expression
    pre-decrement-expression
    cast-expression
    -split new-lines~opt~ unary-expression
    -join new-lines~opt~ unary-expression

pre-increment-expression:
    ++ new-lines~opt~ unary-expression

pre-decrement-expression:
    dashdash new-lines~opt~ unary-expression

cast-expression:
    type-literal unary-expression

attributed-expression:
    type-literal variable

primary-expression:
    value
    member-access
    element-access
    invocation-expression
    post-increment-expression
    post-decrement-expression

value:
    parenthesized-expression
    sub-expression
    array-expression
    script-block-expression
    hash-literal-expression
    literal
    type-literal
    variable

parenthesized-expression:
    ( new-lines~opt~ pipeline new-lines~opt~ )

sub-expression:
    $( new-lines~opt~ statement-list~opt~ new-lines~opt~ )

array-expression:
    @( new-lines~opt~ statement-list~opt~ new-lines~opt~ )

script-block-expression:
    { new-lines~opt~ script-block new-lines~opt~ }
```

```
hash-literal-expression:
    @{ new-lines~opt~ hash-literal-body~opt~ new-lines~opt~ }

hash-literal-body:
    hash-entry
    hash-literal-body statement-terminators hash-entry

hash-entry:
    key-expression = new-lines~opt~ statement

key-expression:
    simple-name
    unary-expression

post-increment-expression:
    primary-expression ++

post-decrement-expression:
    primary-expression dashdash

member-access: Note no whitespace is allowed after
    primary-expression.
    primary-expression . member-name
    primary-expression :: member-name

element-access: Note no whitespace is allowed between primary-expression and [.
    primary-expression [ new-lines~opt~ expression new-lines~opt~ ]

invocation-expression: Note no whitespace is allowed after
    primary-expression.
    primary-expression . member-name argument-list
    primary-expression :: member-name argument-list

argument-list:
    ( argument-expression-list~opt~ new-lines~opt~ )

argument-expression-list:
    argument-expression
    argument-expression new-lines~opt~ , argument-expression-list

argument-expression:
    new-lines~opt~ logical-argument-expression

logical-argument-expression:
    bitwise-argument-expression
    logical-argument-expression -and new-lines~opt~ bitwise-argument-expression
    logical-argument-expression -or  new-lines~opt~ bitwise-argument-expression
    logical-argument-expression -xor new-lines~opt~ bitwise-argument-expression

bitwise-argument-expression:
    comparison-argument-expression
    bitwise-argument-expression -band new-lines~opt~ comparison-argument-expression
    bitwise-argument-expression -bor  new-lines~opt~ comparison-argument-expression
    bitwise-argument-expression -bxor new-lines~opt~ comparison-argument-expression
```

```
comparison-argument-expression:
    additive-argument-expression
    comparison-argument-expression comparison-operator
        new-lines~opt~ additive-argument-expression

additive-argument-expression:
    multiplicative-argument-expression
    additive-argument-expression +    new-lines~opt~ multiplicative-argument-
expression
    additive-argument-expression dash new-lines~opt~ multiplicative-argument-
expression

multiplicative-argument-expression:
    format-argument-expression
    multiplicative-argument-expression \ new-lines~opt~ format-argument-expression
    multiplicative-argument-expression / new-lines~opt~ format-argument-expression
    multiplicative-argument-expression % new-lines~opt~ format-argument-expression

format-argument-expression:
    range-argument-expression
    format-argument-expression format-operator new-lines~opt~ range-argument-
expression

range-argument-expression:
    unary-expression
    range-expression .. new-lines~opt~ unary-expression

member-name:
    simple-name
    string-literal
    string-literal-with-subexpression
    expression-with-unary-operator
    value

string-literal-with-subexpression:
    expandable-string-literal-with-subexpr
    expandable-here-string-literal-with-subexpr

expandable-string-literal-with-subexpr:
    expandable-string-with-subexpr-start statement-list~opt~ )
        expandable-string-with-subexpr-characters expandable-string-with-subexpr-
end
    expandable-here-string-with-subexpr-start statement-list~opt~ )
        expandable-here-string-with-subexpr-characters
        expandable-here-string-with-subexpr-end

expandable-string-with-subexpr-characters:
    expandable-string-with-subexpr-part
    expandable-string-with-subexpr-characters expandable-string-with-subexpr-part

expandable-string-with-subexpr-part:
    sub-expression
    expandable-string-part
```

```
expandable-here-string-with-subexpr-characters:
    expandable-here-string-with-subexpr-part
    expandable-here-string-with-subexpr-characters expandable-here-string-with-
subexpr-part

expandable-here-string-with-subexpr-part:
    sub-expression
    expandable-here-string-part

type-literal:
    [ type-spec ]

type-spec:
    array-type-name new-lines~opt~ dimension~opt~ ]
    generic-type-name new-lines~opt~ generic-type-arguments ]
    type-name

dimension:
    ,
    dimension ,

generic-type-arguments:
    type-spec new-lines~opt~
    generic-type-arguments , new-lines~opt~ type-spec
```

B.2.4 Attributes

Syntax

```
attribute-list:
    attribute
    attribute-list new-lines~opt~ attribute

attribute:
    [ new-lines~opt~ attribute-name ( attribute-arguments new-lines~opt~ ) new-
lines~opt~ ]
    type-literal

attribute-name:
    type-spec

attribute-arguments:
    attribute-argument
    attribute-argument new-lines~opt~ , attribute-arguments

attribute-argument:
    new-lines~opt~ expression
    new-lines~opt~ simple-name
    new-lines~opt~ simple-name = new-lines~opt~ expression
```

Last updated on 03/24/2025

C. References

Article • 01/08/2025

Editorial note

> **Important**
>
> The *Windows PowerShell Language Specification 3.0* was published in December 2012 and is based on Windows PowerShell 3.0. This specification does not reflect the current state of PowerShell. There is no plan to update this documentation to reflect the current state. This documentation is presented here for historical reference.
>
> The specification document is available as a Microsoft Word document from the Microsoft Download Center at: https://www.microsoft.com/download/details.aspx?id=36389 That Word document has been converted for presentation here on Microsoft Learn. During conversion, some editorial changes have been made to accommodate formatting for the Docs platform. Some typos and minor errors have been corrected.

ANSI/IEEE 754−2008, *Binary floating-point arithmetic for microprocessor systems*.

ECMA-334, *C# Language Specification*, 4th edition (June 2006), https://www.ecma-international.org/publications-and-standards/standards/ecma-334/. [This Ecma publication is also approved as ISO/IEC 23270:2006.]

The Open Group Base Specifications: Pattern Matching, IEEE Std 1003.1, 2004 Edition. http://www.opengroup.org/onlinepubs/000095399/utilities/xcu_chap02.html#tag_02_13_01

The Open Group Base Specifications: Regular Expressions, IEEE Std 1003.1, 2004 Edition. http://www.opengroup.org/onlinepubs/000095399/basedefs/xbd_chap09.html.

Ecma Technical Report TR/84, *Common Language Infrastructure (CLI) - Information Derived from Partition IV XML File*, 4th edition (June 2006), https://www.ecma-international.org/publications-and-standards/technical-reports/ecma-tr-84/. This TR was also published as ISO/IEC TR 23272:2006.

ISO 639-1, *Codes for the representation of names of languages - Part 1: Alpha-2 code*.

ISO 3166-1, *Codes for the representation of names of countries and their subdivisions - Part 1: Country codes*.

ISO/IEC 10646-1/AMD1:1996, Amendment 1 to ISO/IEC 10646-1:1993, *Transformation Format for 16 planes of group 00 (UTF-16)*.

The Unicode Standard, Edition 5.2. The Unicode Consortium, http://www.unicode.org/standard/standard.html .

Windows PowerShell

Updated: July 8, 2013

Windows PowerShell® is a task-based command-line shell and scripting language designed especially for system administration. Built on the .NET Framework, Windows PowerShell® helps IT professionals and power users control and automate the administration of the Windows operating system and applications that run on Windows.

The documents published here are written primarily for cmdlet, provider, and host application developers who require reference information about the APIs provided by Windows PowerShell. However, system administrators might also find the information provided by these documents useful.

For the basic information needed to start using Windows PowerShell, see Getting Started with Windows PowerShell .

Windows PowerShell Documents

- Installing the Windows PowerShell SDK Provides information about how to install the Windows PowerShell SDK.
- Writing a Windows PowerShell Module Provides information for administrators, script developers, and cmdlet developers who need to package and distribute their Windows PowerShell solutions.
- Writing a Windows PowerShell Cmdlet Provides information for designing and implementing cmdlets.
- Writing a Windows PowerShell Provider Provides information for designing and implementing Windows PowerShell providers. It will help you understand how Windows PowerShell providers work, and it provides sample code that you can use to start designing or writing your own providers.
- Writing a Windows PowerShell Host Application Provides information that can be used by program managers who are designing host applications and by developers who are implementing them. The host application can, define the runspace where commands are run, open sessions on a local or remote computer, and invoke the commands either synchronously or asynchronously based on the needs of the application.

- Writing a PowerShell Formatting File Provides information for the authoring of formatting files, which control the display format for the objects that are returned by commands (cmdlets, functions, and scripts).
- Windows PowerShell Reference Provides reference content for the APIs used in writing cmdlets, providers, and host applications, as well as other supporting APIs.

Last updated on 05/20/2025

Installing the Windows PowerShell SDK

Applies To: Windows PowerShell 2.0, Windows PowerShell 3.0

The following topic describes how to install the PowerShell SDK on different versions of Windows.

Installing Windows PowerShell 3.0 SDK for Windows 8 and Windows Server 2012

Windows PowerShell 3.0 is automatically installed with Windows 8 and Windows Server 2012. In addition, you can download and install the reference assemblies for Windows PowerShell 3.0 as part of the Windows 8 SDK. These assemblies allow you to write cmdlets, providers, and host programs for Windows PowerShell 3.0. When you install the Windows SDK for Windows 8, the Windows PowerShell assemblies are automatically installed in the reference assembly folder, in `\Program Files (x86)\Reference Assemblies\Microsoft\WindowsPowerShell\3.0`. For more information, see the Windows 8 SDK download site. Windows PowerShell code samples are also available in the powershell-sdk-samples repository.

Reference assemblies

Reference assemblies are installed in the following location by default: `C:\Program Files\Reference Assemblies\Microsoft\WindowsPowerShell\V1.0`.

> **Note**
>
> Code that is compiled against the Windows PowerShell 2.0 assemblies cannot be loaded into Windows PowerShell 1.0 installations. However, code that is compiled against the Windows PowerShell 1.0 assemblies can be loaded into Windows PowerShell 2.0 installations.

Samples

Code samples are installed in the following location by default: `C:\Program Files\Microsoft SDKs\Windows\v7.0\Samples\sysmgmt\WindowsPowerShell\`. The following sections provide a brief description of what each sample does.

Cmdlet samples

- GetProcessSample01 - Shows how to write a simple cmdlet that gets all the processes on the local computer.
- GetProcessSample02 - Shows how to add parameters to the cmdlet. The cmdlet takes one or more process names and returns the matching processes.
- GetProcessSample03 - Shows how to add parameters that accept input from the pipeline.
- GetProcessSample04 - Shows how to handle non-terminating errors.
- GetProcessSample05 - Shows how to display a list of specified processes.
- SelectObject - Shows how to write a filter to select only certain objects.
- SelectString - Shows how to search files for specified patterns.
- StopProcessSample01 - Shows how to implement a PassThru parameter, and how to request user feedback by calls to the ShouldProcess and ShouldContinue methods. Users specify the PassThru parameter when they want to force the cmdlet to return an object,
- StopProcessSample02 - Shows how to stop a specific process.
- StopProcessSample03 - Shows how to declare aliases for parameters and how to support wildcards.
- StopProcessSample04 - Shows how to declare parameter sets, the object that the cmdlet takes as input, and how to specify the default parameter set to use.

Remoting samples

- RemoteRunspace01 - Shows how to create a remote runspace that is used to establish a remote connection.
- RemoteRunspacePool01 - Shows how to construct a remote runspace pool and how to run multiple commands concurrently by using this pool.
- Serialization01 - Shows how to look at an existing .NET class and make sure that information from selected public properties of this class is preserved across serialization/deserialization.
- Serialization02 - Shows how to look at an existing .NET class and make sure that information from instance of this class is preserved across serialization/deserialization when the information is not available in public properties of the class.
- Serialization03 - Shows how to look at an existing .NET class and make sure that instances of this class and of derived classes are deserialized (rehydrated) into live .NET objects.

Event samples

- Event01 - Shows how to create a cmdlet for event registration by deriving from ObjectEventRegistrationBase.

- Event02 - Shows how to shows how to receive notifications of Windows PowerShell events that are generated on remote computers. It uses the PSEventReceived event exposed through the Runspace class.

Hosting application samples

- Runspace01 - Shows how to use the PowerShell class to run the `Get-Process` cmdlet synchronously. The `Get-Process` cmdlet returns Process objects for each process running on the local computer.
- Runspace02 - Shows how to use the PowerShell class to run the `Get-Process` and `Sort-Object` cmdlets synchronously. The `Get-Process` cmdlet returns Process objects for each process running on the local computer, and the `Sort-Object` sorts the objects based on their Id property. The results of these commands is displayed by using a DataGridView control.
- Runspace03 - Shows how to use the PowerShell class to run a script synchronously, and how to handle non-terminating errors. The script receives a list of process names and then retrieves those processes. The results of the script, including any non-terminating errors that were generated when running the script, are displayed in a console window.
- Runspace04 - Shows how to use the PowerShell class to run commands, and how to catch terminating errors that are thrown when running the commands. Two commands are run, and the last command is passed a parameter argument that is not valid. As a result, no objects are returned and a terminating error is thrown.
- Runspace05 - Shows how to add a snap-in to an InitialSessionState object so that the cmdlet of the snap-in is available when the runspace is opened. The snap-in provides a Get-Proc cmdlet (defined by the GetProcessSample01 Sample) that is run synchronously using a PowerShell object.
- Runspace06 - Shows how to add a module to an InitialSessionState object so that the module is loaded when the runspace is opened. The module provides a Get-Proc cmdlet (defined by the GetProcessSample02 Sample) that is run synchronously using a PowerShell object.
- Runspace07 - Shows how to create a runspace, and then use that runspace to run two cmdlets synchronously using a PowerShell object.
- Runspace08 - Shows how to add commands and arguments to the pipeline of a PowerShell object and how to run the commands synchronously.
- Runspace09 - Shows how to add a script to the pipeline of a PowerShell object and how to run the script asynchronously. Events are used to handle the output of the script.
- Runspace10 - Shows how to create a default initial session state, how to add a cmdlet to the InitialSessionState, how to create a runspace that uses the initial session state, and

how to run the command using a PowerShell object.

- Runspace11 - Shows how to use the ProxyCommand class to create a proxy command that calls an existing cmdlet, but restricts the set of available parameters. The proxy command is then added to an initial session state that is used to create a constrained runspace. This means that the user can access the functionality of the cmdlet only through the proxy command.
- PowerShell01 - Shows how to create a constrained runspace using an InitialSessionState object.
- PowerShell02 - Shows how to use a runspace pool to run multiple commands concurrently.

Host samples

- Host01 - Shows how to implement a host application that uses a custom host. In this sample a runspace is created that uses the custom host, and then the PowerShell API is used to run a script that calls `exit`. The host application then looks at the output of the script and prints out the results.
- Host02 - Shows how to write a host application that uses the Windows PowerShell runtime along with a custom host implementation. The host application sets the host culture to German, runs the `Get-Process` cmdlet and displays the results as you would see them by using pwrsh.exe, and then prints out the current data and time in German.
- Host03 - Shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console.
- Host04 - Shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. This host application also supports displaying prompts that allow the user to specify multiple choices.
- Host05 - Shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. This host application also supports calls to remote computers by using the `Enter-PSSession` and `Exit-PSSession` cmdlets.
- Host06 - Shows how to build an interactive console-based host application that reads commands from the command line, executes the commands, and then displays the results to the console. In addition, this sample uses the Tokenizer APIs to specify the color of the text that is entered by the user.

Provider samples

- AccessDBProviderSample01 - Shows how to declare a provider class that derives directly from the CmdletProvider class. It is included here only for completeness.
- AccessDBProviderSample02 - Shows how to overwrite the NewDrive and RemoveDrive methods to support calls to the `New-PSDrive` and `Remove-PSDrive` cmdlets. The provider class in this sample derives from the DriveCmdletProvider class.
- AccessDBProviderSample03 - Shows how to overwrite the GetItem and SetItem methods to support calls to the `Get-Item` and `Set-Item` cmdlets. The provider class in this sample derives from the ItemCmdletProvider class.
- AccessDBProviderSample04 - Shows how to overwrite container methods to support calls to the `Copy-Item`, `Get-ChildItem`, `New-Item`, and `Remove-Item` cmdlets. These methods should be implemented when the data store contains items that are containers. A container is a group of child items under a common parent item. The provider class in this sample derives from the ItemCmdletProvider class.
- AccessDBProviderSample05 - Shows how to overwrite container methods to support calls to the `Move-Item` and `Join-Path` cmdlets. These methods should be implemented when the user needs to move items within a container and if the data store contains nested containers. The provider class in this sample derives from the NavigationCmdletProvider class.
- AccessDBProviderSample06 - Shows how to overwrite content methods to support calls to the `Clear-Content`, `Get-Content`, and `Set-Content` cmdlets. These methods should be implemented when the user needs to manage the content of the items in the data store. The provider class in this sample derives from the NavigationCmdletProvider class, and it implements the IContentCmdletProvider interface.

Last updated on 05/20/2025

Windows PowerShell Reference

Windows PowerShell is a Microsoft .NET Framework-connected environment designed for administrative automation. Windows PowerShell provides a new approach to building commands, composing solutions, and creating graphical user interface-based management tools.

Windows PowerShell enables a system administrator to automate the administration of system resources by the execution of commands either directly or through scripts.

Developer Audience

The Windows PowerShell Software Development Kit (SDK) is written for command developers who require reference information about the APIs provided by Windows PowerShell. Command developers use Windows PowerShell to create both commands and providers that extend the tasks that can be performed by Windows PowerShell.

Windows PowerShell Resources

In addition to the Windows PowerShell SDK, the following resources provide more information.

Getting Started with Windows PowerShell Provides an introduction to Windows PowerShell: the language, the cmdlets, the providers, and the use of objects.

Writing a Windows PowerShell Module Provides information and examples for administrators, script developers, and cmdlet developers who need to package and distribute their Windows PowerShell solutions using Windows PowerShell modules.

Writing a Windows PowerShell Cmdlet Provides information and code examples for program managers who are designing cmdlets and for developers who are implementing cmdlet code.

Windows PowerShell Team Blog The best resource for learning from and collaborating with other Windows PowerShell users. Read the Windows PowerShell Team blog, and then join the Windows PowerShell User Forum (microsoft.public.windows.powershell). Use Windows Live Search to find other Windows PowerShell blogs and resources. Then, as you develop your expertise, freely contribute your ideas.

PowerShell module browser Provides the latest versions of the command-line Help topics.

Class Libraries

System.Management.Automation This namespace is the root namespace for Windows PowerShell. It contains the classes, enumerations, and interfaces required to implement custom cmdlets. In particular, the System.Management.Automation.Cmdlet class is the base class from which all cmdlet classes must be derived. For more information about cmdlets, see.

System.Management.Automation.Provider This namespace contains the classes, enumerations, and interfaces required to implement a Windows PowerShell provider. In particular, the System.Management.Automation.Provider.CmdletProvider class is the base class from which all Windows PowerShell provider classes must be derived.

Microsoft.PowerShell.Commands This namespace contains the classes for the cmdlets and providers implemented by Windows PowerShell. Similarly, it is recommended that you create a *YourName*.Commands namespace for those cmdlets that you implement.

System.Management.Automation.Host This namespace contains the classes, enumerations, and interfaces that the cmdlet uses to define the interaction between the user and Windows PowerShell.

System.Management.Automation.Internal This namespace contains the base classes used by other namespace classes. For example, the System.Management.Automation.Internal.CmdletMetadataAttribute class is the base class for the System.Management.Automation.CmdletAttribute class.

System.Management.Automation.Runspaces This namespace contains the classes, enumerations, and interfaces used to create a Windows PowerShell runspace. In this context, the Windows PowerShell runspace is the context in which one or more Windows PowerShell pipelines invoke cmdlets. That is, cmdlets work within the context of a Windows PowerShell runspace. For more information aboutWindows PowerShell runspaces, see Windows PowerShell Runspaces.

Last updated on 05/20/2025

What's New

Windows PowerShell 2.0 provides the following new features for use when writing cmdlets, providers, and host applications.

Modules

You can now package and distribute Windows PowerShell solutions by using modules. Modules allow you to partition, organize, and abstract your Windows PowerShell code into self-contained, reusable units. For more information about modules, see Writing a Windows PowerShell Module.

The PowerShell class

The PowerShell class provides a simpler solution for creating applications, referred to as host applications, that programmatically run commands. This class allows you to create a pipeline of commands, specify the runspace that is used to run the commands, and specify invoking the commands synchronously or asynchronously.

The RunspacePool class

Runspace pools allow you to create multiple runspaces by using a single call. The CreateRunspacePool method provides several overloads that can be used to create runspaces that have the same features, such as the same host, initial session state, and connection information.

The InitialSessionState class

The InitialSessionState class allows you to create a session state configuration that is used when a runspace is opened. You can create a custom configuration, a default configuration that includes the commands provided by mshshort, and a configuration whose commands are restricted based on the capabilities of the session.

Remote runspaces

You can now create runspaces that can be opened on remote computers, allowing you to run commands on the remote machine and collect the results locally. To create a remote runspace,

you must specify information about the remote connection when creating the runspace. See the CreateRunspace and CreateRunspacePool methods for examples. The connection information is defined by the RunspaceConnectionInfo class.

Private runspace elements

You can now create runspaces whose elements are public or private. This allows you to create runspaces whose elements are available to the runspace, but are not available to the user. See the ConstrainedSessionStateEntry class to find out which elements of the runspace can be made private.

Runspace threading modes and apartment state

You can now specify how threads are created and used when running commands in a runspace. See the System.Management.Automation.Runspaces.Runspace.ThreadOptions and System.Management.Automation.Runspaces.RunspacePool.ThreadOptions properties.

You can now get the apartment state of the threads that are used to run commands in a runspace. See the System.Management.Automation.Runspaces.Runspace.ApartmentState and System.Management.Automation.Runspaces.RunspacePool.ApartmentState properties.

Transaction cmdlets

You can now create cmdlets that can be used within a transaction. When a cmdlet is used in a transaction, its actions are temporary, and they can be accepted or rejected by the transaction cmdlets provided by Windows PowerShell.

For more information about transactions, see How to Support Transactions.

Transaction provider

You can now create providers that can be used within a transaction. Similar to cmdlets, when a provider is used in a transaction, its actions are temporary, and they can be accepted or rejected by the transaction cmdlets provided by Windows PowerShell.

For more information about specifying support for transaction within a provider class, see the System.Management.Automation.Provider.CmdletProviderAttribute.ProviderCapabilities property.

Job cmdlets

You can now write cmdlets that can perform their action as a job. These jobs are run in the background without interacting with the current session. For more information about how Windows PowerShell supports jobs, see Background Jobs.

Cmdlet output types

You can now specify the .NET Framework types that are returned by your cmdlets by declaring the OutputType attribute when writing your cmdlets. This will allow others to determine what type of objects are returned by a cmdlet by looking at the OutputType property of the cmdlet.

Event support

You can now write cmdlets that add and consume events. See the PSEvent class.

Proxy commands

You can now write proxy commands that can be used to run another command. A proxy command allows you to control what functionality of the source cmdlet is available to the user. For example, you can create a proxy command that removes a parameter that is supplied by the source command. See the ProxyCommand class.

Multiple choice prompts

You can now write applications that can provide prompts that allow the user to select multiple choices. See the IHostUISupportsMultipleChoiceSelection interface

Interactive sessions

You can now write applications that can start and stop an interactive session on a remote computer. See the IHostSupportsInteractiveSession interface.

Custom Cmdlet Help for Providers

You can now create customized Help topics for the provider cmdlets. Custom cmdlet help topics can explain how the cmdlet works in the provider path and document special features, including the dynamic parameters that the provider adds to the cmdlet.

Last updated on 05/20/2025

Cmdlet Overview

Commands native to PowerShell are known as cmdlets (pronounced command-lets). A cmdlet is implemented in a .NET class that's compiled into a .NET assembly. The PowerShell runtime invokes these cmdlets within the context of automation scripts that are provided at the command line. The PowerShell runtime also invokes them programmatically through PowerShell APIs.

Cmdlets

Cmdlets perform an action and typically return a .NET object to the next command in the pipeline. A cmdlet is a single command that participates in the pipeline semantics of PowerShell. This includes binary (C#) cmdlets, advanced script functions, and CDXML commands.

This SDK documentation describes how to create binary cmdlets written in C#. For information about script-based cmdlets, see:

- about_Functions_Advanced
- about_Functions_CmdletBindingAttribute
- about_Functions_Advanced_Methods

To create a binary cmdlet, you must implement a cmdlet class that derives from one of two specialized cmdlet base classes. The derived class must:

- Declare an attribute that identifies the derived class as a cmdlet.
- Define public properties that are decorated with attributes that identify the public properties as cmdlet parameters.
- Override one or more of the input processing methods to process records.

You can load the assembly that contains the class directly by using the Import-Module cmdlet, or you can create a host application that loads the assembly by using the System.Management.Automation.Runspaces.InitialSessionState API. Both methods provide programmatic and command-line access to the functionality of the cmdlet.

Cmdlet Terms

The following terms are used frequently in the PowerShell cmdlet documentation:

Cmdlet attribute

A .NET attribute that's used to declare a cmdlet class as a cmdlet. Although PowerShell uses several other attributes that are optional, the Cmdlet attribute is required. For more information about this attribute, see Cmdlet Attribute Declaration.

Cmdlet parameter

The public properties that define the parameters that are available to the user or to the application that's running the cmdlet. Cmdlets can have required, named, positional, and `[switch]` parameters. `[switch]` parameters allow you to define parameters that are evaluated only if the parameters are specified in the call. For more information about the different types of parameters, see Cmdlet Parameters.

Parameter set

A group of parameters that can be used in the same command to perform a specific action. A cmdlet can have multiple parameter sets, but each parameter set must have at least one parameter that's unique. Good cmdlet design strongly suggests that the unique parameter also be a required parameter. For more information about parameter sets, see Cmdlet Parameter Sets.

Dynamic parameter

A parameter that's added to the cmdlet at runtime. Typically, the dynamic parameters are added to the cmdlet when another parameter is set to a specific value. For more information about dynamic parameters, see Cmdlet Dynamic Parameters.

Input processing methods

The System.Management.Automation.Cmdlet class provides the following virtual methods that are used to process records. All the derived cmdlet classes must override one or more of the first three methods:

- System.Management.Automation.Cmdlet.BeginProcessing: Used to provide optional one-time, pre-processing functionality for the cmdlet.
- System.Management.Automation.Cmdlet.ProcessRecord: Used to provide record-by-record processing functionality for the cmdlet. The System.Management.Automation.Cmdlet.ProcessRecord method might be called any number of times, or not at all, depending on the input of the cmdlet.
- System.Management.Automation.Cmdlet.EndProcessing: Used to provide optional one-time, post-processing functionality for the cmdlet.

- System.Management.Automation.Cmdlet.StopProcessing: Used to stop processing when the user stops the cmdlet asynchronously (for example, by pressing `CTRL` + `C`).

For more information about these methods, see Cmdlet Input Processing Methods.

When you implement a cmdlet, you must override at least one of these input processing methods. Typically, the **ProcessRecord()** is the method that you override because it's called for every record that the cmdlet processes. In contrast, the **BeginProcessing()** method and the **EndProcessing()** method are called one time to perform pre-processing or post-processing of the records. For more information about these methods, see Input Processing Methods.

ShouldProcess feature

PowerShell allows you to create cmdlets that prompt the user for feedback before the cmdlet makes a change to the system. To use this feature, the cmdlet must declare that it supports the `ShouldProcess` feature when you declare the Cmdlet attribute, and the cmdlet must call the System.Management.Automation.Cmdlet.ShouldProcess and System.Management.Automation.Cmdlet.ShouldContinue methods from within an input processing method. For more information about how to support the `ShouldProcess` functionality, see Requesting Confirmation.

Transaction

A logical group of commands that are treated as a single task. The task automatically fails if any command in the group fails, and the user has the choice to accept or reject the actions performed within the transaction. To participate in a transaction, the cmdlet must declare that it supports transactions when the Cmdlet attribute is declared. Support for transactions was introduced in Windows PowerShell 2.0. For more information about transactions, see How to Support Transactions.

How Cmdlets Differ from Commands

Cmdlets differ from commands in other command-shell environments in the following ways:

- Cmdlets are instances of .NET classes; they aren't stand-alone executables.
- Cmdlets can be created from as few as a dozen lines of code.
- Cmdlets don't generally do their own parsing, error presentation, or output formatting. Parsing, error presentation, and output formatting are handled by the PowerShell runtime.
- Cmdlets process input objects from the pipeline rather than from streams of text, and cmdlets typically deliver objects as output to the pipeline.

- Cmdlets are record-oriented because they process a single object at a time.

Cmdlet Base Classes

Windows PowerShell supports cmdlets that are derived from the following two base classes.

- Most cmdlets are based on .NET classes that derive from the System.Management.Automation.Cmdlet base class. Deriving from this class allows a cmdlet to use the minimum set of dependencies on the Windows PowerShell runtime. This has two benefits. The first benefit is that the cmdlet objects are smaller, and you are less likely to be affected by changes to the PowerShell runtime. The second benefit is that, if you have to, you can directly create an instance of the cmdlet object and then invoke it directly instead of invoking it through the PowerShell runtime.

- The more-complex cmdlets are based on .NET classes that derive from the System.Management.Automation.PSCmdlet base class. Deriving from this class gives you much more access to the PowerShell runtime. This access allows your cmdlet to call scripts, to access providers, and to access the current session state. (To access the current session state, you get and set session variables and preferences.) However, deriving from this class increases the size of the cmdlet object, and it means that your cmdlet is more tightly coupled to the current version of the PowerShell runtime.

In general, unless you need the extended access to the PowerShell runtime, you should derive from the System.Management.Automation.Cmdlet class. However, the PowerShell runtime has extensive logging capabilities for the execution of cmdlets. If your auditing model depends on this logging, you can prevent the execution of your cmdlet from within another cmdlet by deriving from the System.Management.Automation.PSCmdlet class.

Cmdlet Attributes

PowerShell defines several .NET attributes that are used to manage cmdlets and to specify common functionality that's provided by PowerShell and that might be required by the cmdlet. For example, attributes are used to designate a class as a cmdlet, to specify the parameters of the cmdlet, and to request the validation of input so that cmdlet developers don't have to implement that functionality in their cmdlet code. For more information about attributes, see PowerShell Attributes.

Cmdlet Names

PowerShell uses a verb-and-noun name pair to name cmdlets. For example, the `Get-Command` cmdlet included in PowerShell is used to get all the cmdlets that are registered in the command shell. The verb identifies the action that the cmdlet performs, and the noun identifies the resource on which the cmdlet performs its action.

These names are specified when the .NET class is declared as a cmdlet. For more information about how to declare a .NET class as a cmdlet, see Cmdlet Attribute Declaration.

Writing Cmdlet Code

This document provides two ways to discover how cmdlet code is written. If you prefer to see the code without much explanation, see Examples of Cmdlet Code. If you prefer more explanation about the code, see the GetProc Tutorial, StopProc Tutorial, or SelectStr Tutorial topics.

For more information about the guidelines for writing cmdlets, see Cmdlet Development Guidelines.

See Also

- PowerShell Cmdlet Concepts
- Writing a PowerShell Cmdlet
- PowerShell SDK

Last updated on 07/13/2026

Windows PowerShell Cmdlet Concepts

This section describes how cmdlets work.

In This Section

This section includes the following topics.

Cmdlet Development Guidelines This topic provides development guidelines that can be used to produce well-formed cmdlets.

Cmdlet Class Declaration This topic describes cmdlet class declaration.

Approved Verbs for Windows PowerShell Commands This topic lists the predefined cmdlet verbs that you can use when you declare a cmdlet class.

Cmdlet Input Processing Methods This topic describes the methods that allow a cmdlet to perform preprocessing operations, input processing operations, and post processing operations.

Cmdlet Parameters This section describes the different types of parameters that you can add to cmdlets.

Cmdlet Attributes This section describes the attributes that are used to declare .NET Framework classes as cmdlets, to declare fields as cmdlet parameters, and to declare input validation rules for parameters.

Cmdlet Aliases This topic describes cmdlet aliases.

Cmdlet Output This section describes the type of output that cmdlets can return and how to define and display the objects that are returned by cmdlets.

Registering Cmdlets This section describes how to register cmdlets by using modules and snap-ins.

Requesting Confirmation This section describes how cmdlets request confirmation from a user before they make a change to the system.

Windows PowerShell Error Reporting This section describes how cmdlets report terminating errors and non-terminating errors, and it describes how to interpret error records.

Background Jobs This topic describes how cmdlets can perform their work within background jobs that do not interfere with the commands that are executing in the current session.

Invoking Cmdlets and Scripts Within a Cmdlet This topic describes how cmdlets can invoke other cmdlets and scripts from within their input processing methods.

Cmdlet Sets This topic describes using base classes to create sets of cmdlets.

Windows PowerShell Session State This topic describes Windows PowerShell session state.

Last updated on 05/20/2025

Cmdlet Development Guidelines

The topics in this section provide development guidelines that you can use to produce well-formed cmdlets. By leveraging the common functionality provided by the Windows PowerShell runtime and by following these guidelines, you can develop robust cmdlets with minimal effort and provide the user with a consistent experience. Additionally, you will reduce the test burden because common functionality does not require retesting.

In This Section

- Required Development Guidelines
- Strongly Encouraged Development Guidelines
- Advisory Development Guidelines

See Also

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK

Last updated on 05/20/2025

Required Development Guidelines

The following guidelines must be followed when you write your cmdlets. They're separated into guidelines for designing cmdlets and guidelines for writing your cmdlet code. If you don't follow these guidelines, your cmdlets could fail, and your users might have a poor experience when they use your cmdlets.

In this Topic

Design Guidelines

- Use only approved verbs (RD01)
- Cmdlet names: characters that can't be used (RD02)
- Parameter names that can't be used (RD03)
- Support confirmation requests (RD04)
- Support force parameter for interactive sessions (RD05)
- Document output objects (RD06)

Code Guidelines

- Derive from the Cmdlet or PSCmdlet classes (RC01)
- Specify the Cmdlet attribute (RC02)
- Override an input processing method (RC03)
- Specify the OutputType attribute (RC04)
- Don't retain handles to output objects (RC05)
- Handle errors robustly (RC06)
- Use a Windows PowerShell module to deploy your cmdlets (RC07)

Design Guidelines

The following guidelines must be followed when designing cmdlets to ensure a consistent user experience between using your cmdlets and other cmdlets. When you find a Design guideline that applies to your situation, be sure to look at the Code guidelines for similar guidelines.

Use Only Approved Verbs (RD01)

The verb specified in the Cmdlet attribute must come from the recognized set of verbs provided by Windows PowerShell. It must not be one of the prohibited synonyms. Use the constant strings that are defined by the following enumerations to specify cmdlet verbs:

- System.Management.Automation.VerbsCommon
- System.Management.Automation.VerbsCommunications
- System.Management.Automation.VerbsData
- System.Management.Automation.VerbsDiagnostic
- System.Management.Automation.VerbsLifecycle
- System.Management.Automation.VerbsSecurity
- System.Management.Automation.VerbsOther

For more information about the approved verb names, see Cmdlet Verbs.

Users need a set of discoverable and expected cmdlet names. Use the appropriate verb so that the user can make a quick assessment of what a cmdlet does and to easily discover the capabilities of the system. For example, the following command-line command gets a list of all the commands on the system whose names begin with "Start": `Get-Command Start-*`. Use the nouns in your cmdlets to differentiate your cmdlets from other cmdlets. The noun indicates the resource on which the operation will be performed. The operation itself is represented by the verb.

Cmdlet Names: Characters that can't be Used (RD02)

When you name cmdlets, don't use any of the following special characters.

Expand table

Character	Name
`#`	number sign
`,`	comma
`( )`	parentheses
`{ }`	braces
`[ ]`	brackets
`&`	ampersand
`-`	hyphen

Character	Name
`/`	slash mark
`\`	backslash
`$`	dollar sign
`^`	caret
`;`	semicolon
`:`	colon
`"`	double quotation mark
`'`	single quotation mark
`< >`	angle brackets
`|`	vertical bar
`?`	question mark
`@`	at sign
`` ` ``	back tick (grave accent)
`*`	asterisk
`%`	percent sign
`+`	plus sign
`=`	equals sign
`~`	tilde

Parameter names that can't be used (RD03)

Windows PowerShell provides a common set of parameters to all cmdlets plus additional parameters that are added in specific situations. When designing your own cmdlets you can't use the following names: `Confirm`, `Debug`, `ErrorAction`, `ErrorVariable`, `OutBuffer`, `OutVariable`, `WarningAction`, `WarningVariable`, `WhatIf`, `UseTransaction`, and `Verbose`. For more information about these parameters, see Common Parameter Names.

Support confirmation requests (RD04)

For cmdlets that perform an operation that modifies the system, they should call the System.Management.Automation.Cmdlet.ShouldProcess* method to request confirmation, and in special cases call the System.Management.Automation.Cmdlet.ShouldContinue* method. (The System.Management.Automation.Cmdlet.ShouldContinue* method should be called only after the System.Management.Automation.Cmdlet.ShouldProcess* method is called.)

To make these calls the cmdlet must specify that it supports confirmation requests by setting the `SupportsShouldProcess` keyword of the Cmdlet attribute. For more information about setting this attribute, see Cmdlet Attribute Declaration.

> **Note**
>
> If the Cmdlet attribute of the cmdlet class indicates that the cmdlet supports calls to the System.Management.Automation.Cmdlet.ShouldProcess* method, and the cmdlet fails to make the call to the System.Management.Automation.Cmdlet.ShouldProcess* method, the user could modify the system unexpectedly.

Use the System.Management.Automation.Cmdlet.ShouldProcess* method for any system modification. A user preference and the `WhatIf` parameter control the System.Management.Automation.Cmdlet.ShouldProcess* method. In contrast, the System.Management.Automation.Cmdlet.ShouldContinue* call performs an additional check for potentially dangerous modifications. This method isn't controlled by any user preference or the `WhatIf` parameter. If your cmdlet calls the System.Management.Automation.Cmdlet.ShouldContinue* method, it should have a `Force` parameter that bypasses the calls to these two methods and that proceeds with the operation. This is important because it allows your cmdlet to be used in non-interactive scripts and hosts.

If your cmdlets support these calls, the user can determine whether the action should actually be performed. For example, the Stop-Process cmdlet calls the System.Management.Automation.Cmdlet.ShouldContinue* method before it stops a set of critical processes, including the System, Winlogon, and Spoolsv processes.

For more information about supporting these methods, see Requesting Confirmation.

Support Force parameter for interactive sessions (RD05)

If your cmdlet is used interactively, always provide a Force parameter to override the interactive actions, such as prompts or reading lines of input. This is important because it allows your cmdlet

to be used in non-interactive scripts and hosts. The following methods can be implemented by an interactive host.

- System.Management.Automation.Host.PSHostUserInterface.Prompt*
- System.Management.Automation.Host.PSHostUserInterface.PromptForChoice
- System.Management.Automation.Host.IHostUISupportsMultipleChoiceSelection.PromptForChoice
- System.Management.Automation.Host.PSHostUserInterface.PromptForCredential*
- System.Management.Automation.Host.PSHostUserInterface.ReadLine*
- System.Management.Automation.Host.PSHostUserInterface.ReadLineAsSecureString*

Document output objects (RD06)

Windows PowerShell uses the objects that are written to the pipeline. In order for users to take advantage of the objects that are returned by each cmdlet, you must document the objects that are returned, and you must document what the members of those returned objects are used for.

Code guidelines

The following guidelines must be followed when writing cmdlet code. When you find a Code guideline that applies to your situation, be sure to look at the Design guidelines for similar guidelines.

Derive from the Cmdlet or PSCmdlet classes (RC01)

A cmdlet must derive from either the System.Management.Automation.Cmdlet or System.Management.Automation.PSCmdlet base class. Cmdlets that derive from the System.Management.Automation.Cmdlet class don't depend on the Windows PowerShell runtime. They can be called directly from any Microsoft .NET Framework language. Cmdlets that derive from the System.Management.Automation.PSCmdlet class depend on the Windows PowerShell runtime. Therefore, they execute within a runspace.

All cmdlet classes that you implement must be public classes. For more information about these cmdlet classes, see Cmdlet Overview.

Specify the Cmdlet attribute (RC02)

For a cmdlet to be recognized by Windows PowerShell, its .NET Framework class must be decorated with the Cmdlet attribute. This attribute specifies the following features of the cmdlet.

- The verb-and-noun pair that identifies the cmdlet.
- The default parameter set that's used when multiple parameter sets are specified. The default parameter set is used when Windows PowerShell doesn't have enough information to determine which parameter set to use.
- Indicates if the cmdlet supports calls to the System.Management.Automation.Cmdlet.ShouldProcess* method. This method displays a confirmation message to the user before the cmdlet makes a change to the system. For more information about how confirmation requests are made, see Requesting Confirmation.
- Indicate the impact level (or severity) of the action associated with the confirmation message. In most cases, the default value of Medium should be used. For more information about how the impact level affects the confirmation requests that are displayed to the user, see Requesting Confirmation.

For more information about how to declare the cmdlet attribute, see CmdletAttribute Declaration.

Override an input processing method (RC03)

For the cmdlet to participate in the Windows PowerShell environment, it must override at least one of the following *input processing methods*.

- System.Management.Automation.Cmdlet.BeginProcessing This method is called one time, and it is used to provide pre-processing functionality.
- System.Management.Automation.Cmdlet.ProcessRecord This method is called multiple times, and it's used to provide record-by-record functionality.
- System.Management.Automation.Cmdlet.EndProcessing This method is called one time, and it's used to provide post-processing functionality.

Specify the OutputType attribute (RC04)

The OutputType attribute (introduced in Windows PowerShell 2.0) specifies the .NET Framework type that your cmdlet returns to the pipeline. By specifying the output type of your cmdlets you make the objects returned by your cmdlet more discoverable by other cmdlets. For more information about decorating the cmdlet class with this attribute, see OutputType Attribute Declaration.

Don't retain handles to output objects (RC05)

Your cmdlet shouldn't retain any handles to the objects that are passed to the System.Management.Automation.Cmdlet.WriteObject* method. These objects are passed to the next cmdlet in the pipeline, or they're used by a script. If you retain the handles to the objects, two entities will own each object, which causes errors.

Handle errors robustly (RC06)

An administration environment inherently detects and makes important changes to the system that you are administering. Therefore, it's vital that cmdlets handle errors correctly. For more information about error records, see Windows PowerShell Error Reporting.

- When an error prevents a cmdlet from continuing to process any more records, it's a terminating error. The cmdlet must call the System.Management.Automation.Cmdlet.ThrowTerminatingError* method that references an System.Management.Automation.ErrorRecord object. If an exception isn't caught by the cmdlet, the Windows PowerShell runtime itself throws a terminating error that contains less information.

- For a non-terminating error that doesn't stop operation on the next record that's coming from the pipeline (for example, a record produced by a different process), the cmdlet must call the System.Management.Automation.Cmdlet.WriteError* method that references an System.Management.Automation.ErrorRecord object. An example of a non-terminating error is the error that occurs if a particular process fails to stop. Calling the System.Management.Automation.Cmdlet.WriteError* method allows the user to consistently perform the actions requested and to retain the information for particular actions that fail. Your cmdlet should handle each record as independently as possible.

- The System.Management.Automation.ErrorRecord object that's referenced by the System.Management.Automation.Cmdlet.ThrowTerminatingError* and System.Management.Automation.Cmdlet.WriteError* methods requires an exception at its core. Follow the .NET Framework design guidelines when you determine the exception to use. If the error is semantically the same as an existing exception, use that exception or derive from that exception. Otherwise, derive a new exception or exception hierarchy directly from the System.Exception type.

An System.Management.Automation.ErrorRecord object also requires an error category that groups errors for the user. The user can view errors based on the category by setting the value of the `$ErrorView` shell variable to CategoryView. The possible categories are defined by the System.Management.Automation.ErrorCategory enumeration.

- If a cmdlet creates a new thread, and if the code that's running in that thread throws an unhandled exception, Windows PowerShell can't catch the error and terminates the process.
- If an object has code in its destructor that causes an unhandled exception, Windows PowerShell can't catch the error and terminates the process. This also occurs if an object calls Dispose methods that cause an unhandled exception.

Use a Windows PowerShell module to deploy your cmdlets (RC07)

Create a Windows PowerShell module to package and deploy your cmdlets. Support for modules is introduced in Windows PowerShell 2.0. You can use the assemblies that contain your cmdlet classes directly as binary module files (this is very useful when testing your cmdlets), or you can create a module manifest that references the cmdlet assemblies. You can also add existing snap-in assemblies when using modules. For more information about modules, see Writing a Windows PowerShell Module.

See also

- Strongly Encouraged Development Guidelines
- Advisory Development Guidelines
- Writing a Windows PowerShell Cmdlet

Last updated on 06/16/2026

Strongly Encouraged Development Guidelines

This section describes guidelines that you should follow when you write your cmdlets. They are separated into guidelines for designing cmdlets and guidelines for writing your cmdlet code. You might find that these guidelines are not applicable for every scenario. However, if they do apply and you do not follow these guidelines, your users might have a poor experience when they use your cmdlets.

Design Guidelines

The following guidelines should be followed when designing cmdlets to ensure a consistent user experience between using your cmdlets and other cmdlets. When you find a Design guideline that applies to your situation, be sure to look at the Code guidelines for similar guidelines.

Use a Specific Noun for a Cmdlet Name (SD01)

Nouns used in cmdlet naming need to be very specific so that the user can discover your cmdlets. Prefix generic nouns such as "server" with a shortened version of the product name. For example, if a noun refers to a server that is running an instance of Microsoft SQL Server, use a noun such as "SQLServer". The combination of specific nouns and the short list of approved verbs enable the user to quickly discover and anticipate functionality while avoiding duplication among cmdlet names.

To enhance the user experience, the noun that you choose for a cmdlet name should be singular. For example, use the name `Get-Process` instead of `Get-Processes`. It is best to follow this rule for all cmdlet names, even when a cmdlet is likely to act upon more than one item.

Use Pascal Case for Cmdlet Names (SD02)

Use Pascal case for parameter names. In other words, capitalize the first letter of verb and all terms used in the noun. For example, "`Clear-ItemProperty`".

Parameter Design Guidelines (SD03)

A cmdlet needs parameters that receive the data on which it must operate, and parameters that indicate information that is used to determine the characteristics of the operation. For example, a cmdlet might have a `Name` parameter that receives data from the pipeline, and the cmdlet might have a `Force` parameter to indicate that the cmdlet can be forced to perform its operation. There is no limit to the number of parameters that a cmdlet can define.

Use Standard Parameter Names

Your cmdlet should use standard parameter names so that the user can quickly determine what a particular parameter means. If a more specific name is required, use a standard parameter name, and then specify a more specific name as an alias. For example, the `Get-Service` cmdlet has a parameter that has a generic name (`Name`) and a more specific alias (`ServiceName`). Both terms can be used to specify the parameter.

For more information about parameter names and their data types, see Cmdlet Parameter Name and Functionality Guidelines.

Use Singular Parameter Names

Avoid using plural names for parameters whose value is a single element. This includes parameters that take arrays or lists because the user might supply an array or list with only one element.

Plural parameter names should be used only in those cases where the value of the parameter is always a multiple-element value. In these cases, the cmdlet should verify that multiple elements are supplied, and the cmdlet should display a warning to the user if multiple elements are not supplied.

Use Pascal Case for Parameter Names

Use Pascal case for parameter names. In other words, capitalize the first letter of each word in the parameter name, including the first letter of the name. For example, the parameter name `ErrorAction` uses the correct capitalization. The following parameter names use incorrect capitalization:

- `errorAction`
- `erroraction`

Parameters That Take a List of Options

There are two ways to create a parameter whose value can be selected from a set of options.

- Define an enumeration type (or use an existing enumeration type) that specifies the valid values. Then, use the enumeration type to create a parameter of that type.
- Add the **ValidateSet** attribute to the parameter declaration. For more information about this attribute, see ValidateSet Attribute Declaration.

Use Standard Types for Parameters

To ensure consistency with other cmdlets, use standard types for parameters where ever possible. For more information about the types that should be used for different parameter, see Standard Cmdlet Parameter Names and Types. This topic provides links to several topics that describe the names and .NET Framework types for groups of standard parameters, such as the "activity parameters".

Use Strongly-Typed .NET Framework Types

Parameters should be defined as .NET Framework types to provide better parameter validation. For example, parameters that are restricted to one value from a set of values should be defined as an enumeration type. To support a Uniform Resource Identifier (URI) value, define the parameter as a System.Uri type. Avoid basic string parameters for all but free-form text properties.

Use Consistent Parameter Types

When the same parameter is used by multiple cmdlets, always use the same parameter type. For example, if the `Process` parameter is a System.Int16 type for one cmdlet, do not make the `Process` parameter for another cmdlet a System.Uint16 type.

Parameters That Take True and False

If your parameter takes only `true` and `false`, define the parameter as type System.Management.Automation.SwitchParameter. A `[switch]` parameter is treated as `true` when it is specified in a command. If the parameter is not included in a command, Windows PowerShell considers the value of the parameter to be `false`. Do not define Boolean parameters.

If your parameter needs to differentiate between 3 values: $true, $false and "unspecified", then define a parameter of type Nullable<bool>. The need for a 3rd, "unspecified" value typically occurs when the cmdlet can modify a Boolean property of an object. In this case "unspecified" means to not change the current value of the property.

Support Arrays for Parameters

Frequently, users must perform the same operation against multiple arguments. For these users, a cmdlet should accept an array as parameter input so that a user can pass the arguments into the parameter as a Windows PowerShell variable. For example, the Get-Process cmdlet uses an array for the strings that identify the names of the processes to retrieve.

Support the PassThru Parameter

By default, many cmdlets that modify the system, such as the Stop-Process cmdlet, act as "sinks" for objects and do not return a result. These cmdlet should implement the `PassThru` parameter to force the cmdlet to return an object. When the `PassThru` parameter is specified, the cmdlet returns an object by using a call to the System.Management.Automation.Cmdlet.WriteObject method. For example, the following command stops the Calc (CalculatorApp.exe) and passes the resultant process to the pipeline.

PowerShell

```
Stop-Process -Name CalculatorApp -PassThru
```

In most cases, Add, Set, and New cmdlets should support a `PassThru` parameter.

Support Parameter Sets

A cmdlet is intended to accomplish a single purpose. However, there is frequently more than one way to describe the operation or the operation target. For example, a process might be identified by its name, by its identifier, or by a process object. The cmdlet should support all the reasonable representations of its targets. Normally, the cmdlet satisfies this requirement by specifying sets of parameters (referred to as parameter sets) that operate together. A single parameter can belong to any number of parameter sets. For more information about parameter sets, see Cmdlet Parameter Sets.

When you specify parameter sets, set only one parameter in the set to ValueFromPipeline. For more information about how to declare the **Parameter** attribute, see ParameterAttribute Declaration.

When parameter sets are used, the default parameter set is defined by the **Cmdlet** attribute. The default parameter set should include the parameters most likely to be used in an interactive Windows PowerShell session. For more information about how to declare the **Cmdlet** attribute, see CmdletAttribute Declaration.

Provide Feedback to the User (SD04)

Use the guidelines in this section to provide feedback to the user. This feedback allows the user to be aware of what is occurring in the system and to make better administrative decisions.

The Windows PowerShell runtime allows a user to specify how to handle output from each call to the `Write` method by setting a preference variable. The user can set several preference variables, including a variable that determines whether the system should display information and a variable that determines whether the system should query the user before taking further action.

Support the WriteWarning, WriteVerbose, and WriteDebug Methods

A cmdlet should call the System.Management.Automation.Cmdlet.WriteWarning method when the cmdlet is about to perform an operation that might have an unintended result. For example, a cmdlet should call this method if the cmdlet is about to overwrite a read-only file.

A cmdlet should call the System.Management.Automation.Cmdlet.WriteVerbose method when the user requires some detail about what the cmdlet is doing. For example, a cmdlet should call this information if the cmdlet author feels that there are scenarios that might require more information about what the cmdlet is doing.

The cmdlet should call the System.Management.Automation.Cmdlet.WriteDebug method when a developer or product support engineer must understand what has corrupted the cmdlet operation. It is not necessary for the cmdlet to call the System.Management.Automation.Cmdlet.WriteDebug method in the same code that calls the System.Management.Automation.Cmdlet.WriteVerbose method because the `Debug` parameter presents both sets of information.

Support WriteProgress for Operations that take a Long Time

Cmdlet operations that take a long time to complete and that cannot run in the background should support progress reporting through periodic calls to the System.Management.Automation.Cmdlet.WriteProgress method.

Use the Host Interfaces

Occasionally, a cmdlet must communicate directly with the user instead of by using the various Write or Should methods supported by the System.Management.Automation.Cmdlet class. In this case, the cmdlet should derive from the System.Management.Automation.PSCmdlet class

and use the System.Management.Automation.PSCmdlet.Host* property. This property supports different levels of communication type, including the PromptForChoice, Prompt, and WriteLine/ReadLine types. At the most specific level, it also provides ways to read and write individual keys and to deal with buffers.

Unless a cmdlet is specifically designed to generate a graphical user interface (GUI), it should not bypass the host by using the System.Management.Automation.PSCmdlet.Host* property. An example of a cmdlet that is designed to generate a GUI is the Out-GridView cmdlet.

> **Note**
>
> Cmdlets should not use the System.Console API.

Create a Cmdlet Help File (SD05)

For each cmdlet assembly, create a Help.xml file that contains information about the cmdlet. This information includes a description of the cmdlet, descriptions of the cmdlet's parameters, examples of the cmdlet's use, and more.

Code Guidelines

The following guidelines should be followed when coding cmdlets to ensure a consistent user experience between using your cmdlets and other cmdlets. When you find a Code guideline that applies to your situation, be sure to look at the Design guidelines for similar guidelines.

Coding Parameters (SC01)

Define a parameter by declaring a public property of the cmdlet class that is decorated with the **Parameter** attribute. Parameters do not have to be static members of the derived .NET Framework class for the cmdlet. For more information about how to declare the **Parameter** attribute, see Parameter Attribute Declaration.

Support Windows PowerShell Paths

The Windows PowerShell path is the mechanism for normalizing access to namespaces. When you assign a Windows PowerShell path to a parameter in the cmdlet, the user can define a custom "drive" that acts as a shortcut to a specific path. When a user designates such a drive, stored data, such as data in the Registry, can be used in a consistent way.

If your cmdlet allows the user to specify a file or a data source, it should define a parameter of type System.String. If more than one drive is supported, the type should be an array. The name of the parameter should be `Path`, with an alias of `PSPath`. Additionally, the `Path` parameter should support wildcard characters. If support for wildcard characters is not required, define a `LiteralPath` parameter.

If the data that the cmdlet reads or writes has to be a file, the cmdlet should accept Windows PowerShell path input, and the cmdlet should use the System.Management.Automation.SessionState.Path property to translate the Windows PowerShell paths into paths that the file system recognizes. The specific mechanisms include the following methods:

- System.Management.Automation.PSCmdlet.GetResolvedProviderPathFromPSPath
- System.Management.Automation.PSCmdlet.GetUnresolvedProviderPathFromPSPath
- System.Management.Automation.PathIntrinsics.GetResolvedProviderPathFromPSPath
- System.Management.Automation.PathIntrinsics.GetUnresolvedProviderPathFromPSPath

If the data that the cmdlet reads or writes is only a set of strings instead of a file, the cmdlet should use the provider content information (`Content` member) to read and write. This information is obtained from the System.Management.Automation.Provider.CmdletProvider.InvokeProvider property. These mechanisms allow other data stores to participate in the reading and writing of data.

Support Wildcard Characters

A cmdlet should support wildcard characters if possible. Support for wildcard characters occurs in many places in a cmdlet (especially when a parameter takes a string to identify one object from a set of objects). For example, the sample `Stop-Proc` cmdlet from the StopProc Tutorial defines a `Name` parameter to handle strings that represent process names. This parameter supports wildcard characters so that the user can easily specify the processes to stop.

When support for wildcard characters is available, a cmdlet operation usually produces an array. Occasionally, it does not make sense to support an array because the user might use only a single item at a time. For example, the Set-Location cmdlet does not need to support an array because the user is setting only a single location. In this instance, the cmdlet still supports wildcard characters, but it forces resolution to a single location.

For more information about wildcard-character patterns, see Supporting Wildcard Characters in Cmdlet Parameters.

Defining Objects

This section contains guidelines for defining objects for cmdlets and for extending existing objects.

Define Standard Members

Define standard members to extend an object type in a custom Types.ps1xml file (use the Windows PowerShell Types.ps1xml file as a template). Standard members are defined by a node with the name PSStandardMembers. These definitions allow other cmdlets and the Windows PowerShell runtime to work with your object in a consistent way.

Define ObjectMembers to Be Used as Parameters

If you are designing an object for a cmdlet, ensure that its members map directly to the parameters of the cmdlets that will use it. This mapping allows the object to be easily sent to the pipeline and to be passed from one cmdlet to another.

Preexisting .NET Framework objects that are returned by cmdlets are frequently missing some important or convenient members that are needed by the script developer or user. These missing members can be particularly important for display and for creating the correct member names so that the object can be correctly passed to the pipeline. Create a custom Types.ps1xml file to document these required members. When you create this file, we recommend the following naming convention: <*Your_Product_Name*>.Types.ps1xml.

For example, you could add a `Mode` script property to the System.IO.FileInfo type to display the attributes of a file more clearly. Additionally, you could add a `Count` alias property to the System.Array type to allow the consistent use of that property name (instead of `Length`).

Implement the IComparable Interface

Implement a System.IComparable interface on all output objects. This allows the output objects to be easily piped to various sorting and analysis cmdlets.

Update Display Information

If the display for an object does not provide the expected results, create a custom <*YourProductName*>.Format.ps1xml file for that object.

Support Well Defined Pipeline Input (SC02)

Implement for the Middle of a Pipeline

Implement a cmdlet assuming that it will be called from the middle of a pipeline (that is, other cmdlets will produce its input or consume its output). For example, you might assume that the `Get-Process` cmdlet, because it generates data, is used only as the first cmdlet in a pipeline. However, because this cmdlet is designed for the middle of a pipeline, this cmdlet allows previous cmdlets or data in the pipeline to specify the processes to retrieve.

Support Input from the Pipeline

In each parameter set for a cmdlet, include at least one parameter that supports input from the pipeline. Support for pipeline input allows the user to retrieve data or objects, to send them to the correct parameter set, and to pass the results directly to a cmdlet.

A parameter accepts input from the pipeline if the **Parameter** attribute includes the `ValueFromPipeline` keyword, the `ValueFromPipelineByPropertyName` keyword attribute, or both keywords in its declaration. If none of the parameters in a parameter set support the `ValueFromPipeline` or `ValueFromPipelineByPropertyName` keywords, the cmdlet cannot meaningfully be placed after another cmdlet because it will ignore any pipeline input.

Support the ProcessRecord Method

To accept all the records from the preceding cmdlet in the pipeline, your cmdlet must implement the System.Management.Automation.Cmdlet.ProcessRecord method. Windows PowerShell calls this method multiple times, once for every record that is sent to your cmdlet.

Write Single Records to the Pipeline (SC03)

When a cmdlet returns objects, the cmdlet should write the objects immediately as they are generated. The cmdlet should not hold them in order to buffer them into a combined array. The cmdlets that receive the objects as input will then be able to process, display, or process and display the output objects without delay. A cmdlet that generates output objects one at a time should call the System.Management.Automation.Cmdlet.WriteObject method. A cmdlet that generates output objects in batches (for example, because an underlying API returns an array of output objects) should call the System.Management.Automation.Cmdlet.WriteObject Method with its second parameter set to `true`.

Make Cmdlets Case-Insensitive and Case-Preserving (SC04)

By default, Windows PowerShell itself is case-insensitive. However, because it deals with many preexisting systems, Windows PowerShell does preserve case for ease of operation and compatibility. In other words, if a character is supplied in uppercase letters, Windows

PowerShell keeps it in uppercase letters. For systems to work well, a cmdlet needs to follow this convention. If possible, it should operate in a case-insensitive way. It should, however, preserve the original case for cmdlets that occur later in a command or in the pipeline.

See Also

Required Development Guidelines

Advisory Development Guidelines

Writing a Windows PowerShell Cmdlet

Last updated on 04/08/2026

Advisory Development Guidelines

Article • 05/22/2025

This section describes guidelines that you should consider to ensure good development and user experiences. Sometimes they might apply, and sometimes they might not.

Design Guidelines

The following guidelines should be considered when designing cmdlets. When you find a Design guideline that applies to your situation, be sure to look at the Code guidelines for similar guidelines.

Support an InputObject Parameter (AD01)

Because Windows PowerShell works directly with Microsoft .NET Framework objects, a .NET Framework object is often available that exactly matches the type the user needs to perform a particular operation. `InputObject` is the standard name for a parameter that takes such an object as input. For example, the sample `Stop-Proc` cmdlet in the StopProc Tutorial defines an `InputObject` parameter of type Process that supports the input from the pipeline. The user can get a set of process objects, manipulate them to select the exact objects to stop, and then pass them to the `Stop-Proc` cmdlet directly.

Support the Force Parameter (AD02)

Occasionally, a cmdlet needs to protect the user from performing a requested operation. Such a cmdlet should support a **Force** parameter to allow the user to override that protection if the user has permissions to perform the operation.

For example, the Remove-Item cmdlet doesn't normally remove a read-only file. However, this cmdlet supports a **Force** parameter so a user can force removal of a read-only file. If the user already has permission to modify the read-only attribute, and the user removes the file, use of the **Force** parameter simplifies the operation. However, if the user doesn't have permission to remove the file, the **Force** parameter has no effect.

Handle Credentials Through Windows PowerShell (AD03)

A cmdlet should define a `Credential` parameter to represent credentials. This parameter must be of type System.Management.Automation.PSCredential and must be defined using a Credential attribute declaration. This support automatically prompts the user for the user

name, for the password, or for both when a full credential isn't supplied directly. For more information about the Credential attribute, see Credential Attribute Declaration.

Support Encoding Parameters (AD04)

If your cmdlet reads or writes text to or from a binary form, such as writing to or reading from a file in a filesystem, then your cmdlet has to have Encoding parameter that specifies how the text is encoded in the binary form.

Test Cmdlets Should Return a Boolean (AD05)

Cmdlets that perform tests against their resources should return a System.Boolean type to the pipeline so that they can be used in conditional expressions.

Code Guidelines

The following guidelines should be considered when writing cmdlet code. When you find a guideline that applies to your situation, be sure to look at the Design guidelines for similar guidelines.

Follow Cmdlet Class Naming Conventions (AC01)

By following standard naming conventions, you make your cmdlets more discoverable, and you help the user understand exactly what the cmdlets do. This practice is particularly important for other developers using Windows PowerShell because cmdlets are public types.

Define a Cmdlet in the Correct Namespace

You normally define the class for a cmdlet in a .NET Framework namespace that appends `.Commands` to the namespace that represents the product in which the cmdlet runs. For example, cmdlets that are included with Windows PowerShell are defined in the `Microsoft.PowerShell.Commands` namespace.

Name the Cmdlet Class to Match the Cmdlet Name

When you name the .NET Framework class that implements a cmdlet, name the class `<Verb><Noun>Command`, where you replace the `<Verb>` and `<Noun>` placeholders with the verb and noun used for the cmdlet name. For example, the Get-Process cmdlet is implemented by a class called `GetProcessCommand`.

If No Pipeline Input Override the BeginProcessing Method (AC02)

If your cmdlet doesn't accept input from the pipeline, processing should be implemented in the System.Management.Automation.Cmdlet.BeginProcessing method. Use of this method allows Windows PowerShell to maintain ordering between cmdlets. The first cmdlet in the pipeline always returns its objects before the remaining cmdlets in the pipeline get a chance to start their processing.

To Handle Stop Requests Override the StopProcessing Method (AC03)

Override the System.Management.Automation.Cmdlet.StopProcessing method so that your cmdlet can handle stop signal. Some cmdlets take a long time to complete their operation, and they let a long time pass between calls to the Windows PowerShell runtime, such as when the cmdlet blocks the thread in long-running RPC calls. This includes cmdlets that make calls to the System.Management.Automation.Cmdlet.WriteObject method, to the System.Management.Automation.Cmdlet.WriteError method, and to other feedback mechanisms that may take a long time to complete. For these cases the user might need to send a stop signal to these cmdlets.

Implement the IDisposable Interface (AC04)

If your cmdlet has objects that aren't disposed of (written to the pipeline) by the System.Management.Automation.Cmdlet.ProcessRecord method, your cmdlet might require additional object disposal. For example, if your cmdlet opens a file handle in its System.Management.Automation.Cmdlet.BeginProcessing method and keeps the handle open for use by the System.Management.Automation.Cmdlet.ProcessRecord method, this handle has to be closed at the end of processing.

The Windows PowerShell runtime doesn't always call the System.Management.Automation.Cmdlet.EndProcessing method. For example, the System.Management.Automation.Cmdlet.EndProcessing method might not be called if the cmdlet is canceled midway through its operation or if a terminating error occurs in any part of the cmdlet. Therefore, the .NET Framework class for a cmdlet that requires object cleanup should implement the complete System.IDisposable interface pattern, including the finalizer, so that the Windows PowerShell runtime can call both the System.Management.Automation.Cmdlet.EndProcessing and System.IDisposable.Dispose* methods at the end of processing.

Use Serialization-friendly Parameter Types (AC05)

To support running your cmdlet on remote computers, use types that can be serialized on the client computer and then rehydrated on the server computer. The follow types are serialization-friendly.

Primitive types:

- Byte, SByte, Decimal, Single, Double, Int16, Int32, Int64, Uint16, UInt32, and UInt64.
- Boolean, Guid, Byte[], TimeSpan, DateTime, Uri, and Version.
- Char, String, XmlDocument.

Built-in rehydratable types:

- PSPrimitiveDictionary
- SwitchParameter
- PSListModifier
- PSCredential
- IPAddress, MailAddress
- CultureInfo
- X509Certificate2, X500DistinguishedName
- DirectorySecurity, FileSecurity, RegistrySecurity

Other types:

- SecureString
- Containers (lists and dictionaries of the above type)

Use SecureString for Sensitive Data (AC06)

When handling sensitive data always use the System.Security.SecureString data type. This could include pipeline input to parameters, as well as returning sensitive data to the pipeline.

While .NET recommends against using **SecureString** for new development, PowerShell continues to support the **SecureString** class for backward compatibility. Using a **SecureString** is still more secure than using a plain text string. PowerShell still relies on the **SecureString** type to avoid accidentally exposing the contents to the console or in logs. Use **SecureString** carefully, because it can be easily converted to a plain text string. For a full discussion about using **SecureString**, see the System.Security.SecureString class documentation.

See Also

Required Development Guidelines

Strongly Encouraged Development Guidelines

Writing a Windows PowerShell Cmdlet

Cmdlet Class Declaration

A Microsoft .NET Framework class is declared as a cmdlet by specifying the **Cmdlet** attribute as metadata for the class. (The **Cmdlet** attribute is the only required attribute for all cmdlets). When you specify the **Cmdlet** attribute, you must specify the verb-and-noun pair that identifies the cmdlet to the user. And, you must describe the Windows PowerShell functionality that the cmdlet supports. For more information about the declaration syntax that is used to specify the **Cmdlet** attribute, see Cmdlet Attribute Declaration.

> **Note**
>
> The **Cmdlet** attribute is defined by the System.Management.Automation.CmdletAttribute class. The properties of this class correspond to the declaration parameters that are used when you declare the attribute.

Nouns

The noun of the cmdlet specifies the resources upon which the cmdlet acts. The noun differentiates your cmdlets from other cmdlets.

Nouns in cmdlet names must be specific, and in the case of generic nouns, such as *server*, it is best to add a short prefix that differentiates your resource from other similar resources. For example, a cmdlet name that includes a noun with a prefix is `Get-SQLServer`. The combination of a specific noun with a more general verb enables the user to quickly locate the cmdlet by its action and then identify the cmdlet by its resource while avoiding unnecessary cmdlet name duplication.

For a list of special characters that cannot be used in cmdlet names, see Required Development Guidelines.

Verbs

When you specify a verb, the development guidelines require you to use one of the predefined verbs provided by Windows PowerShell. By using one of these predefined verbs, you will ensure consistency between the cmdlets that you write and the cmdlets that are written by Microsoft and by others. For example, the "Get" verb is used for cmdlets that retrieve data.

For more information about guidelines for verbs, see Cmdlet Verb Names. For a list of special characters that cannot be used in cmdlet names, see Required Development Guidelines.

Supporting Windows PowerShell Functionality

The **Cmdlet** attribute also allows you to specify that your cmdlet supports some of the common functionality that is provided by Windows PowerShell. This includes support for common functionality such as user feedback confirmation (referred to as support for the ShouldProcess feature) and support for transactions. (Support for transactions was introduced in Windows PowerShell 2.0).

For more information about the declaration syntax that is used to specify the **Cmdlet** attribute, see Cmdlet Attribute Declaration.

Cmdlet Class Definition

The following code is the definition for a GetProc cmdlet class. Notice that Pascal casing is used and that the name of the class includes the verb and noun of the cmdlet.

C#

```
[Cmdlet(VerbsCommon.Get, "Proc")]
public class GetProcCommand : Cmdlet
```

Pascal Casing

When you name cmdlets, use Pascal casing. For example, the `Get-Item` and `Get-ItemProperty` cmdlets show the correct way to use capitalization when you are naming cmdlets.

See Also

System.Management.Automation.CmdletAttribute

CmdletAttribute Declaration

Cmdlet Verb Names

Writing a Windows PowerShell Cmdlet

Windows PowerShell SDK